TEMCO LEARN

T3000 Engineering

1.0 · 16 pages · built Nov 13, 2026

1. T3000 Engineering
2. Building T3000
3. Building T3000 from source
4. Compiling T3000 and making the setup file
5. The InstallShield project
6. Debugging the webview
7. Merging the new editor
8. Using C# components in T3000
9. Authoring the help file
10. The GitHub workflow
11. Get started with GitHub
12. Creating a release
13. TestTool: trend logs
14. TestTool: recording data
15. Building the Bravo Controls API
16. Updating the Airlab bootloader

T3000 Engineering

How T3000 is built, debugged and shipped: compiling the solution, making the setup file, attaching to the webview, the GitHub workflow, and the tools used to prove a fix.

Contents

- Building T3000
- Building T3000 from source
- Compiling T3000 and making the setup file
- The InstallShield project
- Debugging the webview
- Merging the new editor
- Using C# components in T3000
- Authoring the help file
- The GitHub workflow
- Get started with GitHub
- Creating a release
- TestTool: trend logs
- TestTool: recording data
- Building the Bravo Controls API
- Updating the Airlab bootloader

Building T3000

T3000 is a C++ application built with MFC, with a web front end in the same process tree and an installer around both. This project is the engineering side of it: how the solution is compiled, how the setup file is produced, how the webview is debugged, and how a change gets into the repository and out to a customer.

It is written for people who work on the product. The screenshots are the source material — the notes around them say what to look for.

Where to start

- **Building** — from a clean checkout to a working build, and then to the setup file an installer ships.
- **Debugging** — attaching to the webview, which is the part that is not the C++ shell.
- **The repository** — the GitHub workflow, and the release that turns a build into a download.
- **Tools** — TestTool for trend logs and recordings, the Bravo Controls API, and the Airlab bootloader.

Building T3000 from source

The steps to get a working build out of the repository: what to install, which solution to open, and what a clean build looks like when it has worked.

Temco Software for development

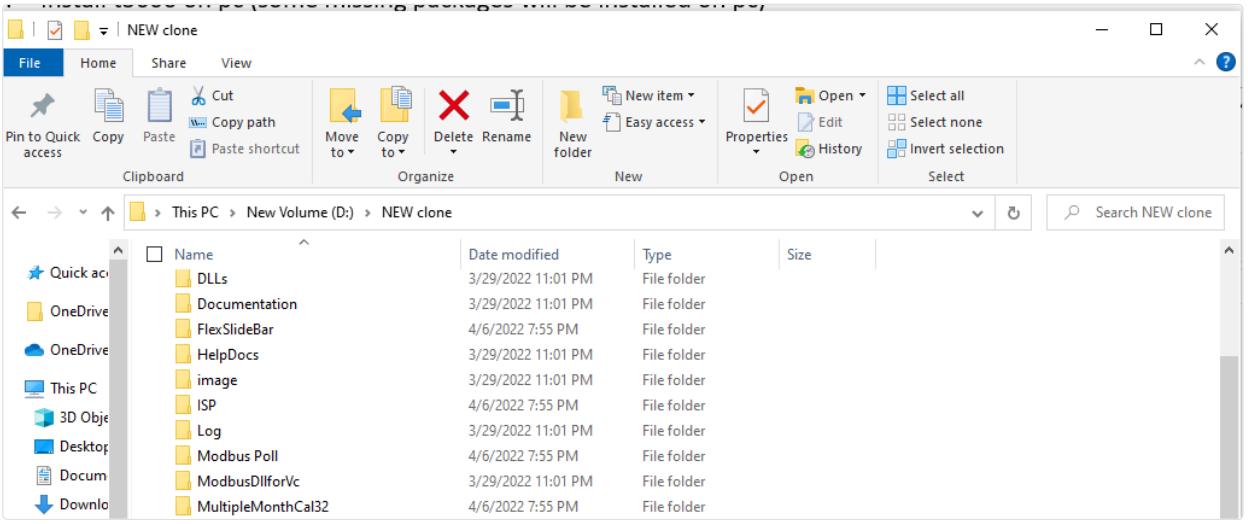
Build all the projects in solution

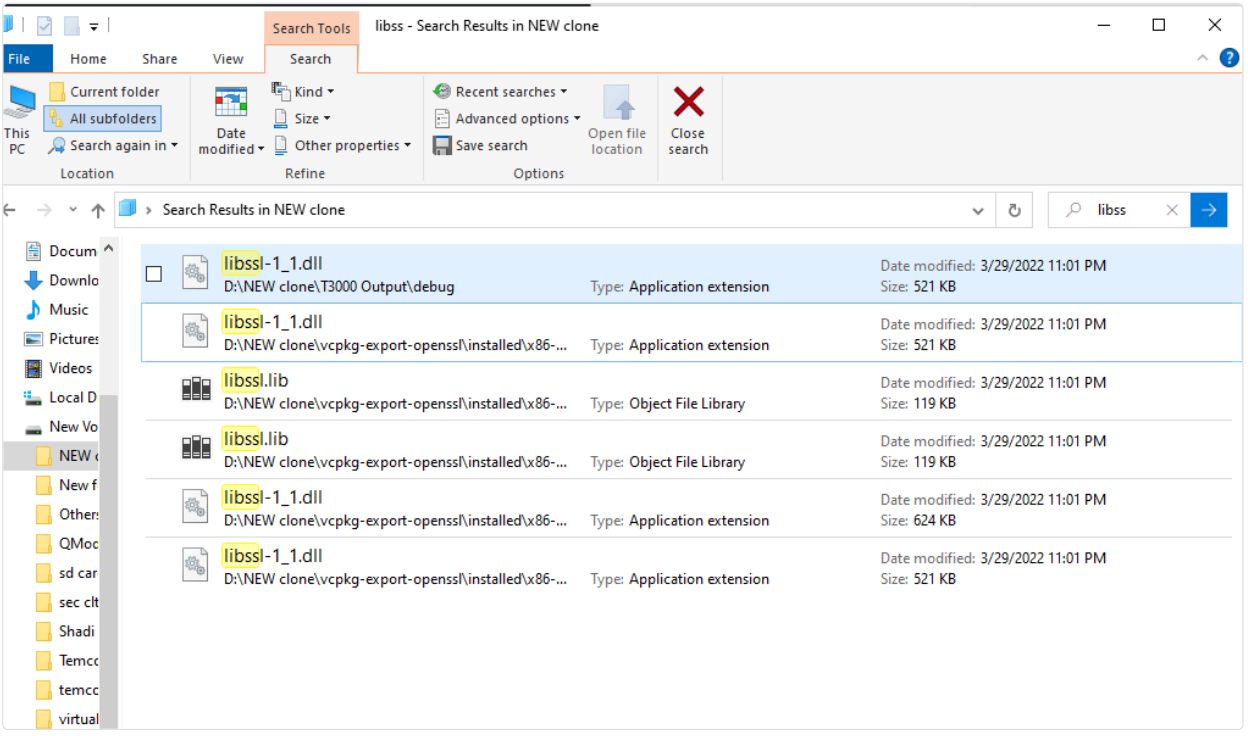
Right click on T3000 project and set as startup project

Install t3000 on pc (some missing packages will be installed on pc)

some libcrypto-1_1.dll and libssl-1_1.dll will be missing paste them in (T3000 Output/debug) folder

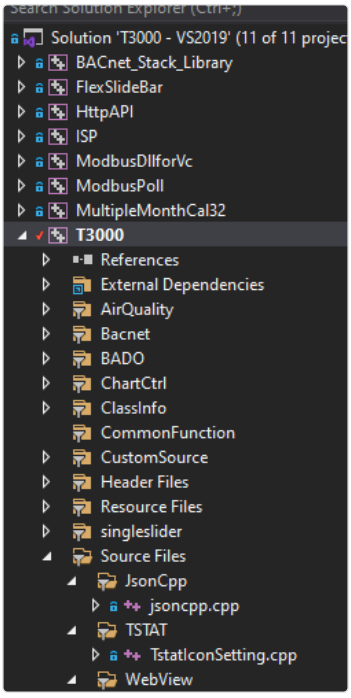
go to main folder and search here for the files



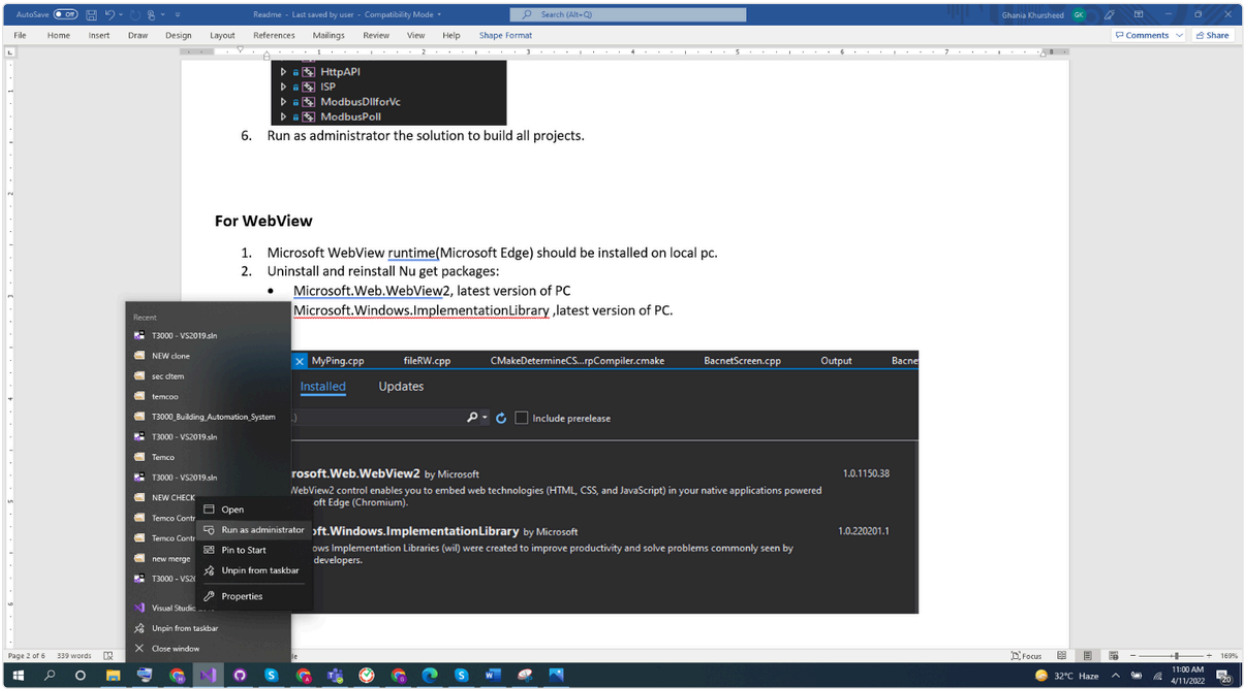


After completion its shown like that2nd step

debug assortion failed C++ mg=fc/occcont.cpp may occur that can be due to flexslideBar project rebuild it after T3000 installation



Run as administrator the solution to build all projects.



Select hereRight click here

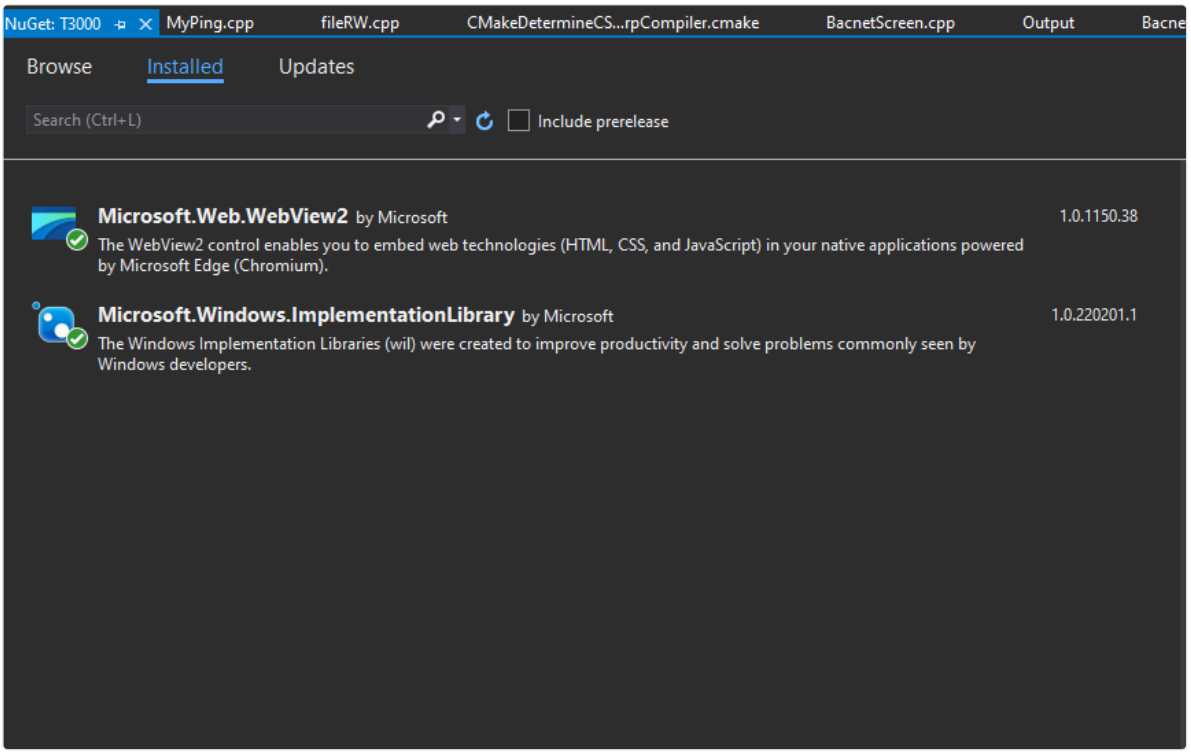
For WebView

Microsoft WebView runtime(Microsoft Edge) should be installed on local pc.

Uninstall and reinstall Nu get packages:

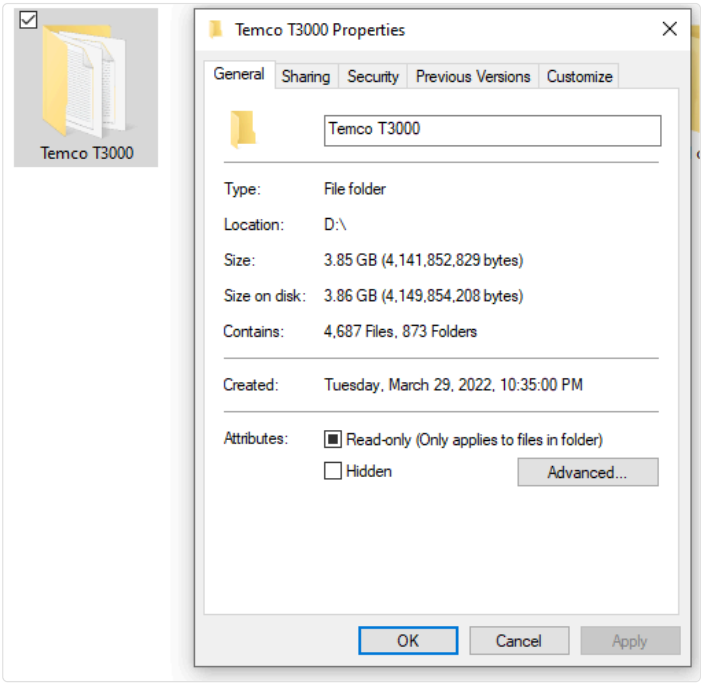
Microsoft.Web.WebView2, latest version of PC

Microsoft.Windows.ImplementationLibrary ,latest version of PC.



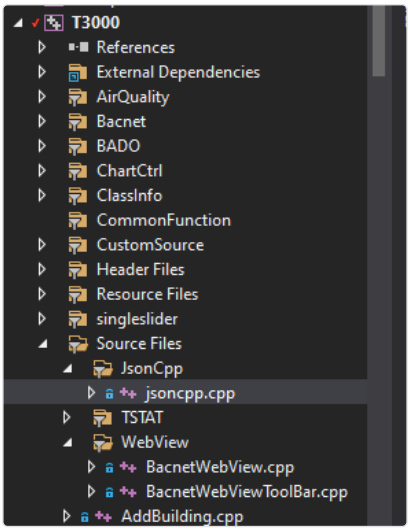
On Aqeeb Branch

Right click on main project file, go to the properties and uncheck read only option, then select apply to folders, subfolders and files and then okay.

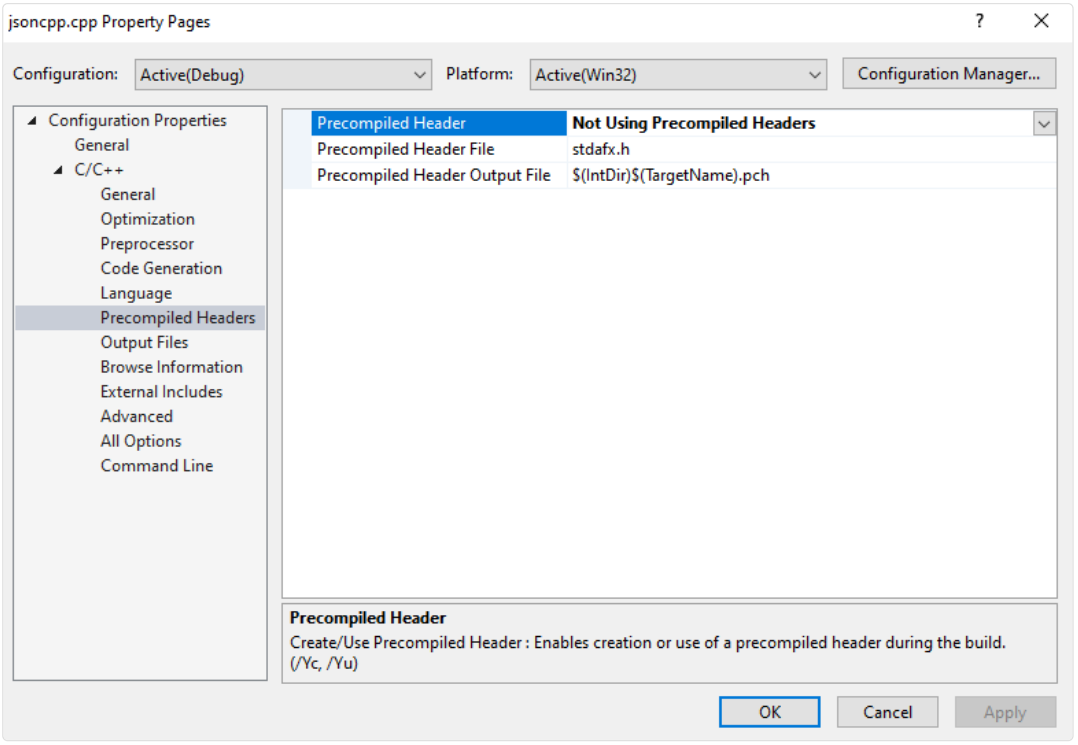


Uncheck hereRight click here

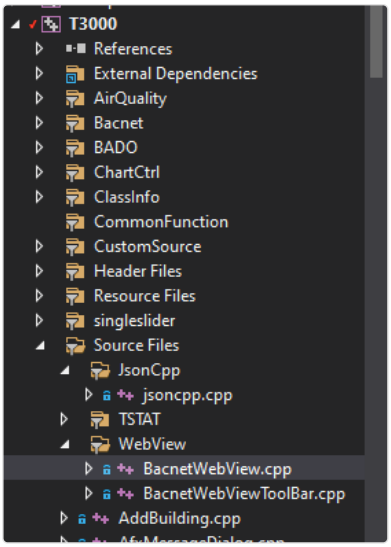
To access the property pages, choose T3000 > jsoncpp.cpp> Properties from the main menu, go to C/C++, on drop down menu go to precompiled header and set not using pre-compiled header



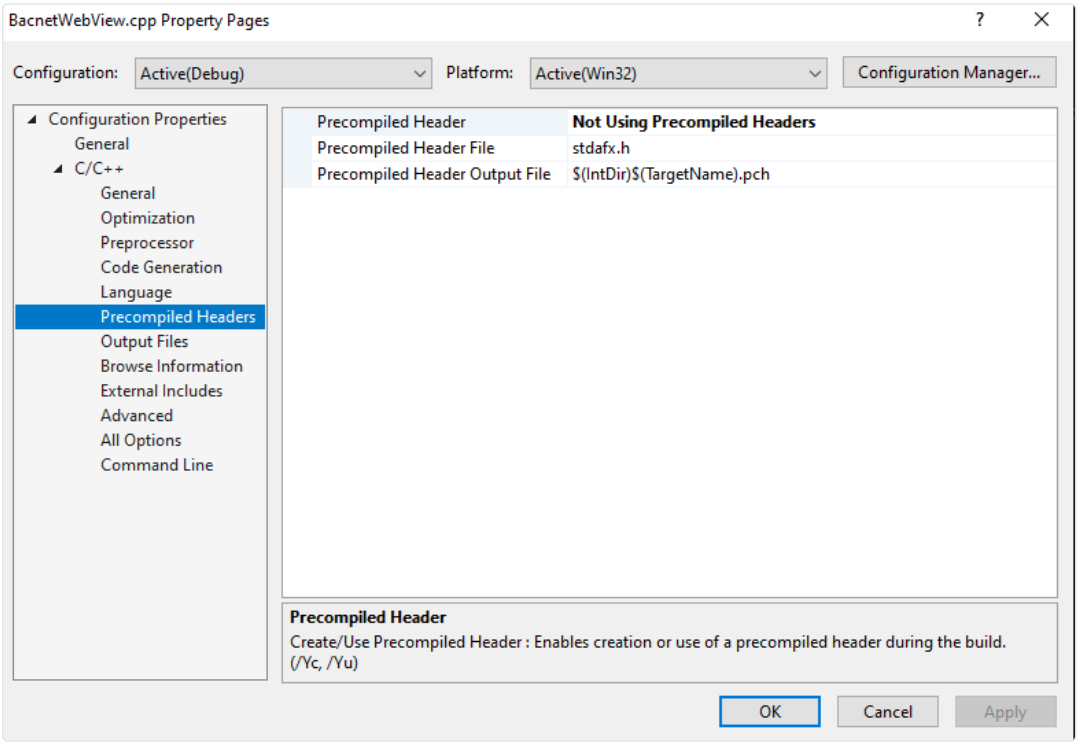
Right click here



To access the property pages, choose T3000 > WebView>BacnetWebView.cpp> Properties from the main menu, go to C/C++, on drop down menu go to precompiled header and set not using pre-compiled header



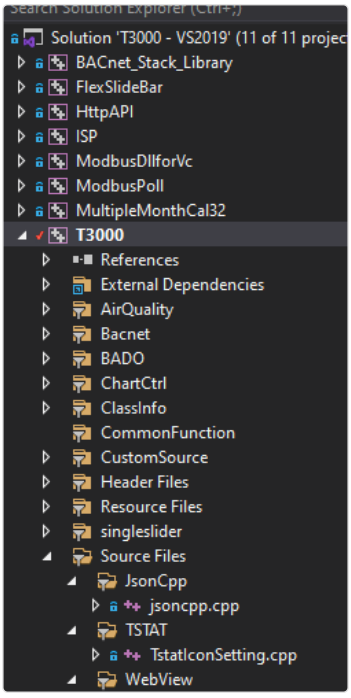
Right click here



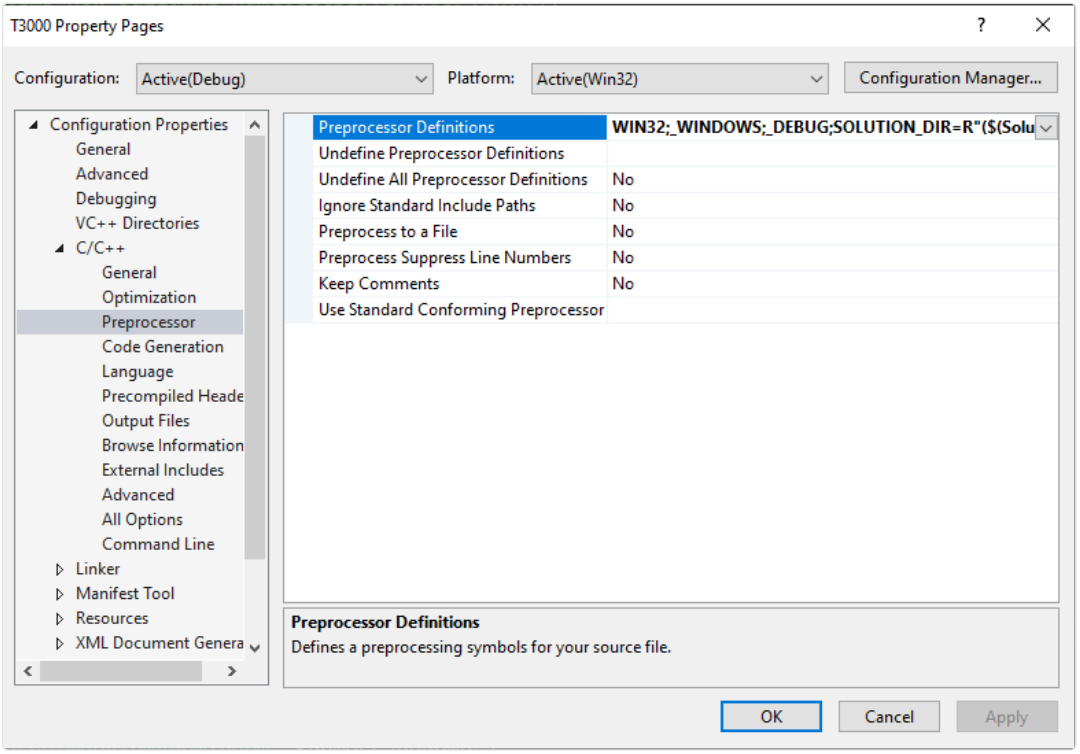
Copy and paste T3000.exe from T3000 folder to T3000 Output>debug

Search WebView2Loader.dll then copy and paste it into T3000 Output>debug

To access the property pages, choose T3000 right-click on the file and choose Properties., go to Prepocessor, on drop down menu go to General and click on Additional Include Directories and give this directory given below:WIN32;_WINDOWS;_DEBUG;SOLUTION_DIR=R"($(SolutionDir))";%(PreprocessorDefinitions)



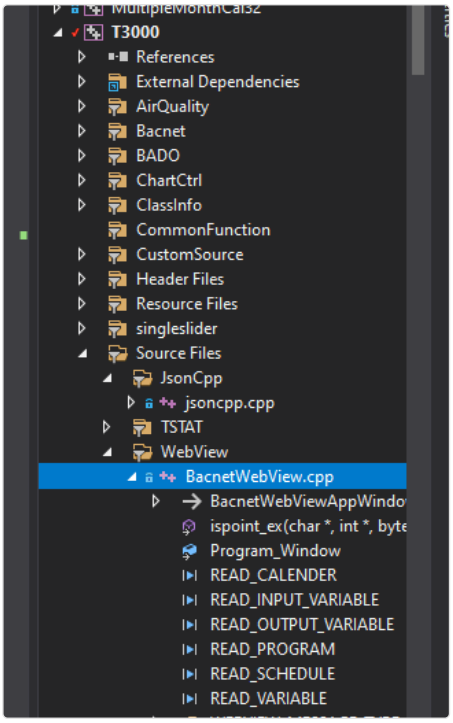
Right click here



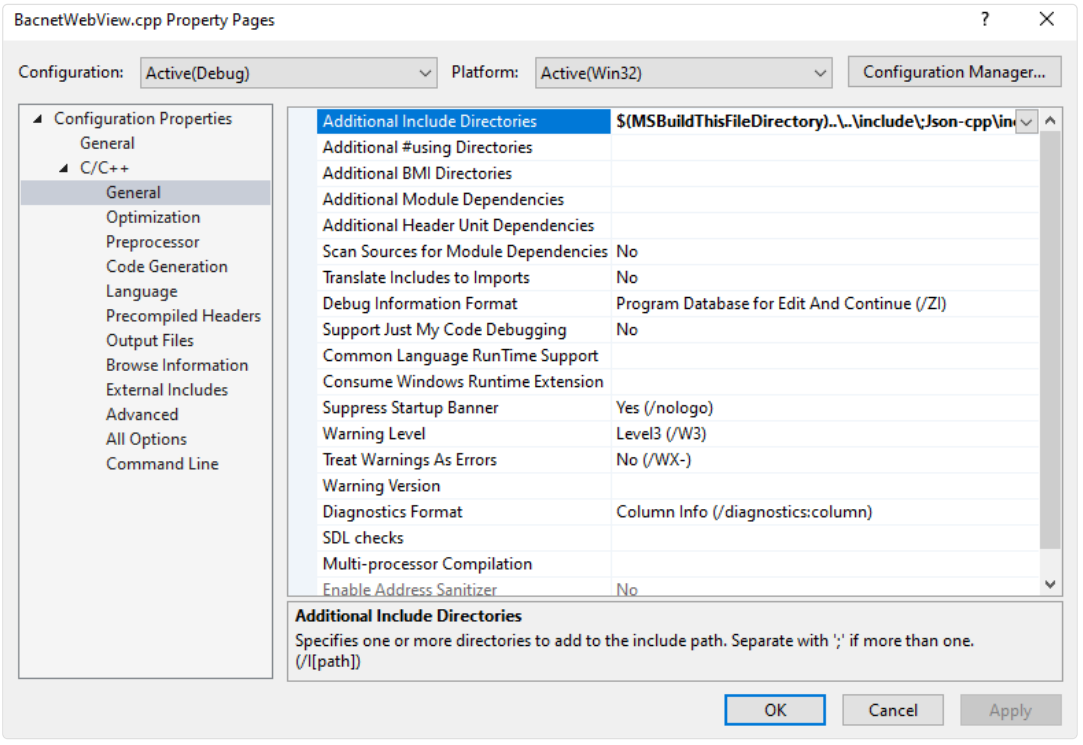
Include here

To access the property pages, choose T3000>source files>Web View>Bacnet WebView.cpp, right-click on the file and choose Properties., go to C/C++, on drop down menu go to General and click on Additional Include Directories and give this directory given below:

$(MSBuildThisFileDirectory)....\include;Json-cpp\include;%(AdditionalIncludeDirectories)



Right click here



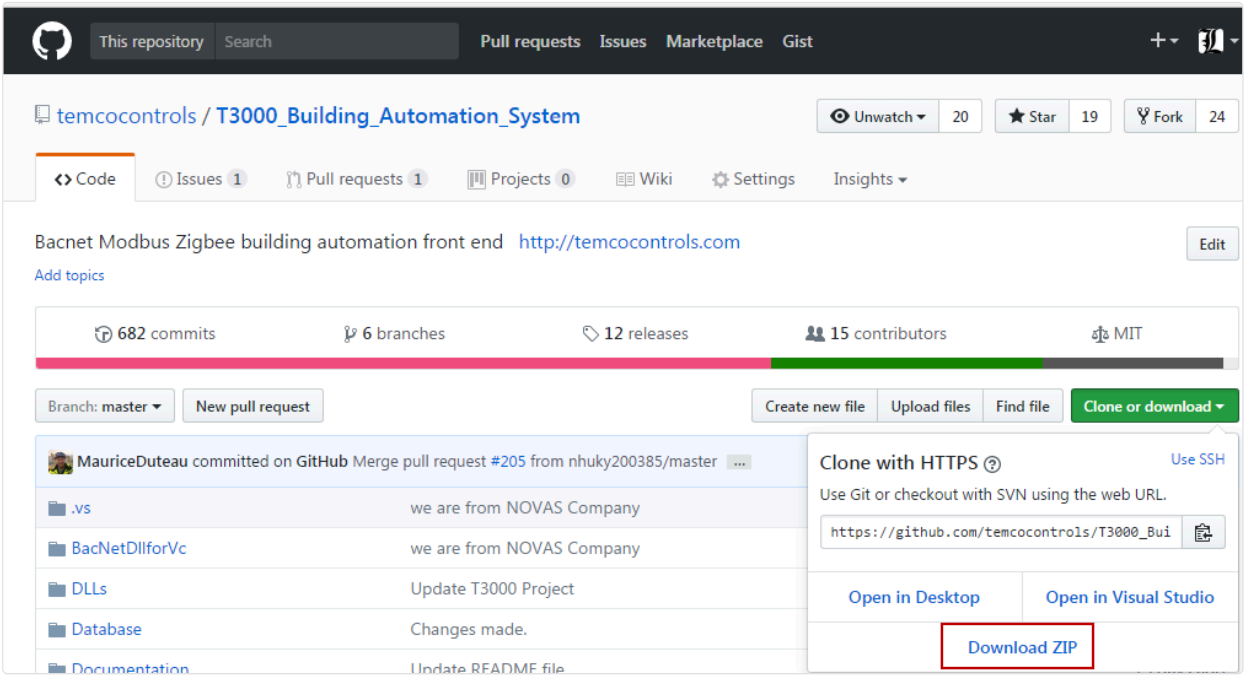
Include here

Compiling T3000 and making the setup file

The build and the installer are one job here: compile the solution, then assemble the setup that ships it. The screens are in the order the process runs.

Compile T3000

1. Download the repo from github to your local working folder.



While cloning the repository of T3000, make sure to get the submodules too. On a command line, here is how to do it:

git clone --recurse-submodules

This would get the T3000_CrossPlatform project as a submodule of T3000_Building_Automation_System project.

Whenever you update your local repository, make sure to update the submodules too.

git pull --recurse-submodules

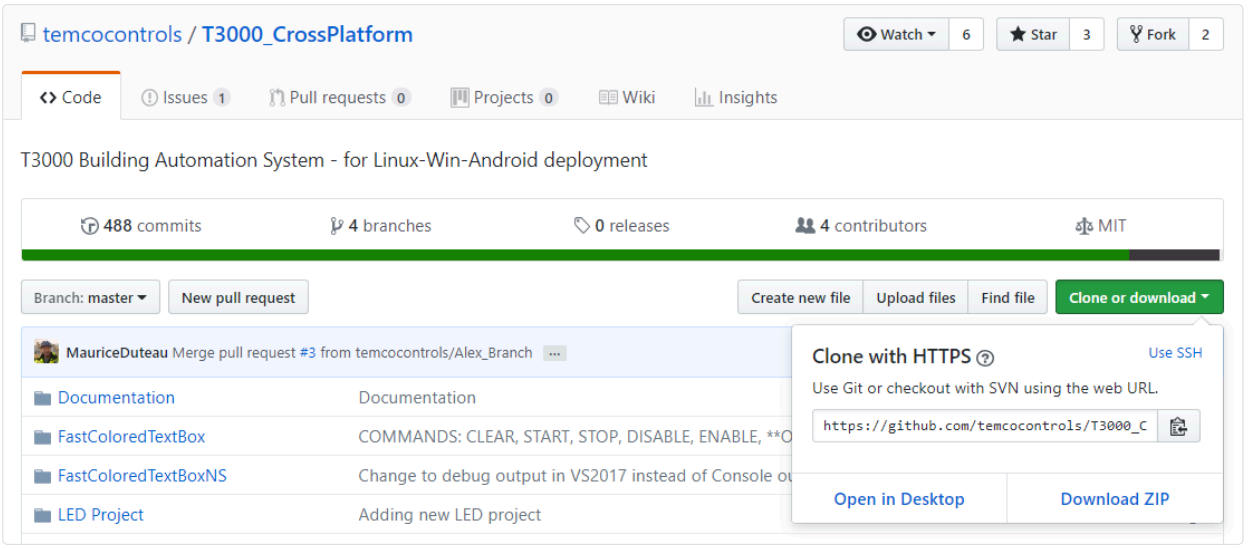
Note: You can download the git command line client for windows from

https://git-scm.com/downloads

If you plan to download the zip file, the submodule won't be included in the download. To cleanly compile the T3000 solution, you need to bring T3000_CrossPlatform project also in the folder where you extracted T3000_Building_Automation_System project. For this go to the T3000_Crossplatform project and download the zip. Extract this zip inside T3000_Building_Automation_System so that it comes as a child to this folder:

T3000_Building_Automation_System

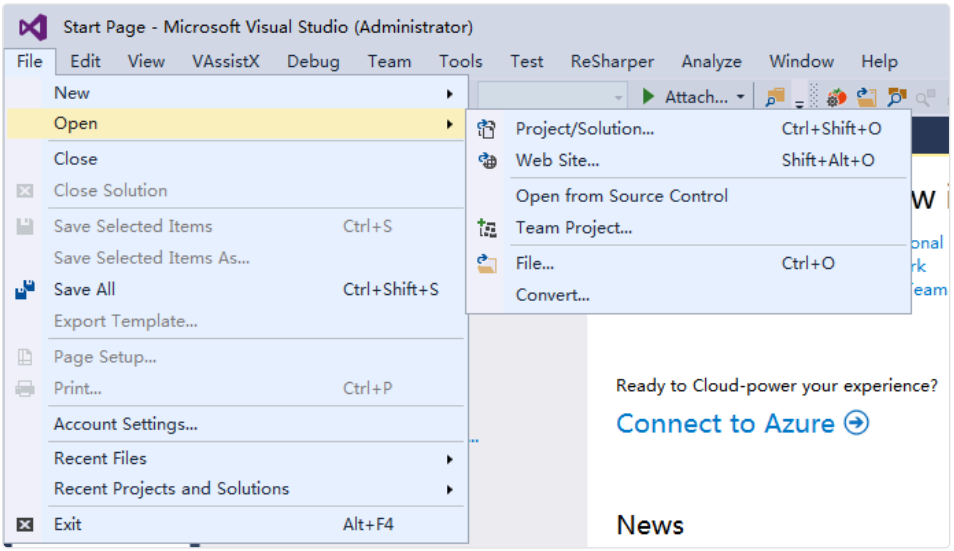
├——T3000_CrossPlatform

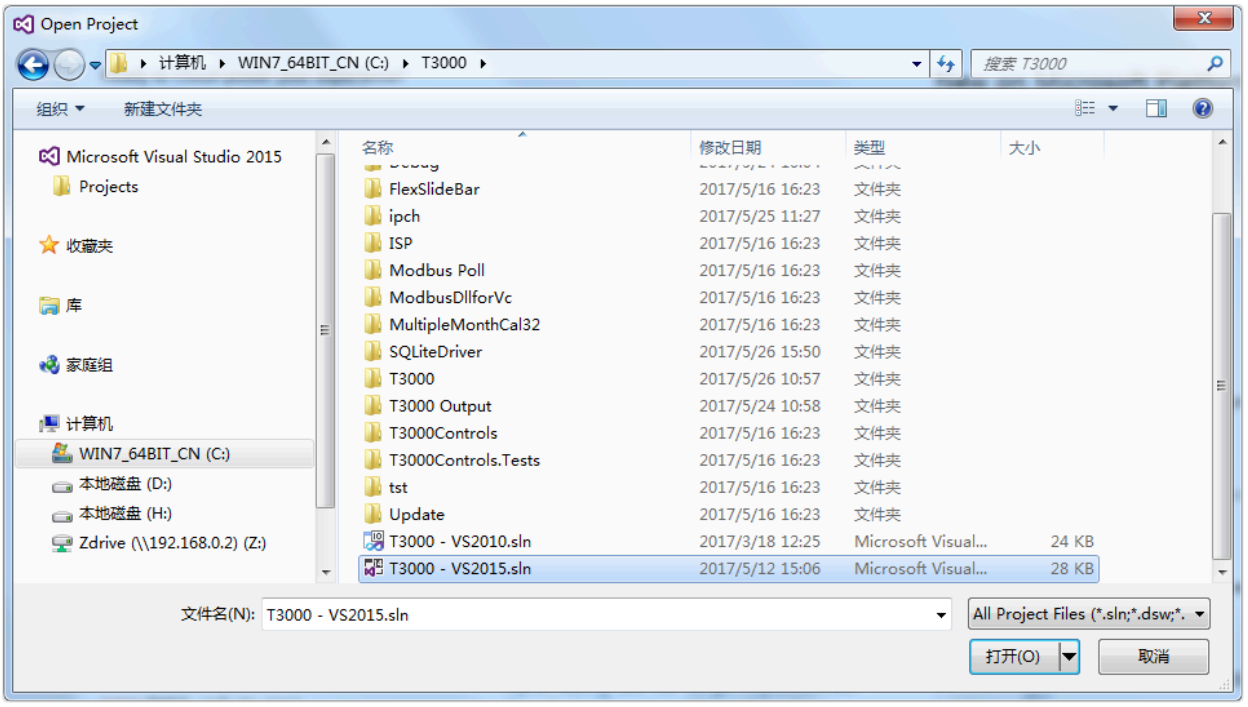


Note: We went by a submodule approach because the ProgramEditor project under T3000 solution requires FastColoredTextBox, T3000Grammer and PRGReaderLibrary projects in T3000_CrossPlatform project.

Install Microsoft Visual Studio VS2010 and VS2015.

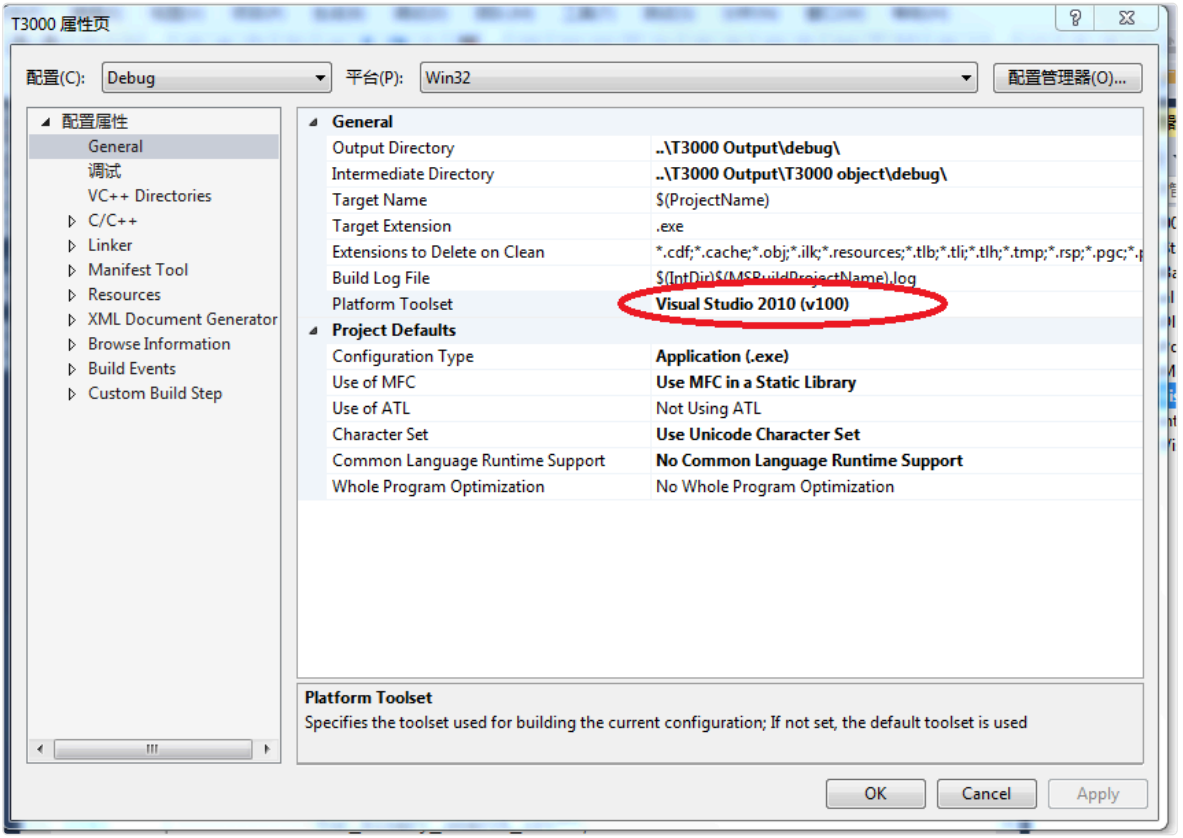
1. OpenT3000Project , using VS2015



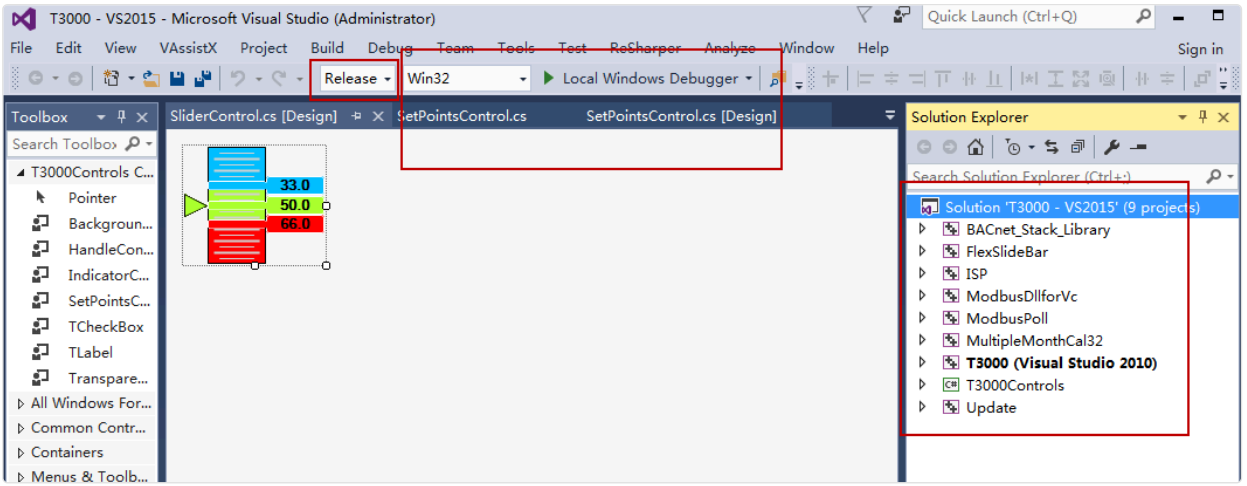




Make sure all these project are using Visual Studio VS2010 . VS2015 not completely compatible with VS2010.Many codes were completed by the old compiler.



We specify that we will create a Win32 application which can run on both 32 and 64bit Windows installations.



2. Compile T3000

Two Modes ,one is debug ,one is Release .

The platform ,please select Win32 OR x86

The build will rebuild the following items in this order:

BACnet_Stack_Library

FlexSlideBar

ModbusDllforVc

MultipleMonthCal32

T3000Controls

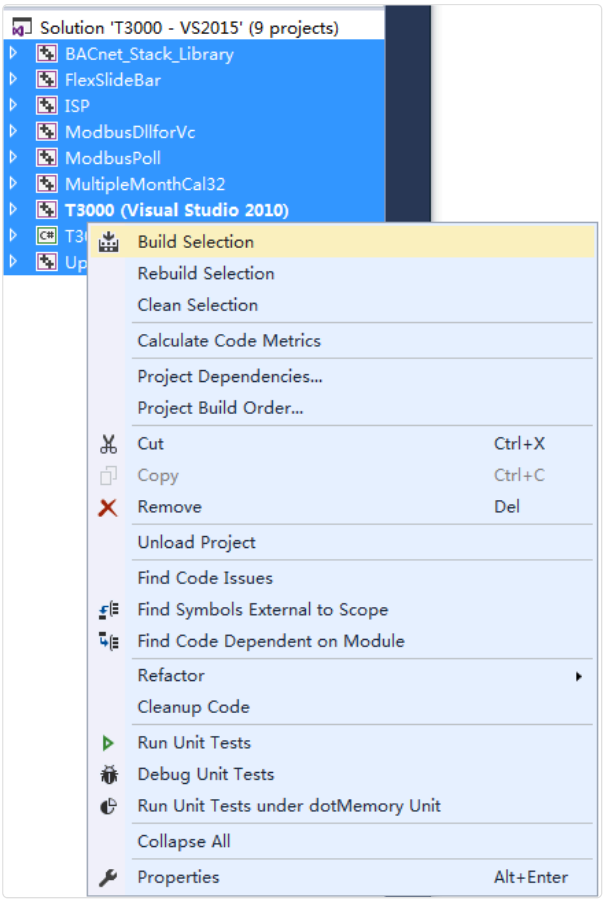
ISP

ModbusPoll

Update

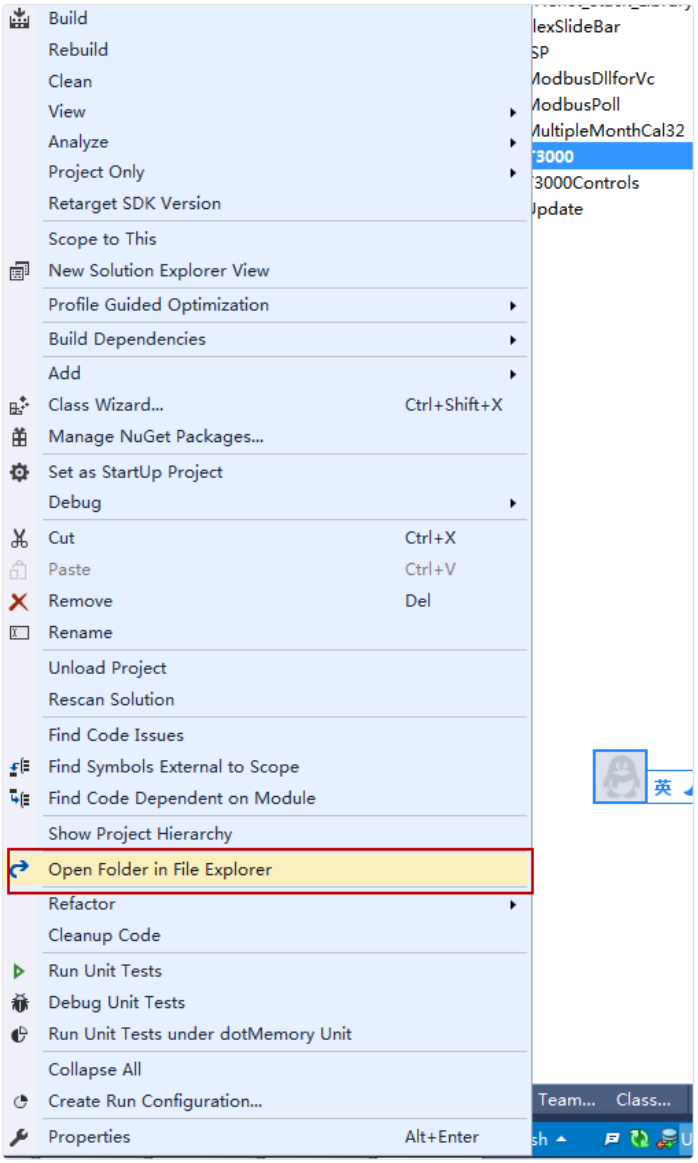
You can select all project to compile .

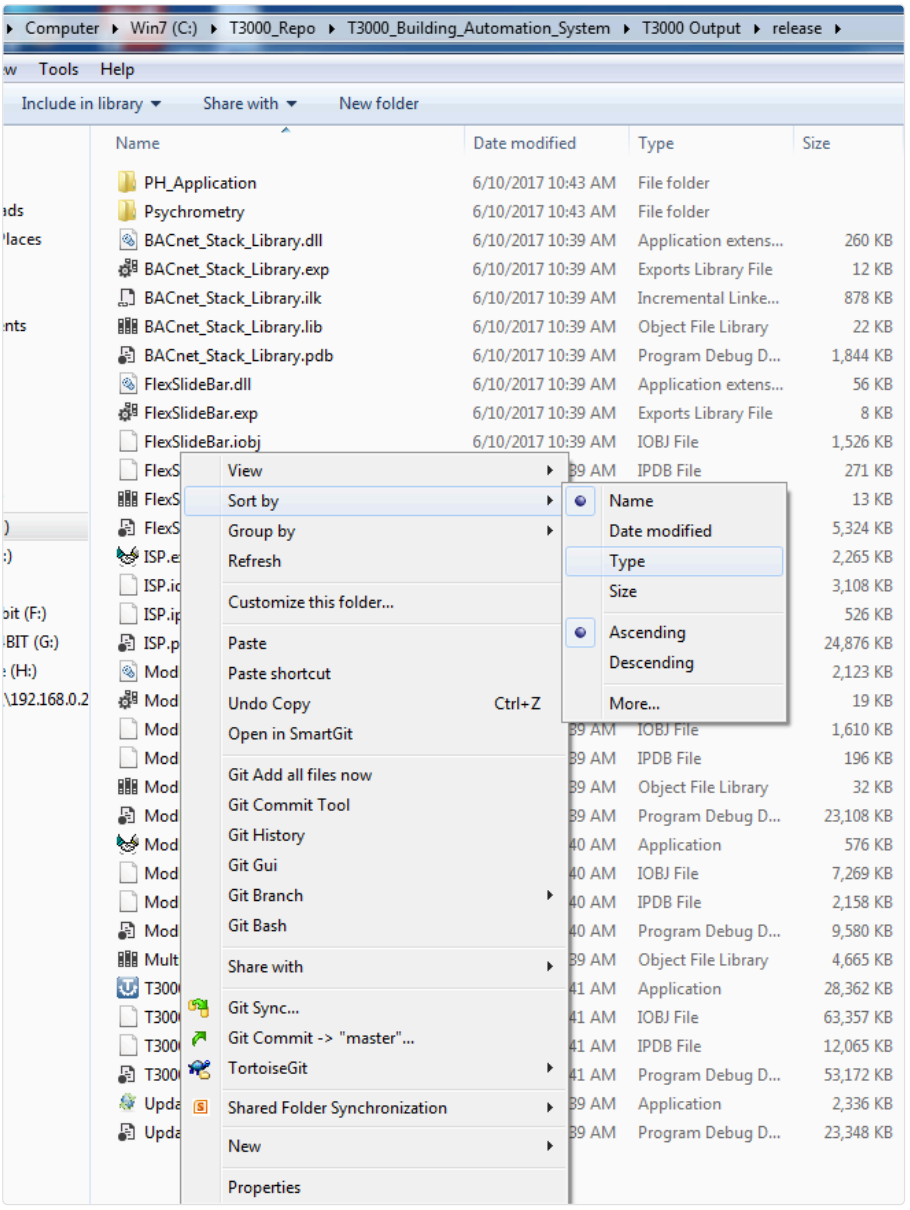
Notes:



You can rebuild all the project or just parts which you know have changed.

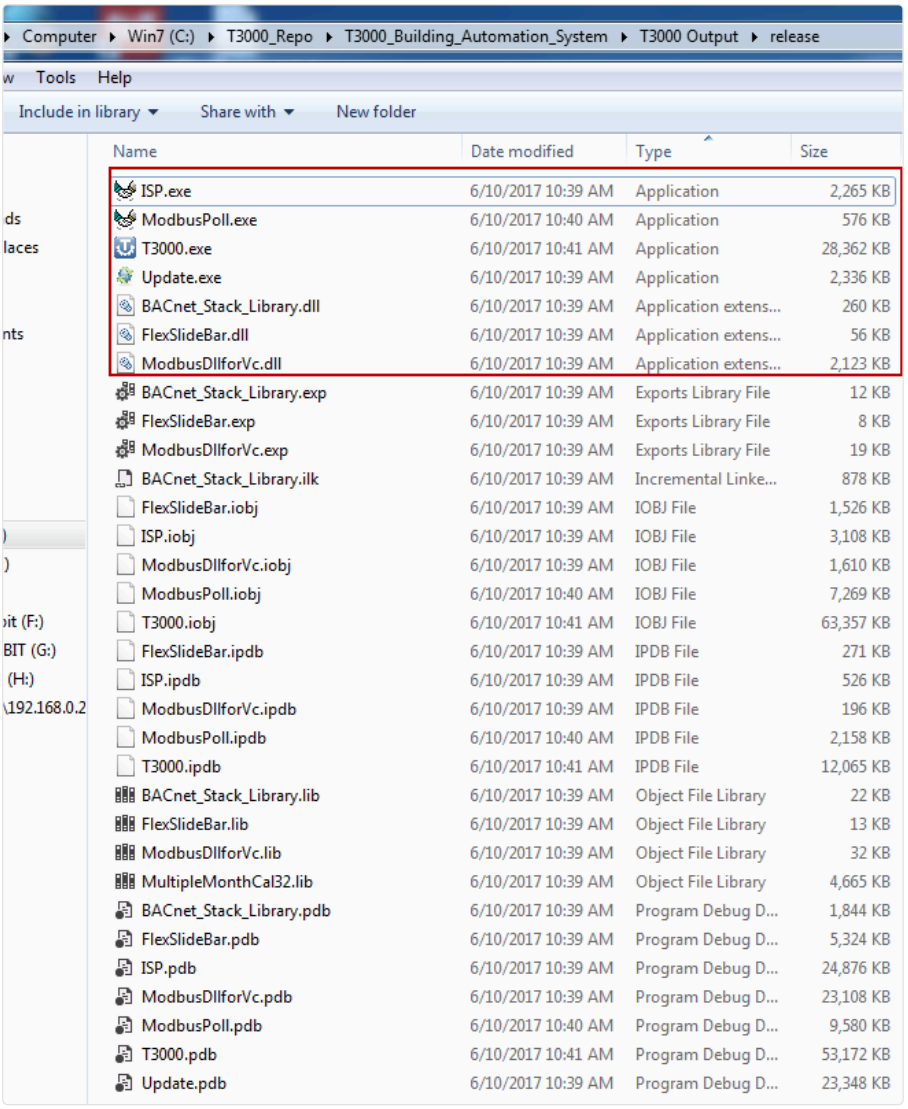
To find your latest release .


Build
Rebuild
Clean
View
Analyze
Project Only
Retarget SDK Version
Scope to This
New Solution Explorer View
Profile Guided Optimization
Build Dependencies
Add
Class Wizard... Ctrl+Shift+X
Manage NuGet Packages...
Set as StartUp Project
Debug
Cut Ctrl+X
Paste Ctrl+V
Remove Del
Rename
Unload Project
Rescan Solution
Find Code Issues
Find Symbols External to Scope
Find Code Dependent on Module
Show Project Hierarchy
Open Folder in File Explorer
Refactor
Cleanup Code
Run Unit Tests
Debug Unit Tests
Run Unit Tests under dotMemory Unit
Collapse All
Create Run Configuration...
Properties Alt+Enter
lexSlideBar
SP
ModbusDllforVc
ModbusPoll
MultipleMonthCal32
3000
3000Controls
Update
英
Team...
Class...



These files should be copied to the Installshield folder on your local machine. Installshield will only look to its local folder for files to be combined into the MSI install file, so this step is necessary.

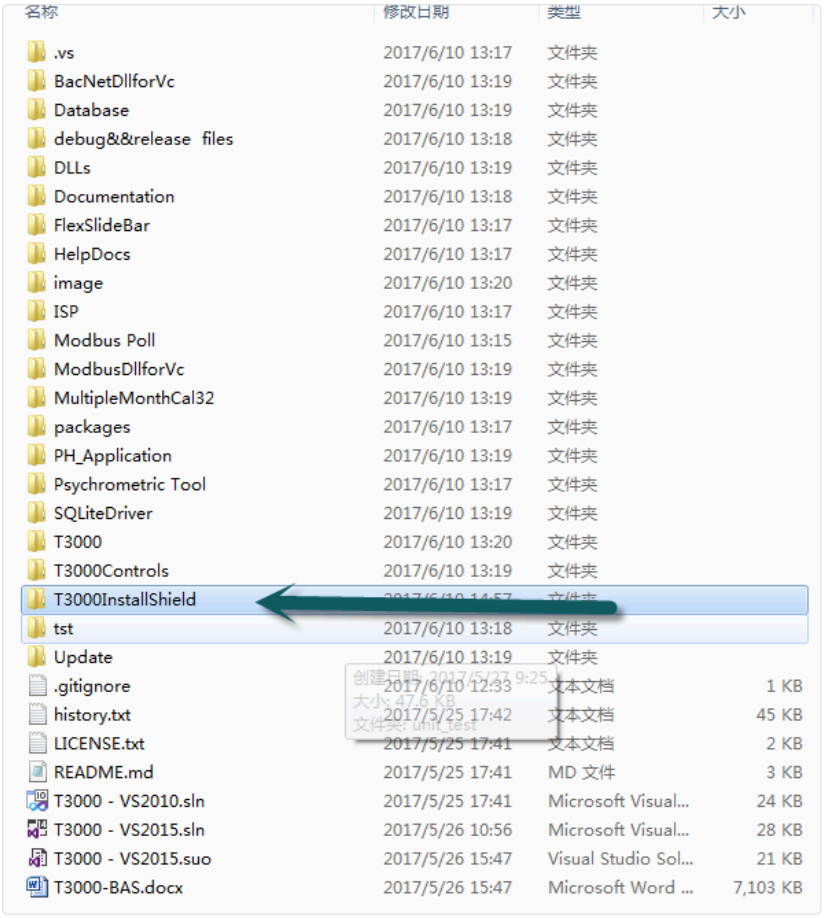
These are the files which the build will produce, EXE and some DLL's.



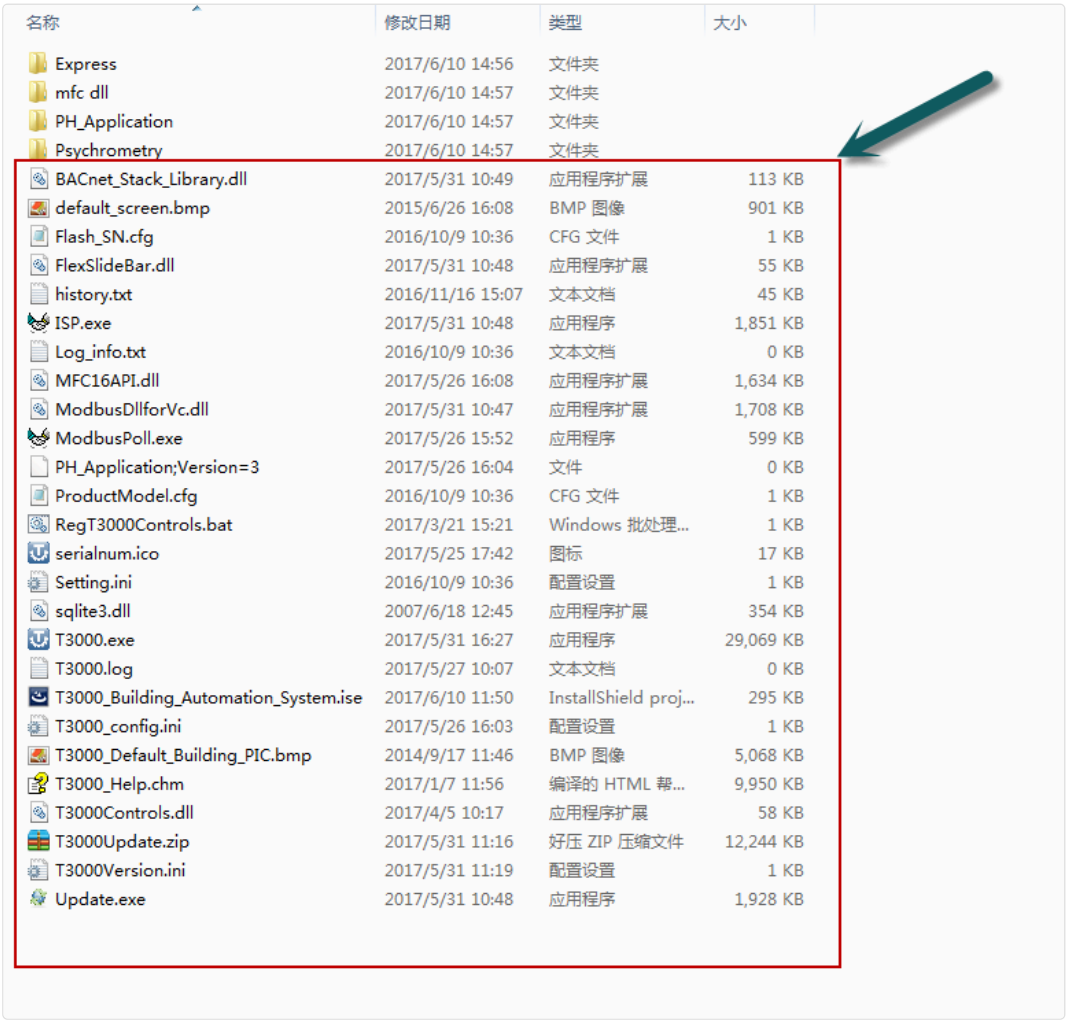
Copy these file to T3000InstallShield , and then replace the old files.

Using the same way to compile Psychrometric Tool && PH_Application

Add these two separate applications as part of the solution, not separate applications like it is now.



名称	修改日期	类型	大小
.vs	2017/6/10 13:17	文件夹	
BacNetDllforVc	2017/6/10 13:19	文件夹	
Database	2017/6/10 13:19	文件夹	
debug&&release files	2017/6/10 13:18	文件夹	
DLLs	2017/6/10 13:19	文件夹	
Documentation	2017/6/10 13:18	文件夹	
FlexSlideBar	2017/6/10 13:17	文件夹	
HelpDocs	2017/6/10 13:17	文件夹	
image	2017/6/10 13:20	文件夹	
ISP	2017/6/10 13:17	文件夹	
Modbus Poll	2017/6/10 13:15	文件夹	
ModbusDllforVc	2017/6/10 13:19	文件夹	
MultipleMonthCal32	2017/6/10 13:19	文件夹	
packages	2017/6/10 13:17	文件夹	
PH_Application	2017/6/10 13:19	文件夹	
Psychrometric Tool	2017/6/10 13:17	文件夹	
SQLiteDriver	2017/6/10 13:19	文件夹	
T3000	2017/6/10 13:20	文件夹	
T3000Controls	2017/6/10 13:19	文件夹	
T3000InstallShield			
tst	2017/6/10 13:18	文件夹	
Update	2017/6/10 13:19	文件夹	
.gitignore	2017/6/10 12:33	文本文档	1 KB
history.txt	2017/5/25 17:42	文本文档	45 KB
LICENSE.txt	2017/5/25 17:41	文本文档	2 KB
README.md	2017/5/25 17:41	MD 文件	3 KB
T3000 - VS2010.sln	2017/5/25 17:41	Microsoft Visual...	24 KB
T3000 - VS2015.sln	2017/5/26 10:56	Microsoft Visual...	28 KB
T3000 - VS2015.suo	2017/5/26 15:47	Visual Studio Sol...	21 KB
T3000-BAS.docx	2017/5/26 15:47	Microsoft Word ...	7,103 KB



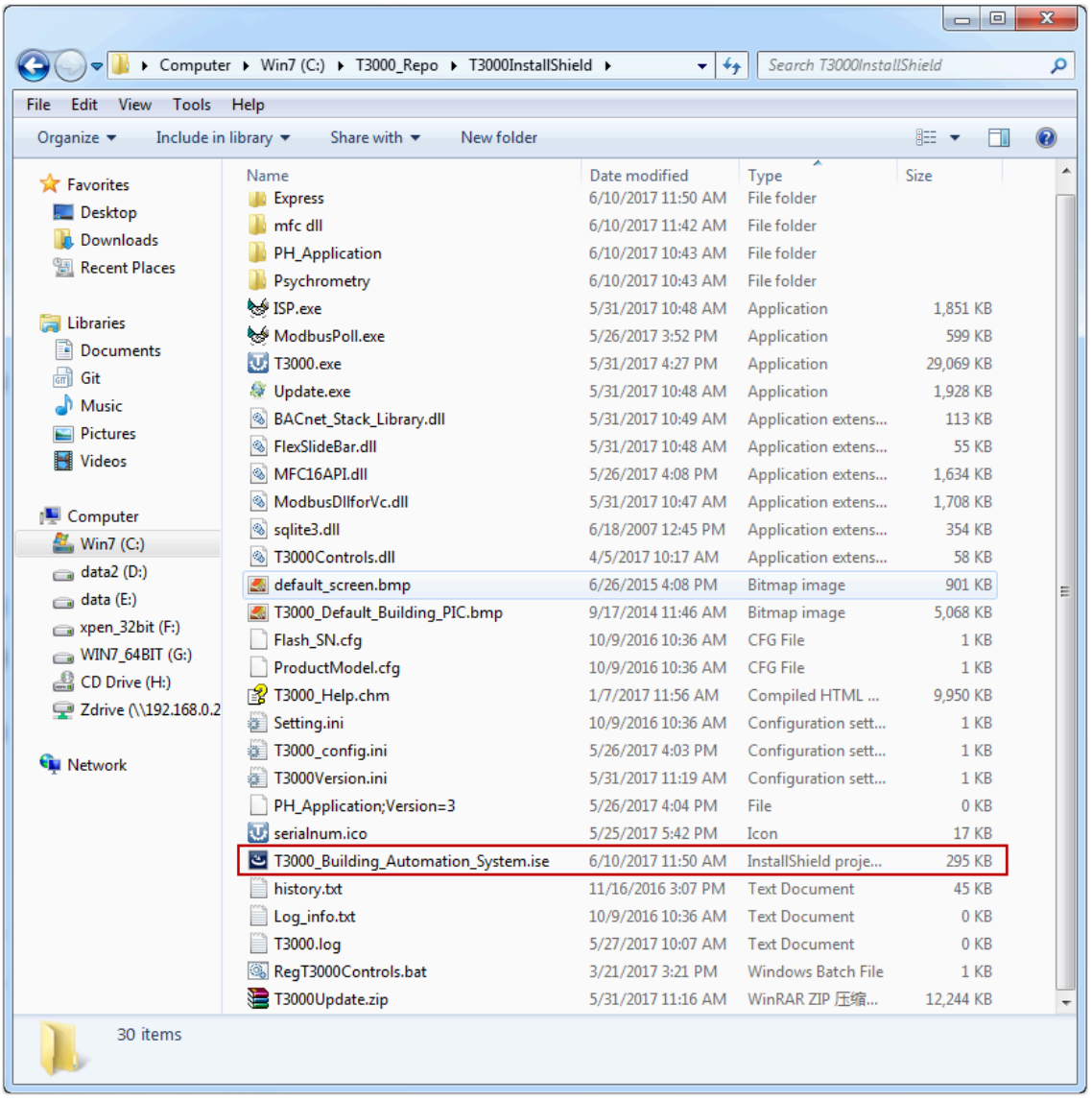
名称	修改日期	类型	大小
Express	2017/6/10 14:56	文件夹	
mfc dll	2017/6/10 14:57	文件夹	
PH_Application	2017/6/10 14:57	文件夹	
Psychrometry	2017/6/10 14:57	文件夹	
BACnet_Stack_Library.dll	2017/5/31 10:49	应用程序扩展	113 KB
default_screen.bmp	2015/6/26 16:08	BMP 图像	901 KB
Flash_SN.cfg	2016/10/9 10:36	CFG 文件	1 KB
FlexSlideBar.dll	2017/5/31 10:48	应用程序扩展	55 KB
history.txt	2016/11/16 15:07	文本文档	45 KB
ISP.exe	2017/5/31 10:48	应用程序	1,851 KB
Log_info.txt	2016/10/9 10:36	文本文档	0 KB
MFC16API.dll	2017/5/26 16:08	应用程序扩展	1,634 KB
ModbusDllforVc.dll	2017/5/31 10:47	应用程序扩展	1,708 KB
ModbusPoll.exe	2017/5/26 15:52	应用程序	599 KB
PH_Application;Version=3	2017/5/26 16:04	文件	0 KB
ProductModel.cfg	2016/10/9 10:36	CFG 文件	1 KB
RegT3000Controls.bat	2017/3/21 15:21	Windows 批处理...	1 KB
serialnum.ico	2017/5/25 17:42	图标	17 KB
Setting.ini	2016/10/9 10:36	配置设置	1 KB
sqlite3.dll	2007/6/18 12:45	应用程序扩展	354 KB
T3000.exe	2017/5/31 16:27	应用程序	29,069 KB
T3000.log	2017/5/27 10:07	文本文档	0 KB
T3000_Building_Automation_System.ise	2017/6/10 11:50	InstallShield proj...	295 KB
T3000_config.ini	2017/5/26 16:03	配置设置	1 KB
T3000_Default_Building_PIC.bmp	2014/9/17 11:46	BMP 图像	5,068 KB
T3000_Help.chm	2017/1/7 11:56	编译的 HTML 帮...	9,950 KB
T3000Controls.dll	2017/4/5 10:17	应用程序扩展	58 KB
T3000Update.zip	2017/5/31 11:16	好压 ZIP 压缩文件	12,244 KB
T3000Version.ini	2017/5/31 11:19	配置设置	1 KB
Update.exe	2017/5/31 10:48	应用程序	1,928 KB

2.How to make an MSI for T3000

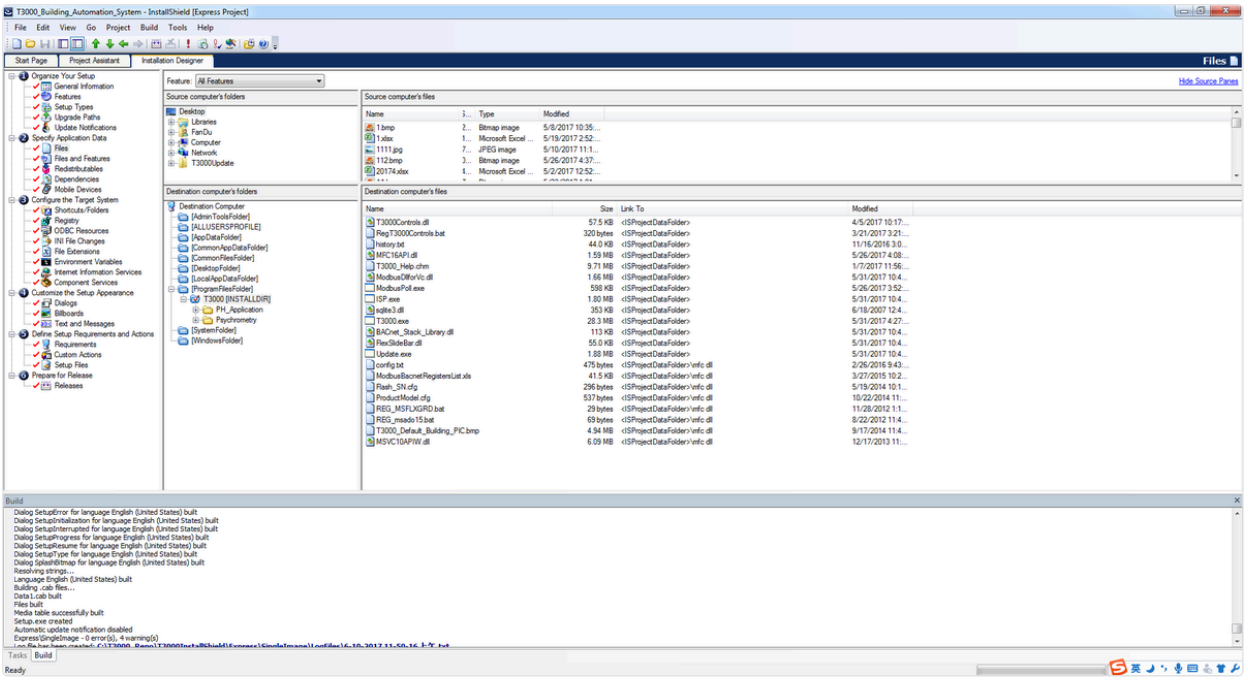
Download Installshield2019 ,and install it.

This MSI will include all the proejcts, T3000 as well as the separate Psychrometric and PH applications.

1. double click installshield project file

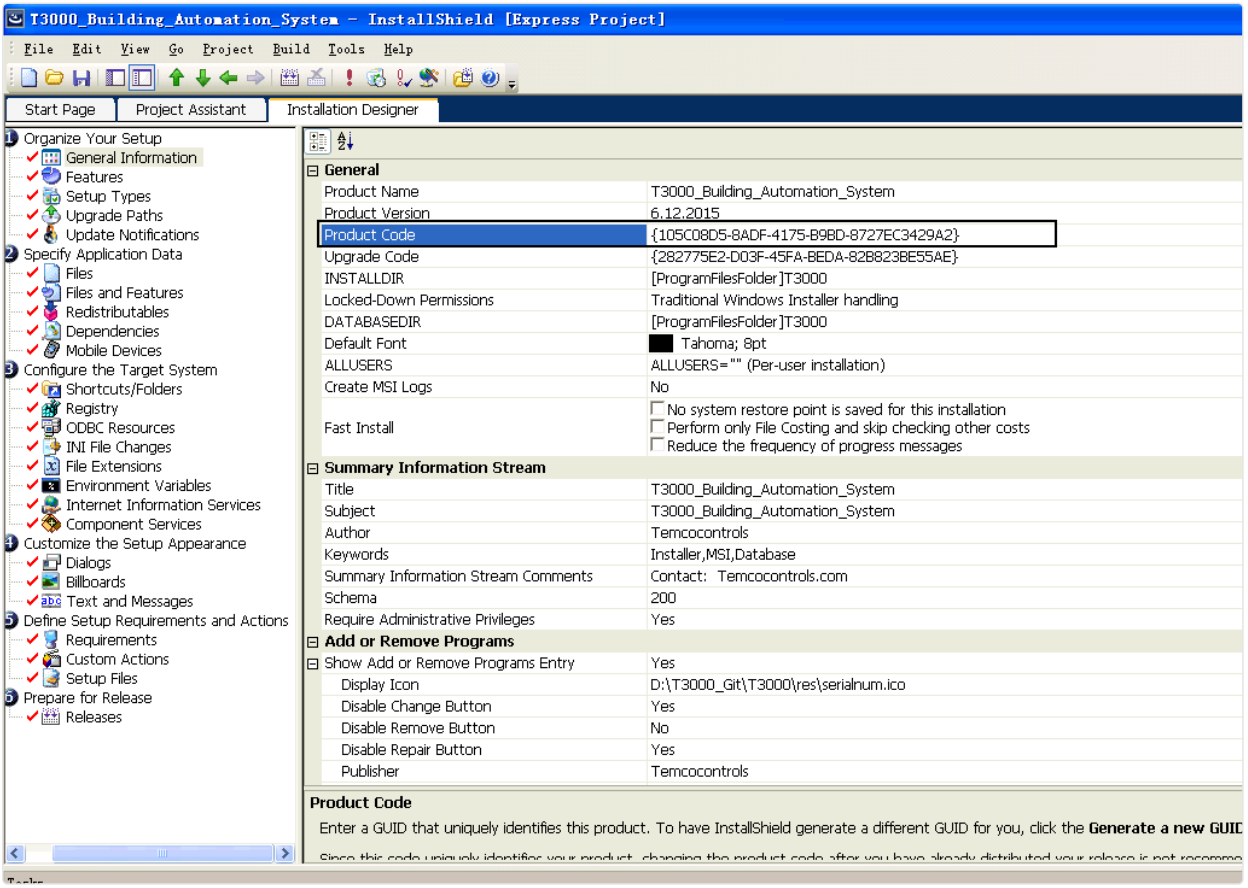


2. InstallSheild showing



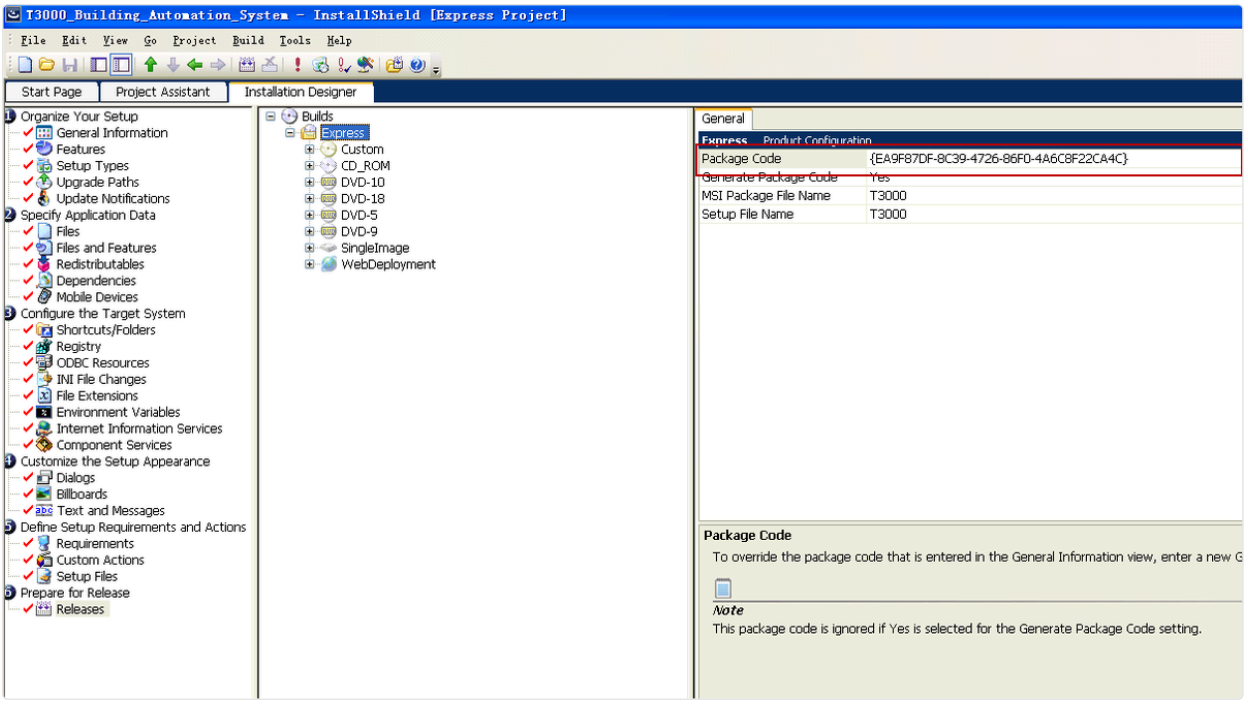
2.Click on the “General Information’ tab, then on the ‘Prooduct Code” cell to generate a GUID which is a code for the application.



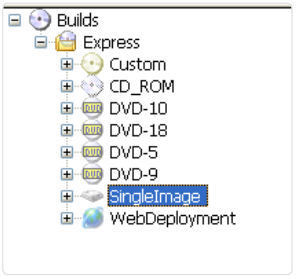


3. Click on the Prepare for release tab, ‘Releases’ itms which generates another GUID, these are just codes which are automatically generated.





4.Sleect what type of image to generate, in our case SingleImage



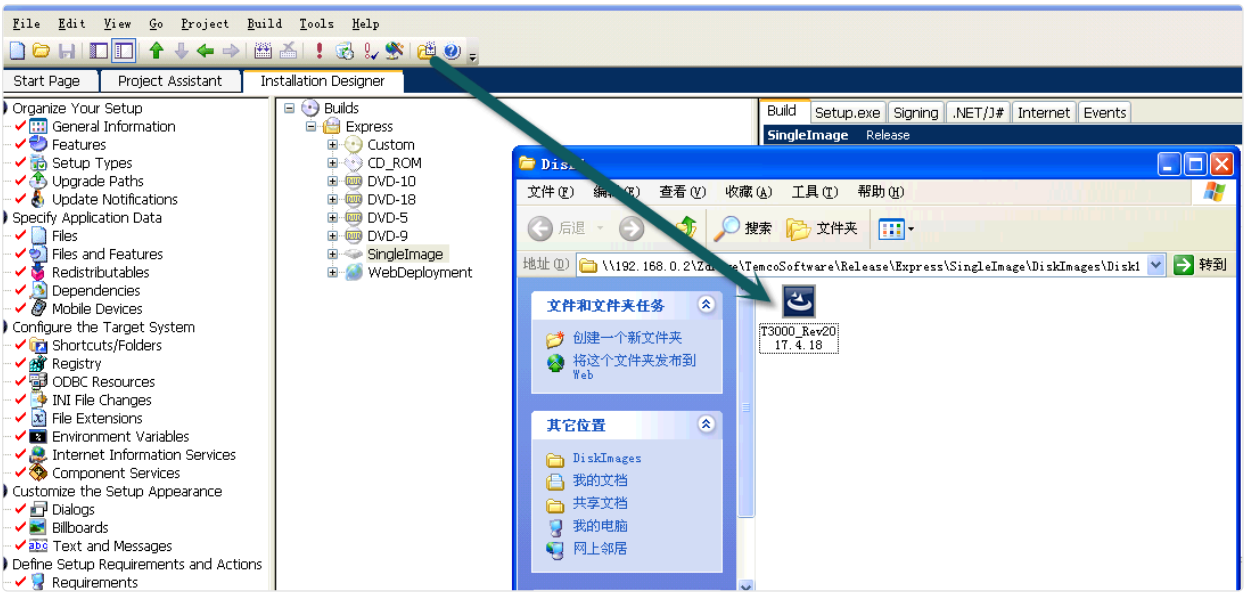
5. Build





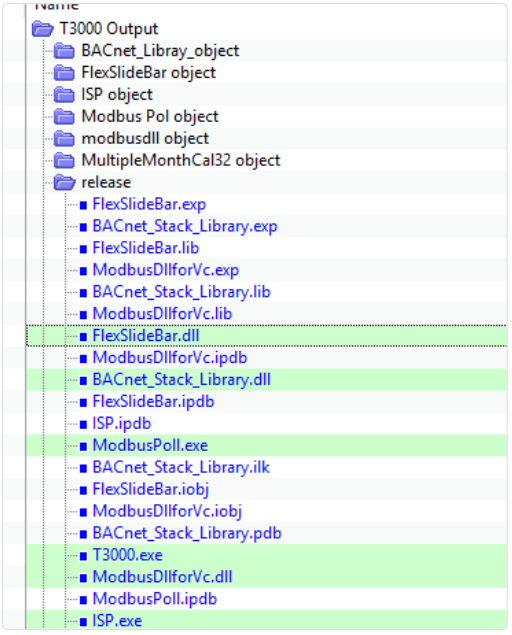
6.Open the setup exe file

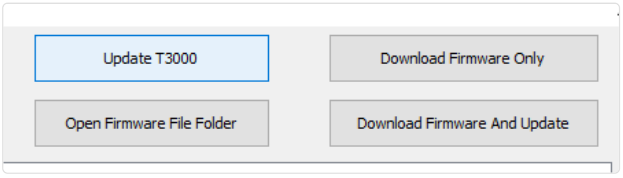




For generating the smaller EXE for updates from within T3000, just copy these files to the a zip file and load the following files to the temco FTP folder.

The files can be found in the T3000 output folder, only the DLL and exe files need to be sent there.





The InstallShield project

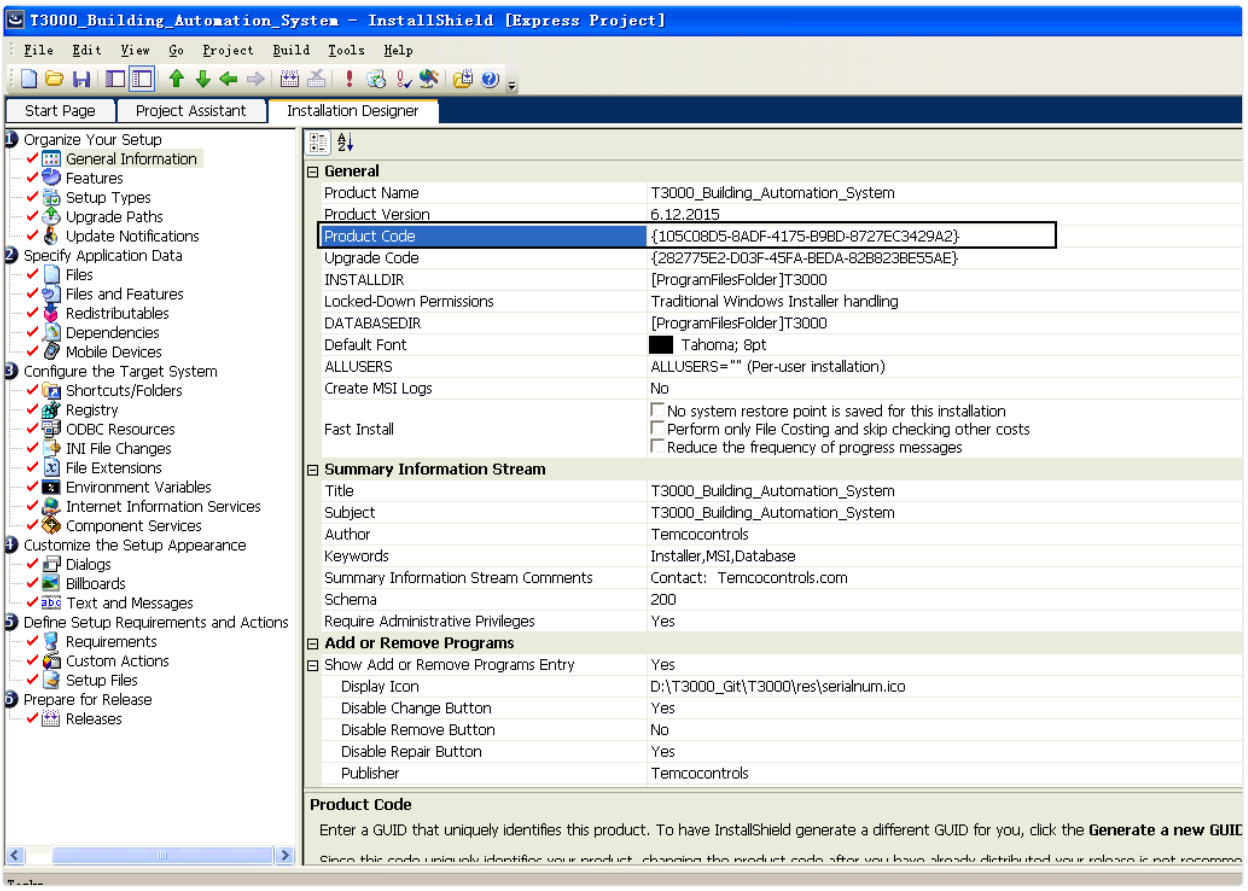
What the installer project carries, and the settings that have to be right before a build is shipped to a customer.

1.双击打开

\192.168.0.2\Zdrive\TemcoSoftware\Release\T3000_Building_Automation_System

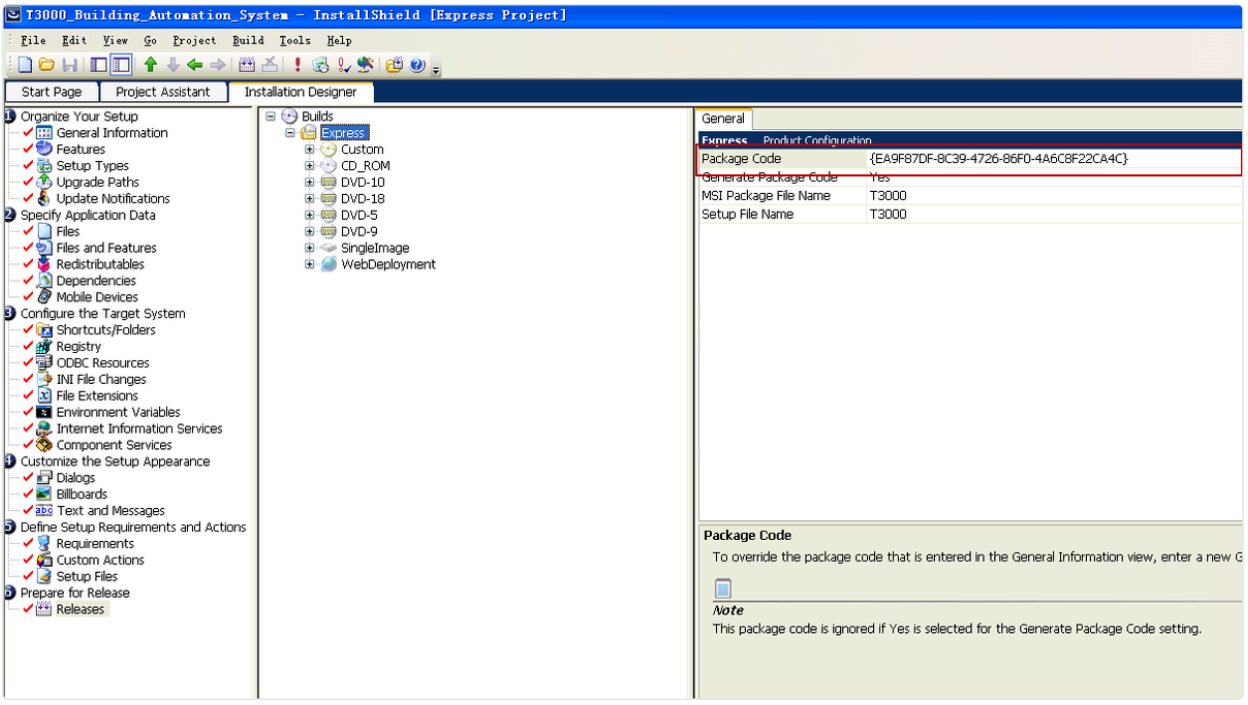
2.点击随机生成一个GUID



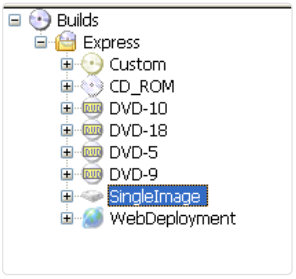


3. 点击随机生成一个GUID





4.选择SingleImage

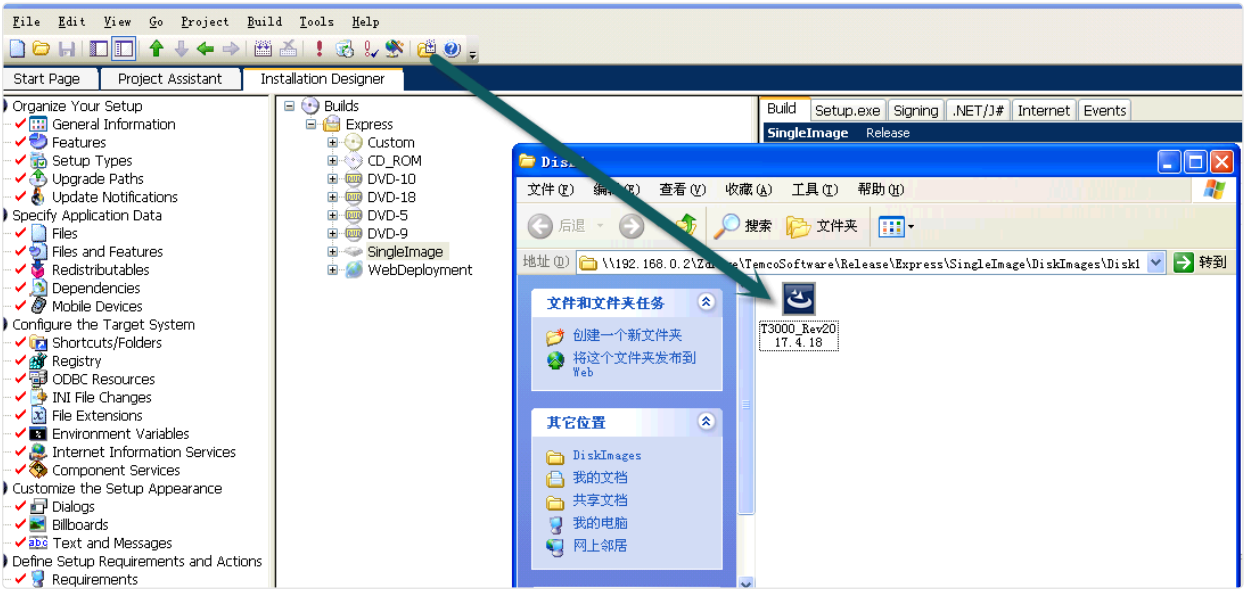


5.Build



6.





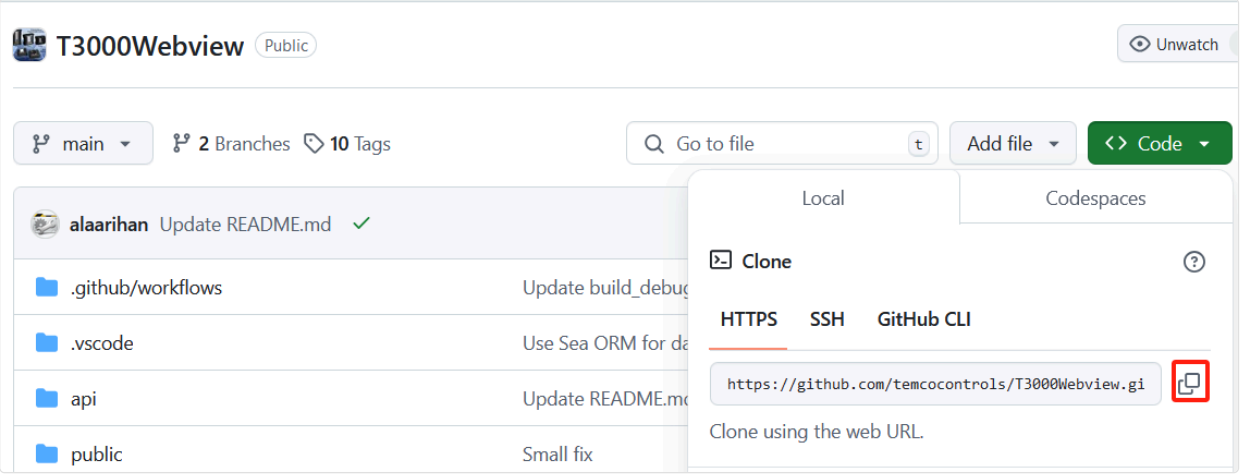
Debugging the webview

The webview is a separate process from the C++ shell, so debugging it means attaching to the right one. This is the setup, screen by screen.

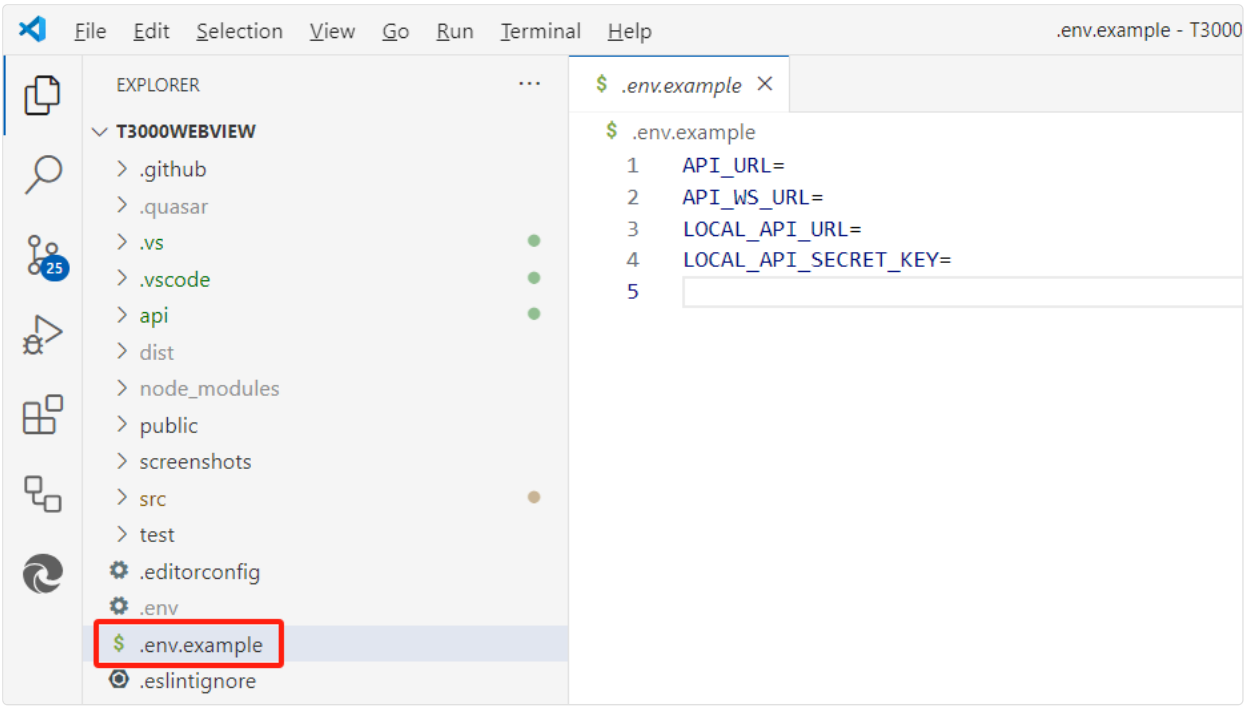
Debugging T3000Webview

1. clone the T3000Webview source code to local with below command.

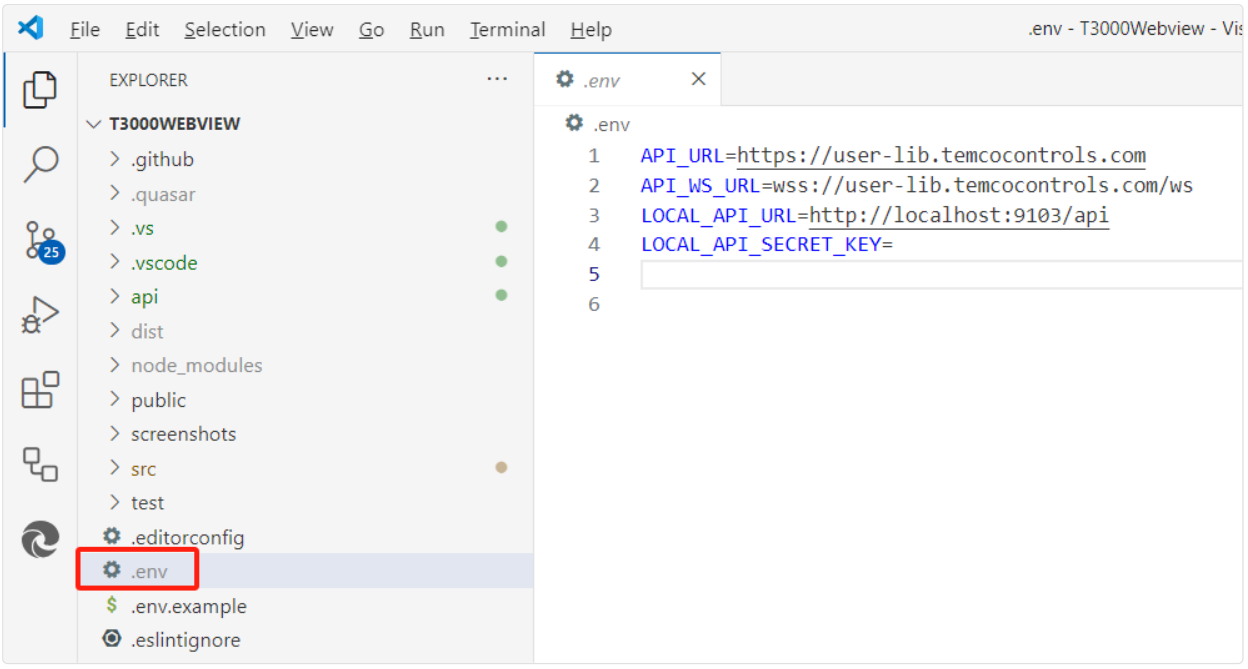
git clone https://github.com/temcocontrols/T3000Webview.git



2. open the source code via Visual Studio Code (or some other tool), find the .env.example file in the root folder "\T3000WEBVIEW"



3. add a new config file with name ".env" and copy the keys from ".env.example" and set the new values as below.



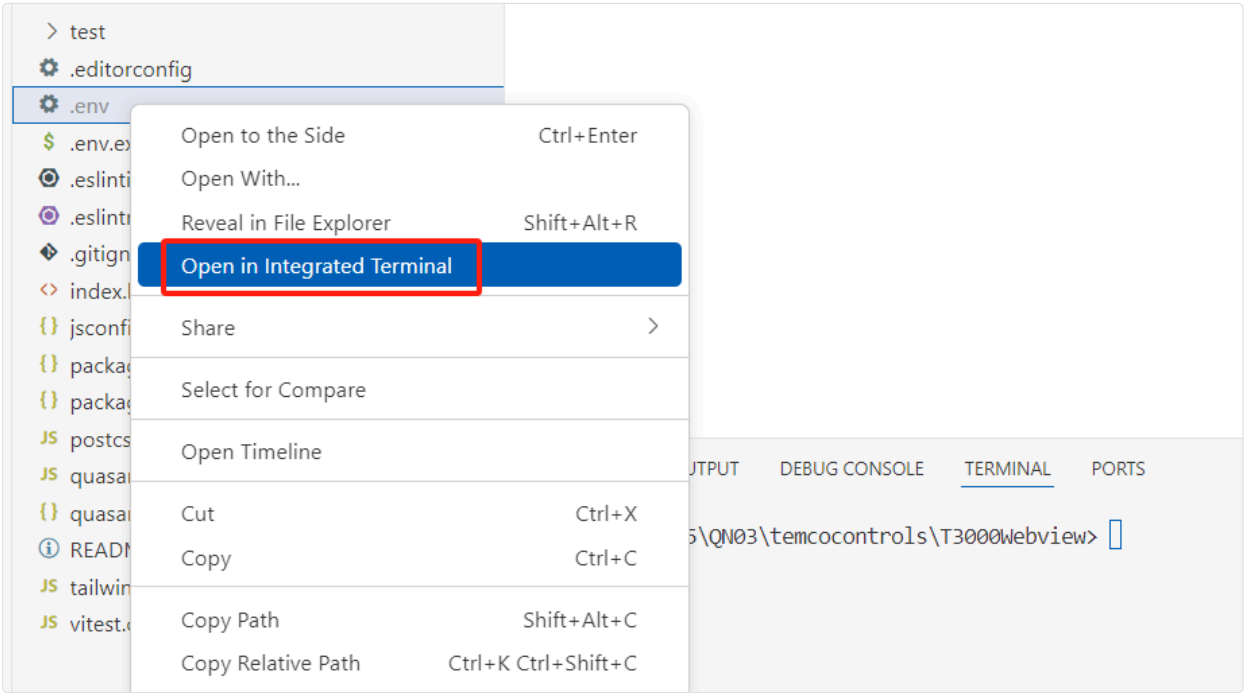
API_URL=https://user-lib.temcocontrols.com

API_WS_URL=wss://user-lib.temcocontrols.com/ws

LOCAL_API_URL=http://localhost:9103/api

LOCAL_API_SECRET_KEY=

4. right click to open the “Integrated Terminal”.



5. install the packages with below command.

npm i

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\QN\1025\QN03\temcocontrols\T3000Webview> npm i

up to date in 2s

131 packages are looking for funding
  run `npm fund` for details
PS D:\QN\1025\QN03\temcocontrols\T3000Webview>
```

6. build the web project with below command.

npm run build

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\QN\1025\QN03\temcocontrols\T3000Webview> npm i

up to date in 2s

131 packages are looking for funding
  run `npm fund` for details
PS D:\QN\1025\QN03\temcocontrols\T3000Webview> npm run build

> t3-webview@0.8.1 build
> quasar build


 .d88888b.
d88P" "Y88b
888     888
888     888 888  888  8888b.  .d8888b   8888b.  888d888
888     888 888  888     "88b 88K          "88b 888P"
888 Y8b 888 888  888 .d888888 "Y8888b. .d888888 888
```

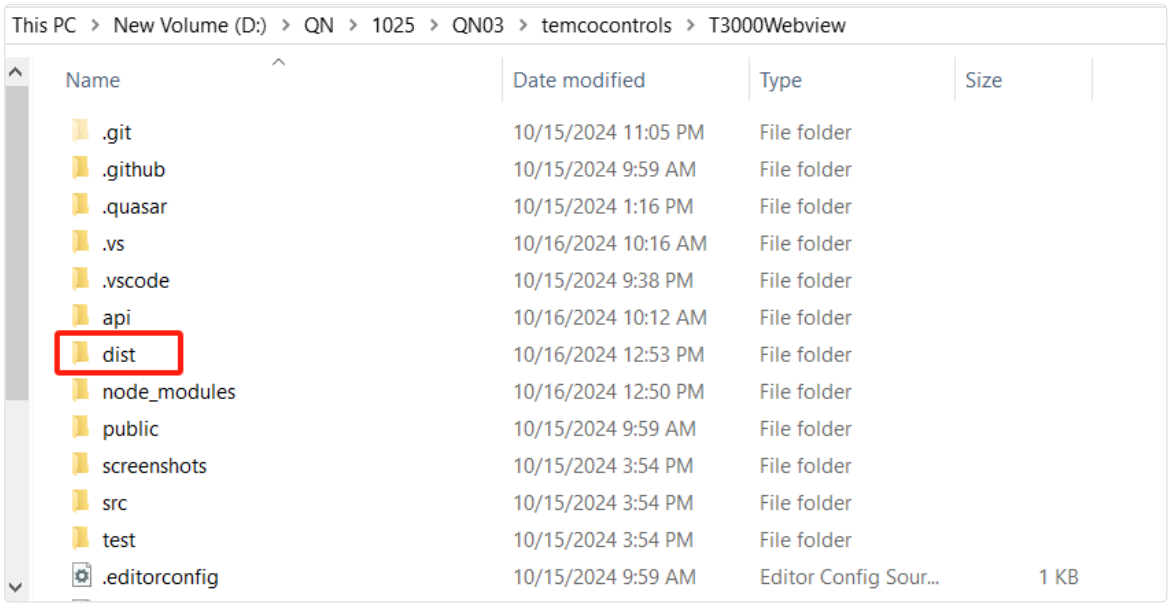
```
Build succeeded

Build mode.............. spa
Pkg quasar.............. v2.16.4
Pkg @quasar/app-vite... v1.9.3
Pkg vite................ v2.9.18
Debugging............... no
Publishing.............. no
Browser target......... es2019|edge88|firefox78|chrome87|safari13.1
========================
Output folder.......... D:\QN\1025\QN03\temcocontrols\T3000Webview\dist\spa

Tip: Built files are meant to be served over an HTTP server
     Opening index.html over file:// won't work

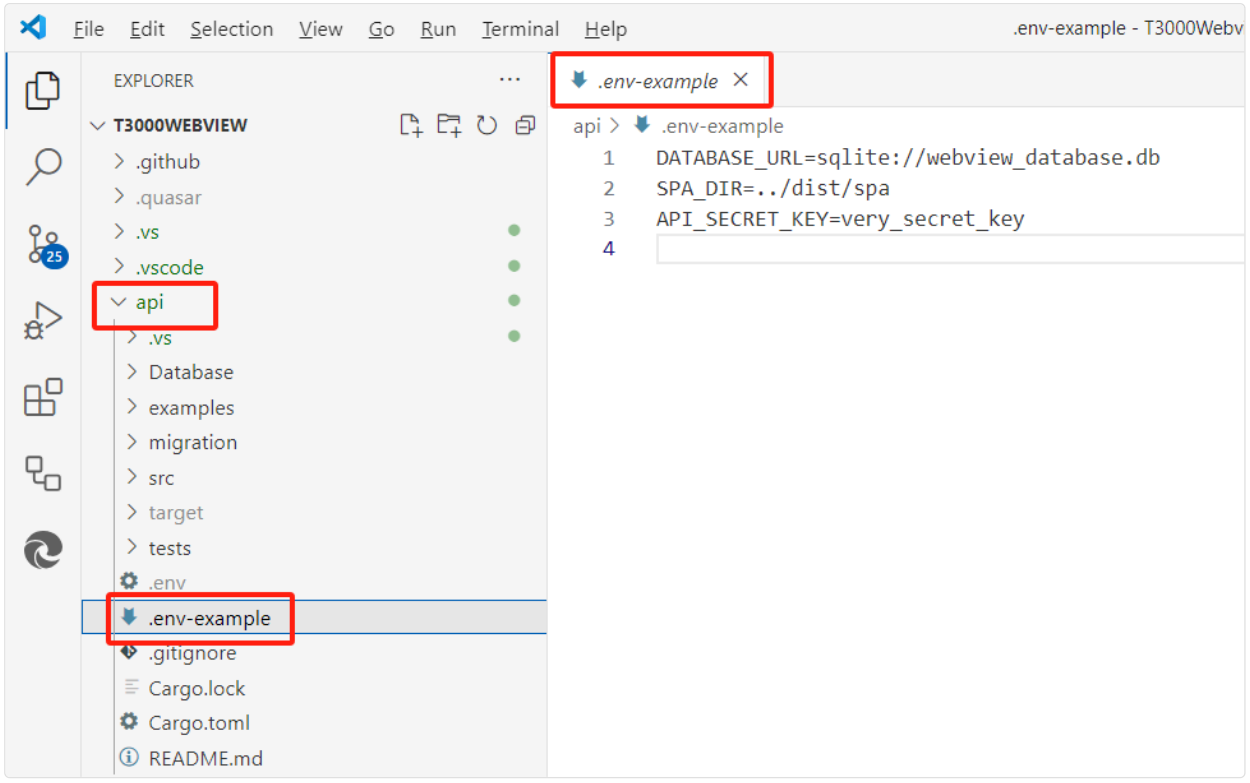
Tip: You can use "$ quasar serve" command to create a web server,
     both for testing or production. Type "$ quasar serve -h" for
     parameters. Also, an npm script (usually named "start") can
     be added for deployment environments.
     If you're using Vue Router "history" mode, don't forget to
     specify the "--history" parameter: "$ quasar serve --history"

PS D:\QN\1025\QN03\temcocontrols\T3000Webview>
```

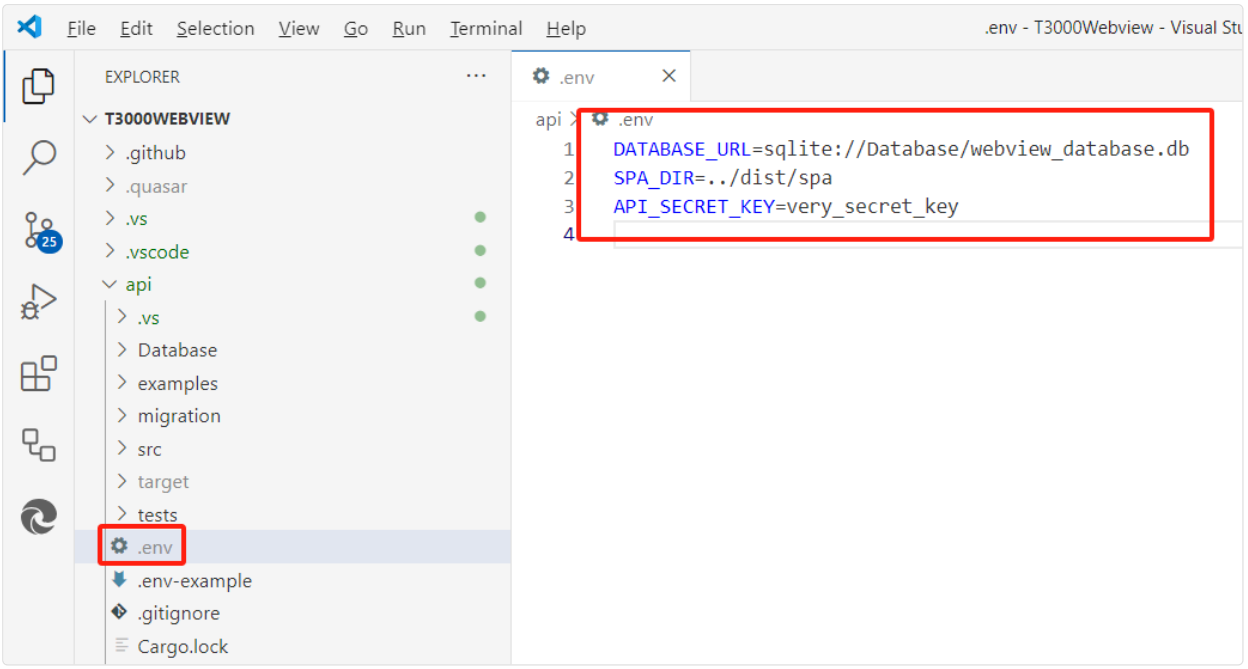


7. open the ".env-example" file under api folder. (full path should be \T3000Webview\api)

The api folder stores the source code of Rust back-end api



8. and a new config file under "\T3000Webview\api" with name ".env" and copy the keys from ".env-example", update the values as below.



DATABASE_URL=sqlite://Database/webview_database.db

SPA_DIR=../dist/spa

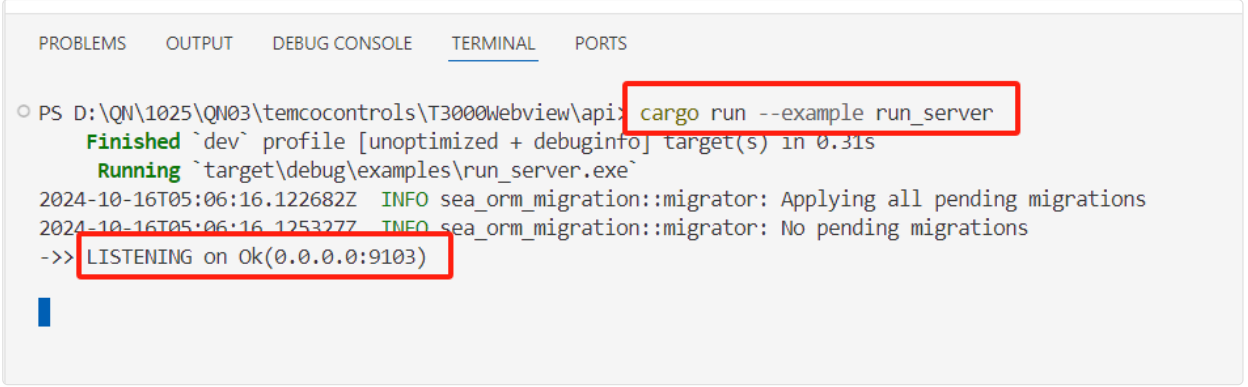
API_SECRET_KEY=very_secret_key

9. open the “Integrated Terminal” again, and cd into api folder



10. execute following command to start the rust api server.

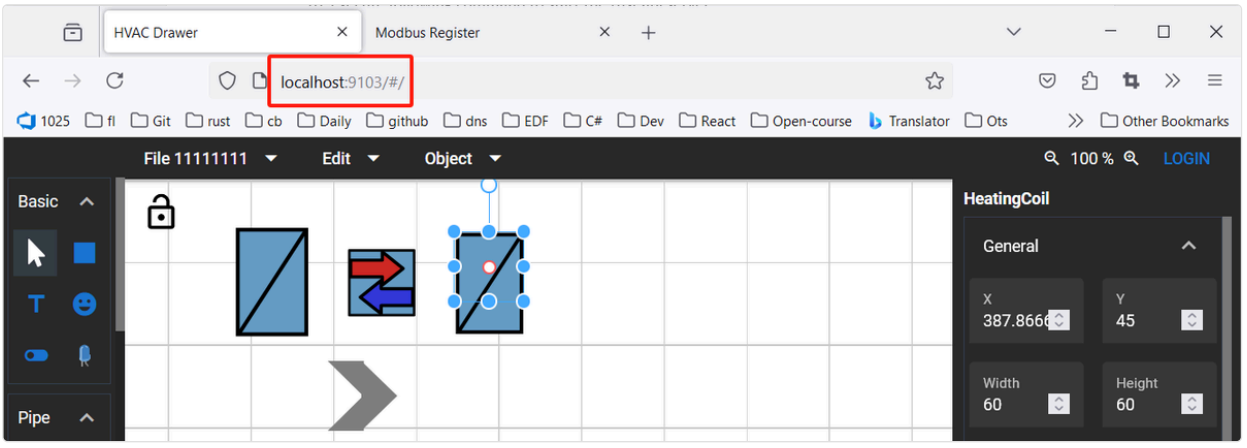
cargo run --example run_server

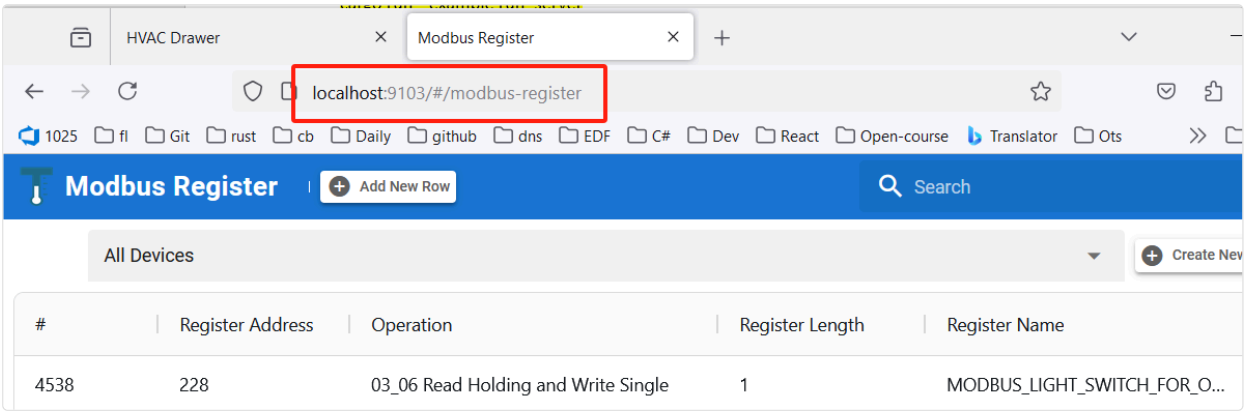


check with below urls to make sure the server has been started and working well.

http://localhost:9103/#/

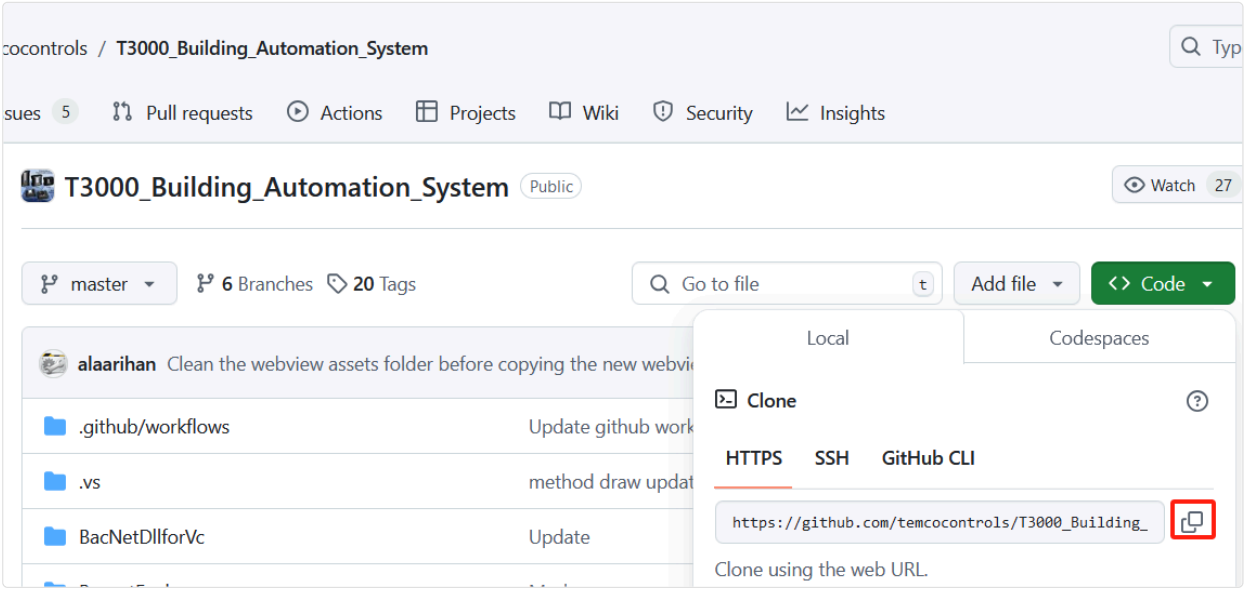
http://localhost:9103/#/modbus-register





Notes: if no need to debug it via T3000, can ignore below steps 11 & 12, can just use any web browser to do it.

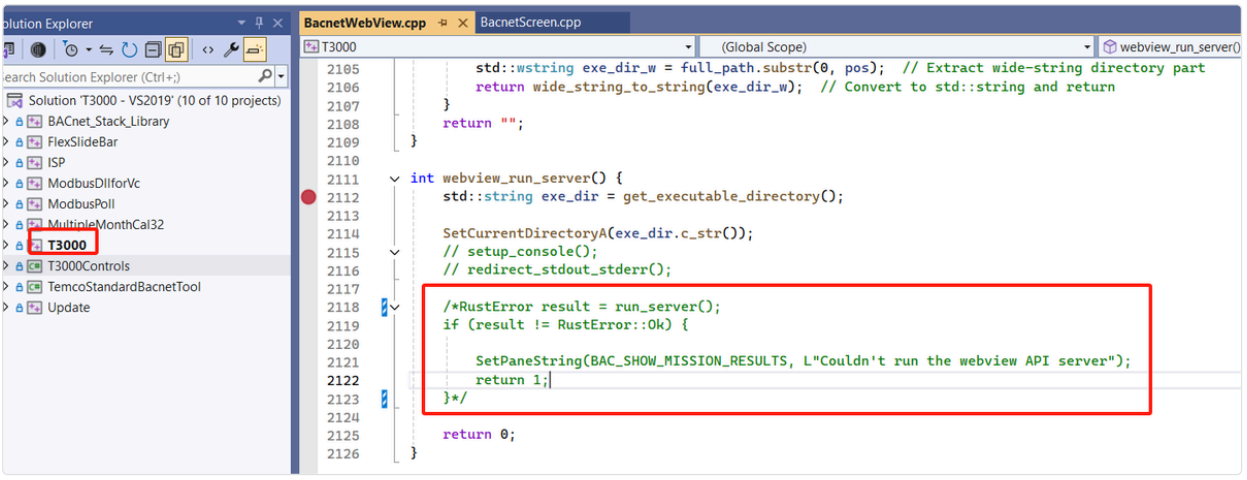
11. clone or update the source code of T3000_Buiding_Automation_System, make sure it's updated.



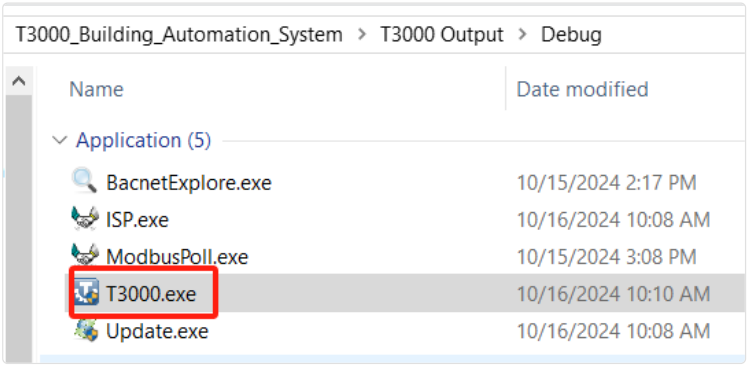
12. open the "T3000 - VS2019.sln" solution file with visual studio as administrator.

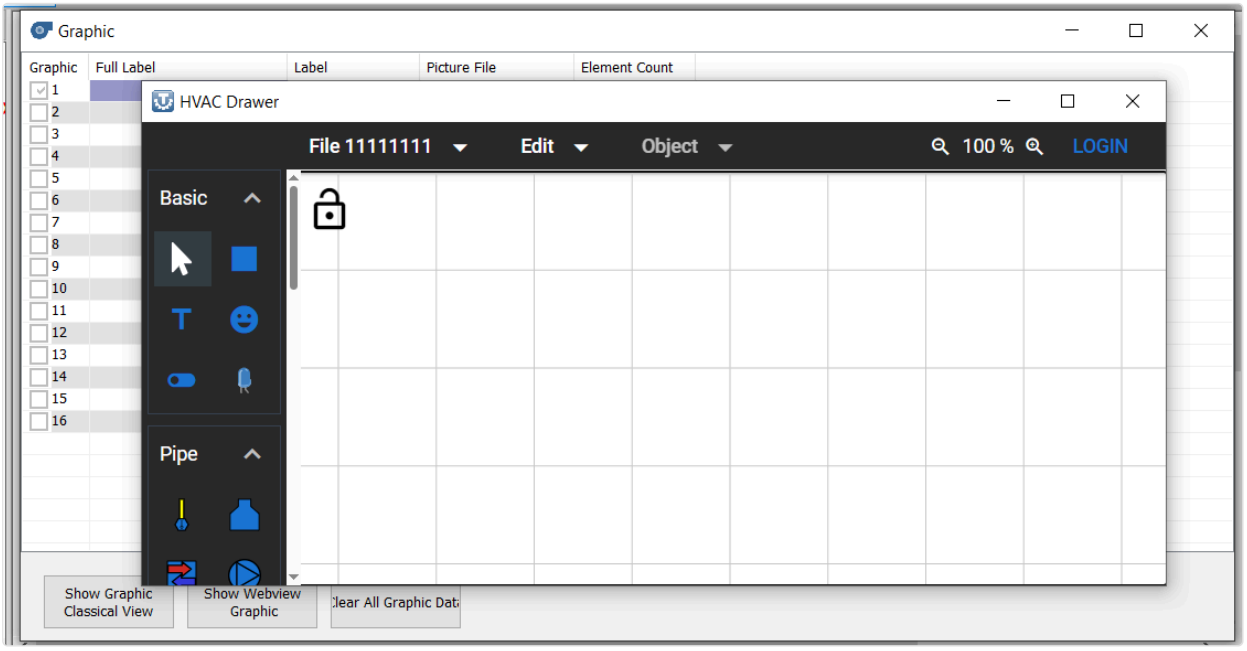
comment the code as shown below.

Make sure the T3000.exe does not start the rust server in background.

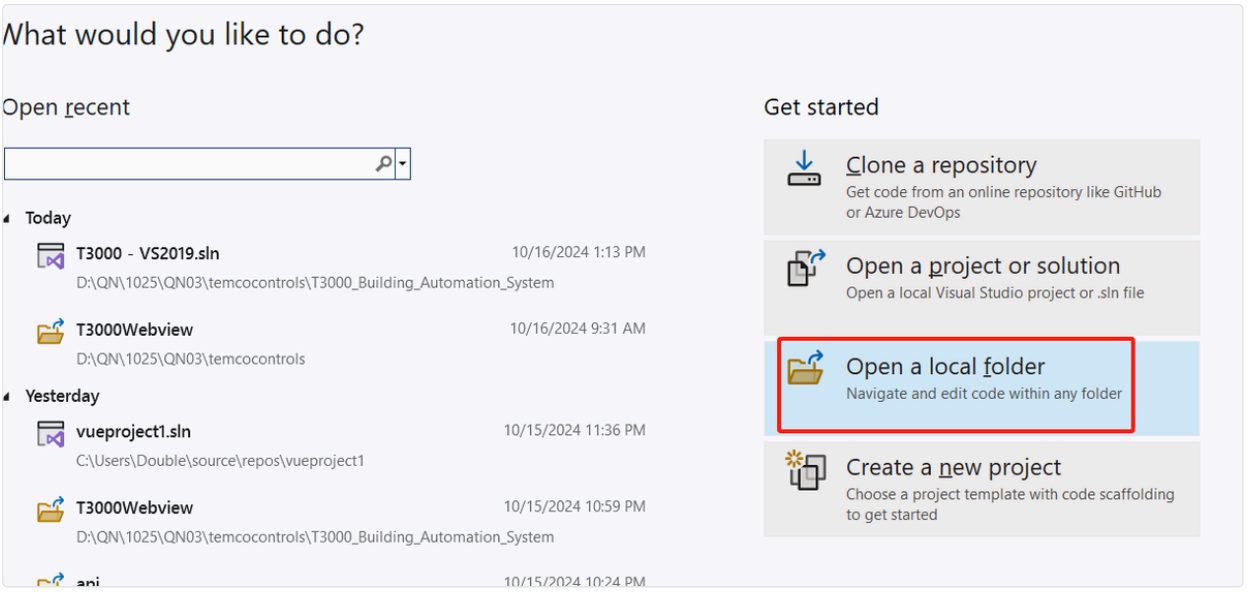


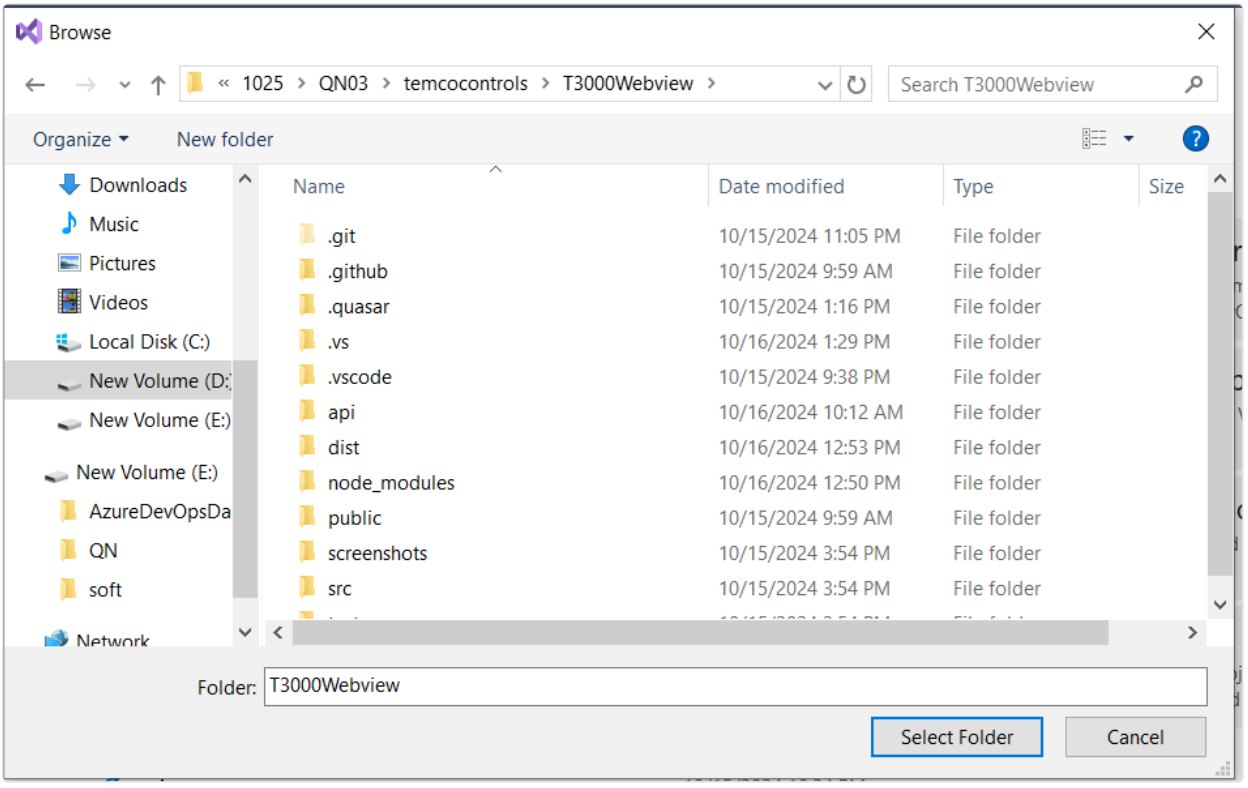
13. rebuild the solutions and run T3000.exe under T3000 Ouput\Debug, and check whether the "Show Webview Graphic" can access the rust server which was started by step 10



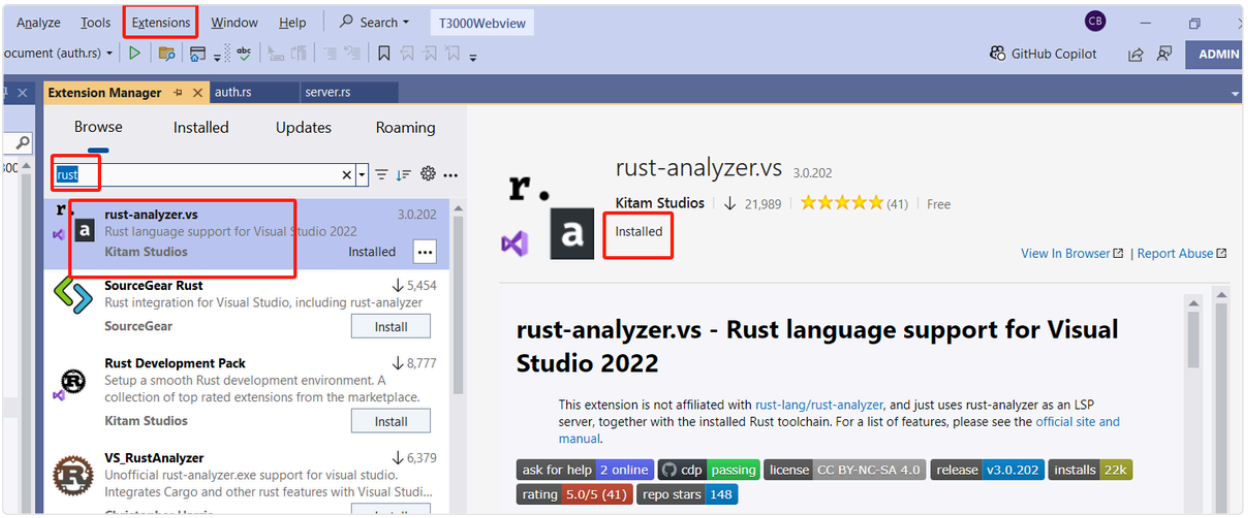


14. open the T3000Webview folder with Visual Studio 2022.

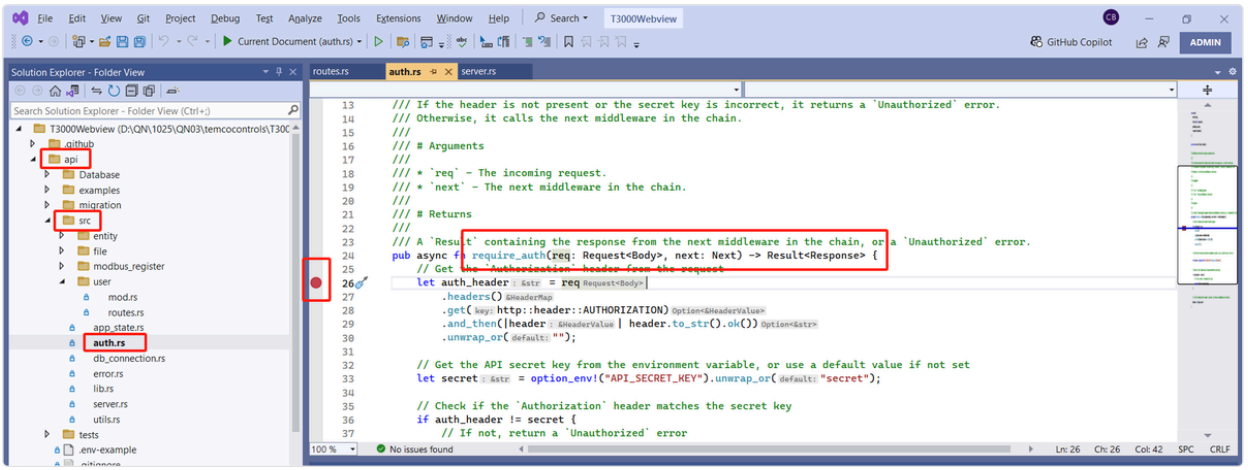




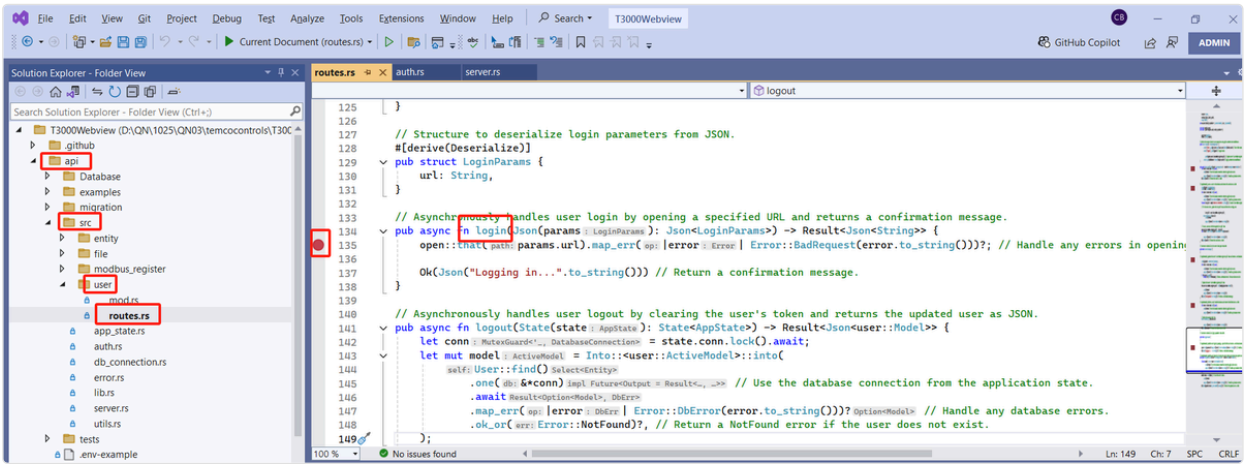
15. add rust-analyzer.vs extension to visual studio 2022.



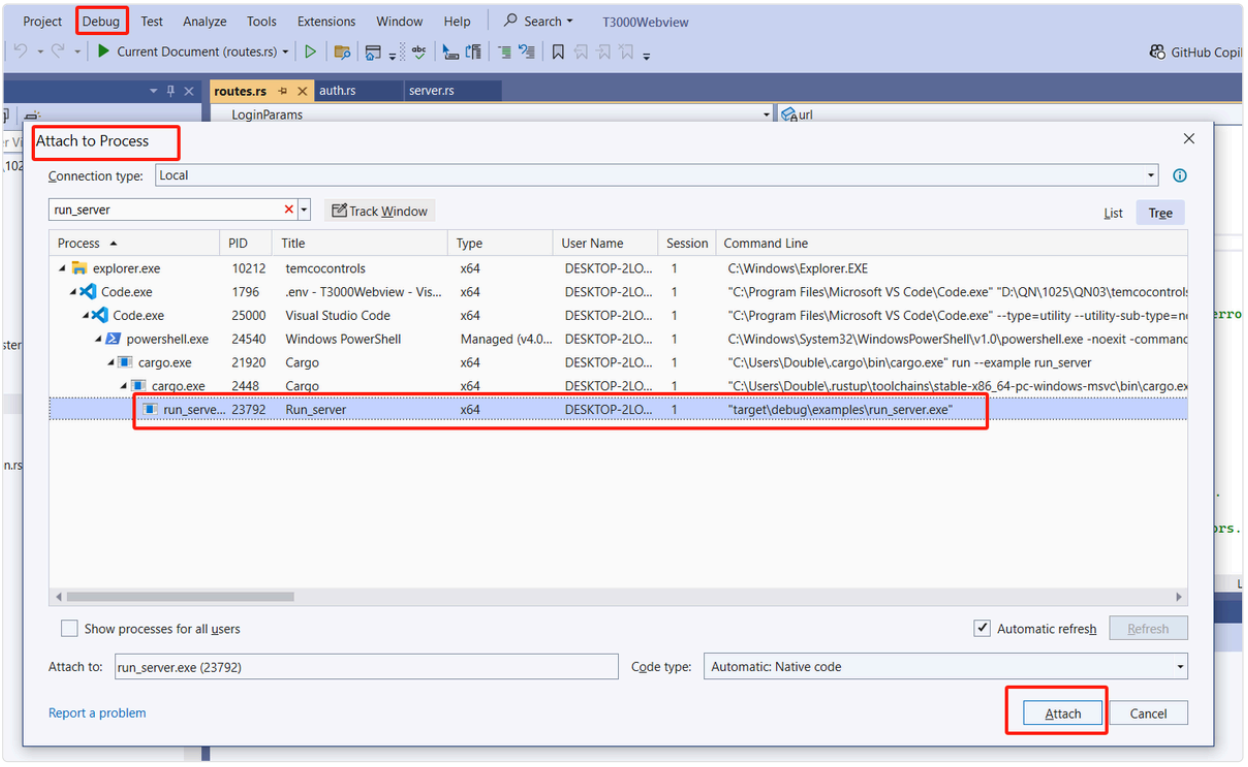
16. find auth.rs file under /api/src/auth.rs, and add a breakpoint for “require_auth” function.



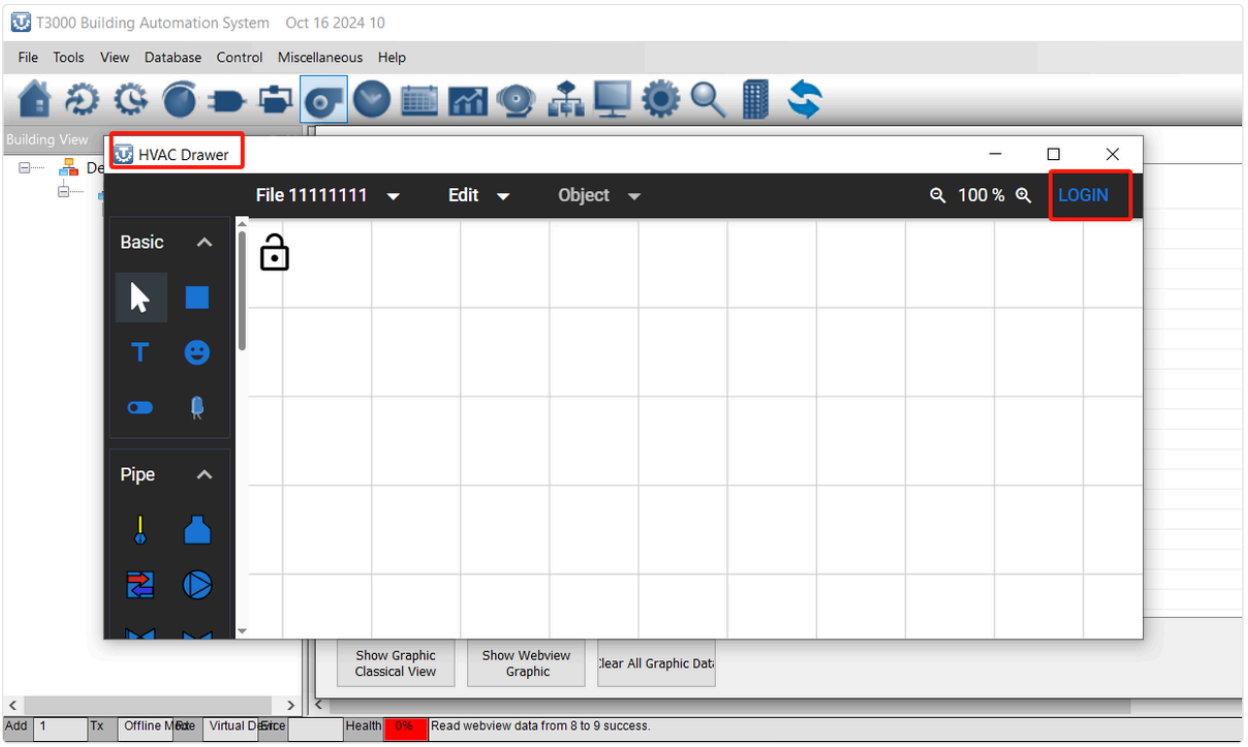
add another breakpoint for “login” function

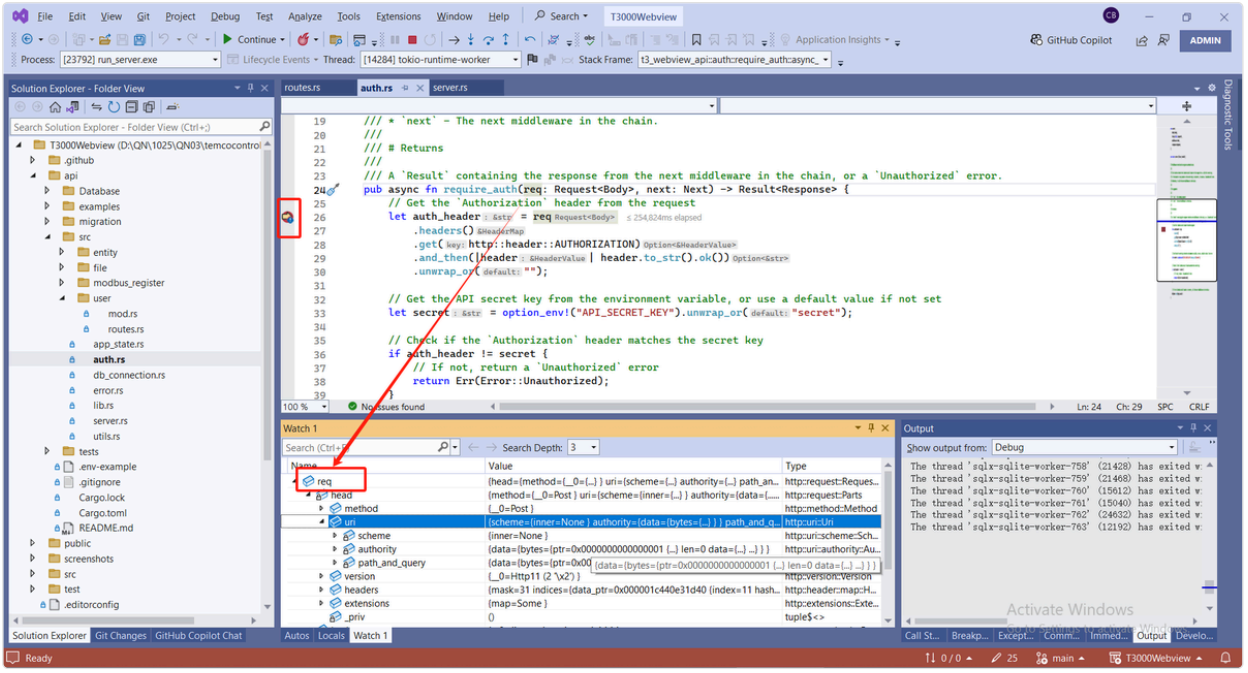


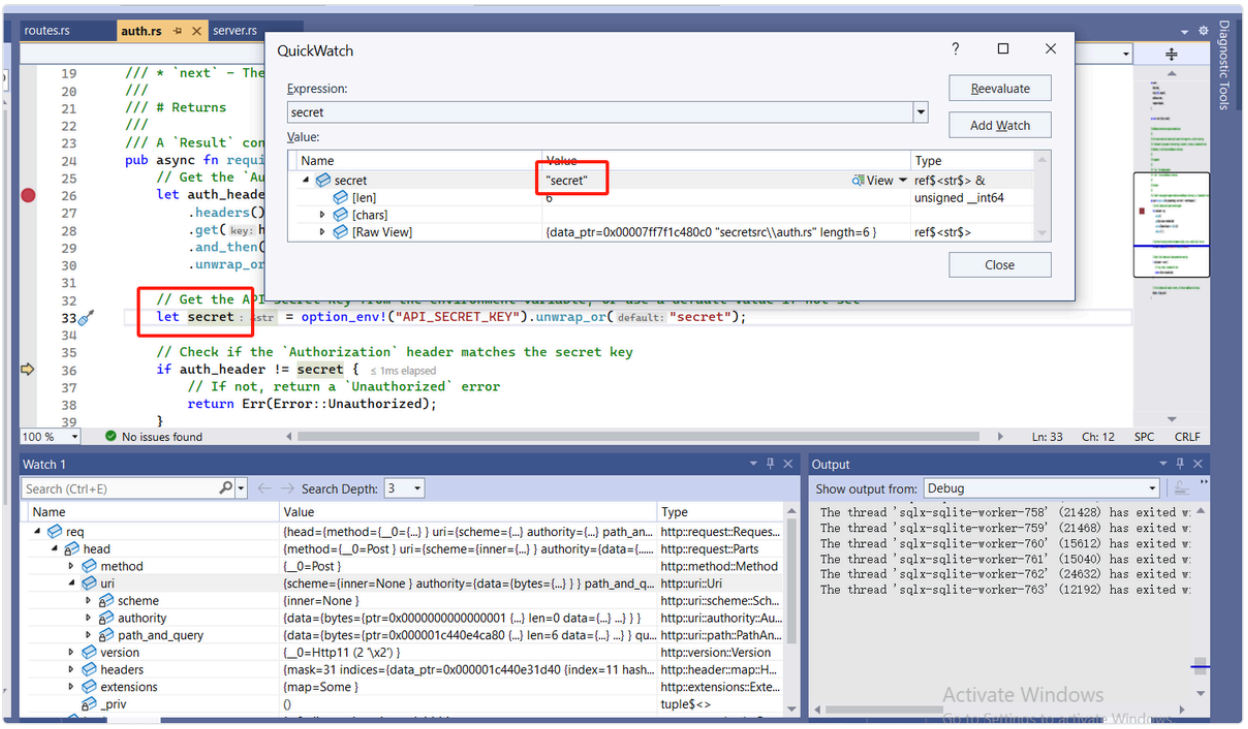
17. open the “Attach to Process” dialog, “Debug->Attach to Process”, and find the “run_server” process to attach.



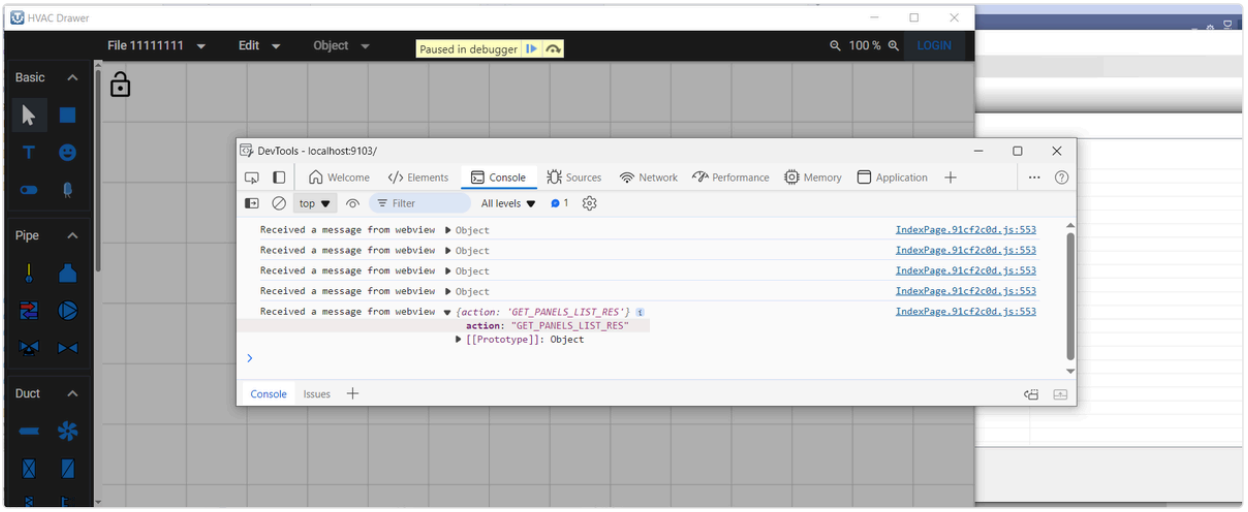
18. open “HVAC Drawer” and click the “LOGIN” button, it will redirect to the source code for debugging. You can check the details in visual studio.

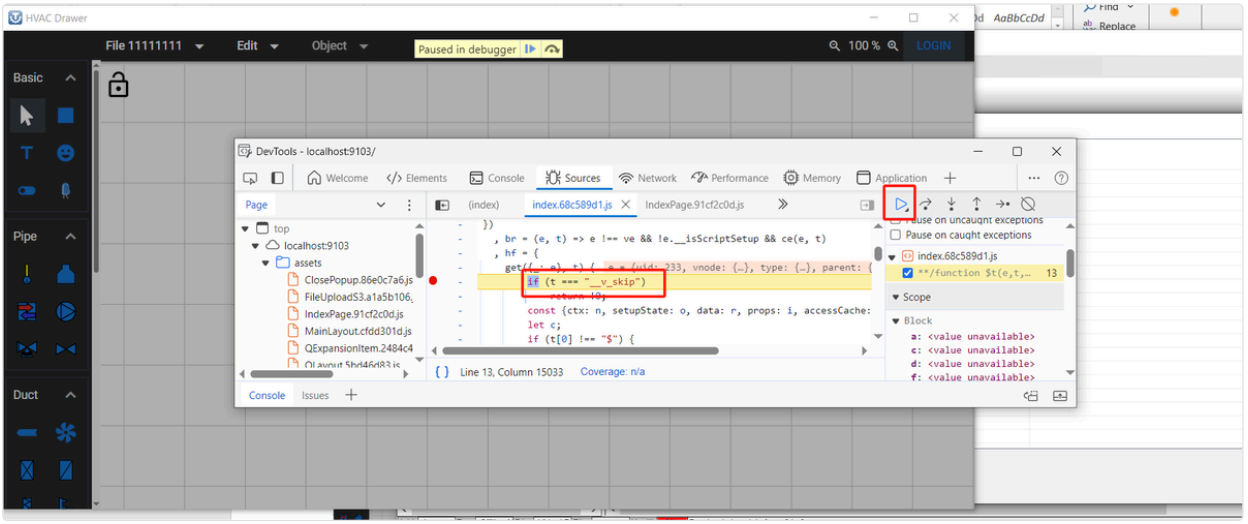

T3000 Building Automation System Oct 16 2024 10
File Tools View Database Control Miscellaneous Help
Building View
HVAC Drawer
File 11111111
Edit
Object
100 %
LOGIN
Basic
Pipe
Show Graphic Classical View
Show Webview Graphic
Add
Tx
Offline Mode
Virtual Device
Health
0%
Read webview data from 8 to 9 success.


T3000Webview
GitHub Copilot
ADMIN
Solution Explorer - Folder View
T3000Webview (D:\QN\1025\QN03\temcocontrol
.github
api
Database
examples
migration
src
entity
file
modbus_register
user
mod.rs
routes.rs
app_state.rs
auth.rs
db_connection.rs
error.rs
lib.rs
server.rs
utils.rs
tests
.env-example
.gitignore
Cargo.lock
Cargo.toml
README.md
public
screenshots
src
test
.editorconfig
routes.rs
auth.rs
server.rs
/// * `next` - The next middleware in the chain.
/// # Returns
/// A `Result` containing the response from the next middleware in the chain, or a `Unauthorized` error.
pub async fn require_auth(req: Request<Body>, next: Next) -> Result<Response> {
// Get the `Authorization` header from the request
let auth_header = req.headers().get(http::header::AUTHORIZATION).and_then(|header| header.to_str().ok()).unwrap_or("");
// Get the API secret key from the environment variable, or use a default value if not set
let secret = option_env!("API_SECRET_KEY").unwrap_or("secret");
// Check if the `Authorization` header matches the secret key
if auth_header != secret {
// If not, return a `Unauthorized` error
return Err(Error::Unauthorized);
}
No issues found
Ln: 24 Ch: 29 SPC CRLF
Watch 1
Search Depth: 3
Name
Value
Type
req
head
method
uri
scheme
authority
path_and_query
version
headers
extensions
_priv
Output
Show output from: Debug
The thread 'sqlx-sqlite-worker-758' (21428) has exited w
The thread 'sqlx-sqlite-worker-759' (21468) has exited w
The thread 'sqlx-sqlite-worker-760' (15612) has exited w
The thread 'sqlx-sqlite-worker-761' (15040) has exited w
The thread 'sqlx-sqlite-worker-762' (24632) has exited w
The thread 'sqlx-sqlite-worker-763' (12192) has exited w
Activate Windows
Solution Explorer
Git Changes
GitHub Copilot Chat
Autos
Locals
Watch 1
Call St...
Breakp...
Except...
Comm...
Immed...
Output
Develo...
Ready
main
T3000Webview



19. for the JavaScript and Html files, can open the Edge Dev Tools to check the log and some other details.

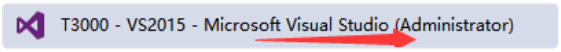




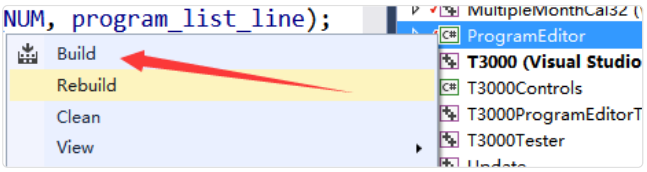
Merging the new editor

Replacing the editor inside the shell is a merge, not a copy: the screenshots show the files and the order the change goes in.

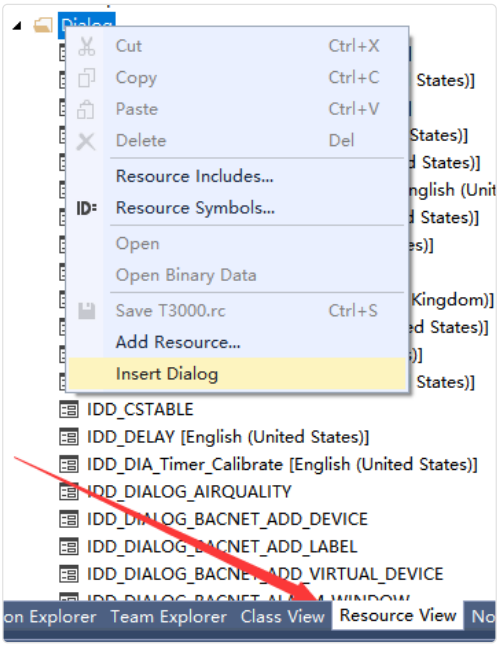
Run VS as Administrtor

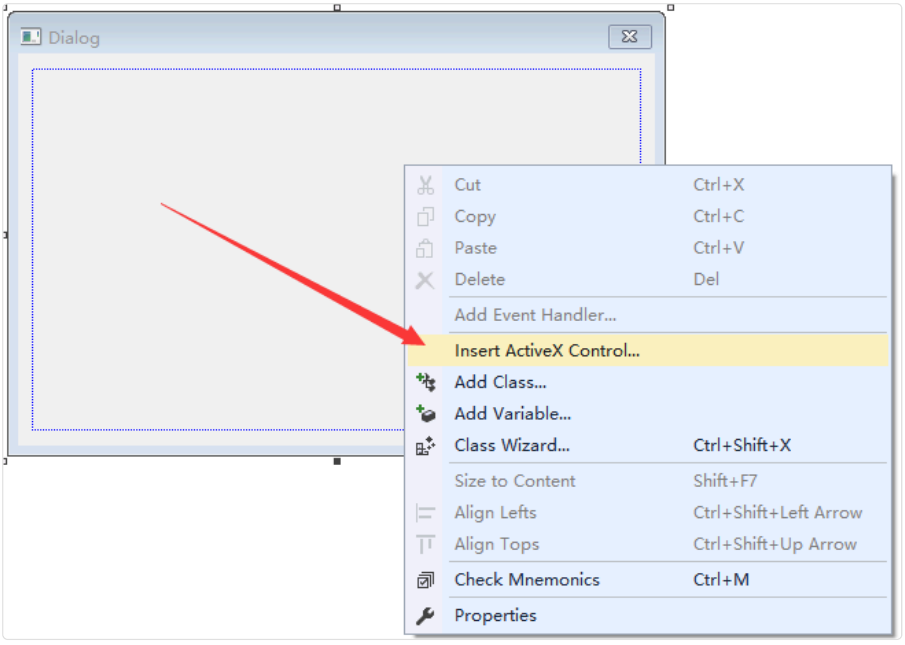


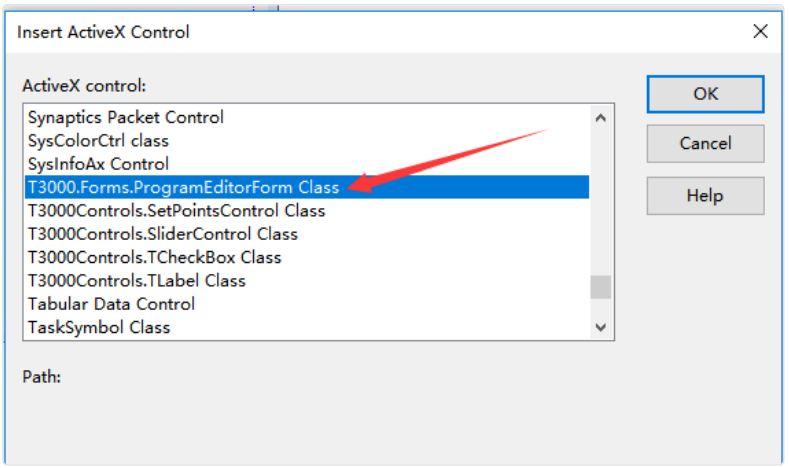
Compile “ProgramEditor”

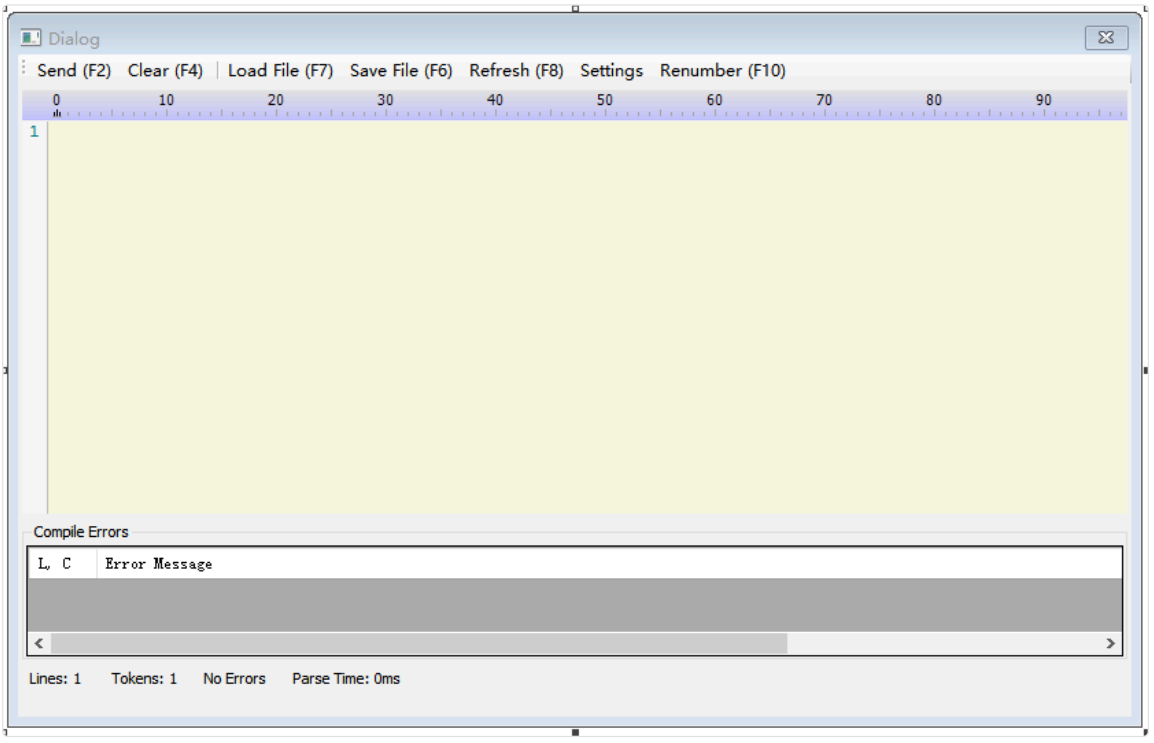


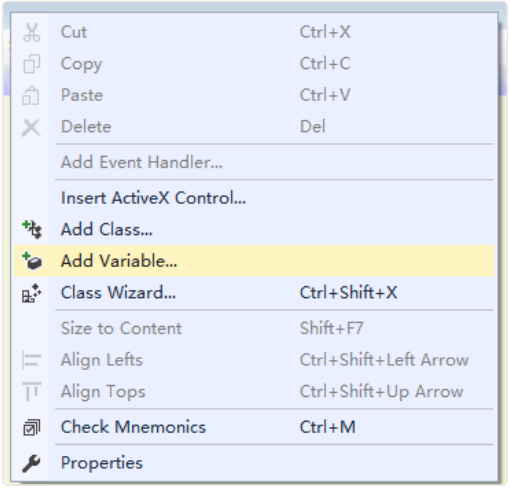
Add a Dialog for new editor



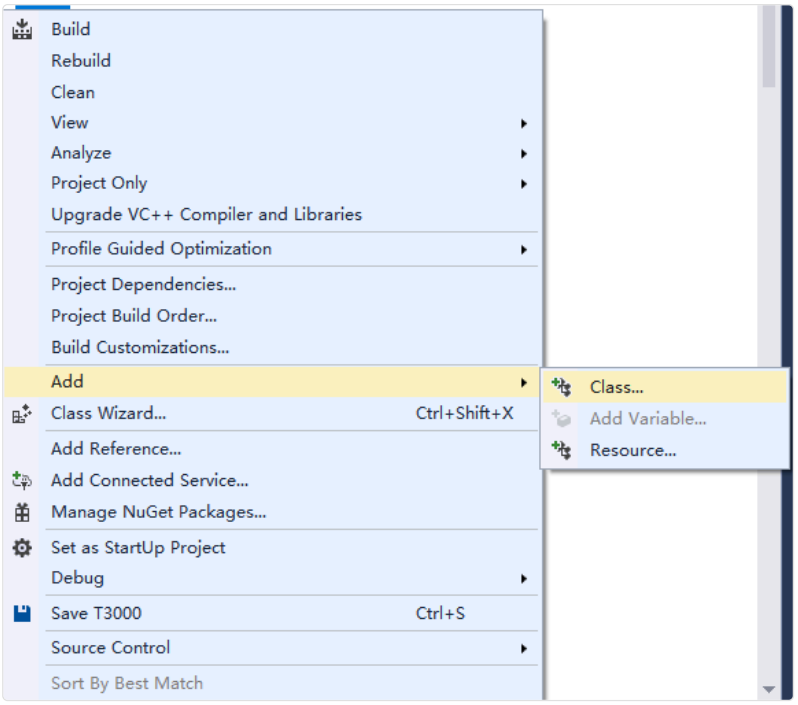


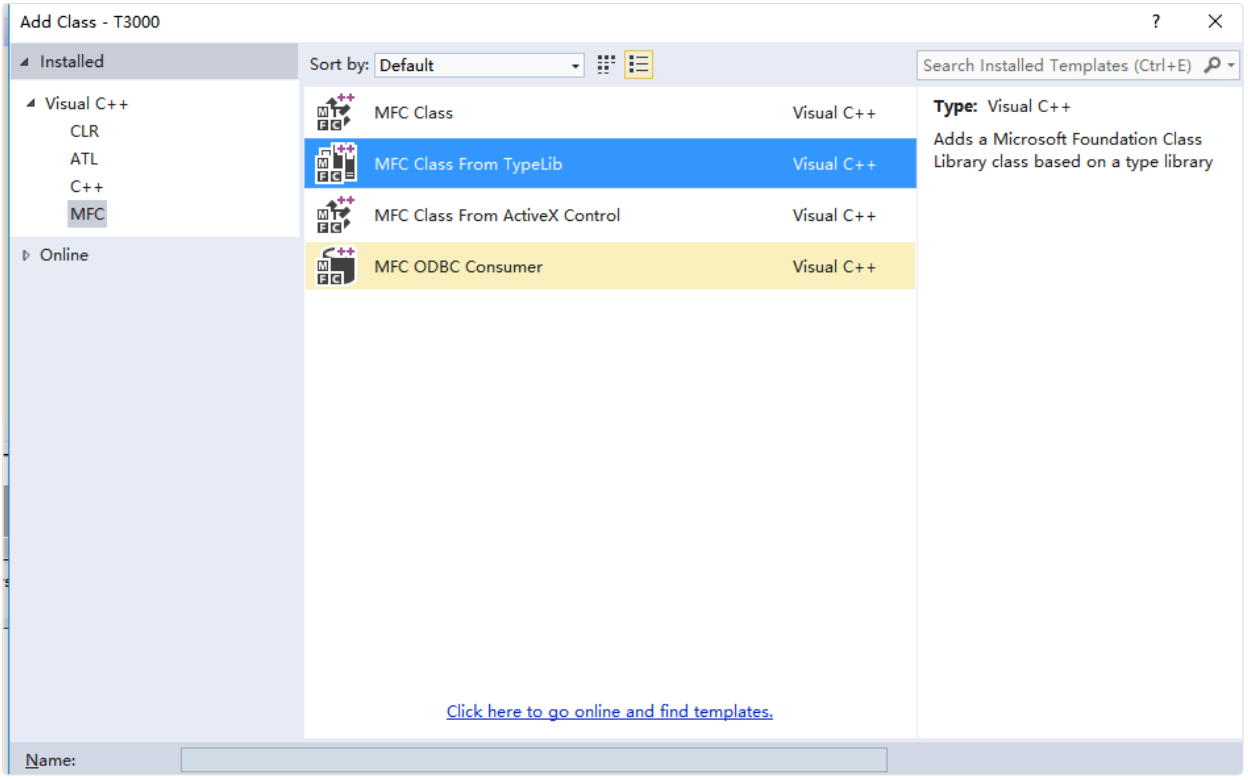


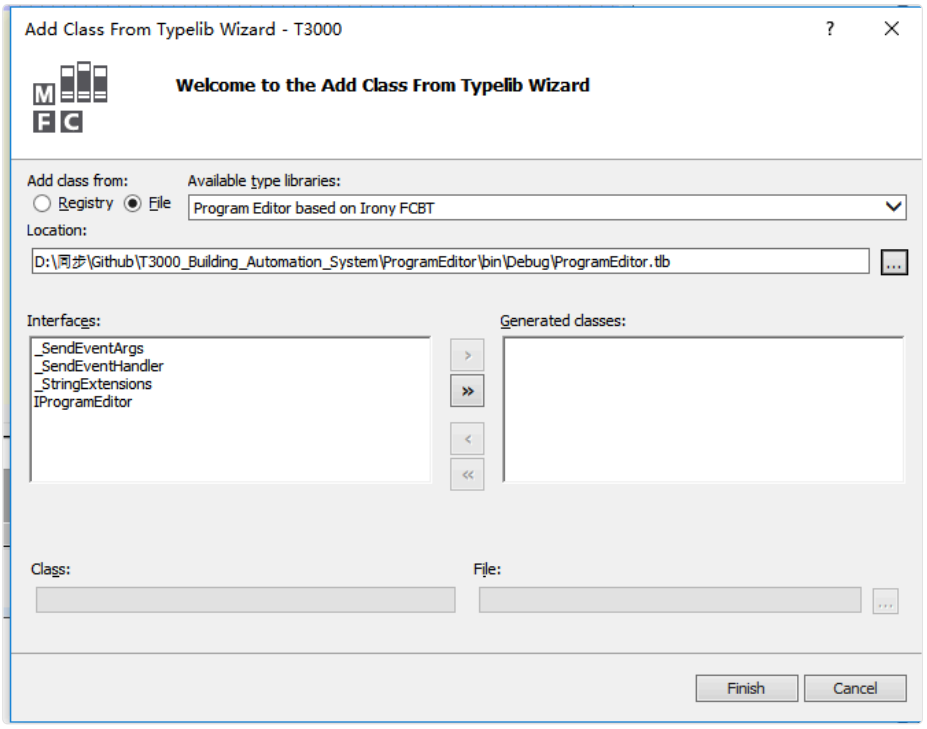


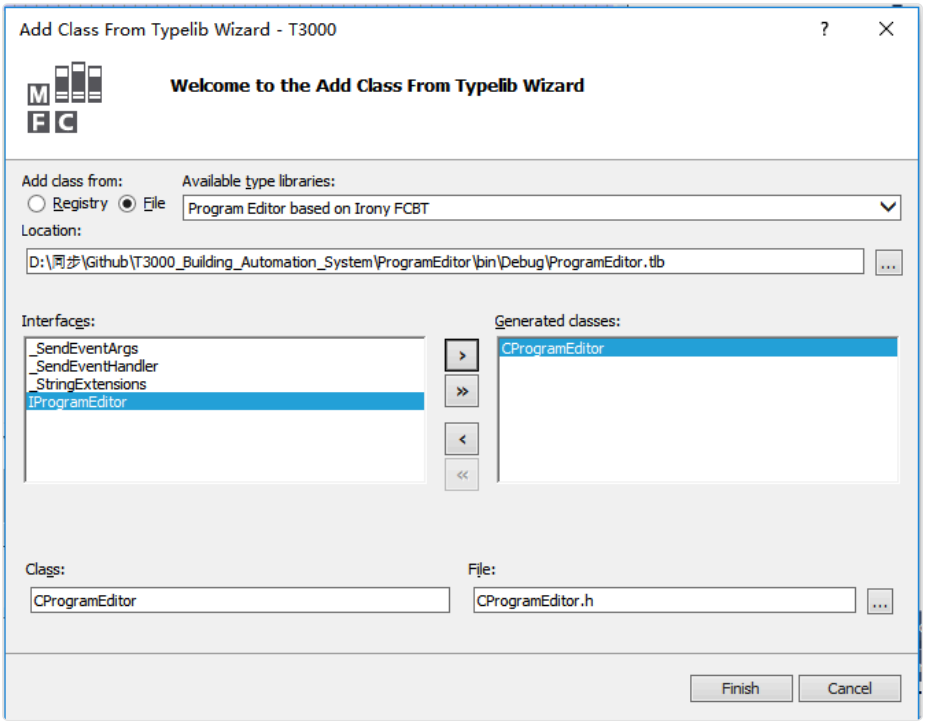


Add the programeeditor class .

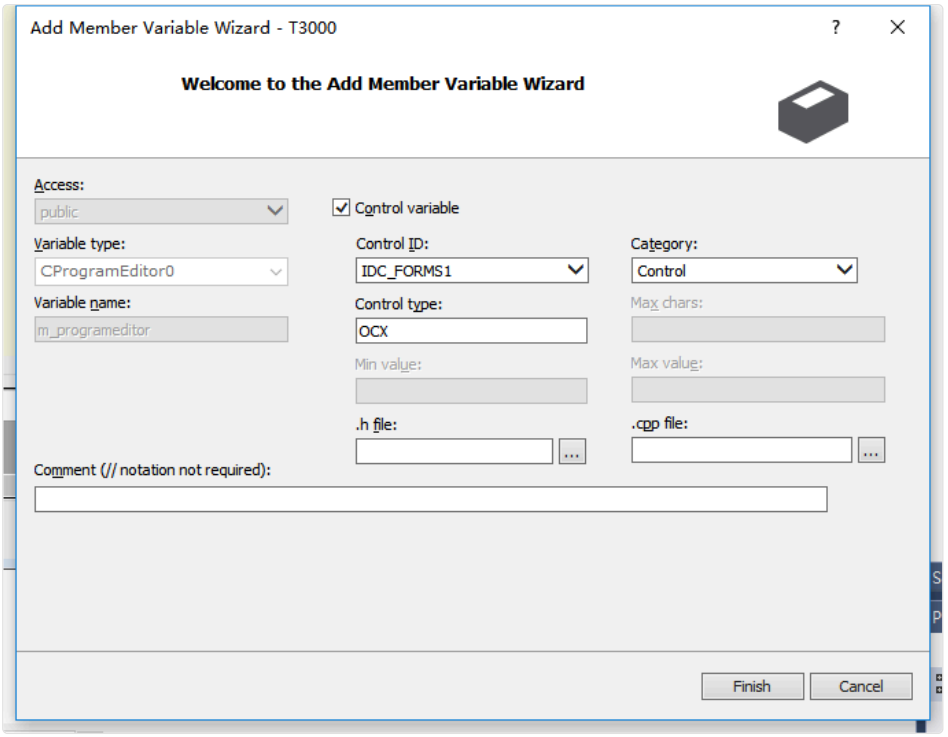

Build
Rebuild
Clean
View
Analyze
Project Only
Upgrade VC++ Compiler and Libraries
Profile Guided Optimization
Project Dependencies...
Project Build Order...
Build Customizations...
Add
Class Wizard... Ctrl+Shift+X
Add Reference...
Add Connected Service...
Manage NuGet Packages...
Set as StartUp Project
Debug
Save T3000 Ctrl+S
Source Control
Sort By Best Match
Class...
Add Variable...
Resource...


Add Class - T3000
Installed
Visual C++
CLR
ATL
C++
MFC
Online
Sort by: Default
Search Installed Templates (Ctrl+E)
MFC Class Visual C++
MFC Class From TypeLib Visual C++
MFC Class From ActiveX Control Visual C++
MFC ODBC Consumer Visual C++
Type: Visual C++
Adds a Microsoft Foundation Class Library class based on a type library
Click here to go online and find templates.
Name:

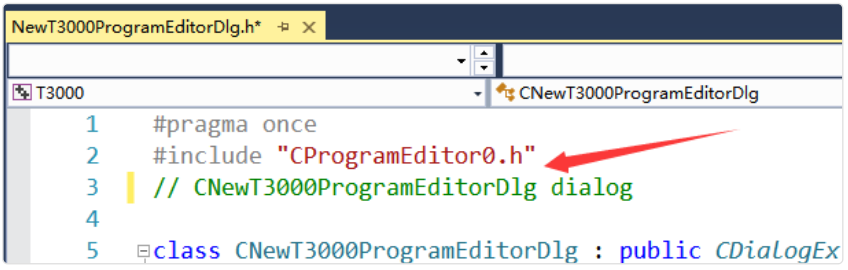


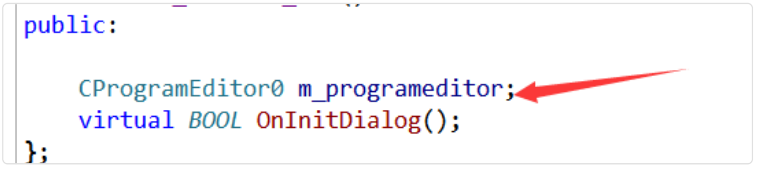


Add a variable

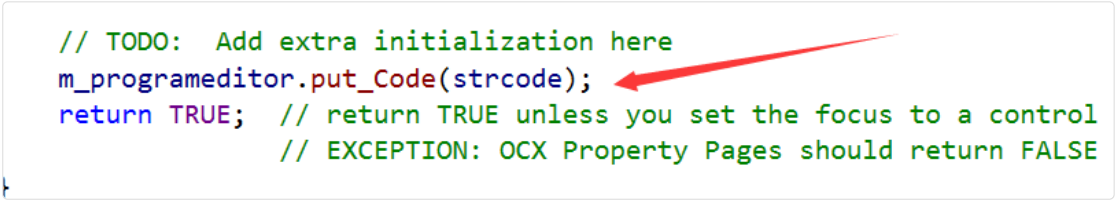


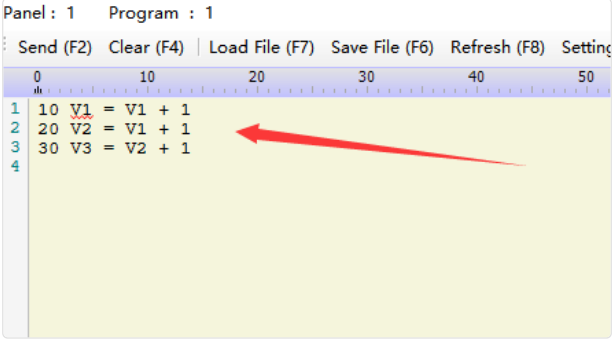
add a variable for the program editor





Call the put_code() function ,you can set the code string into the editor.





Using C# components in T3000

The shell is C++, and not everything around it is. This is how a C# component is added to a project that is built as C++.

Project Name: T3000_CrossPlatform

This document describes how to use a COM component written in C# , in the main C++/MFC application. Specifically, the control basic editor, it is done in C# and can be called from within the T3000

GITHUB: https://github.com/temcocontrols/T3000_CrossPlatform

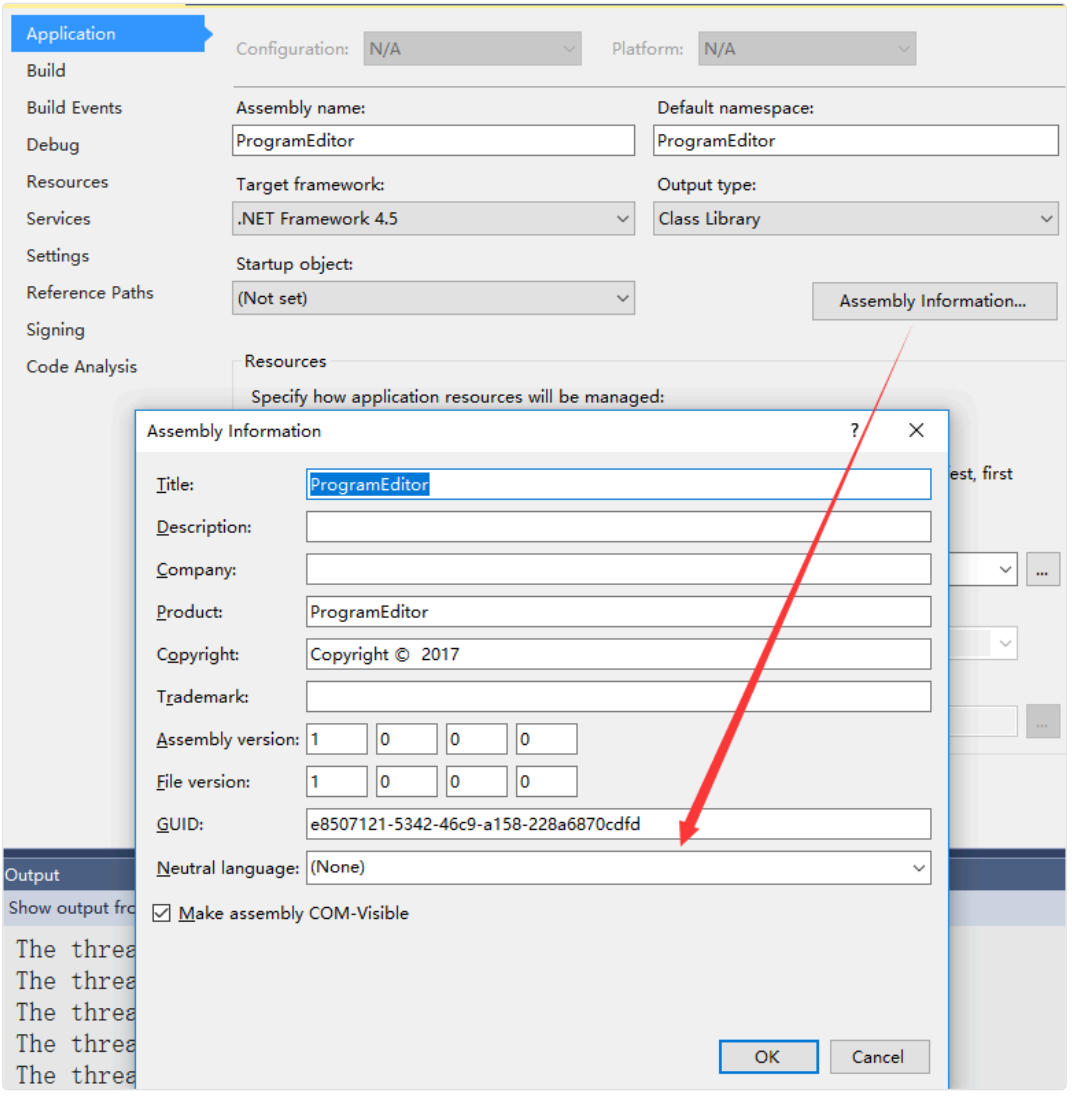
Here is a document about program editor

https://github.com/temcocontrols/T3000_CrossPlatform/tree/master/T3000Documenter/Help

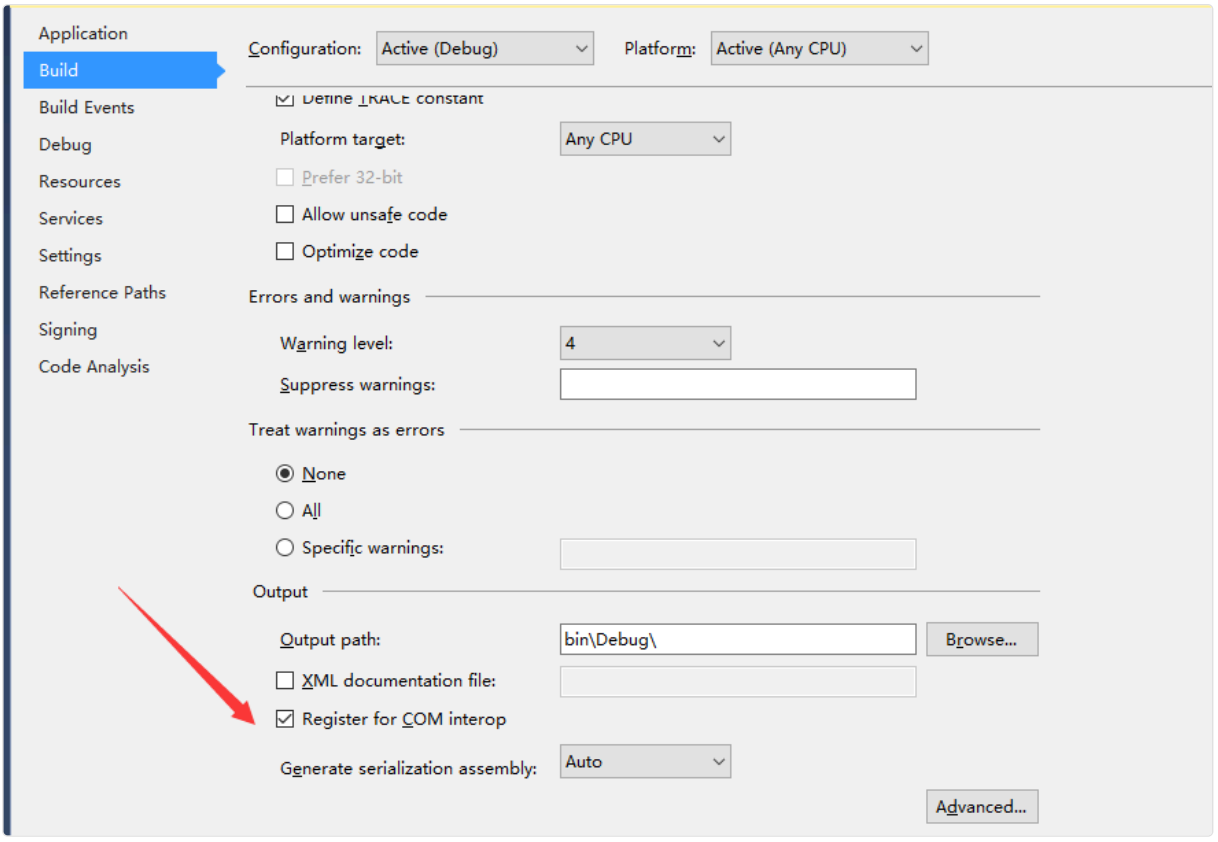
TBD: Move this to the \Documentation folder of the cross platform project.

How to Make C# Component use in MFC

Make the com visible



2.Register for COM interop



Put the ComUtilities class into the project .the code is in the background in the document .

Select the component what you want to use in other language .

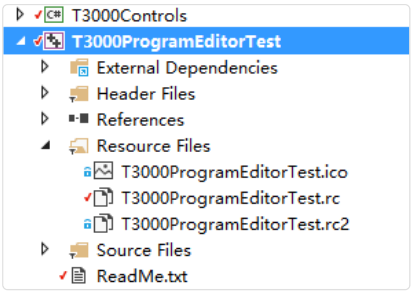
```
using Irony.Parsing;
using System.Drawing;
using System.ComponentModel;
using PRGReaderLibrary;
using System.Runtime.InteropServices;
[Guid("C5B9E15C-F44C-465F-BA06-D35F248D7D8D")]
[ClassInterface(ClassInterfaceType.None)]

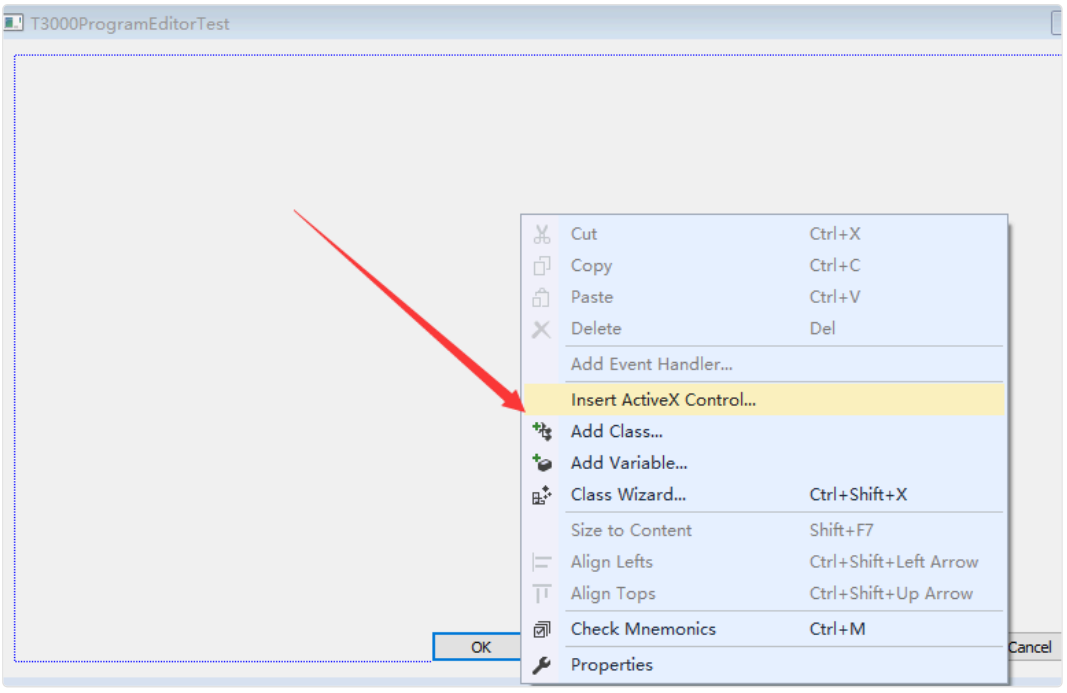
7 references | Alex Zhang, 188 days ago | 1 author, 1 change
public partial class ProgramEditorForm : Form
{
    4 references | Alex Zhang, 188 days ago | 1 author, 1 change
    public string Code { get; set; }
    Grammar _grammar;
    LanguageData _language;
    ParseTree _parseTree;
```

```
[ComRegisterFunction()]
0 references | 0 changes | 0 authors, 0 changes
public static void RegisterClass(Type type) => ComUtilities.RegisterControlClass(type);

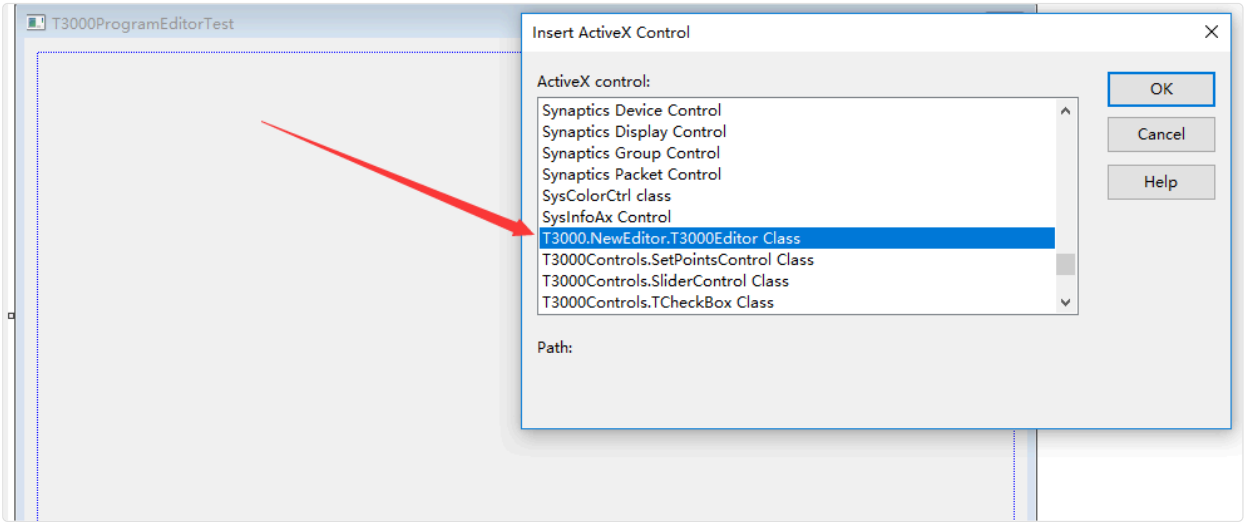
[ComUnregisterFunction()]
0 references | 0 changes | 0 authors, 0 changes
public static void UnregisterClass(Type type) => ComUtilities.UnregisterControlClass(type);
```

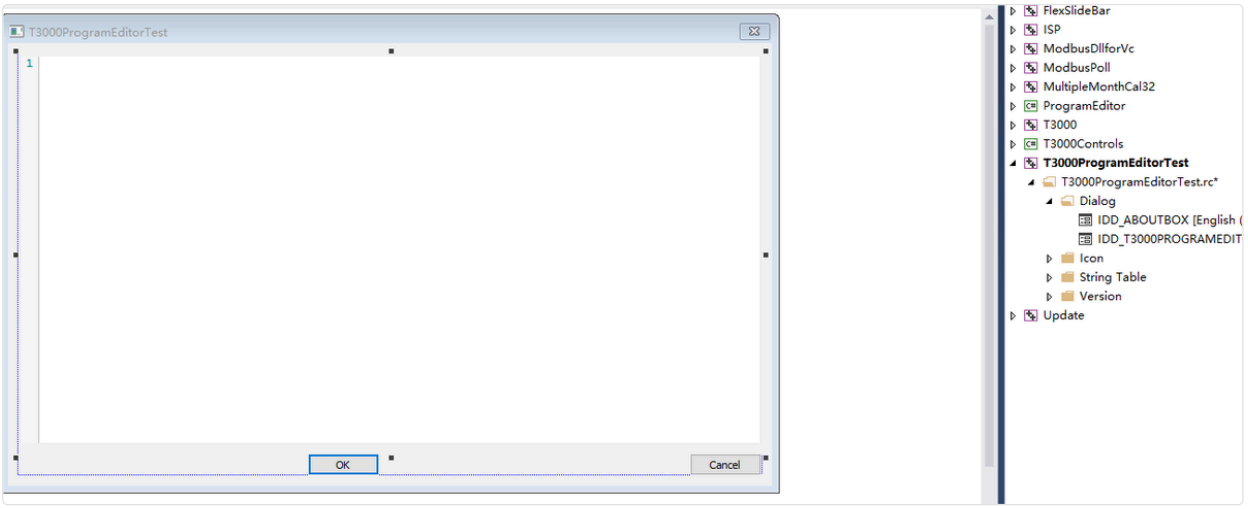
Make a test project



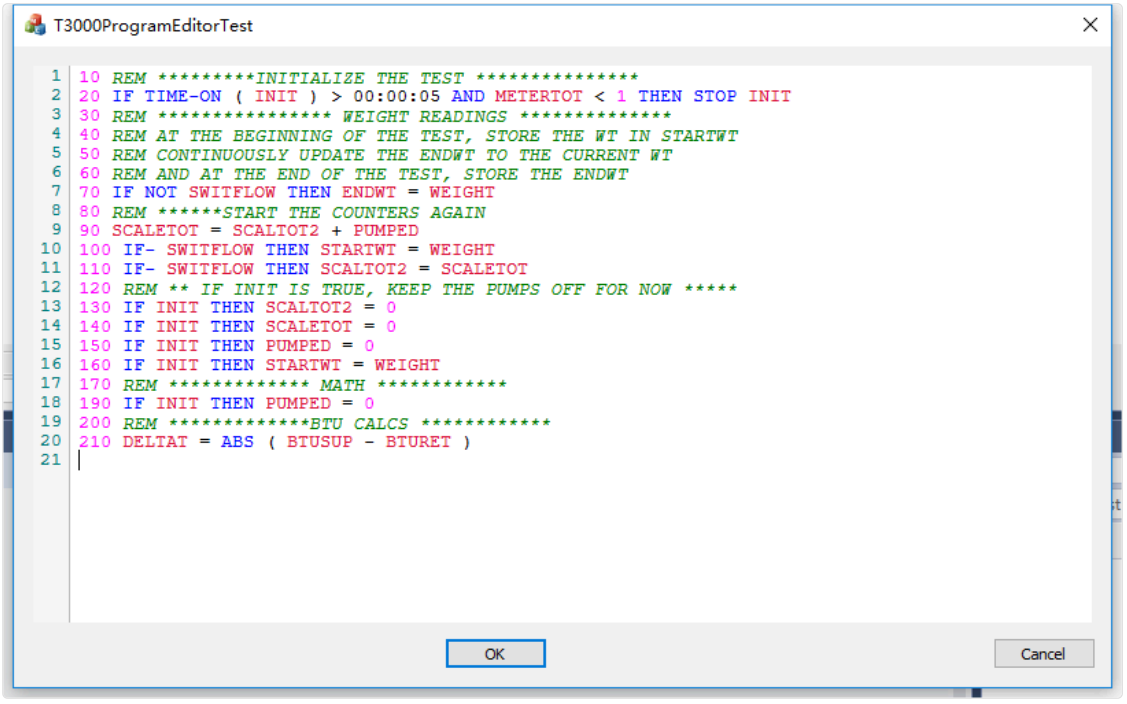


Select the component that we want to use .





F5 ,run the test project .



CODE – ComUtilities

namespace T3000.Forms

{

using System;

using System.Reflection;

using Microsoft.Win32;

using System.Runtime.InteropServices;

internal class NativeMethods

{

internal static class OLEMISC

{

internal const int OLEMISC_RECOMPOSEONRESIZE = 0x1;

internal const int OLEMISC_ONLYICONIC = 0x2;

```
internal const int OLEMISC_INSERTNOTREPLACE = 0x4;

internal const int OLEMISC_STATIC = 0x8;

internal const int OLEMISC_CANTLINKINSIDE = 0x10;

internal const int OLEMISC_CANLINKBYOLE1 = 0x20;

internal const int OLEMISC_ISLINKOBJECT = 0x40;

internal const int OLEMISC_INSIDEOUT = 0x80;

internal const int OLEMISC_ACTIVATEWHENVISIBLE = 0x100;

internal const int OLEMISC_RENDERINGISDEVICEINDEPENDENT = 0x200;

internal const int OLEMISC_INVISIBLEATRUNTIME = 0x400;

internal const int OLEMISC_ALWAYSRUN = 0x800;

internal const int OLEMISC_ACTSLIKEBUTTON = 0x1000;

internal const int OLEMISC_ACTSLIKELABEL = 0x2000;

internal const int OLEMISC_NOUIACTIVATE = 0x4000;

internal const int OLEMISC_ALIGNABLE = 0x8000;

internal const int OLEMISC_SIMPLEFRAME = 0x10000;

internal const int OLEMISC_SETCLIENTSITEFIRST = 0x20000;

internal const int OLEMISC_IMEMODE = 0x40000;

internal const int OLEMISC_IGNOREACTIVATEWHENVISIBLE = 0x80000;

internal const int OLEMISC_WANTSTOMENUMERGE = 0x100000;

internal const int OLEMISC_SUPPORTSMULTILEVELUNDO = 0x200000;

}

}

internal static class ComUtilities

{

///

/// Register the class as a control and set it's CodeBase entry.

/// Key format: "HKEY_CLASSES_ROOT\CLSID{guid}"

///

/// The type of the control

public static void RegisterControlClass(Type type, int bitmapId = 101)

{

var keyName = @"CLSID" + type.GUID.ToString("B");

using (var key = Registry.ClassesRoot.OpenSubKey(keyName, true))

{

key.SetValue(null, type.ToString() + " Class");

key.CreateSubKey("Programmable").Close();

key.CreateSubKey("Control").Close();

key.CreateSubKey("Insertable").Close();
```

```
using (var subkey = key.CreateSubKey("MiscStatus"))
{
var miscStatus =
NativeMethods.OLEMISC.OLEMISC_RECOMPOSEONRESIZE +
NativeMethods.OLEMISC.OLEMISC_INSIDEOUT +
NativeMethods.OLEMISC.OLEMISC_ACTIVATEWHENVISIBLE +
NativeMethods.OLEMISC.OLEMISC_SETCLIENTSITEFIRST;
subkey.SetValue(null, miscStatus.ToString());
}
using (var subkey = key.CreateSubKey("TypeLib"))
{
var libid = Marshal.GetTypeLibGuidForAssembly(type.Assembly);
subkey.SetValue(null, libid.ToString("B"));
}
//key.DeleteSubKeyTree("InprocServer32");
using (var subkey = key.CreateSubKey("InprocServer32"))
{
//subkey.SetValue(null, Environment.SystemDirectory + @"\mscoree.dll");
subkey.SetValue("CodeBase", Assembly.GetExecutingAssembly().CodeBase);
//subkey.SetValue(null, Assembly.GetExecutingAssembly().Location);
//subkey.SetValue("ThreadingModel", "Apartment");
}
//using (var subkey = key.CreateSubKey("Server"))
//{
// subkey.SetValue(null, "mscorld.dll");
//}
//
//using (var subkey = key.CreateSubKey("ToolboxBitmap32"))
//{
//subkey.SetValue(null,
// Assembly.GetExecutingAssembly().Location + ", " +
// bitmapId.ToString());
//}
using (var subkey = key.CreateSubKey("Implemented Categories"))
{
subkey.CreateSubKey("{7DD95801-9882-11CF-9FA9-00AA006C42C4}");
subkey.CreateSubKey("{7DD95802-9882-11CF-9FA9-00AA006C42C4}");
subkey.CreateSubKey("{40FC6ED3-2438-11CF-A3DB-080036F12502}");
```

```
subkey.CreateSubKey("{40FC6ED4-2438-11CF-A3DB-080036F12502}");
subkey.CreateSubKey("{40FC6ED5-2438-11CF-A3DB-080036F12502}");
subkey.CreateSubKey("{4FED769C-D8DB-44ea-99EA-65135757C156}");
subkey.CreateSubKey("{0DE86A52-2BAA-11CF-A229-00AA003D7352}");
subkey.CreateSubKey("{0DE86A53-2BAA-11CF-A229-00AA003D7352}");
subkey.CreateSubKey("{0DE86A57-2BAA-11CF-A229-00AA003D7352}");
}
using (var subkey = key.CreateSubKey("Version"))
{
int majorVersion;
int minorVersion;
Marshal.GetTypeLibVersionForAssembly(
type.Assembly, out majorVersion, out minorVersion);
subkey.SetValue(null, $"{majorVersion}.{minorVersion}");
}
key.CreateSubKey("InstalledVersion").Close();
}
}
///
/// Called to unregister the control.
/// Key format: "HKEY_CLASSES_ROOT\CLSID{guid}"
///
/// The type of the control
public static void UnregisterControlClass(Type type)
{
Registry.ClassesRoot.DeleteSubKeyTree(@"CLSID" + type.GUID.ToString("B"), false);
}
}
}
```

Authoring the help file

The F1 help is a compiled HTML Help file built from topics, a contents tree and a map of context ids. This is how a topic is written and how the app is pointed at the result.

Help File Documentation:

The help file for T3000 is generated using a help authoring tool called Dr. Explain. This document explains how to use the tool and link the help to T3000 software.

Installation

First install Dr.Explain on your PC:

1. Download the latest installation package from the Dr.Explain website: http://www.drexplain.com/download/ and save it to your computer.
2. Extract setup.exe from the downloaded archive. Use WinZip or another archiving utility.
3. Run setup.exe and follow the instructions. If you plan to create PDF files with Dr.Explain then you need to agree to install Java Runtime Environment. You don't need to uninstall the previous version.
4. Run Dr.Explain once the installation is finished.

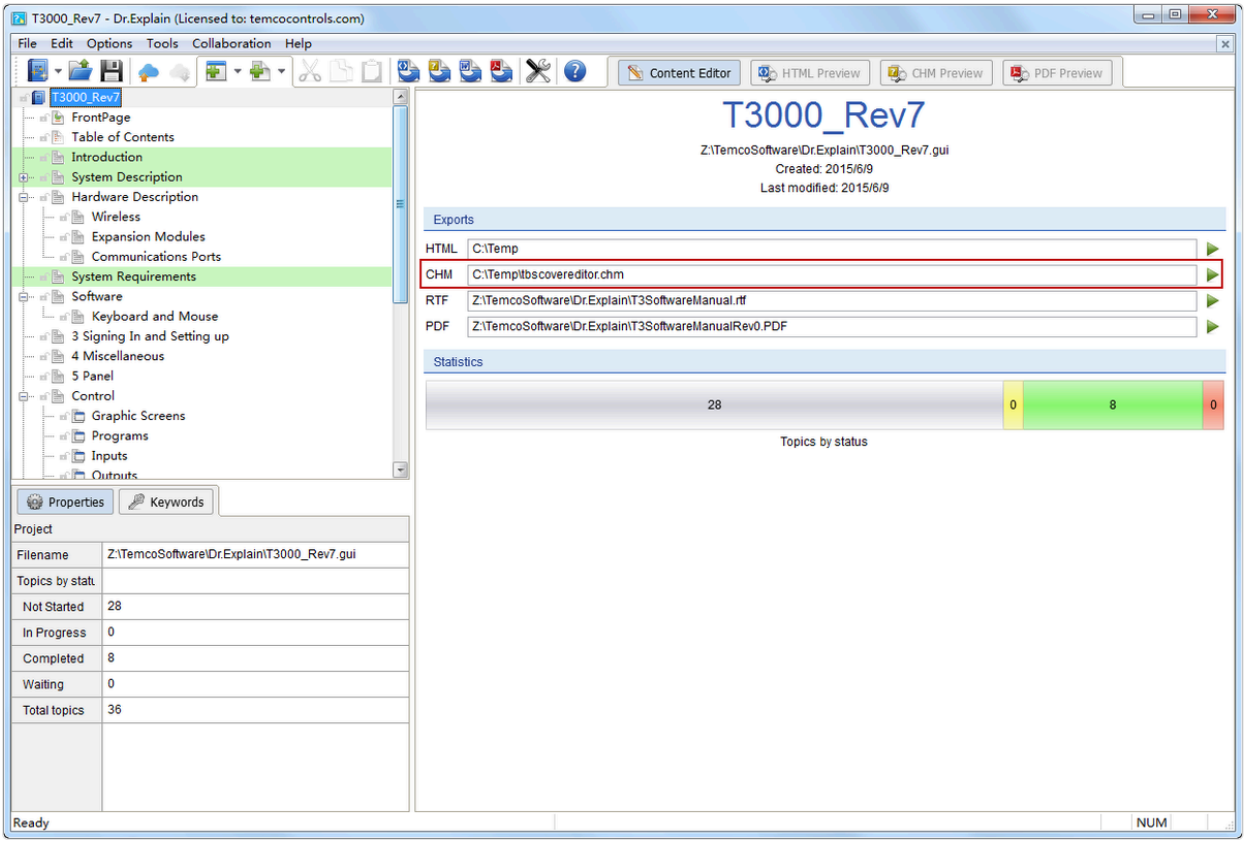
By default, Dr.Explain is installed to C:\Program Files\DrExplain on 32-bit operating systems and to C:\Program Files (x86)\DrExplain on 64-bit ones.

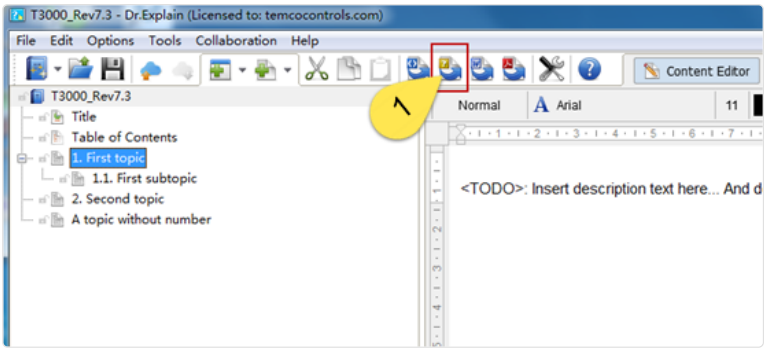
Run Dr.Explain



Open the latest T3000 help file .

Z:\TemcoSoftware\Dr.Explain



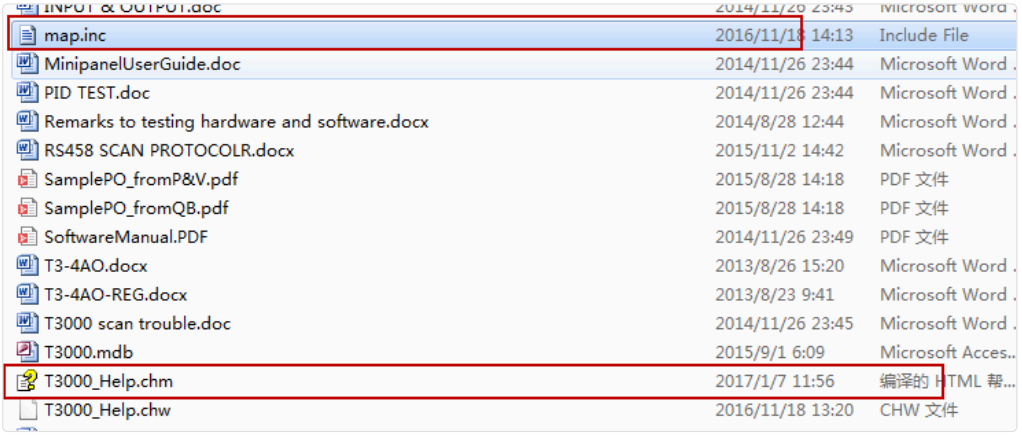


Click the button showing at tab (1) ,it will generate a file named “map.inc”

These files have to be manually copied to the ‘Installshield’ folder which is here at the Temco office:

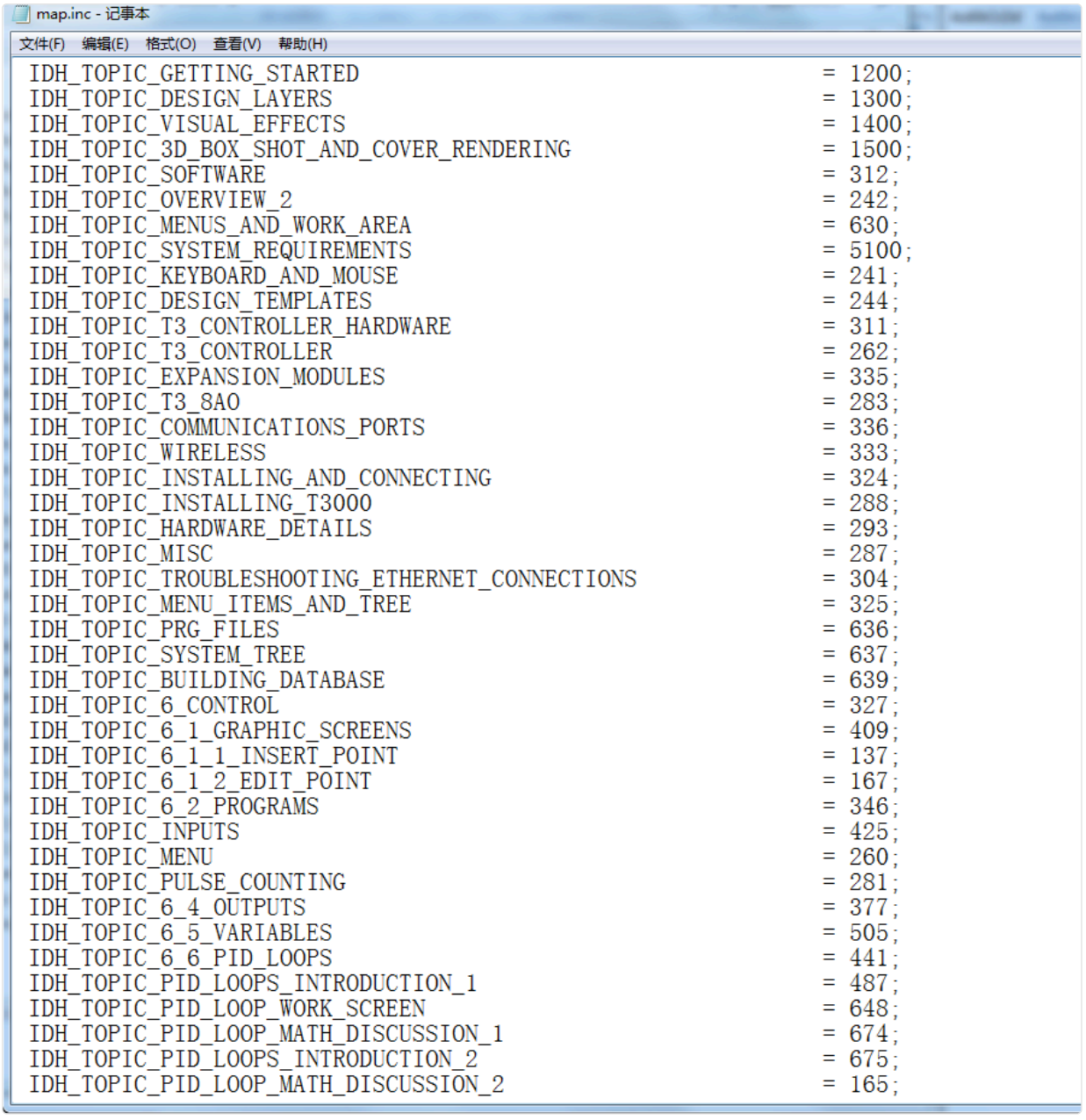
Z:\TemcoSoftware\T3000_Repo\T3000_Building_Automation_System\T3000InstallShield

This way, when the new MSI file is generated, it will also contain the updated help files.



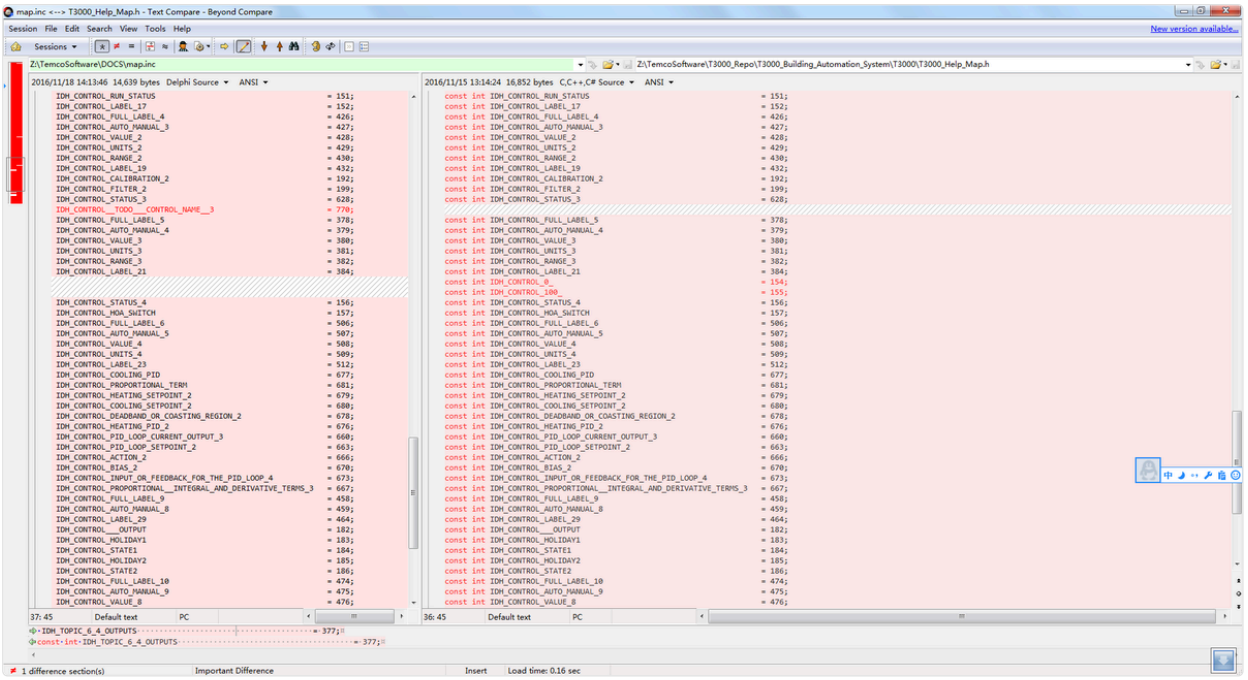
2.How to link T3000 to help file

Copy and paste the contents of map.inc into the file called T3000_Help_MAP.h

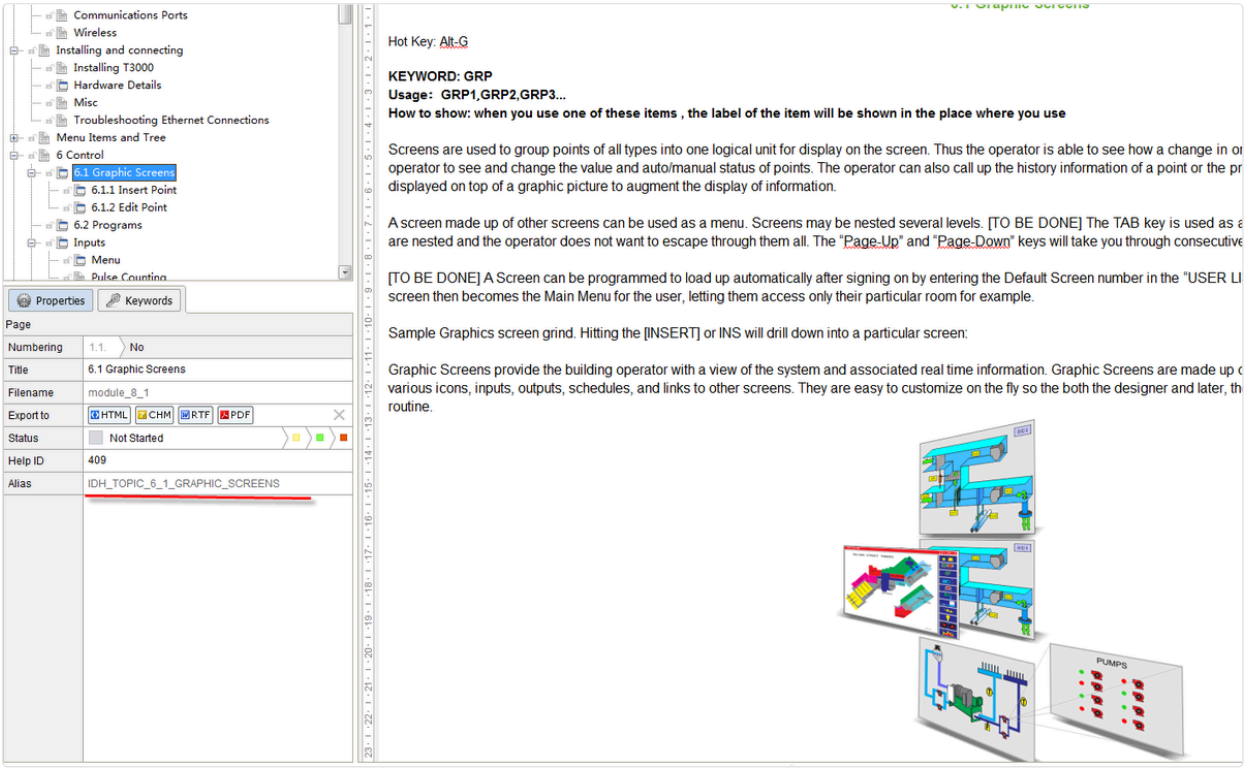


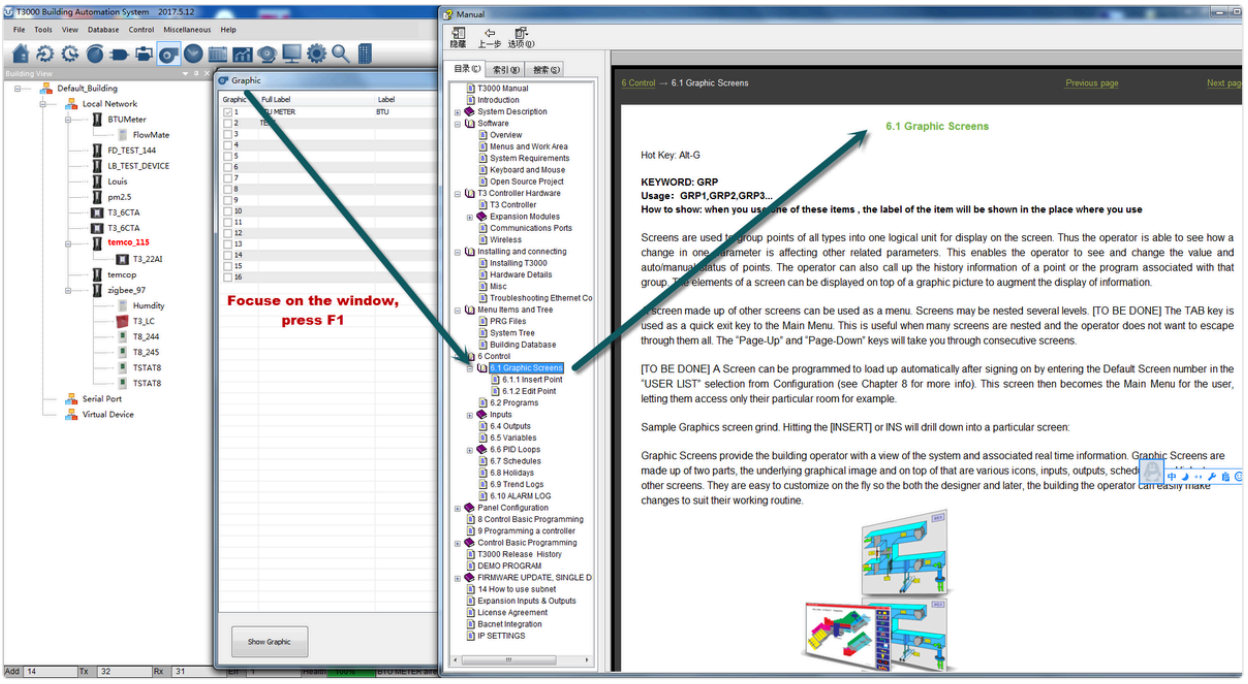
Replace the old content ,using the latest code .

and then adding the const int before key string



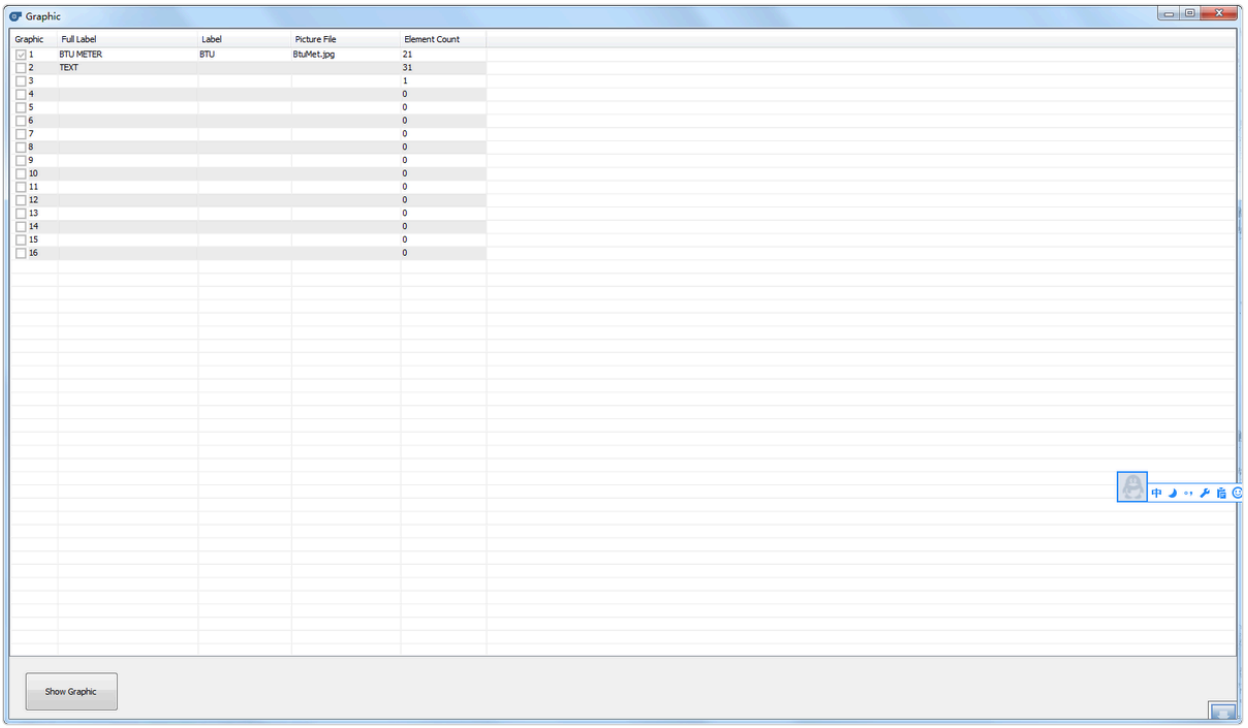
The ID numbers are generated automatically. You can change the name and the ID number in dr Explan: TBD: Show a real example from the T3000 software.





Add a function of a Window

TBD: Show the actual window here please. Screen shot.



For example //Add the code .

BOOL CBacnetGraphic::OnHelpInfo(HELPINFO* pHelpInfo)

{

hWnd = ::HtmlHelp((HWND)pHelpInfo->hItemHandle, theApp.m_szHelpFile,

HH_HELP_CONTEXT, IDH_TOPIC_6_1_GRAPHIC_SCREENS);

return CDialogEx::OnHelpInfo(pHelpInfo);

}

If user press F1 ,T3000 software will go to find the page ,according to the ID of IDH_TOPIC_6_1_GRAPHIC_SCREENS

The GitHub workflow

How the team works in the repository: branches, the merge that lands a change, and the commands used when the graphical client is not the right tool.

Basic Instructions of github

KEY POINTS

Create and use a repository

Start and manage a new branch

Make changes to a file and push them to GitHub as commits

Open and merge a pull request

Step 1. Create a Repository

A repository is usually used to organize a single project. Repositories can contain folders and files, images, videos, spreadsheets, and data sets – anything your project needs. We recommend including a README, or a file with information about your project. GitHub makes it easy to add one at the same time you create your new repository. It also offers other common options such as a license file.

Your hello-world repository can be a place where you store ideas, resources, or even share and discuss things with others.

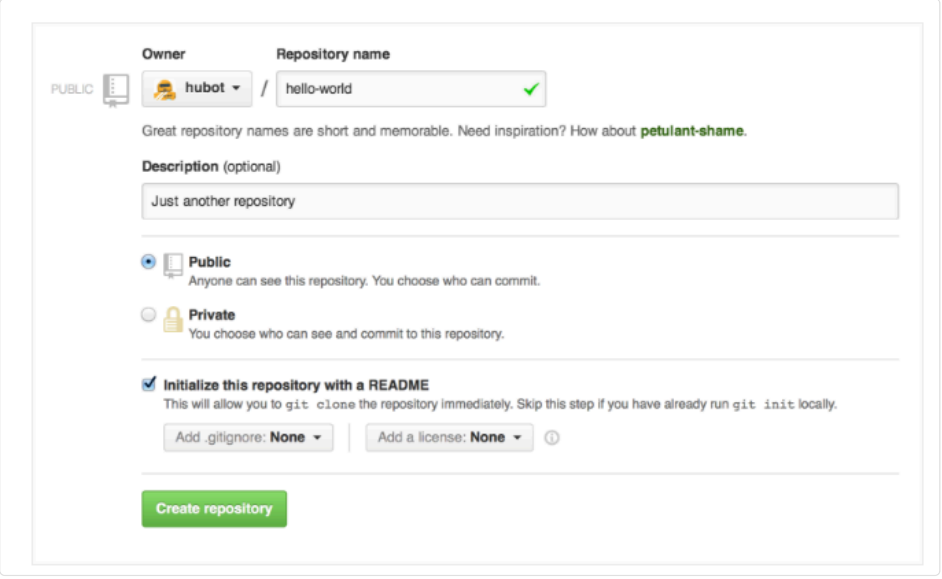
To create a new repository

In the upper right corner, next to your avatar or identicon, click and then select New repository.

Name your repository hello-world.

Write a short description.

Select Initialize this repository with a README.





Click Create repository.

Step 2. Create a Branch

Branching is the way to work on different versions of a repository at one time.

By default your repository has one branch named master which is considered to be the definitive branch. We use branches to experiment and make edits before committing them to master.

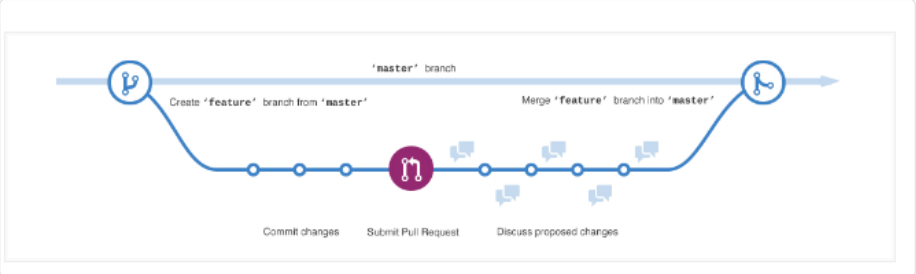
When you create a branch off the master branch, you're making a copy, or snapshot, of master as it was at that point in time. If someone else made changes to the master branch while you were working on your branch, you could pull in those updates.

This diagram shows:

The master branch

A new branch called feature (because we're doing 'feature work' on this branch)

The journey that feature takes before it's merged into master



Have you ever saved different versions of a file? Something like:

story.txt

story-joe-edit.txt

story-joe-edit-reviewed.txt

Branches accomplish similar goals in GitHub repositories.

Here at GitHub, our developers, writers, and designers use branches for keeping bug fixes and feature work separate from our master (production) branch. When a change is ready, they merge their branch into master.

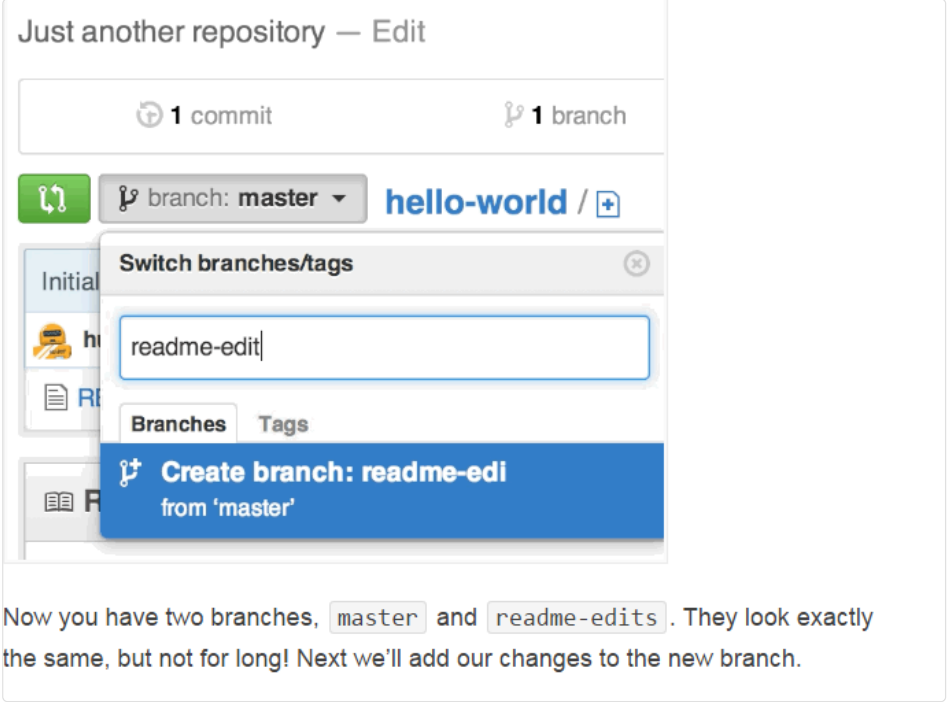
To create a new branch

Go to your new repository hello-world.

Click the drop down at the top of the file list that says branch: master.

Type a branch name, readme-edits, into the new branch text box.

Select the blue Create branch box or hit "Enter" on your keyboard.



Step 3. Make and commit changes

Bravo! Now, you're on the code view for your readme-edits branch, which is a copy of master. Let's make some edits.

On GitHub, saved changes are called commits. Each commit has an associated commit message, which is a description explaining why a particular change was made. Commit messages capture the history of your changes, so other contributors can understand what you've done and why.

Make and commit changes

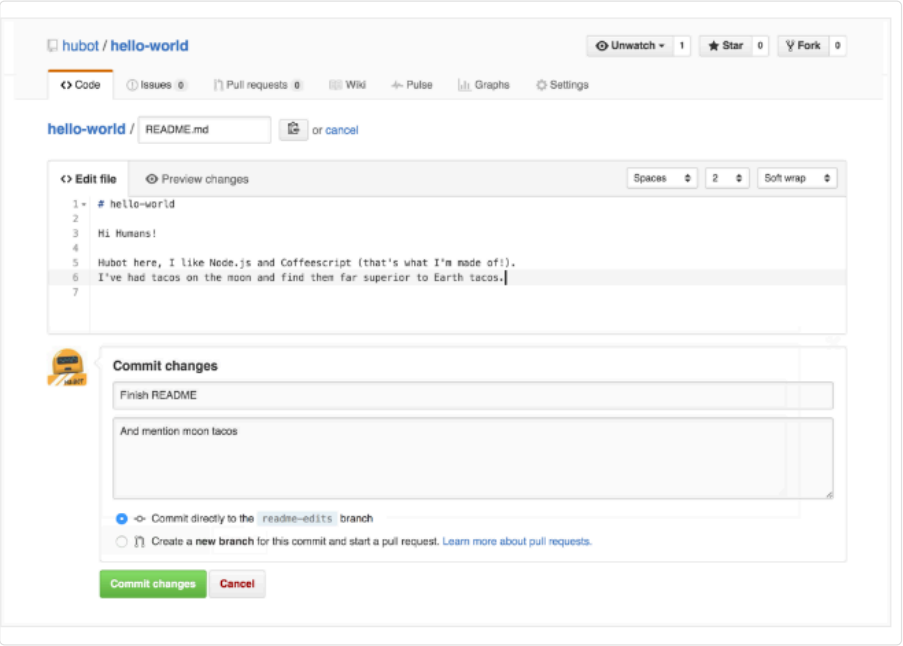
Click the README.md file.

Click the pencil icon in the upper right corner of the file view to edit.

In the editor, write a bit about yourself.

Write a commit message that describes your changes.

Click Commit changes button.



These changes will be made to just the README file on your readme-editsbranch, so now this branch contains content that's different from master.

Step 4. Open a Pull Request

Nice edits! Now that you have changes in a branch off of master, you can open a pull request.

Pull Requests are the heart of collaboration on GitHub. When you open a pull request, you're proposing your changes and requesting that someone review and pull in your contribution and merge them into their branch. Pull requests show diffs, or differences, of the content from both branches. The changes, additions, and subtractions are shown in green and red.

As soon as you make a commit, you can open a pull request and start a discussion, even before the code is finished.

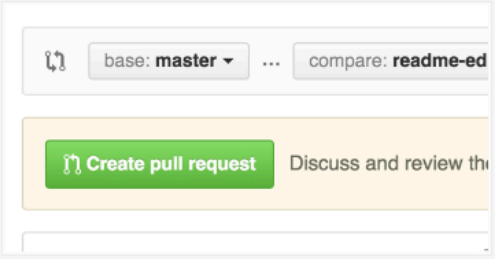
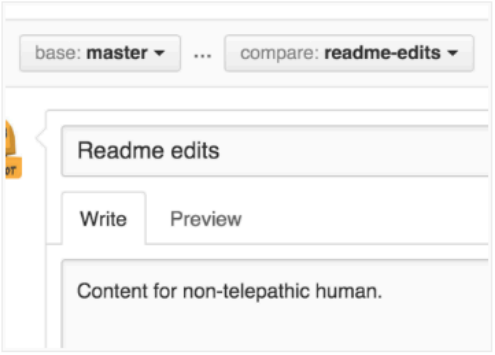
By using GitHub's @mention system in your pull request message, you can ask for feedback from specific people or teams, whether they're down the hall or 10 time zones away.

You can even open pull requests in your own repository and merge them yourself. It's a great way to learn the GitHub Flow before working on larger projects.

Open a Pull Request for changes to the README

Click on the image for a larger version

Click on the image for a larger version

Step	Screenshot
Click the **Pull Request** tab, then from the Pull Request page, click the green **New pull request** button.	
Select the branch you made, `readme-edits`, to compare with `master` (the original).	
Look over your changes in the diffs on the Compare page, make sure they're what you want to submit.	
When you're satisfied that these are the changes you want to submit, click the big green **Create Pull Request** button.	
Give your pull request a title and write a brief description of your changes.	

When you're done with your message, click Create pull request!

Tip: You can use emoji and drag and drop images and gifs onto comments and Pull Requests.

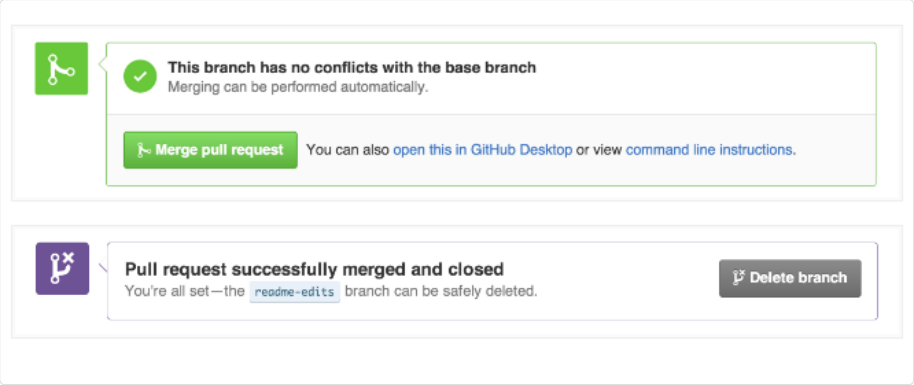
Step 5. Merge your Pull Request

In this final step, it's time to bring your changes together – merging your readme-edits branch into the master branch.

Click the green Merge pull request button to merge the changes into master.

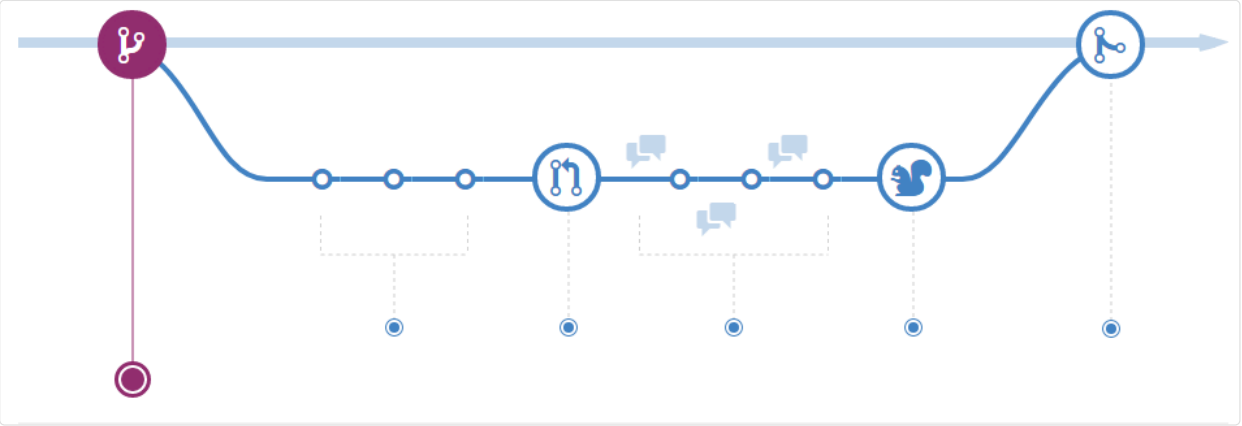
Click Confirm merge.

Go ahead and delete the branch, since its changes have been incorporated, with the Delete branch button in the purple box.



The GitHub Flow

GitHub Flow is a lightweight, branch-based workflow that supports teams and projects where deployments are made regularly. This guide explains how and why GitHub Flow works.



Create a branch

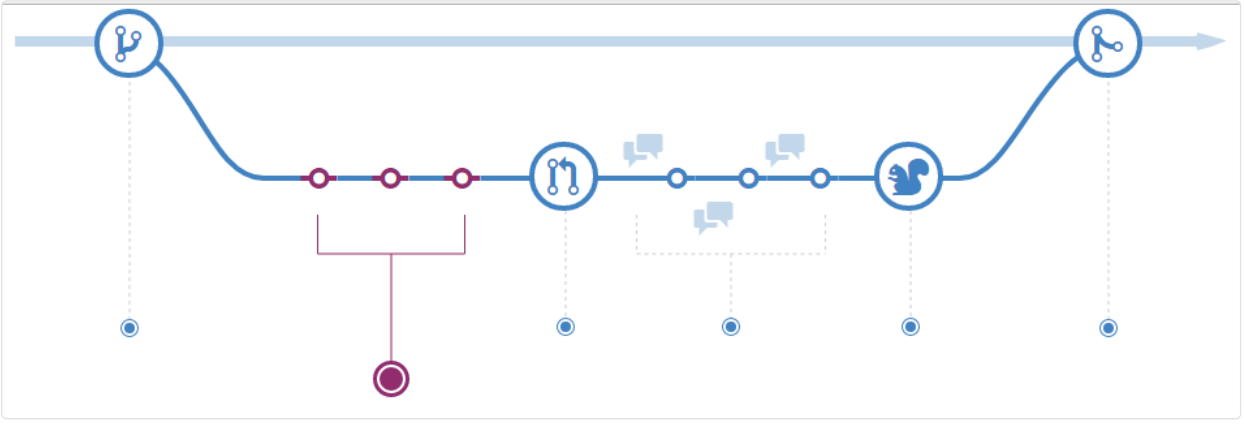
When you're working on a project, you're going to have a bunch of different features or ideas in progress at any given time – some of which are ready to go, and others which are not. Branching exists to help you manage this workflow.

When you create a branch in your project, you're creating an environment where you can try out new ideas. Changes you make on a branch don't affect the master branch, so you're free to experiment and commit changes, safe in the knowledge that your branch won't be merged until it's ready to be reviewed by someone you're collaborating with.

ProTip

Branching is a core concept in Git, and the entire GitHub Flow is based upon it. There's only one rule: anything in the master branch is always deployable.

Because of this, it's extremely important that your new branch is created off of master when working on a feature or a fix. Your branch name should be descriptive (e.g., refactor-authentication, user-content-cache-key, make-retina-avatars), so that others can see what is being worked on.



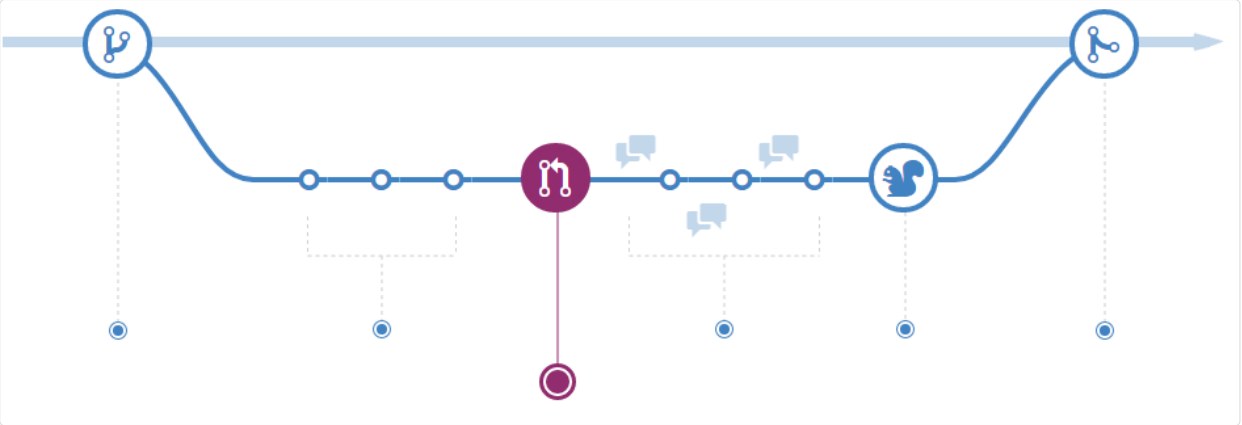
Add commits

Once your branch has been created, it's time to start making changes. Whenever you add, edit, or delete a file, you're making a commit, and adding them to your branch. This process of adding commits keeps track of your progress as you work on a feature branch.

Commits also create a transparent history of your work that others can follow to understand what you've done and why. Each commit has an associated commit message, which is a description explaining why a particular change was made. Furthermore, each commit is considered a separate unit of change. This lets you roll back changes if a bug is found, or if you decide to head in a different direction.

ProTip

Commit messages are important, especially since Git tracks your changes and then displays them as commits once they're pushed to the server. By writing clear commit messages, you can make it easier for other people to follow along and provide feedback.



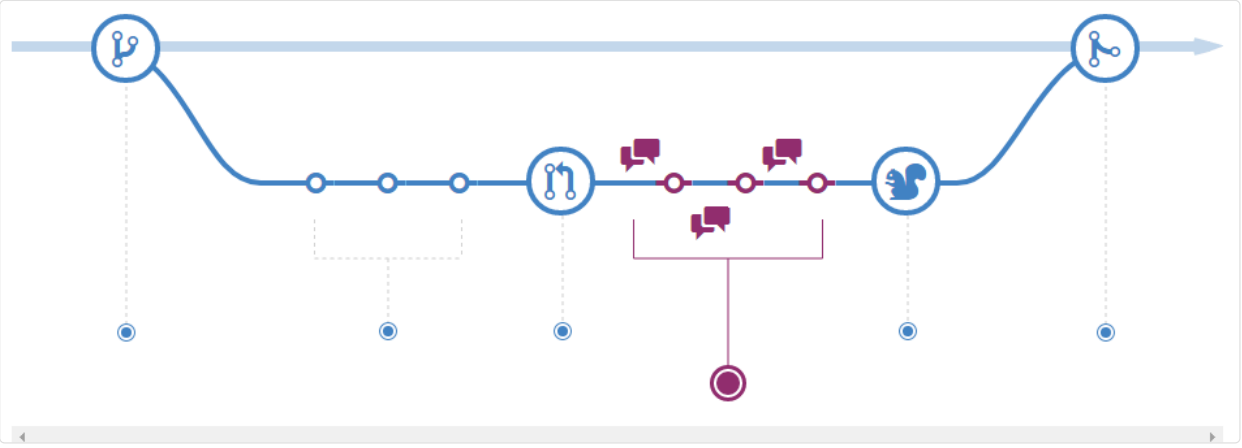
Open a Pull Request

Pull Requests initiate discussion about your commits. Because they're tightly integrated with the underlying Git repository, anyone can see exactly what changes would be merged if they accept your request.

You can open a Pull Request at any point during the development process: when you have little or no code but want to share some screenshots or general ideas, when you're stuck and need help or advice, or when you're ready for someone to review your work. By using GitHub's @mention system in your Pull Request message, you can ask for feedback from specific people or teams, whether they're down the hall or ten time zones away.

ProTip

Pull Requests are useful for contributing to open source projects and for managing changes to shared repositories. If you're using a Fork & Pull Model, Pull Requests provide a way to notify project maintainers about the changes you'd like them to consider. If you're using a Shared Repository Model, Pull Requests help start code review and conversation about proposed changes before they're merged into the master branch.



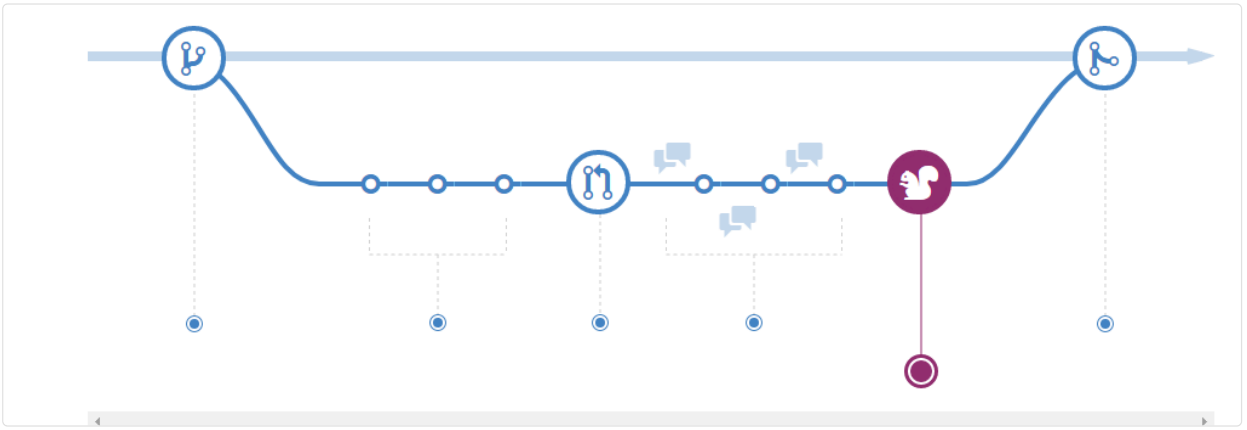
Discuss and review your code

Once a Pull Request has been opened, the person or team reviewing your changes may have questions or comments. Perhaps the coding style doesn't match project guidelines, the change is missing unit tests, or maybe everything looks great and props are in order. Pull Requests are designed to encourage and capture this type of conversation.

You can also continue to push to your branch in light of discussion and feedback about your commits. If someone comments that you forgot to do something or if there is a bug in the code, you can fix it in your branch and push up the change. GitHub will show your new commits and any additional feedback you may receive in the unified Pull Request view.

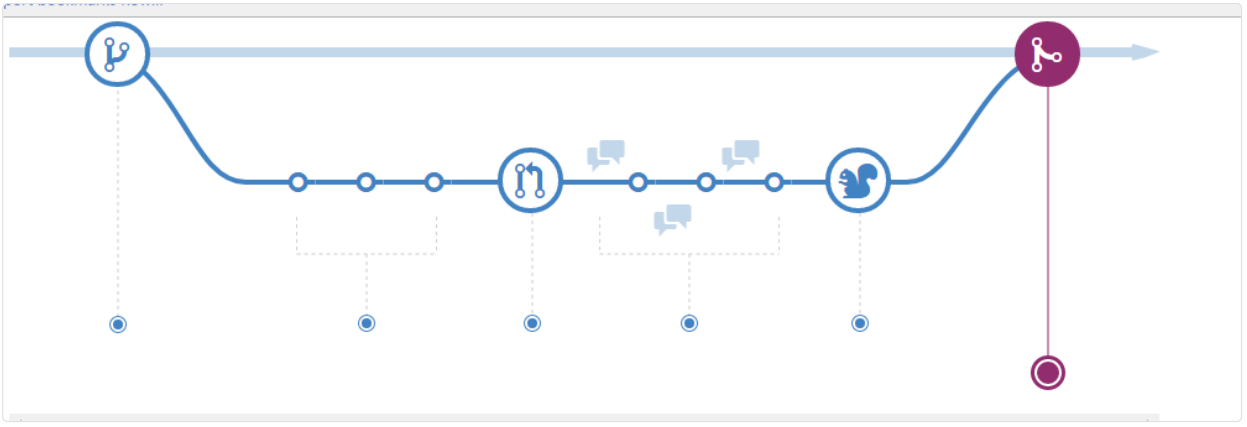
ProTip

Pull Request comments are written in Markdown, so you can embed images and emoji, use pre-formatted text blocks, and other lightweight formatting.



Deploy

Once your pull request has been reviewed and the branch passes your tests, you can deploy your changes to verify them in production. If your branch causes issues, you can roll it back by deploying the existing master into production.



Merge

Now that your changes have been verified in production, it is time to merge your code into the master branch.

Once merged, Pull Requests preserve a record of the historical changes to your code. Because they're searchable, they let anyone go back in time to understand why and how a decision was made.

ProTip

By incorporating certain keywords into the text of your Pull Request, you can associate issues with code. When your Pull Request is merged, the related issues are also closed. For example, entering the phrase Closes #32 would close issue number 32 in the repository. For more information, check out our help article.

Get started with GitHub

The long version of the same subject: installing Git and TortoiseGit, generating an SSH key, signing up, cloning, and the workflow used for the T3000 repository. It is the document a new developer reads first.

Table of Contents

Introduction to Git

Installing Tools

Installation of Git

Installation of Github for Windows

Procedire to Merge using Git Bash

Compilation on Microsoft Visual Studio 2010

Git commands summary

Cloning a repo from github.com using Git Bash Shell

TortoiseGit

Installation of msysgit, the windows version of Git

Installation of TortoiseGit

Sign up for a github account

Setting up SSH key

Working with repository

TortoiseGit andGithub workflow for T3000_Building_Automation_System project

Introduction to Git

While developing software where developers are spread across the globe, issues of version management become critical. What if two developers simultaneously work on the same module and try to upload their changes back? Situations like these are not uncommon in real-world. To solve problems such as these there exist Version Control Systems.

What is Git?

Git is a distributed version control system (dvcs) written in the programming language C. A distributed version control system keeps track of software revisions and allows many developers to work on a given project without necessarily being connected to a common network.

There is no central server which stores the data. Every local copy contains full history of source code.

Git keeps track of all versions. Therefore, you can revert to any point in your source code history.

The logo of git looks like below:



Git is a distributed revision control and source code management (SCM) system with an emphasis on speed. Git was initially designed and developed by Linus Torvalds for Linux kernel development; it has since been adopted by many other projects. Every Git working directory is a full-fledged repository with complete history and full revision tracking capabilities, not dependent on network access or a central server. Git is free software distributed under the terms of the GNU General Public License version 2.

Local and Remote Repositories

In a distributed version control system everyone has a complete copy of the source code (including the complete history of the source code) and can perform version control operations against this local copy. The use of a dvcs does not require a central code repository.

Git commits file changes to your local repository and you can synchronize your repository with other (remote) repositories. Git allows you to clone repositories, e.g. create an exact copy of a repository including the complete history of the source code. Owners of repositories can synchronize

changes via push (transferring changes to a remote repository) or pull (getting changes from a remote repository).

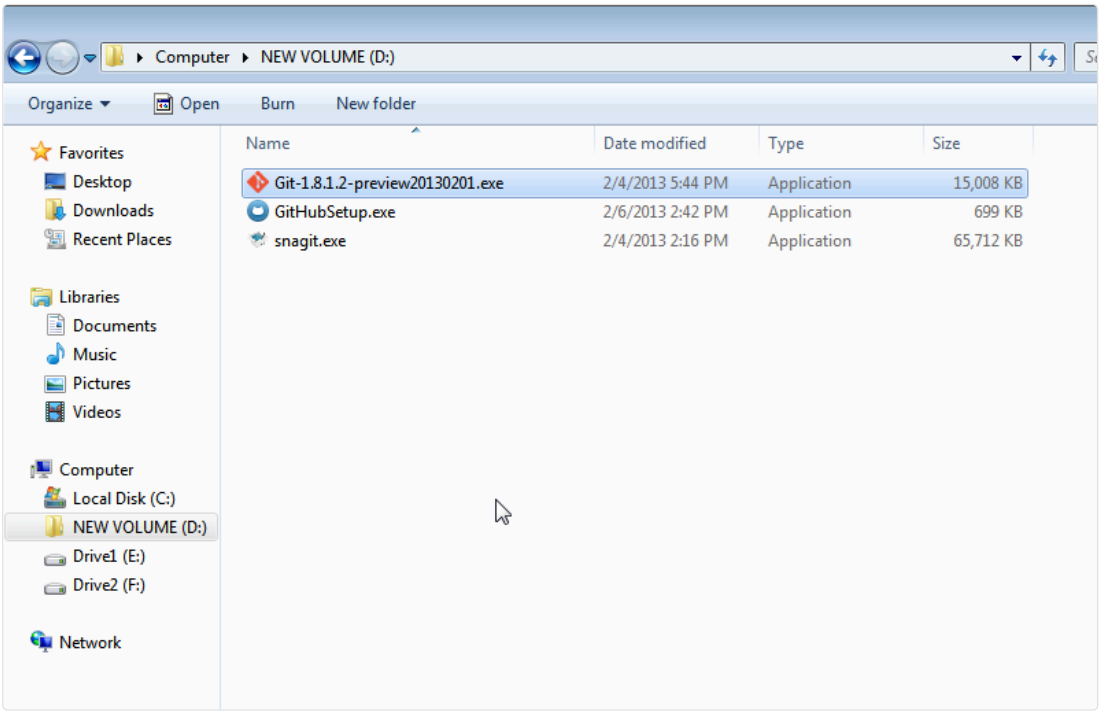
Refer to http://git-scm.com/book for detailed help on understanding git.

Installing Tools

Installation of git

Download git installer.

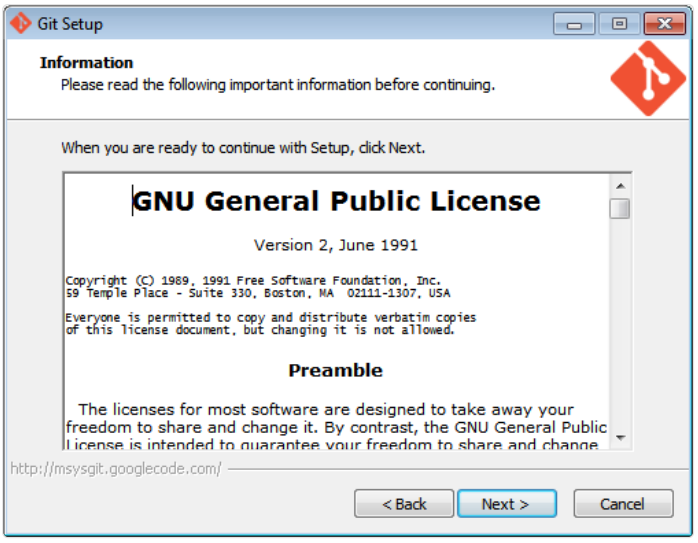
Double click on the setup file.



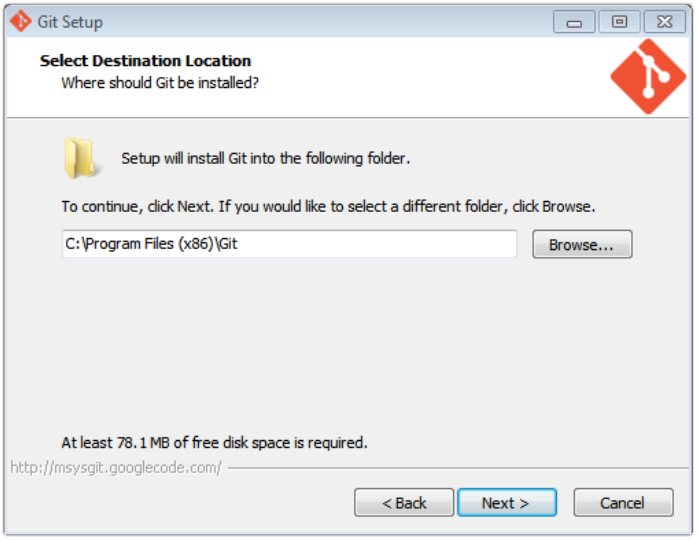
Following dialog box opens. Click Next to continue.



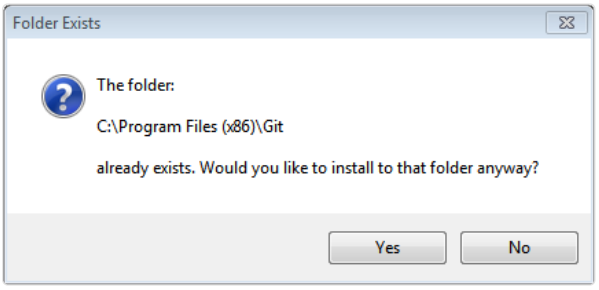
Read GNU General Public License. Click Next.



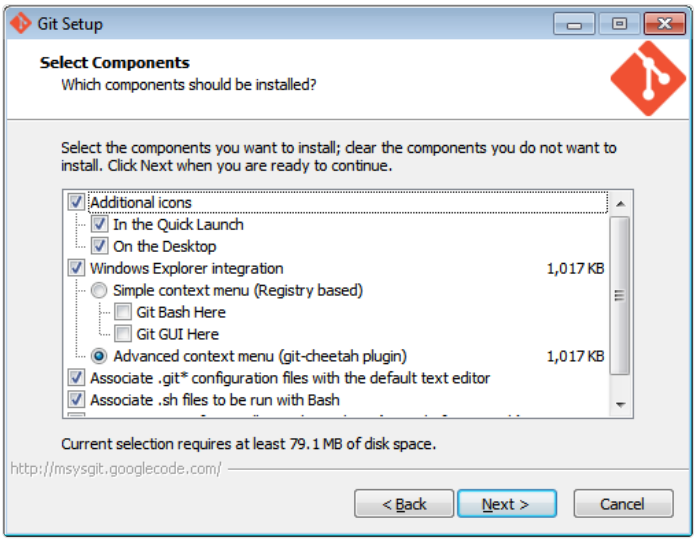
Select destination location. This is the directory where you want to install git. Normally, default directory can be kept as it is. If you wish, you can change the directory by clicking on Browse button. Here, we will install git in the default directory. (C:\Program Files (x86)\Git). Click Next.



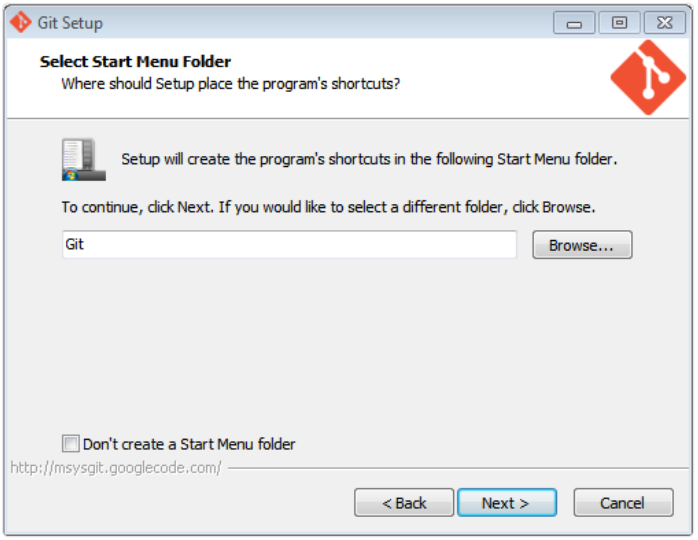
If following dialog box appears, click Yes.



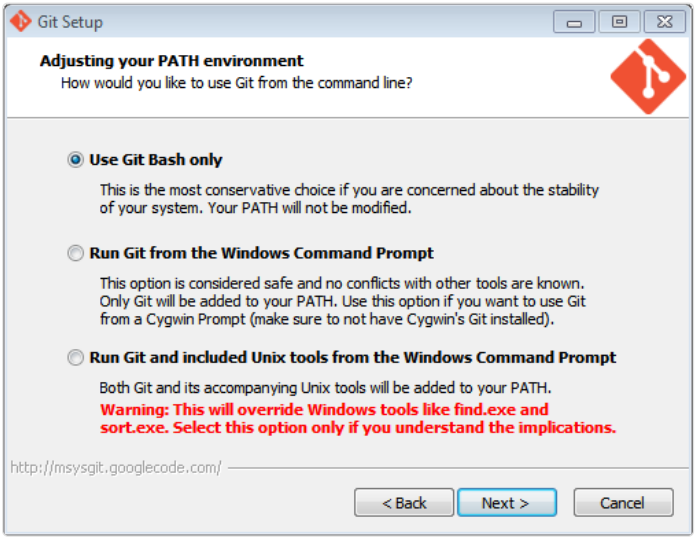
Following screen appears. Leave all the default options as is and click Next.



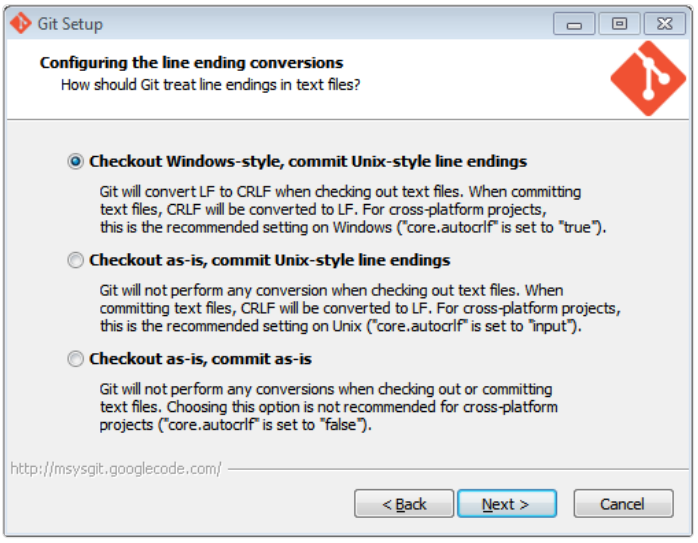
Following screen appears. Click Next.



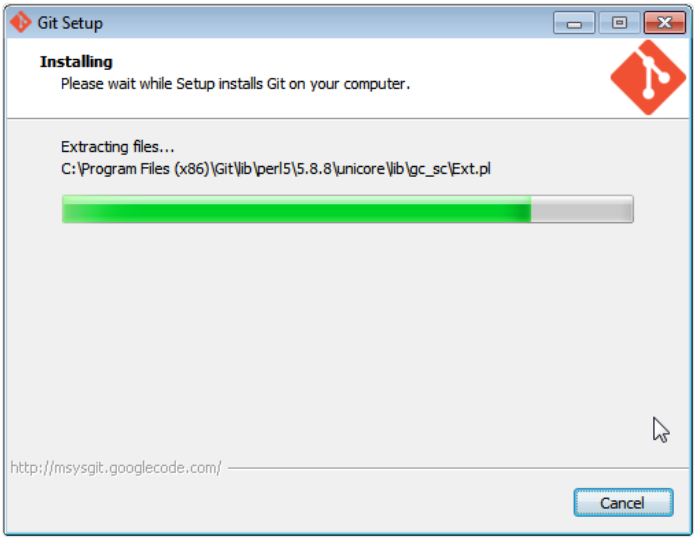
Following screen appears. Click Next to continue.



Following screen appears. Click Next to continue.



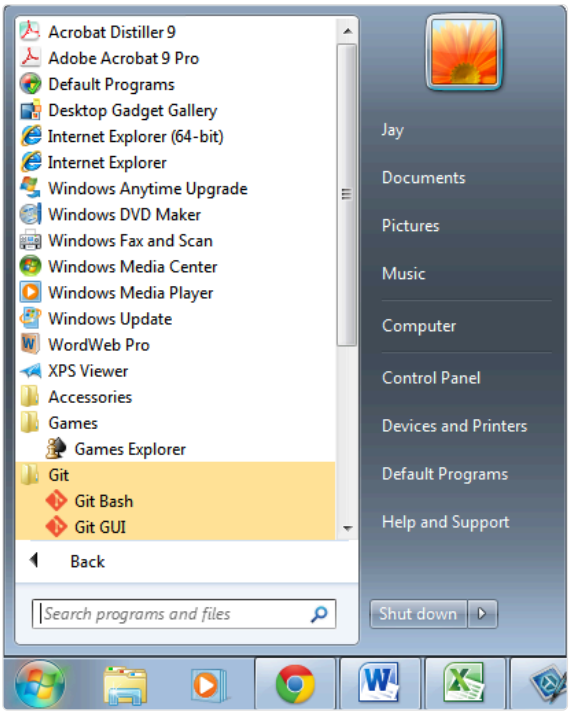
Setup will progress.



Click Finish to complete the setup.



Check installation in Start Menu.



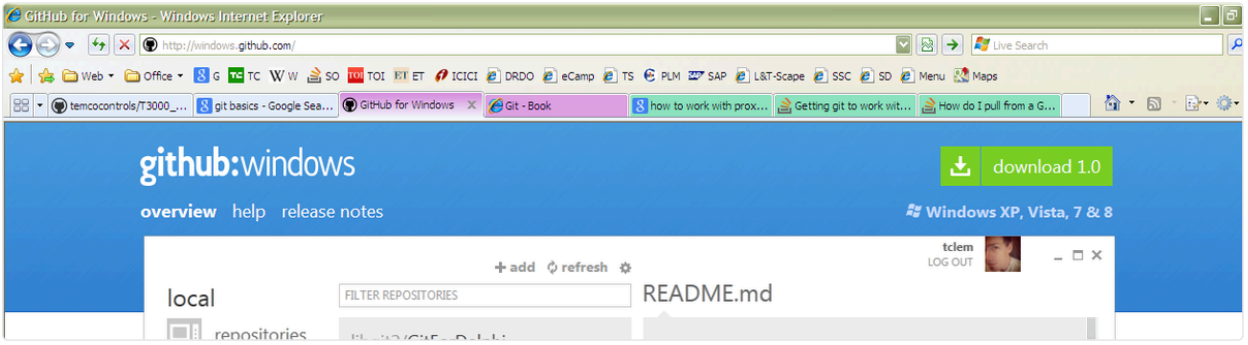
As can be seen, there are two sub menus under Git menu.

Git Bash

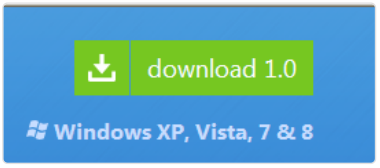
Git GUI

Installation of github for windows

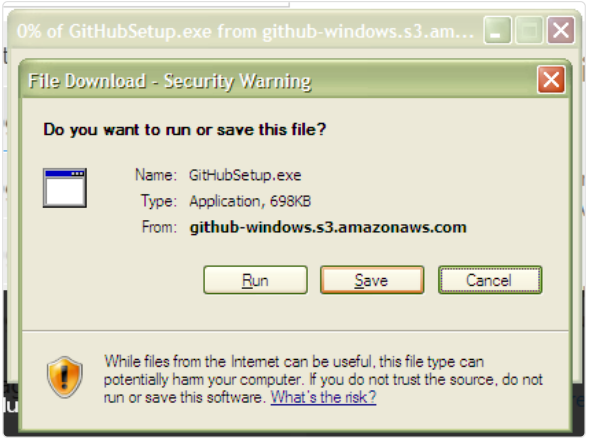
Download github for windows from http://windows.github.com/



Click on the link given for download.

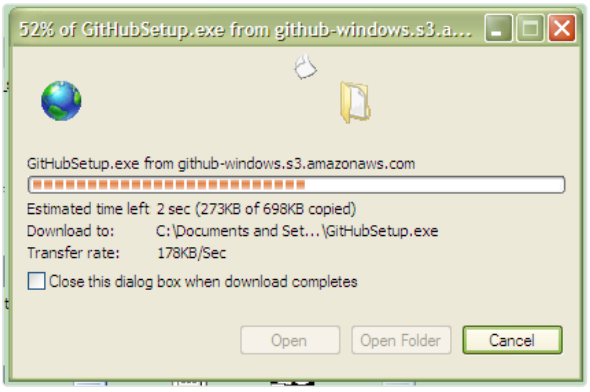


If any confirmation is asked by OS, click Save.



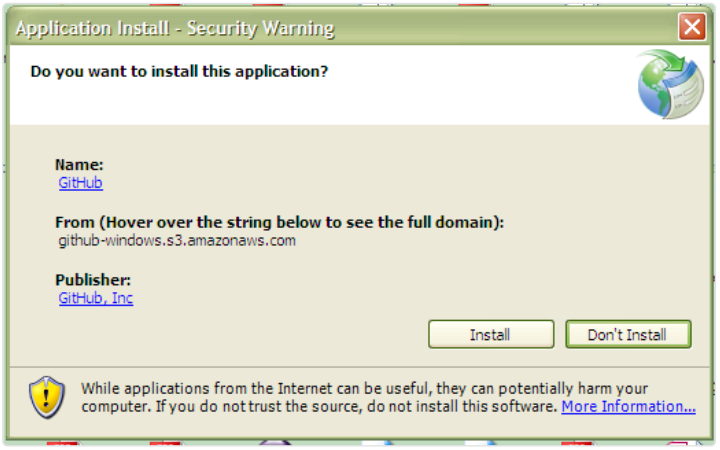
Save the file on your computer at a chosen location.

File download progress bar would be shown.

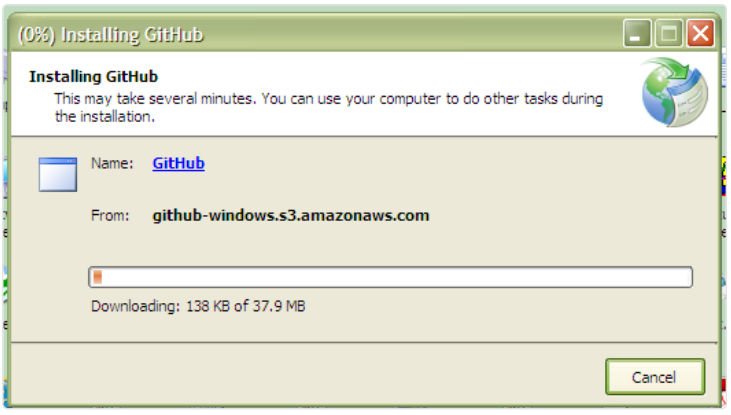


Once the download is complete, double-click on the same to install github for windows on your PC.

A dialog box for Security warning may appear.

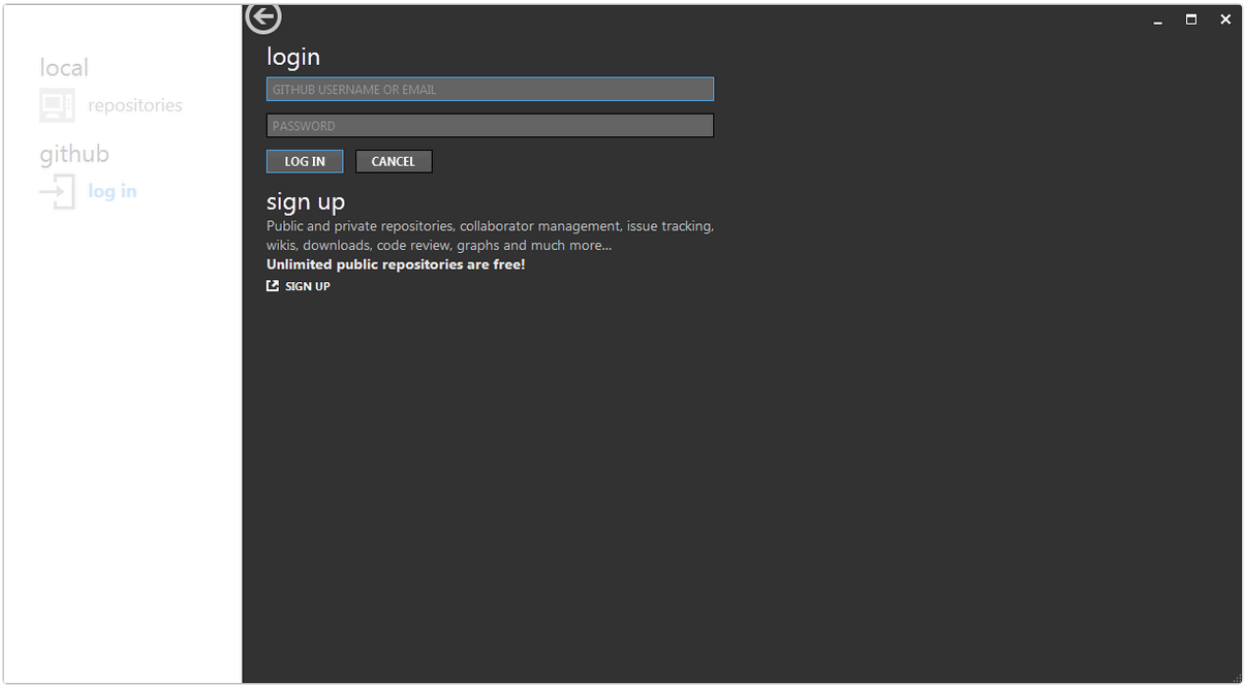


Github for windows starts installing.

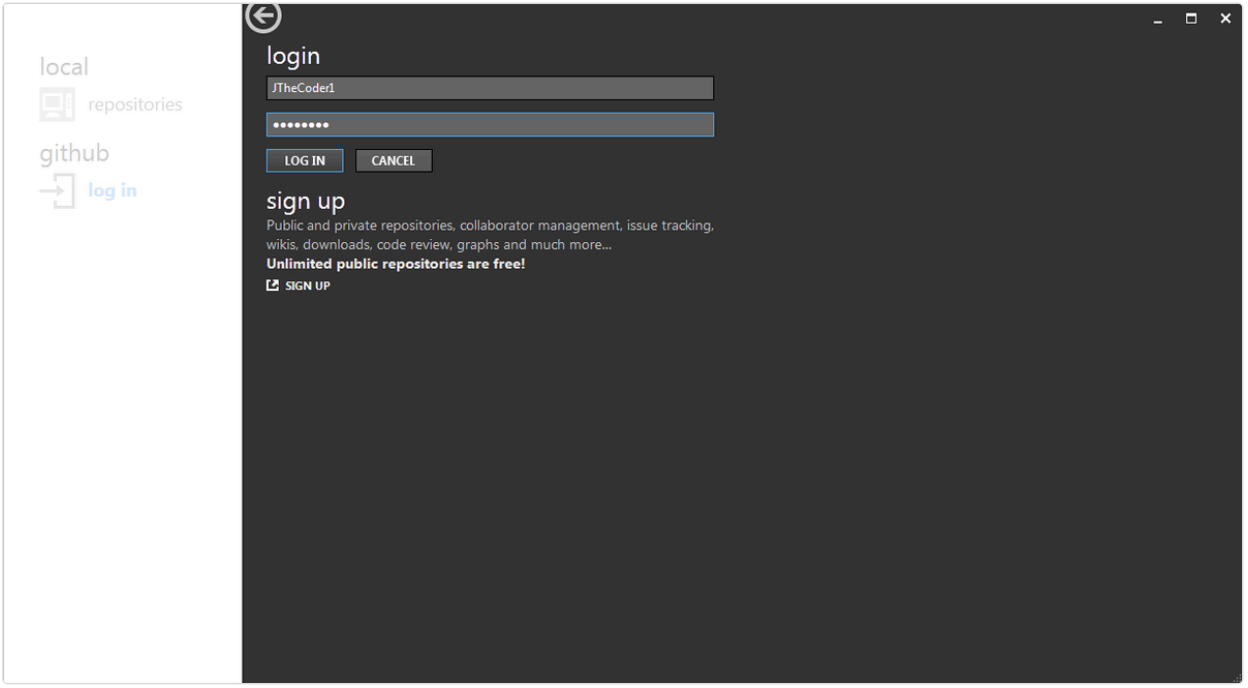


Istallation eventually gets over. Application starts.

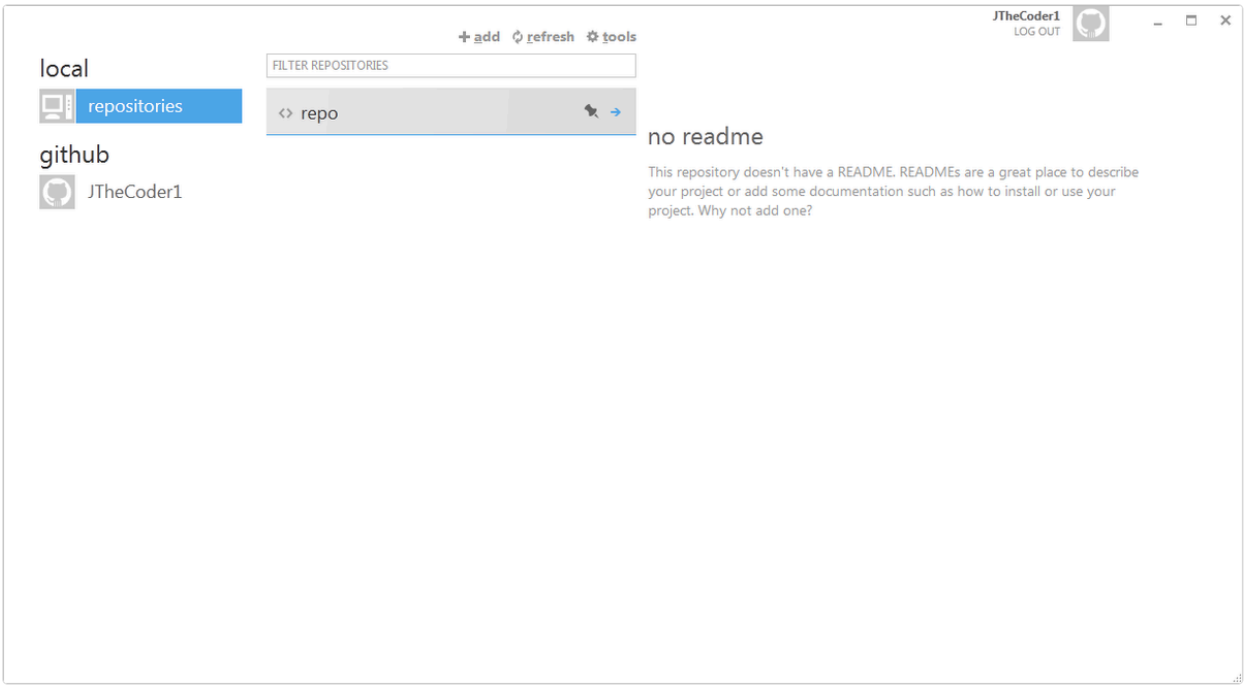
The first screen in the application Github for Windows is as follows.



Enter username and password in the given fields. You have to Sign Up with github to have these information.

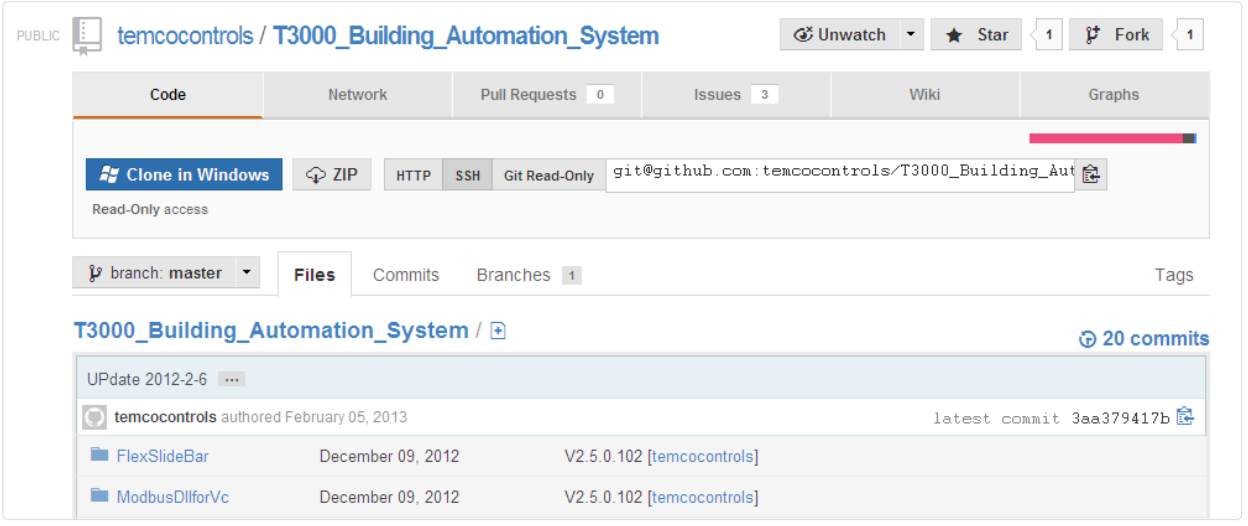


Application screen looks as follows.

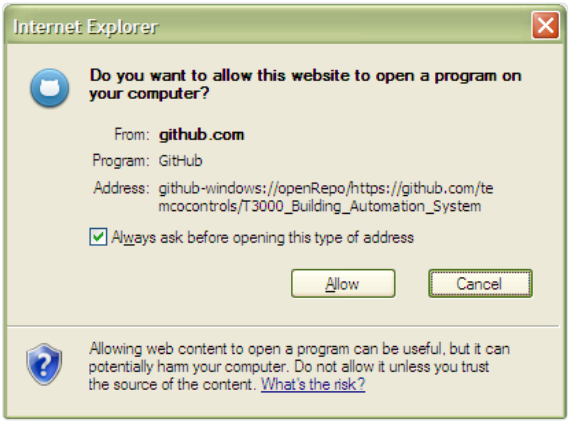


Once the application is installed on the PC, go to github website. Open the project you want to work on. For example, in my case, I am opening repository of T3000 Building Automation System.

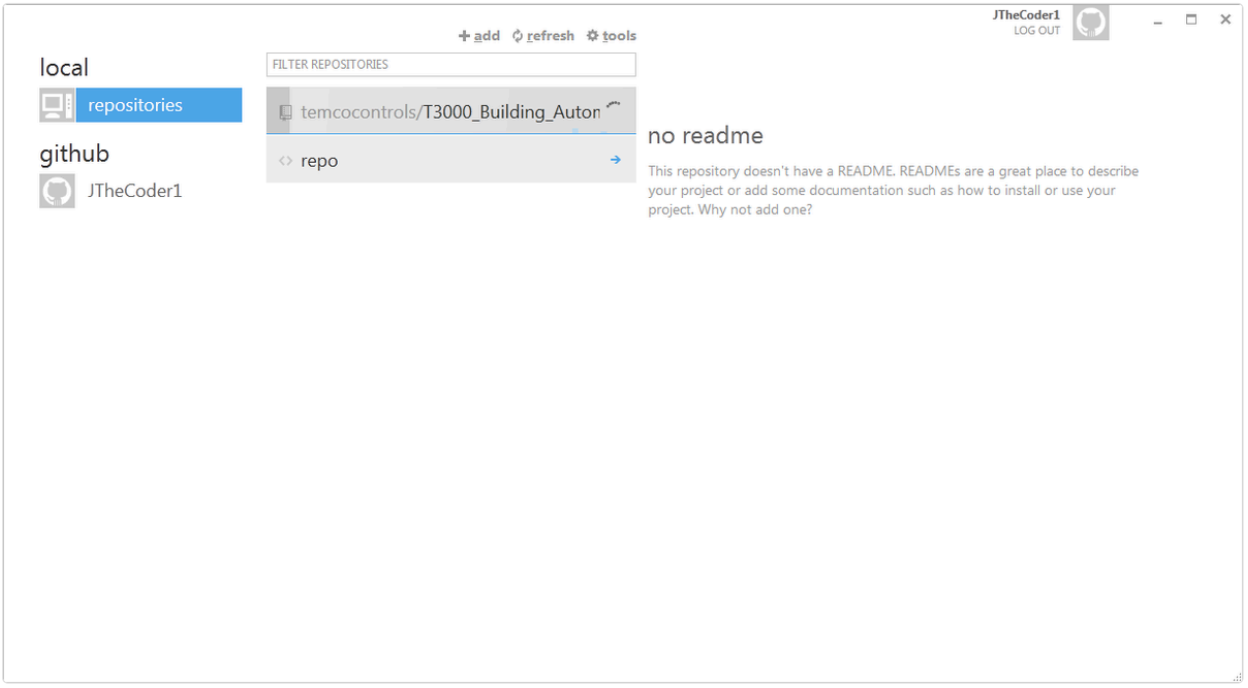
In order to clone the repository from github to local PC, click on Clone in Windows button as shown below.



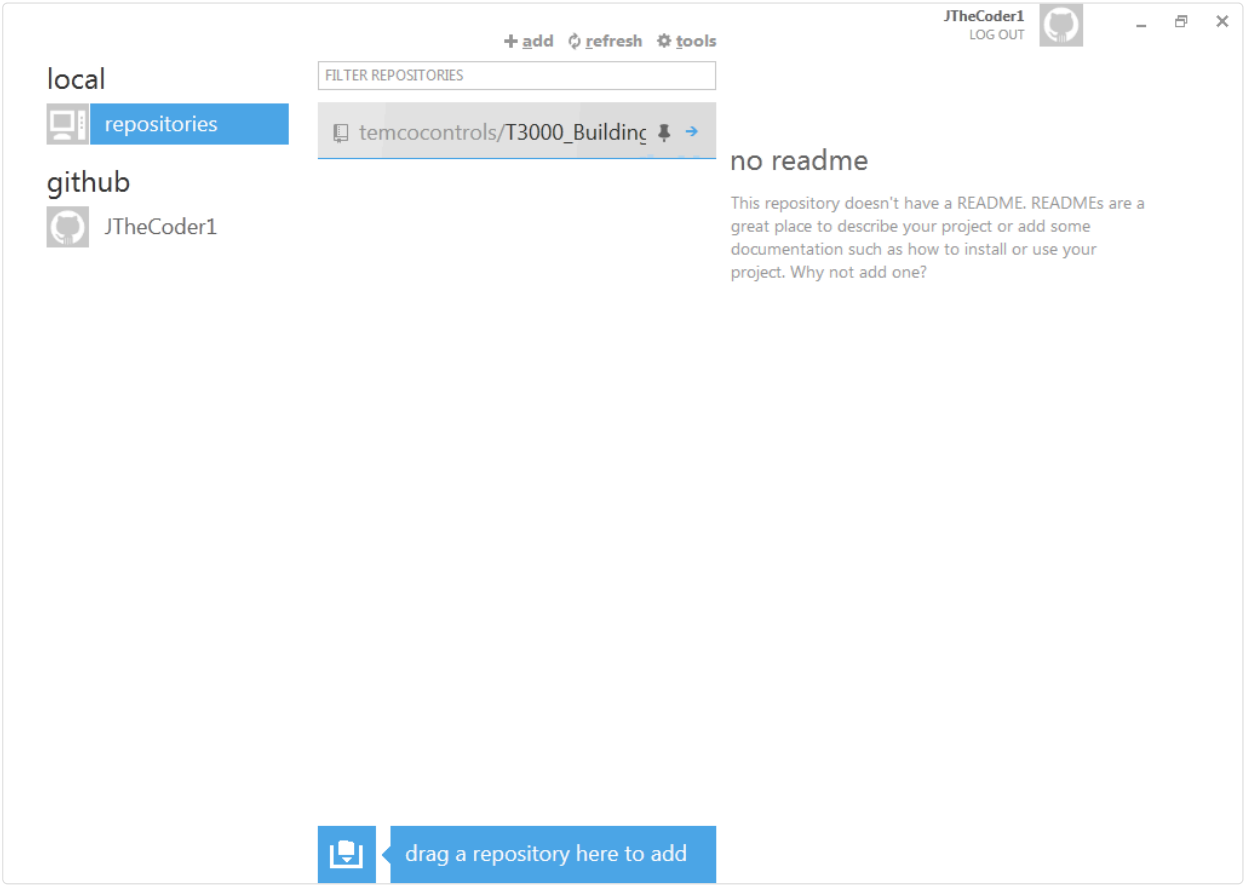
You may get following warning message. Click Allow.



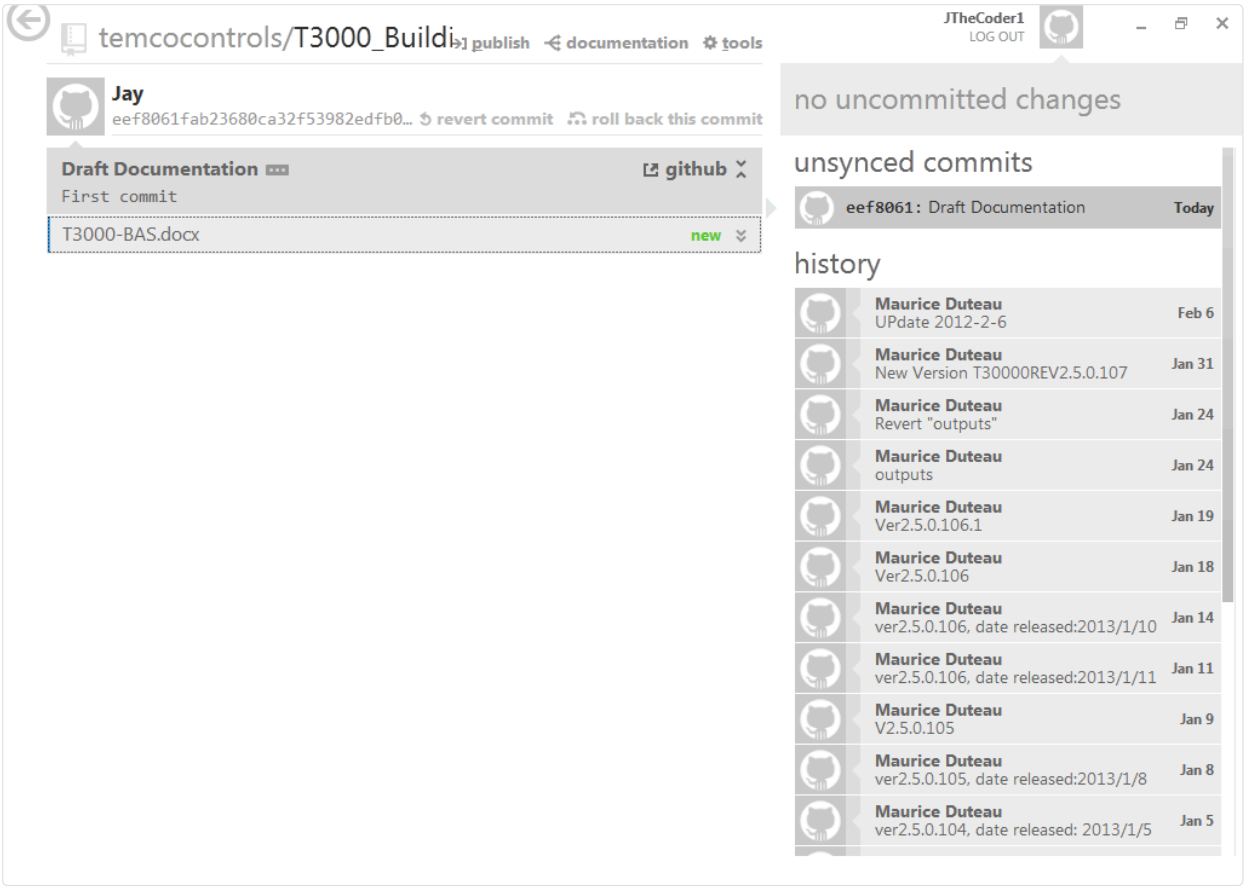
The repository starts getting cloned on local PC. The application shows the progress of the same. Be patient. The repository, depending on its size, takes few minutes to download.



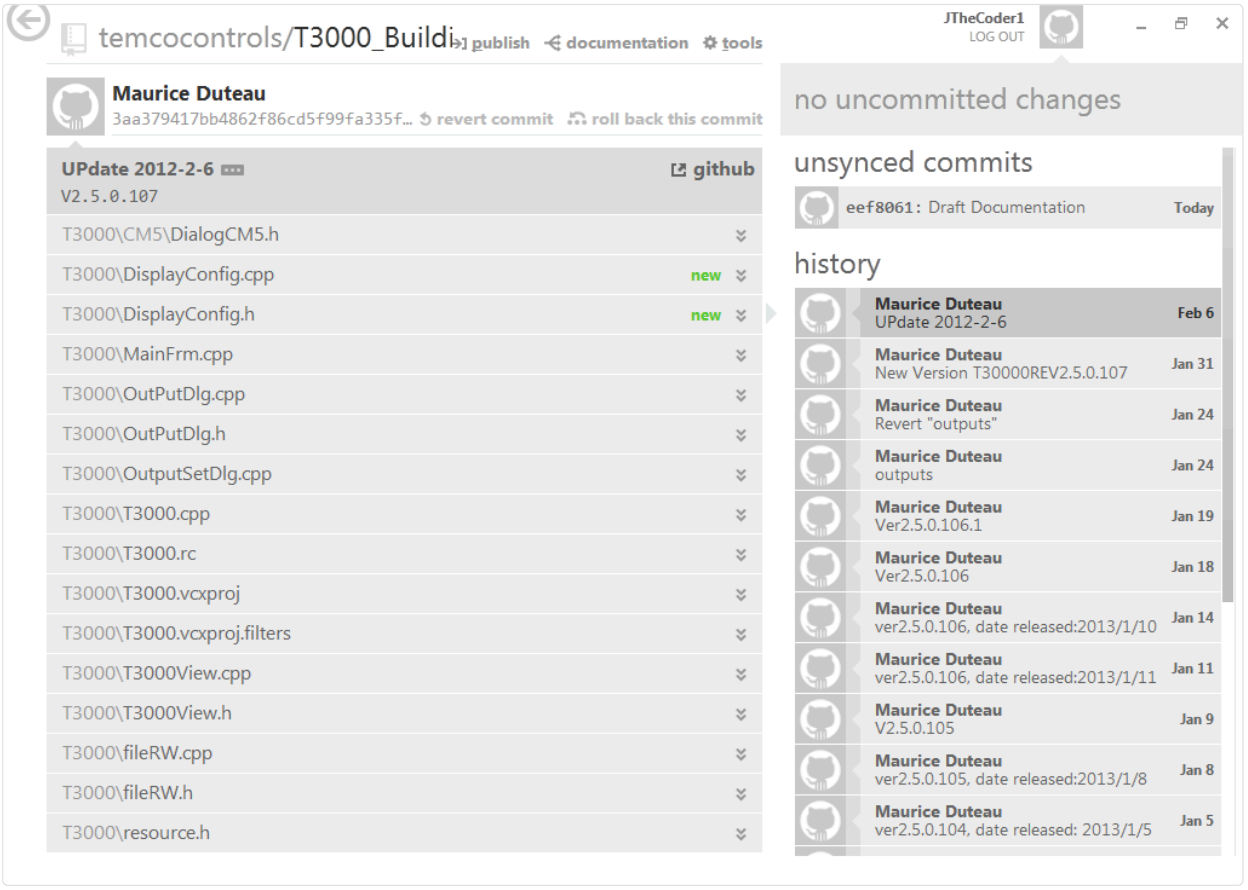
Once cloning is over, the repositories look like this.



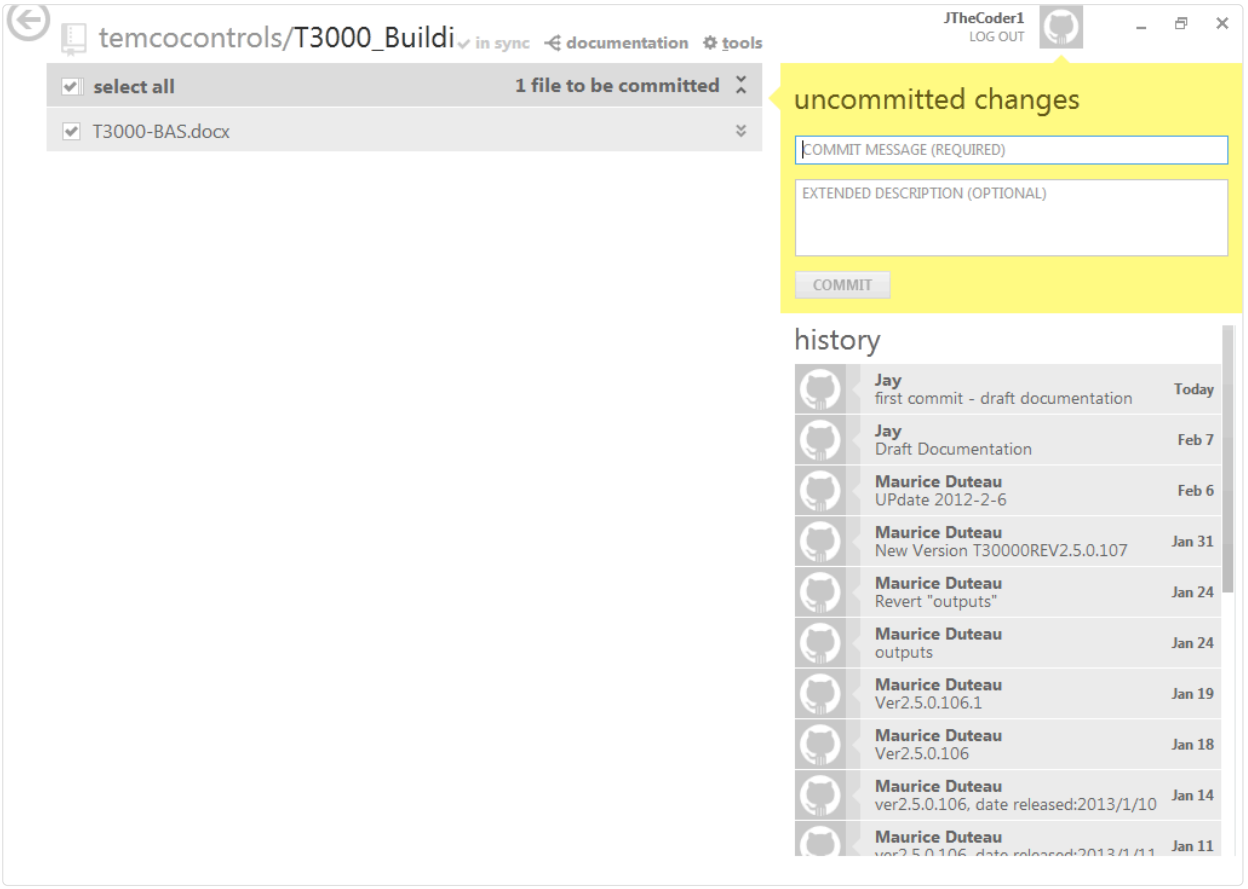
If you click on the small blue arrow on the repository, you would be taken to the following screen.



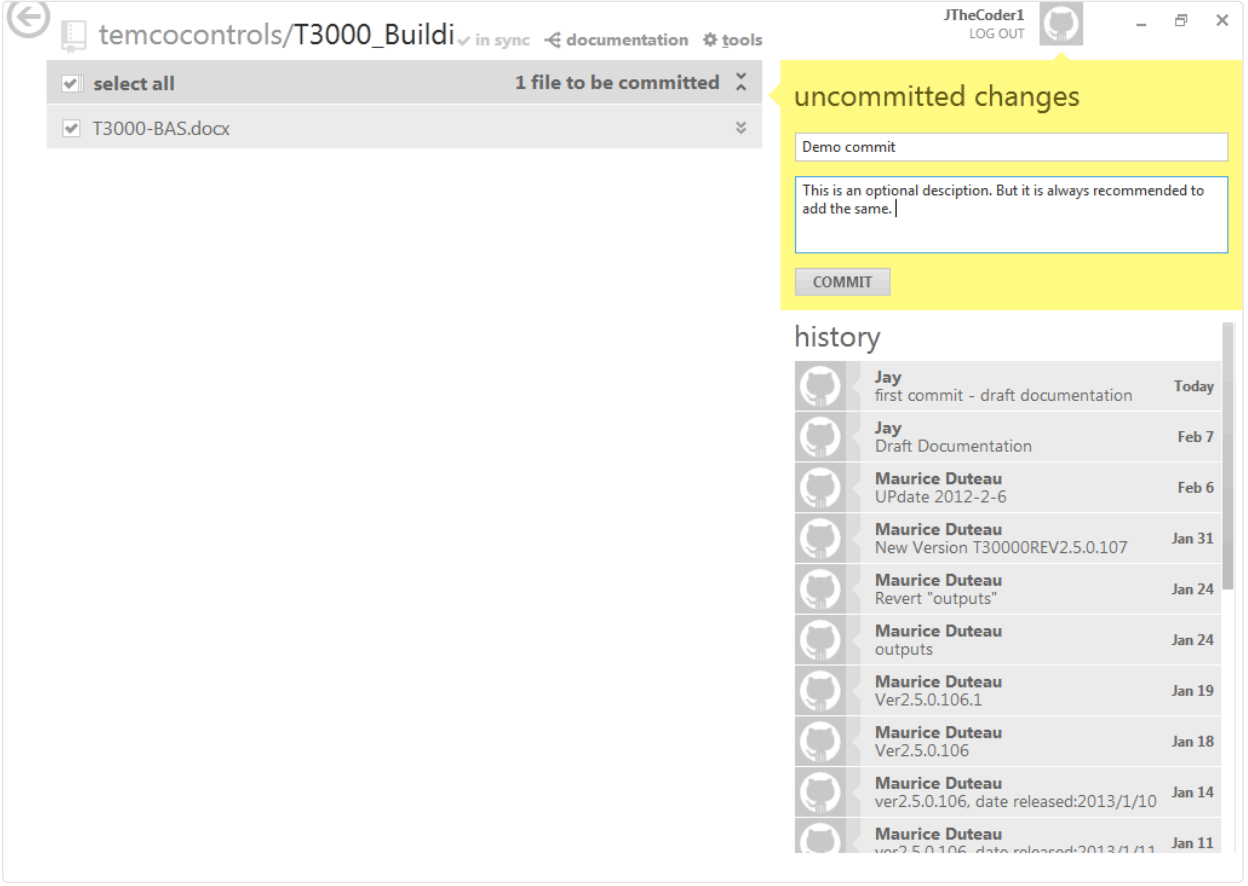
If you click on any of the links on history pane on right side, you get all the files that are there in the repository for that version.



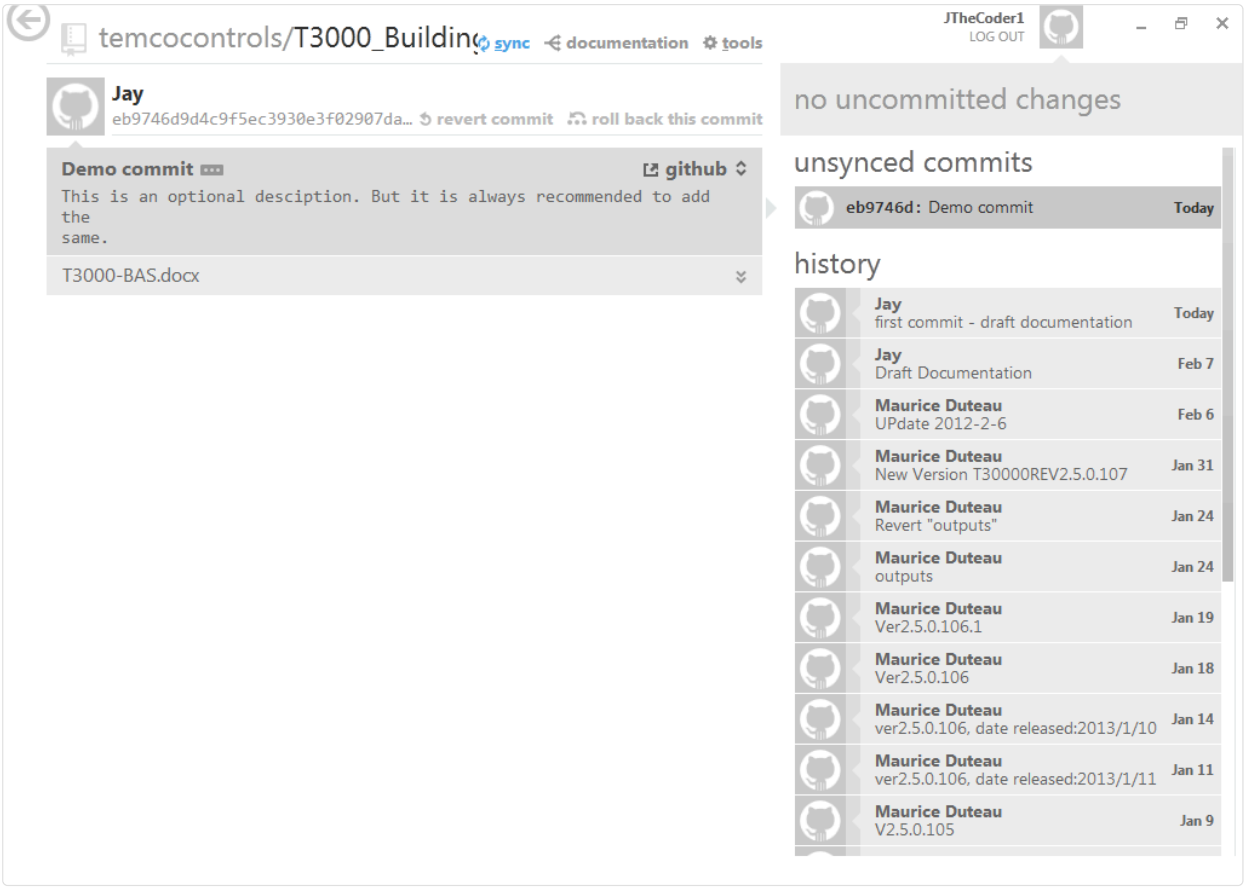
Once the file is modified in the repository, it appears on the screen as follows. I have the current file, which I am typing write now as the one which is to be committed to the repository. The same is shown on the screen.



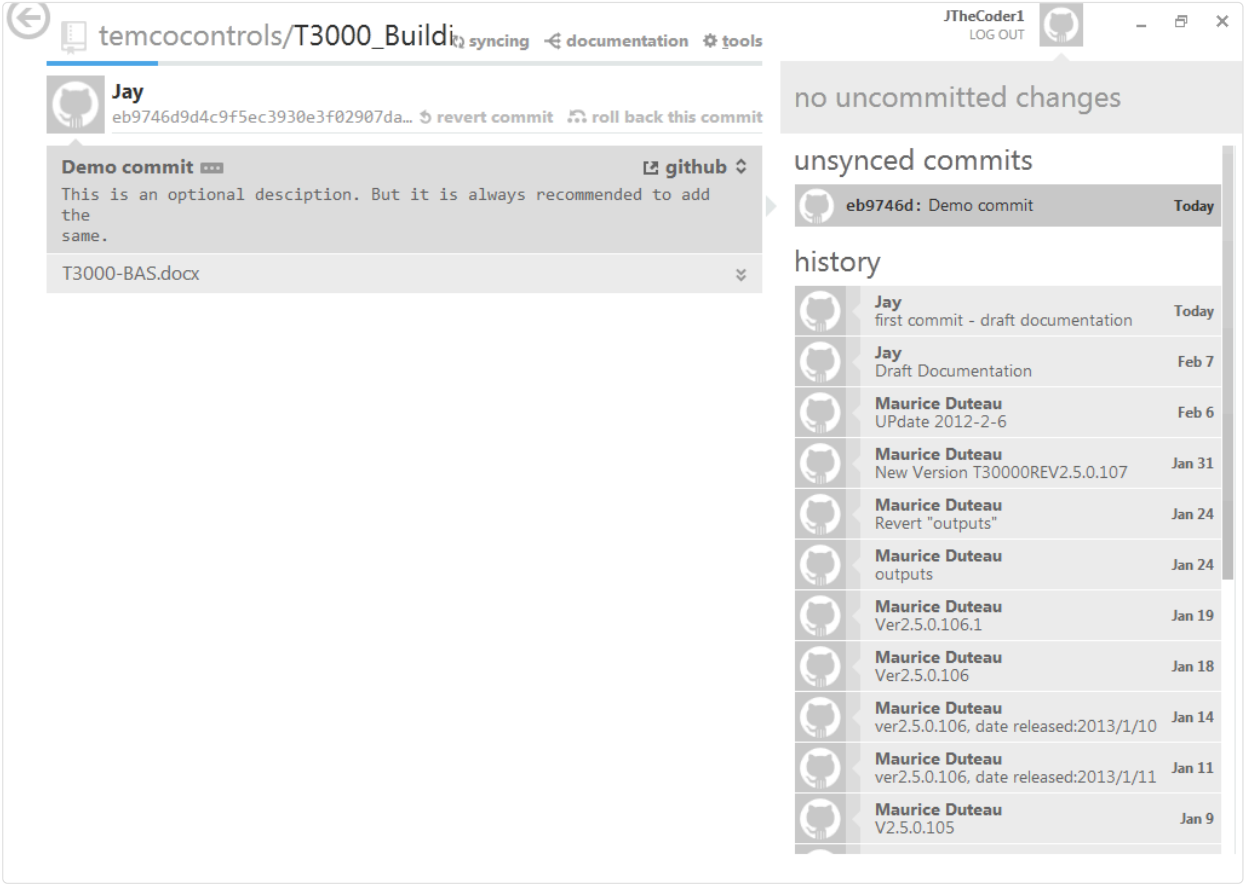
You can commit the change in the repository by adding a commit message and optionally an extended description. And then click on the COMMIT button given in the yellow area on right.



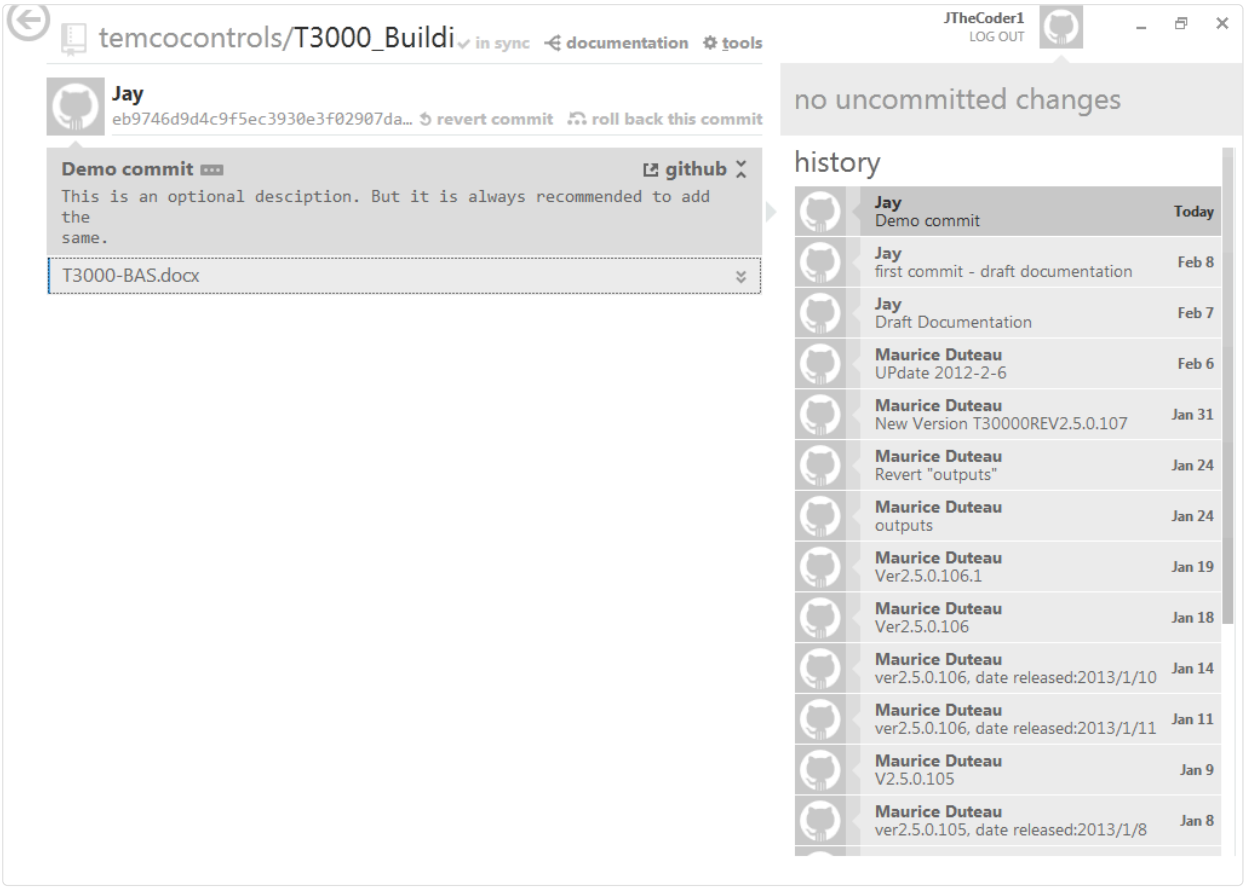
Once the COMMIT button is clicked, changes are committed to the repository. But they are not in sync with the repository on the github server. To push the changes to the github server, click on sync button as shown in blue in the following figure.



Following figure shows sync in progress.



Once in sync, the same is shown as follows.

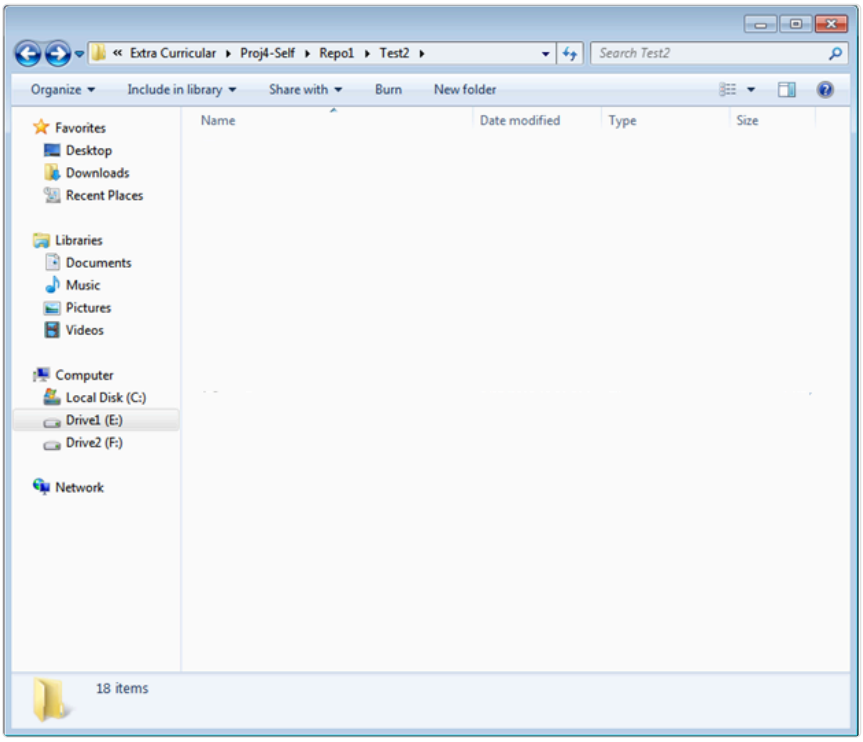


You can work with github and repository in this manner.

Procedure to Merge

First decide a folder on your PC where you want to create local copy of your git repository and git working directory.

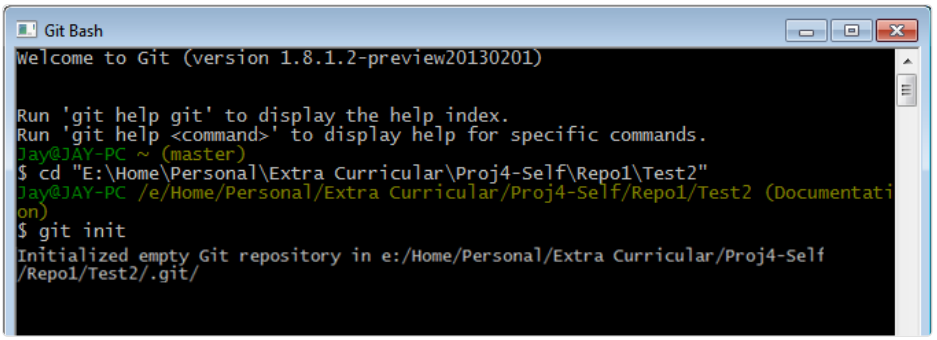
On my PC, this folder is “E:\Home\Personal\Extra Curricular\Proj4-Self\Repo1\Test2”.



Open Git Bash and type following commands.

$ cd <”path of folder”>

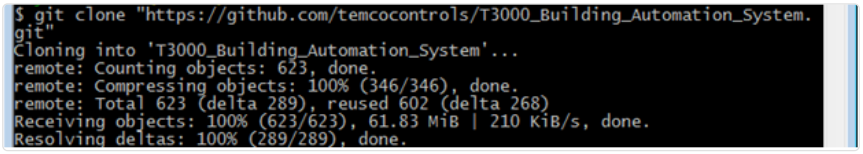
$ git init



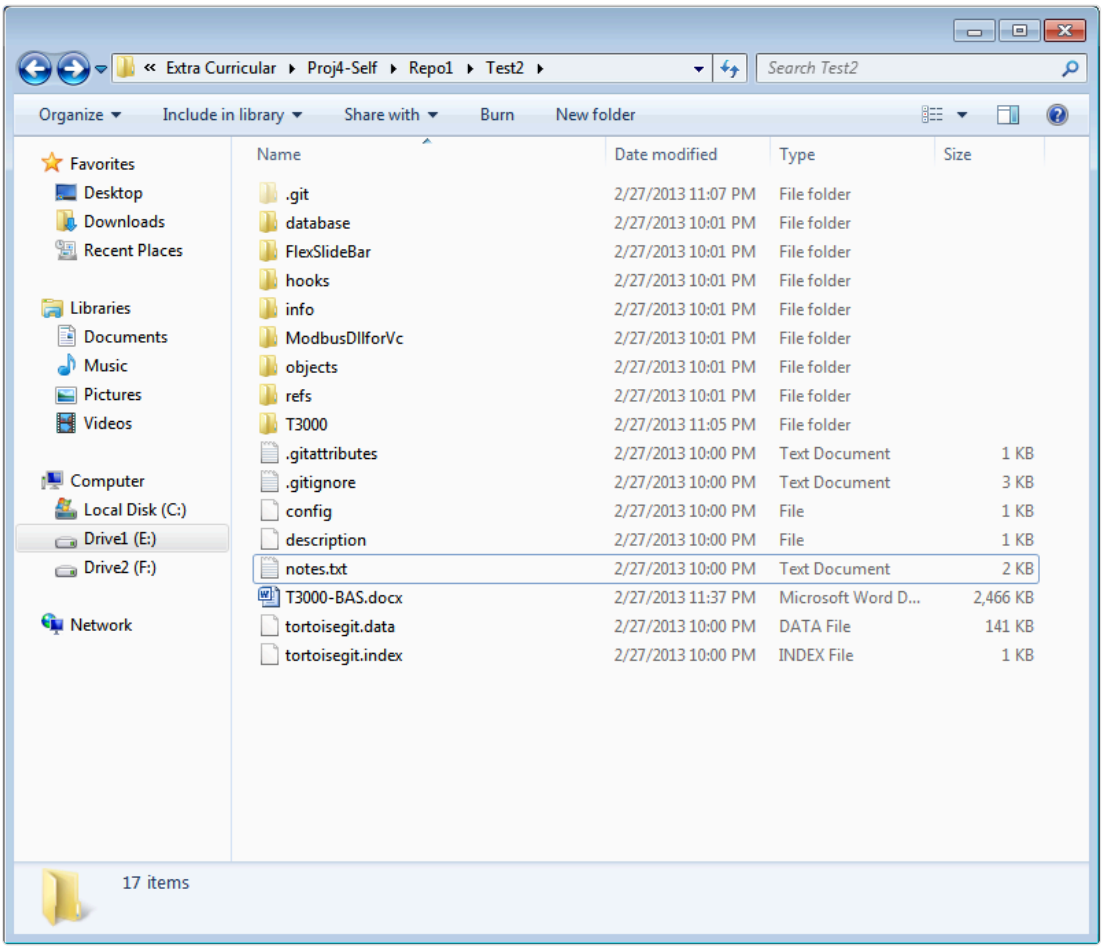
Next step is to clone repository of T3000_Building_Automation_System from github server. To achieve this, type following command.

$ git clone "https://github.com/temcocontrols/T3000_Building_Automation_System.git"

Upon executing this command, a message similar to the one shown in screen shot below appears. It might take 5 to 15 minutes depending on internet speed to clone complete cloning of repository.



Once the cloning is complete, folder on PC looks like as shown below. There would be a hidden sub-folder named ".git" which is local git repository. Outside of it Is working directory.



Now, use following git commands to do tasks as desired. I am documenting normal steps that would be followed by development team (Alex) and documentation team (Jay).

Both documentation and development teams will have to clone git repository from github.com.

Run following commands to determine the branches present in the repository. Remember that once cloning is done, all these commands would work on local repository and not on repository on github server.

$ git branch

This command lists all the branches present in the local repository. In the list which appears following this command, there would be one branch with * marked. This is the current branch of the repository.

For basic concepts of git branch read following material:

http://git-scm.com/book/en/Git-Branching-What-a-Branch-Is

For complete tutorial on branching and merging refer to chapter 3 of Git Book.

http://git-scm.com/book/en/Git-Branching

Let's say there are two branches:

Master and Documentation.

Documentation team shall move to Documentation branch by executing following command.

$ git checkout Documentation

This command loads all files and folders of repository to working directory.

Documentation team shall add its comments to all the source code files. Once, a particular level of commenting is reached, changes shall be committed to git repository by executing following commands.

$ git add "."

$ git commit -m "commit message"

'git add' command adds files to the staging area and marks them for the commit (To understand this concept, read git theory from Git Book: http://git-scm.com/book/en/Getting-Started-Git-Basics). 'git commit' command actually commits staged files to repository.

Once the Documentation branch is updated, documentation team shall push it to github for the world to see it.

[I will update this procedure soon. I am still having some doubts on this, so trying to clarify from various sources of mine.]

Development team can fetch the contents from github server using following command.

$ git fetch

This command fetches all the new changes that have been pushed by documentation team to the local repository of development team.

Then development team may switch between master and documentation branches and go through the changes – that is, the commenting done by documentation team. It may decide to merge the changes of documentation branch to master branch using following command.

$ git checkout master

$ git merge documentation

These commands merge documentation to master branch. Now, the changes done by documentation team should have been reflected in master branch.

Once, this is done, development team shall push the changes to github and inform documentation team.

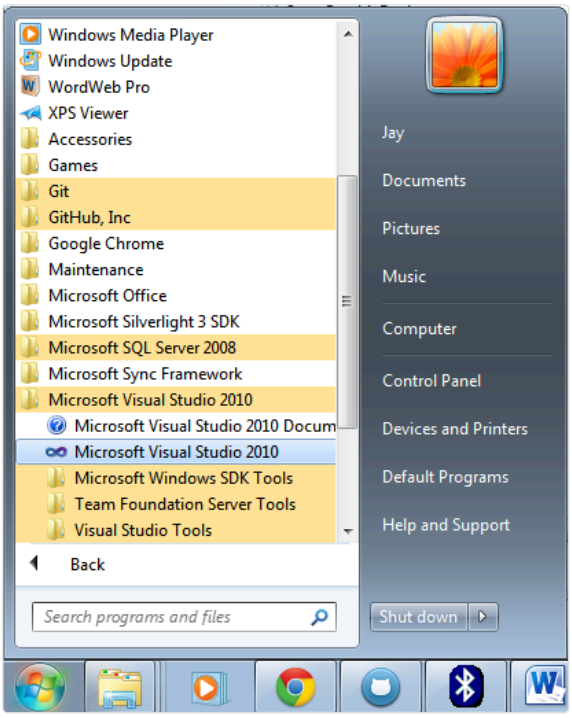
This way, both teams can collaborate.

[This procedure may change in near future as roughness would be smoothened out. Till then keeping this statement as it is.]

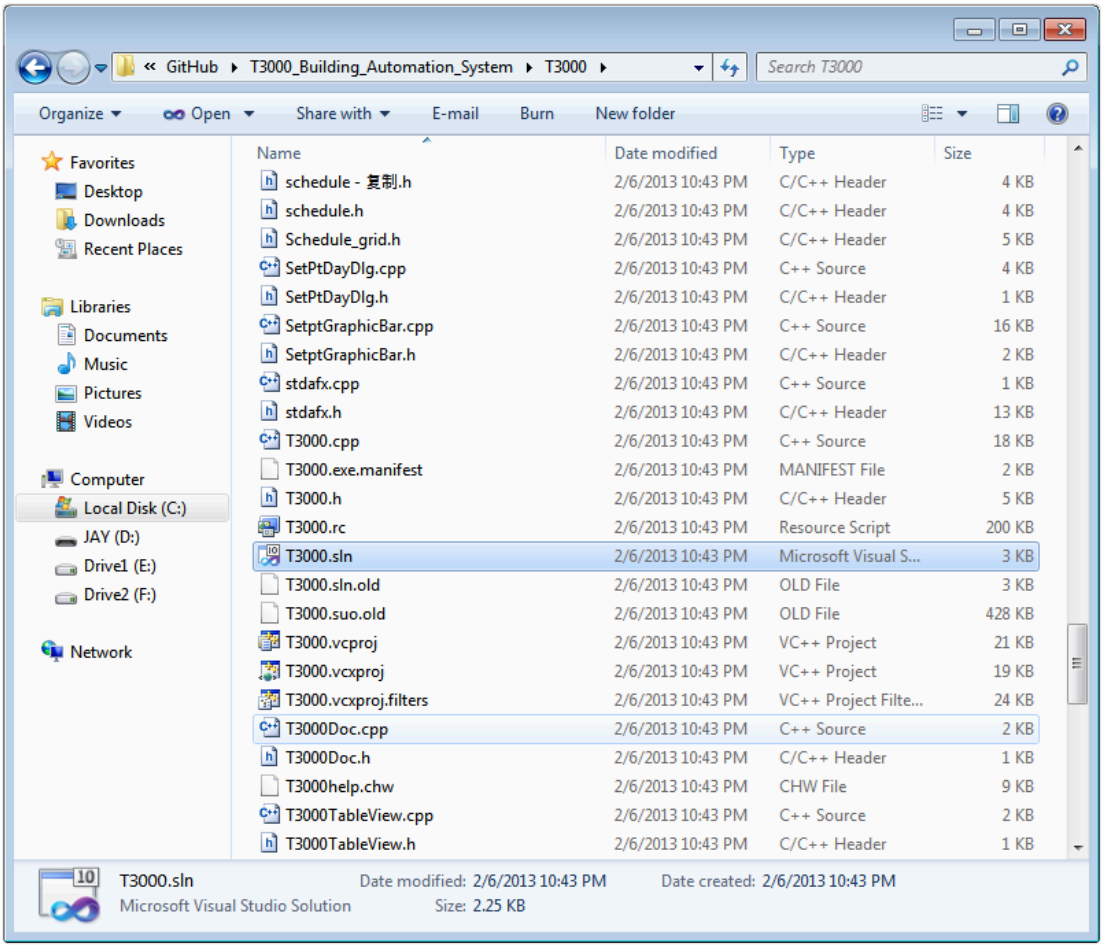
Compilation on Microsoft Visual Studio 2010

Install MS Visual Studio 2010 on your PC.

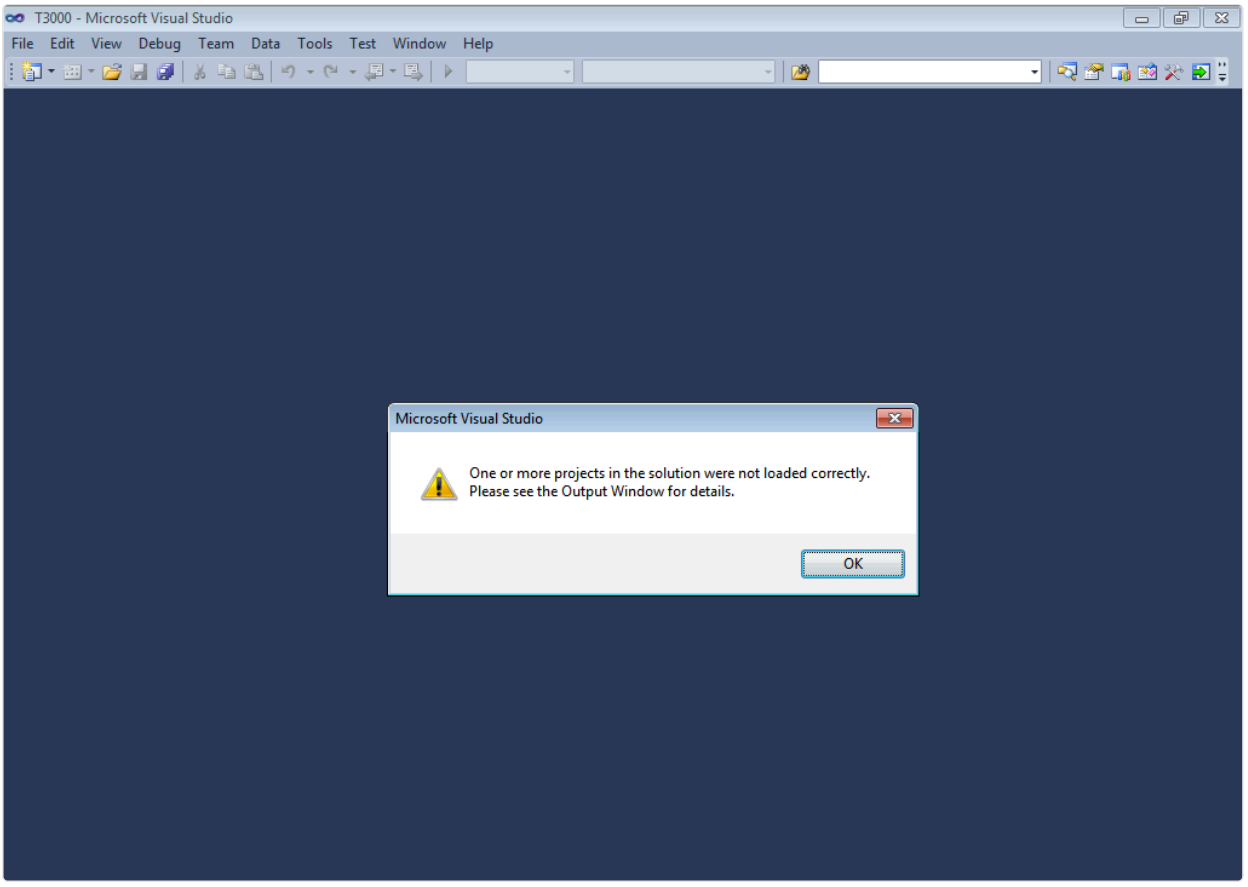
Start MS Visual Studio from the main start menu.



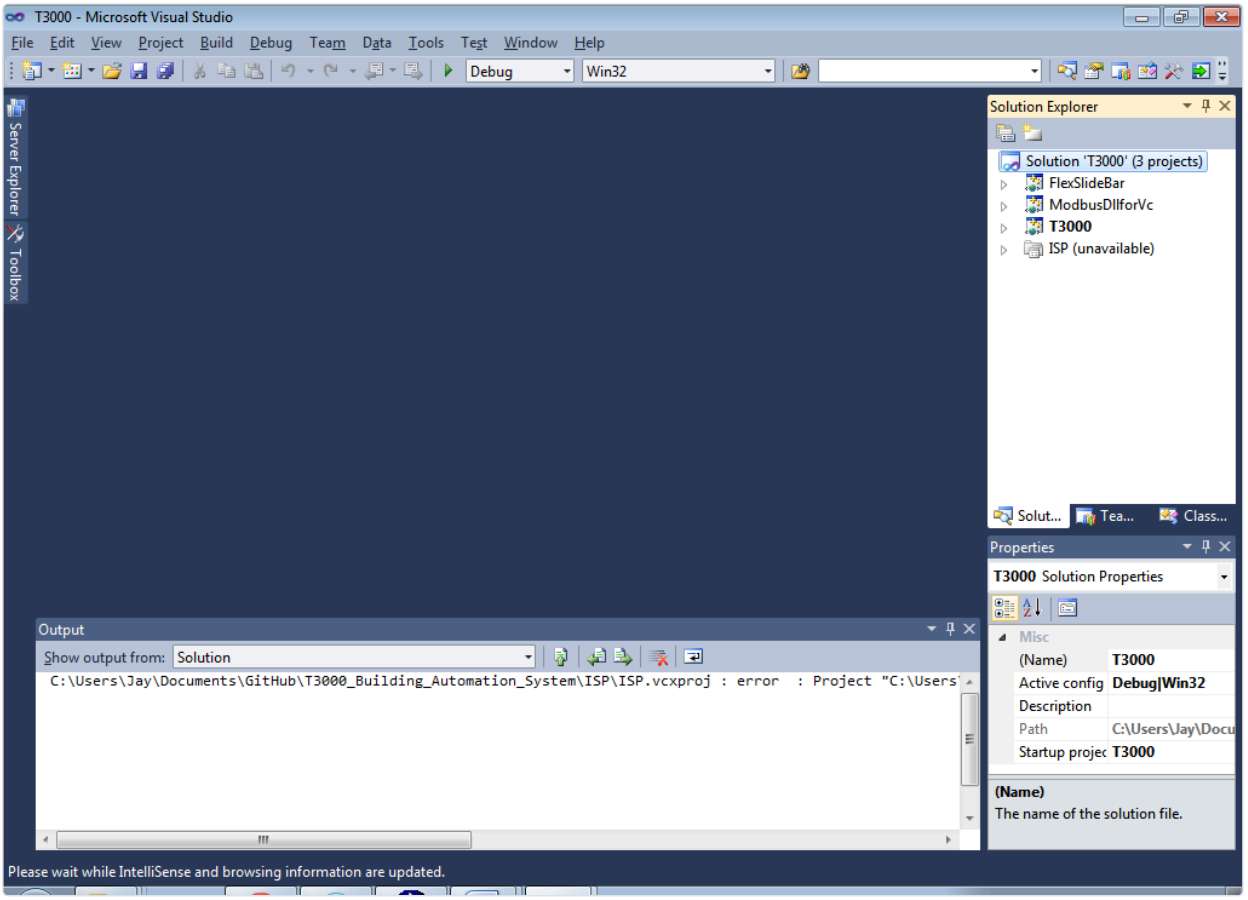
Go to the repository directory. Go to T3000 folder. Click on T3000.sln.



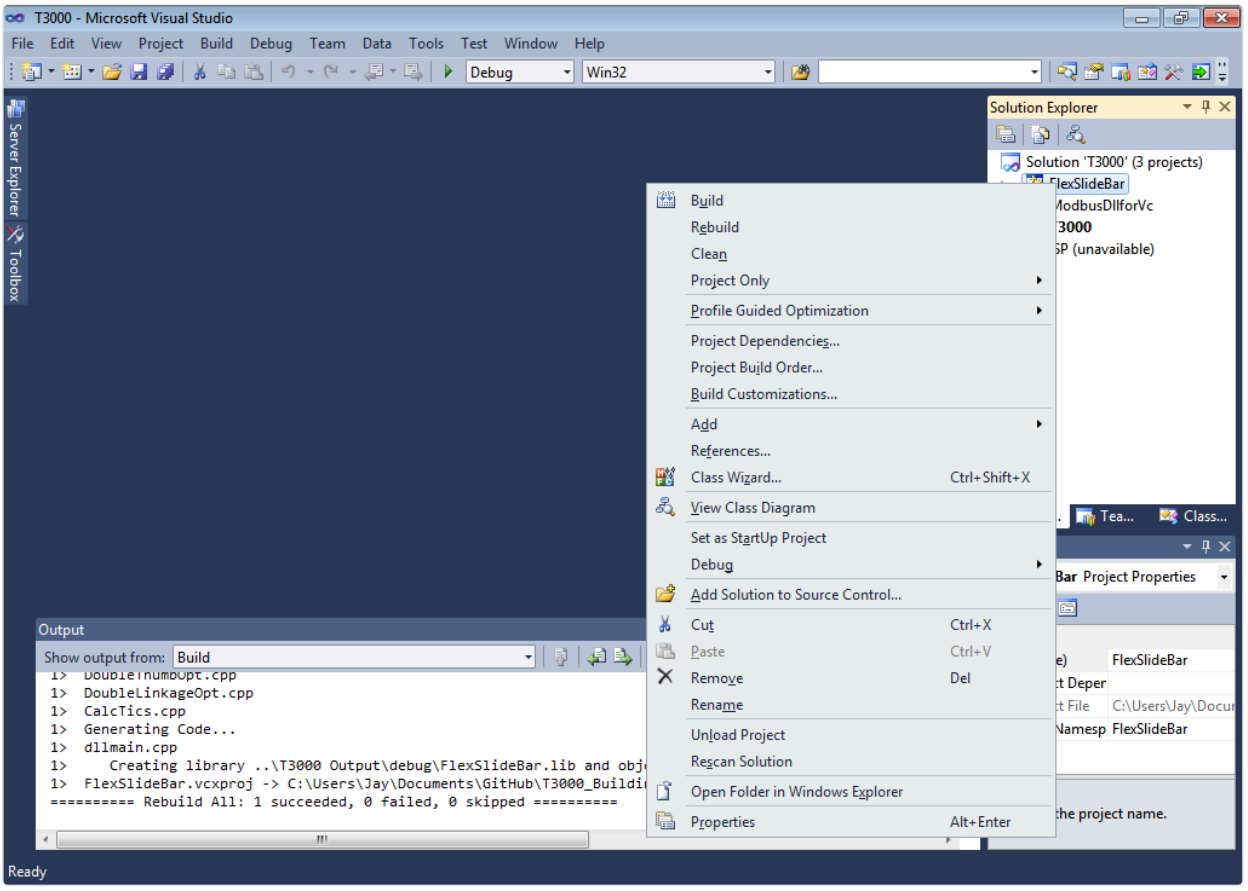
An error window appears as follows. [Right now, not sure why is this error coming? Anyway, capturing it in this doc as of now. We will remove this once the issue is solved.]



The following screen appears.



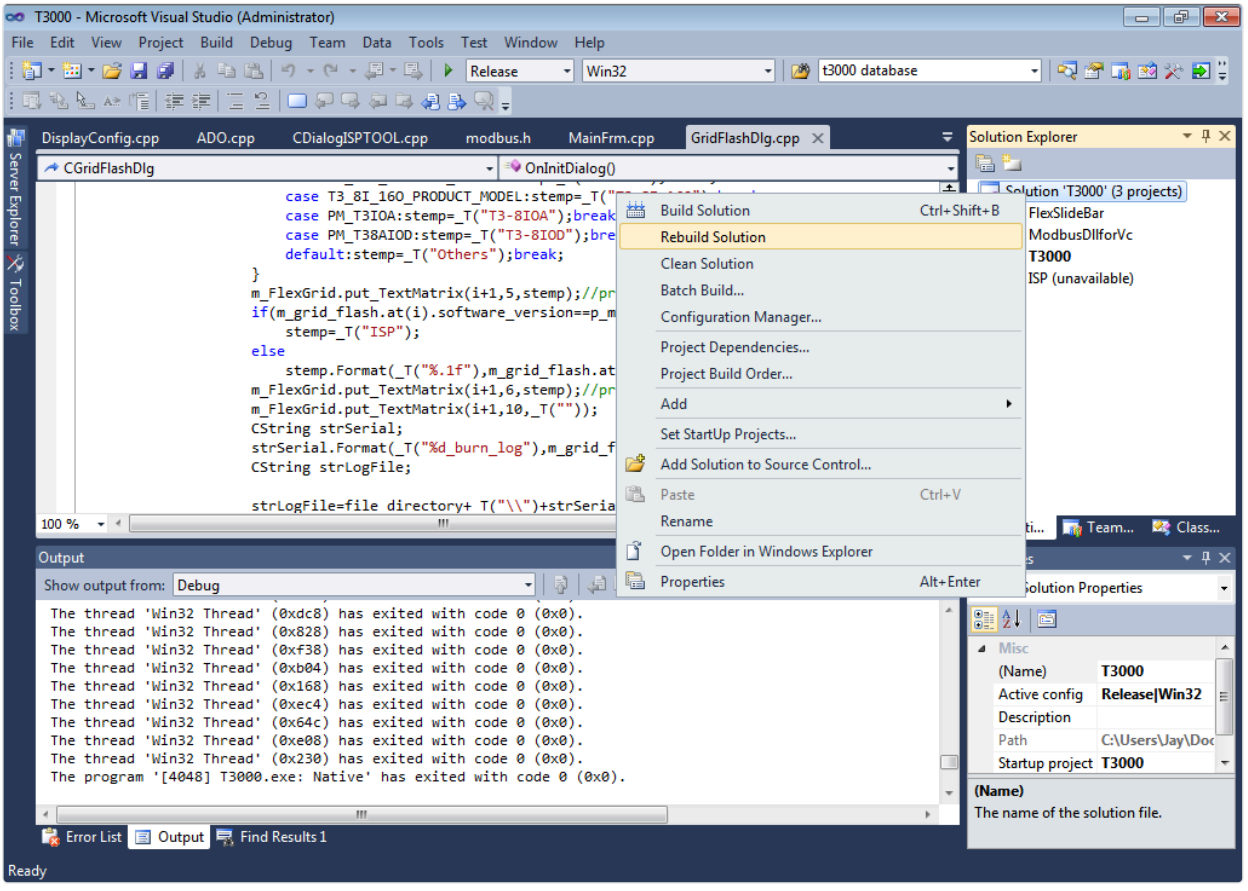
Right-click on FlexSlideBar project. And click Rebuild. This shall compile FlexSlideBar project.



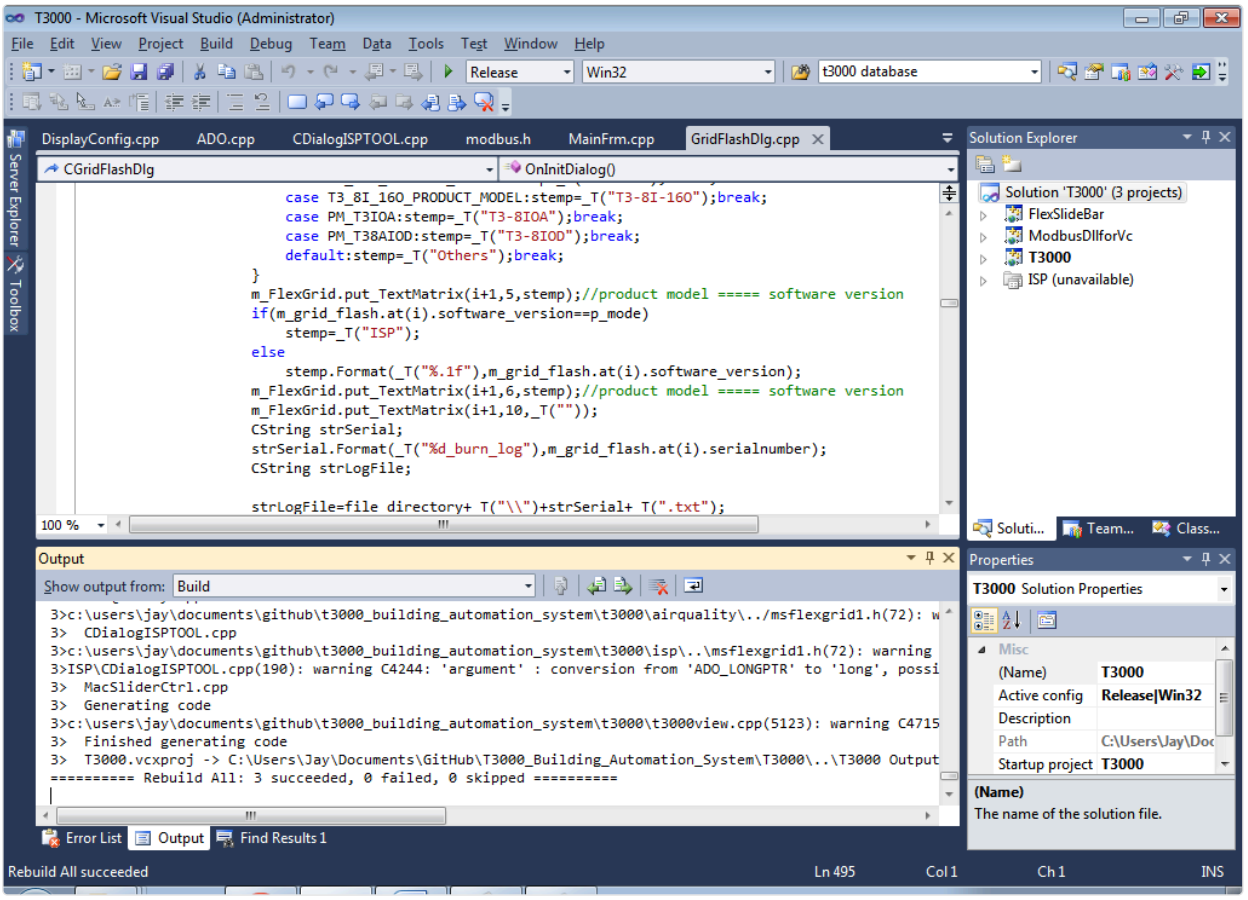
Similarly compile ModbusDllforVc project by right-clicking it and clicking Rebuild.

Similarly compile T3000 project by right-clicking it and clicking Rebuild.

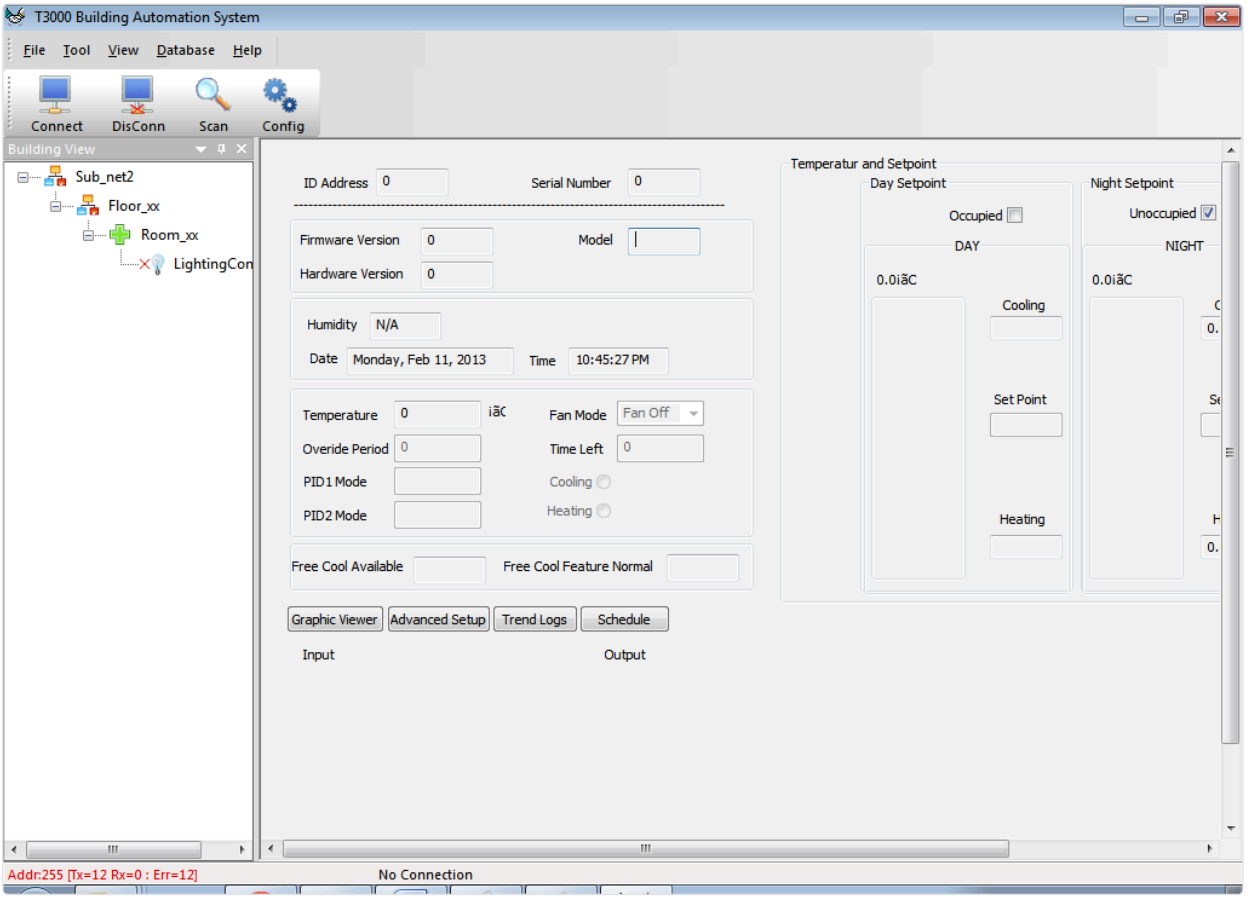
Another way to compile all the projects is to right-click on solution T3000. And click on Rebuild Solution.



Once the code is compiled successfully, Output window of IDE shall show some text as shown below.



Once the code is compiled and it is Run, following application runs.



Git commands summary

git config:

To set configuration parameters of git.

git help

To get help from man pages for git or any of its specific commands.

git clone

To clone a git repo from a remote repository.

git status

To get status of files that are to be staged / committed to the repository.

git add

To mark files to be added to the repository in the next commit.

git commit

To commit files to the repository.

git diff

To check differences between two versions.

git log

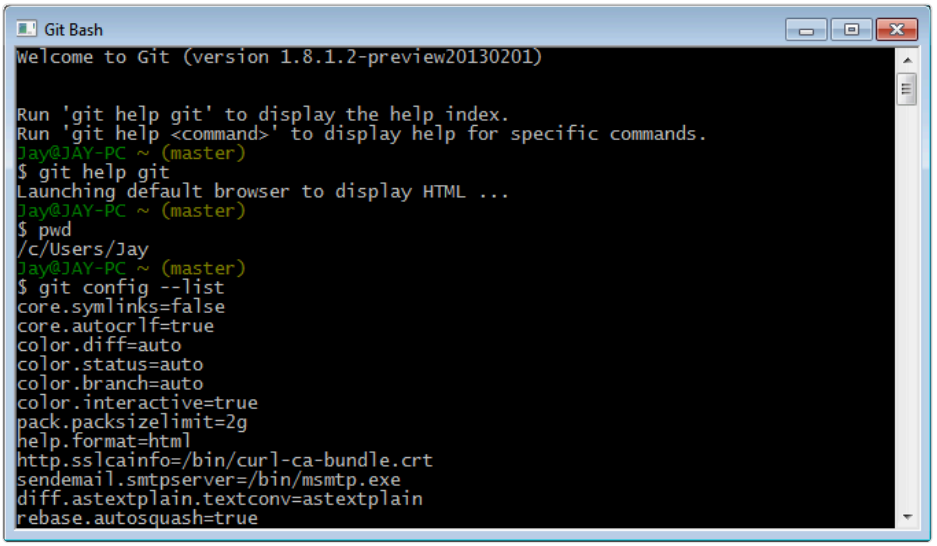
To view commit history.

gitk

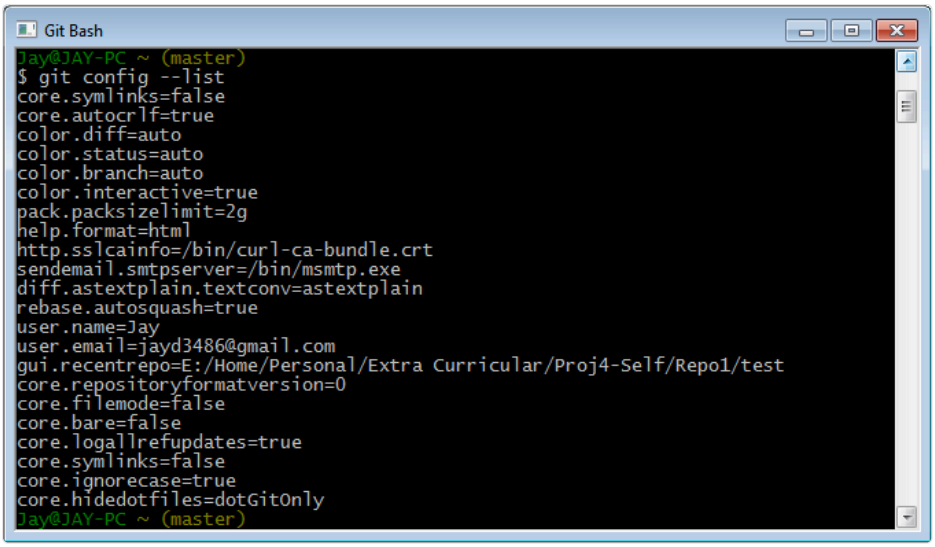
To invoke a GUI to visualize history.

Cloning a repo from github.com using Git Bash Shell

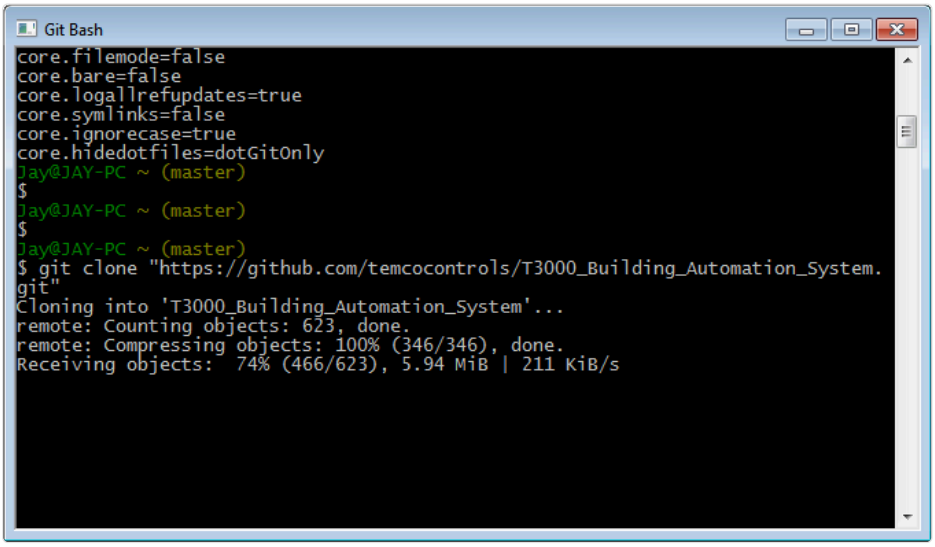
Use pwd (present working directory) command to know your working directory.



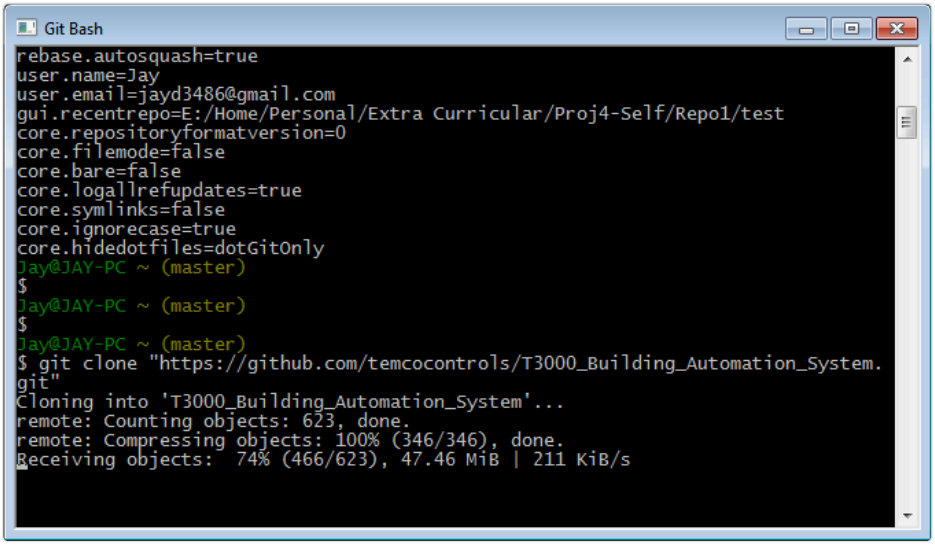
You can use git config –list to know present configuration settings of git.



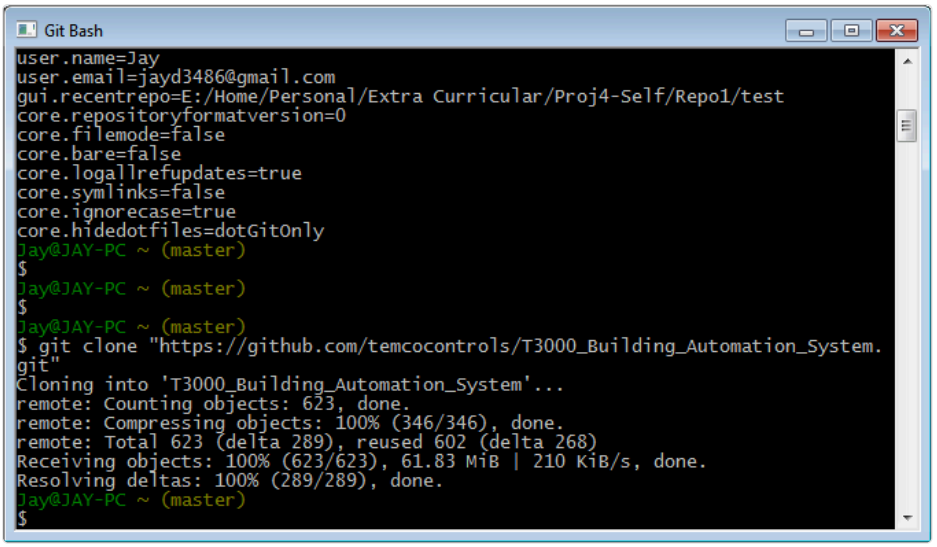
Use clone command as shown below to clone T3000 project from github website.



Continuation of the same.



Cloning done.



TortoiseGit

Setting up of TortoiseGit involves following steps:

Installation of msysgit, the windows version of Git

Installation of TortoiseGit

Sign up for a github account

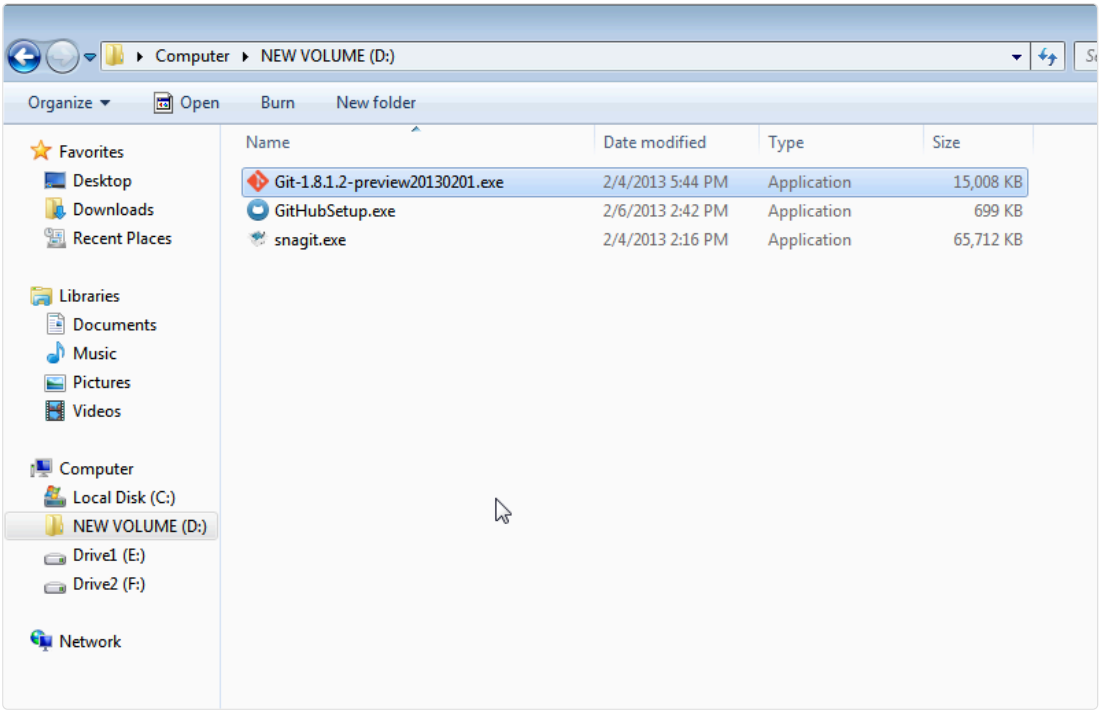
Setting up SSH key

Working with repository

Installation of msysgit

Download git installer.

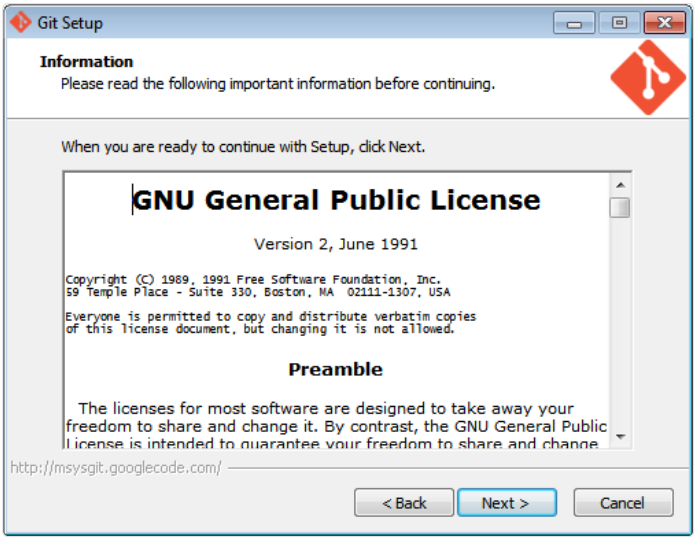
Double click on the setup file.



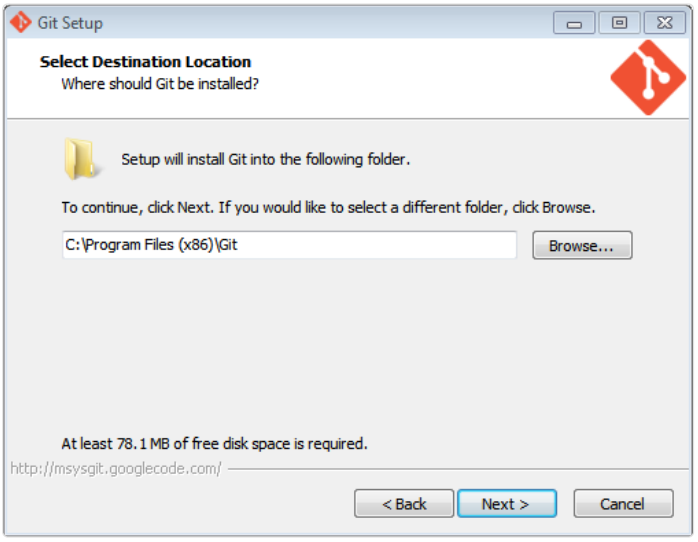
Following dialog box opens. Click Next to continue.



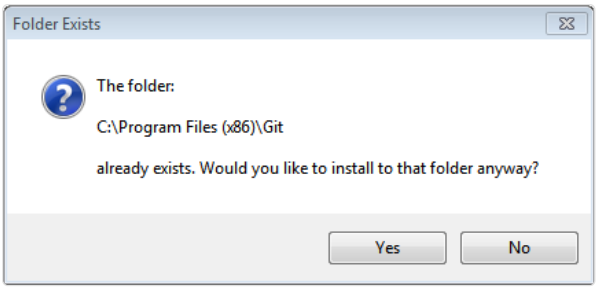
Read GNU General Public License. Click Next.



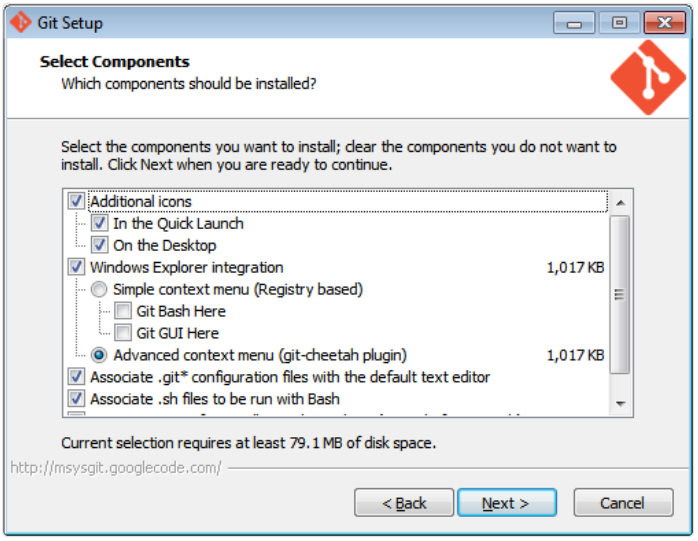
Select destination location. This is the directory where you want to install git. Normally, default directory can be kept as it is. If you wish, you can change the directory by clicking on Browse button. Here, we will install git in the default directory. (C:\Program Files (x86)\Git). Click Next.



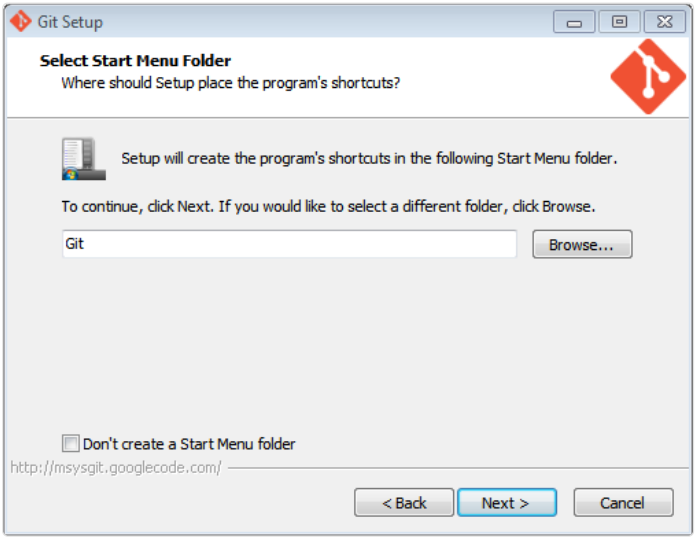
If following dialog box appears, click Yes.



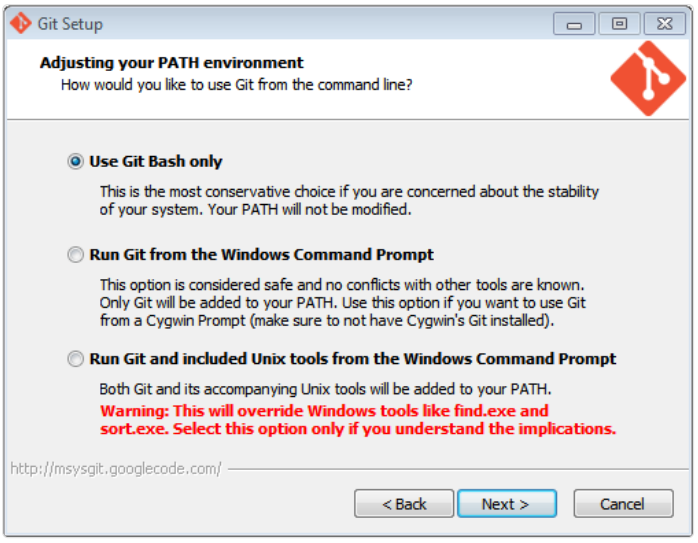
Following screen appears. Leave all the default options as is and click Next.



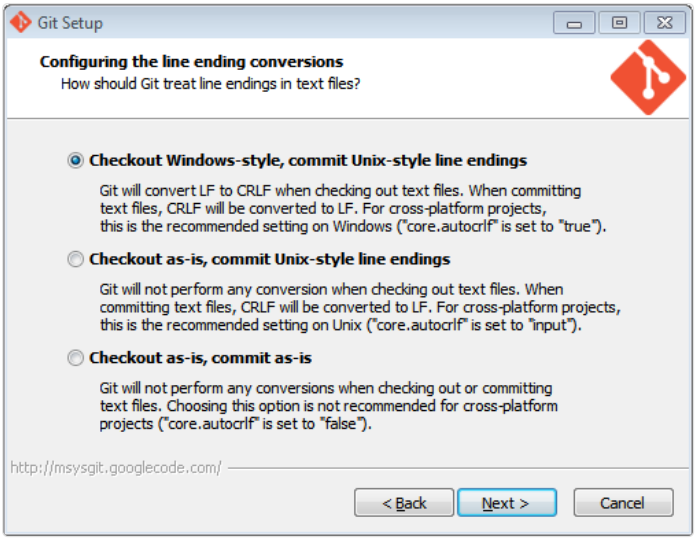
Following screen appears. Click Next.



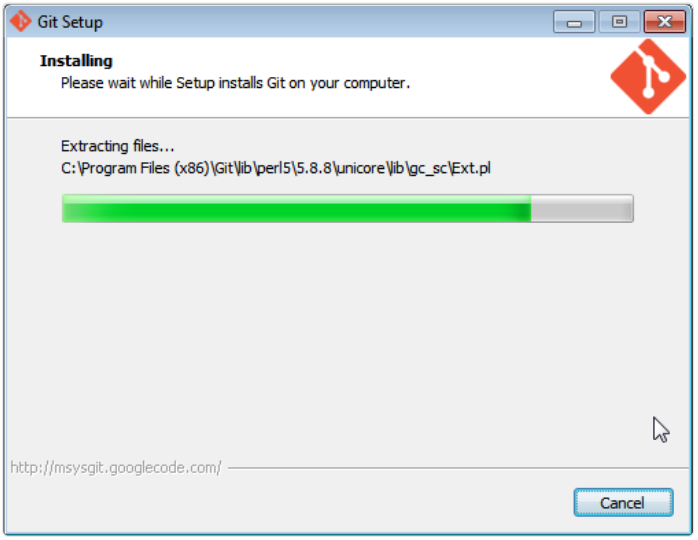
Following screen appears. Click Next to continue.



Following screen appears. Click Next to continue.



Setup will progress.



Click Finish to complete the setup.



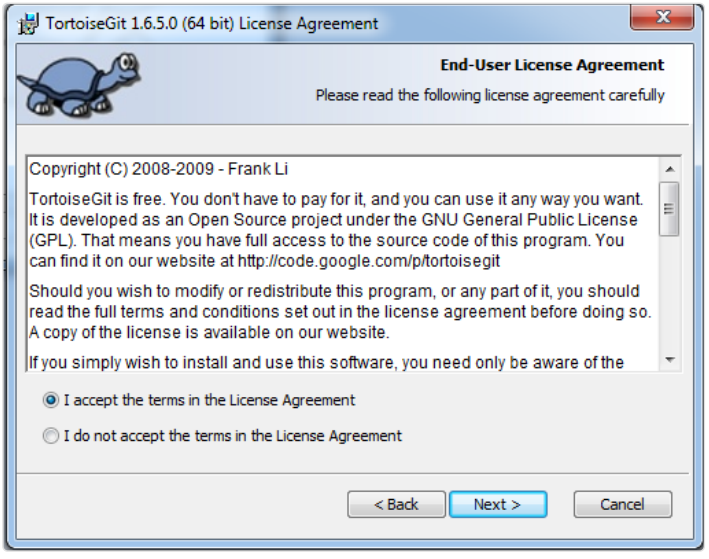
Installation of TortoiseGit

TortoiseGit is a GUI wrapper around git, and provides one of the cleaner user experiences. You'll need to download and install msysgit before installing TortoiseGit.

The first screen you'll be presented with is a splash screen.

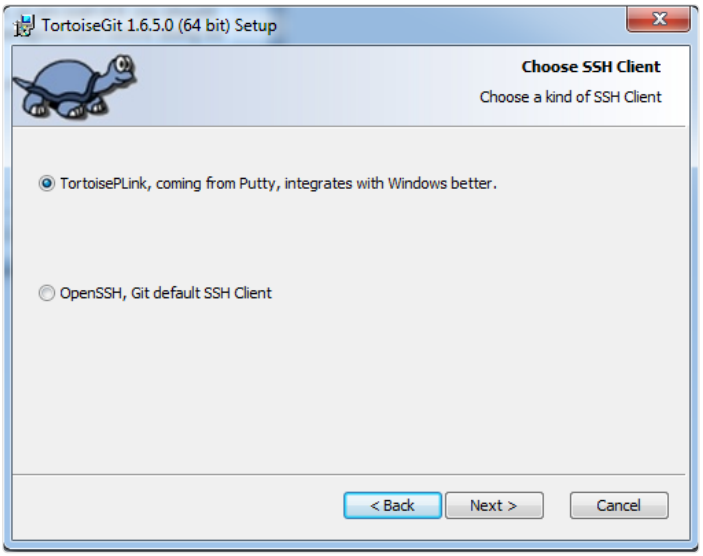


Click Next, and EULA will pop up.



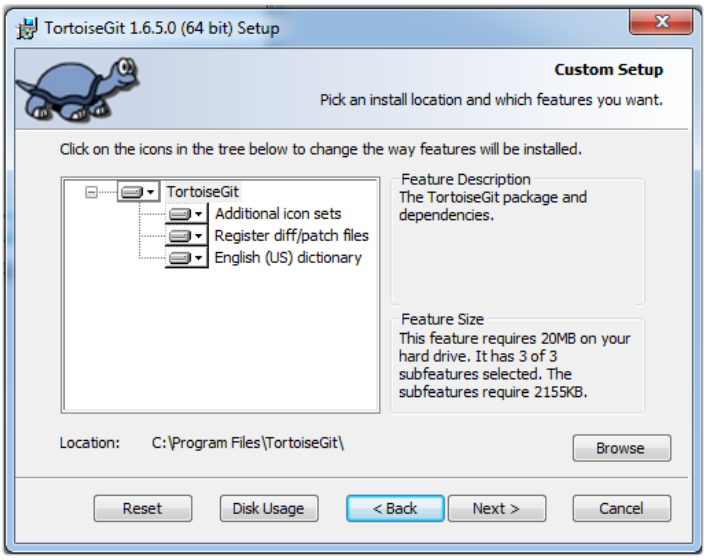
Select 'I accept...' and click Next.

Following screen appears.



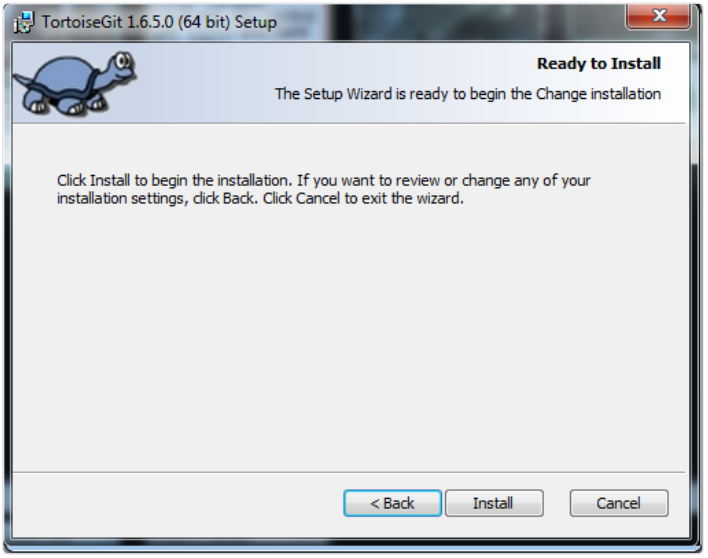
Make sure TortoisePLink is selected, and then press Next.

Screen for custom setup appears.

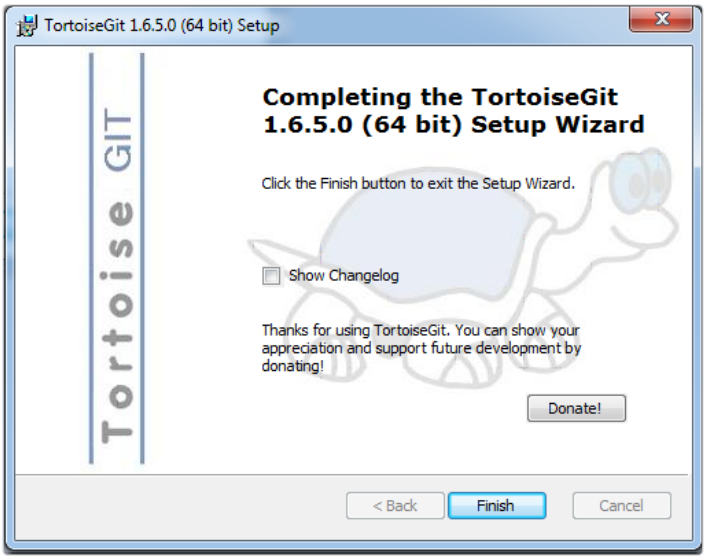


Click Next.

Following screen appears. Click Install button.



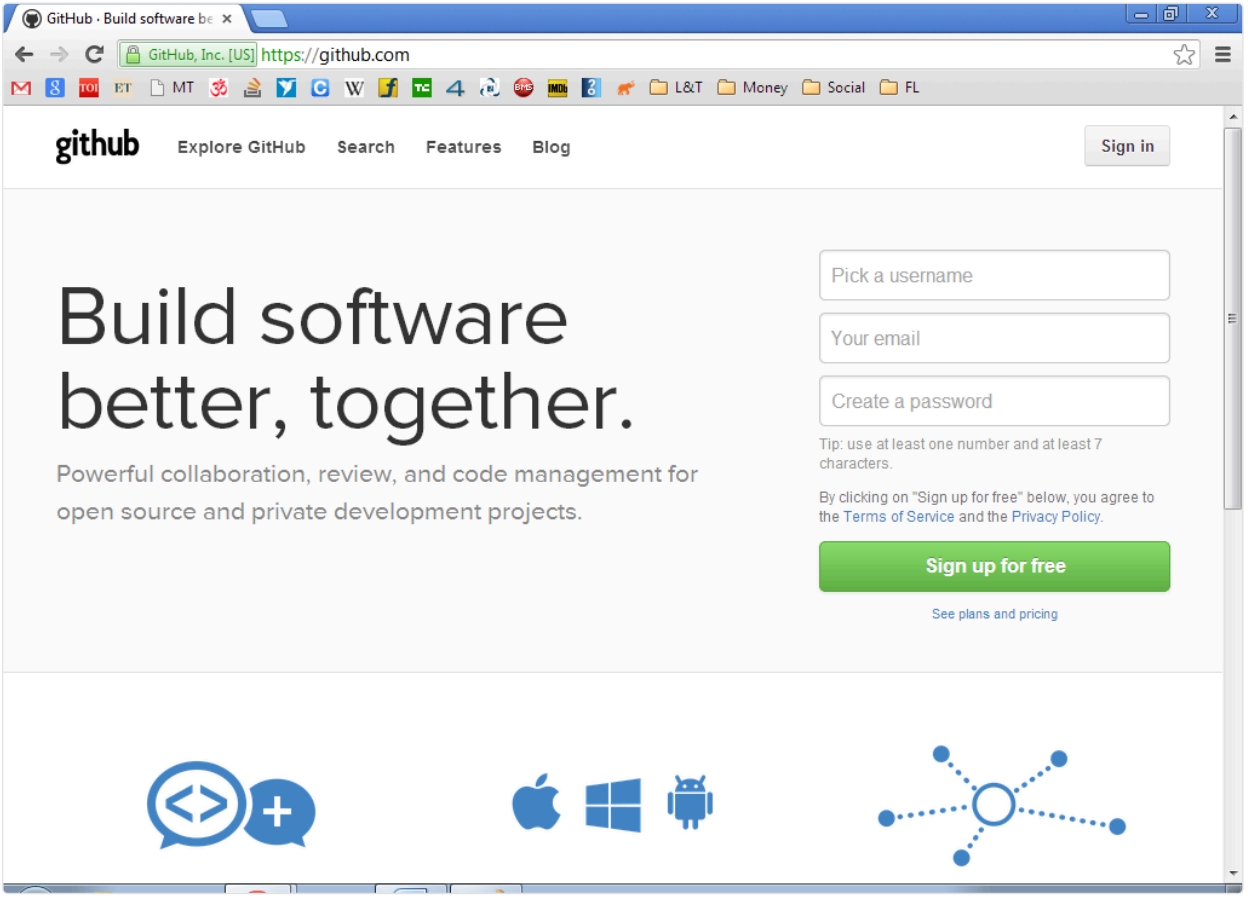
TortoiseGit gets installed and following screen appears. Click Finish.



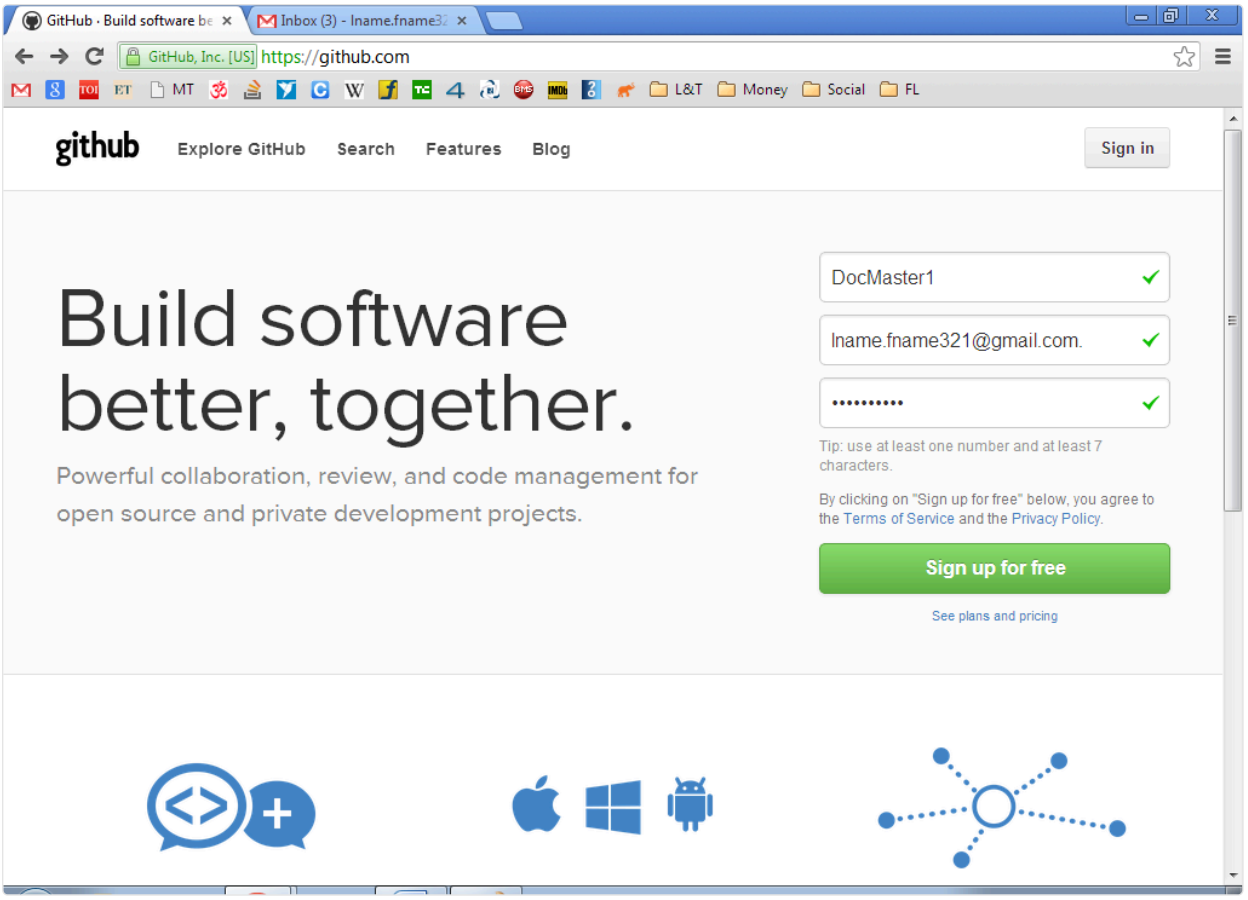
Your machine is now all set-up for playing with code on Github. You just need a Github account.

Sign up for github account

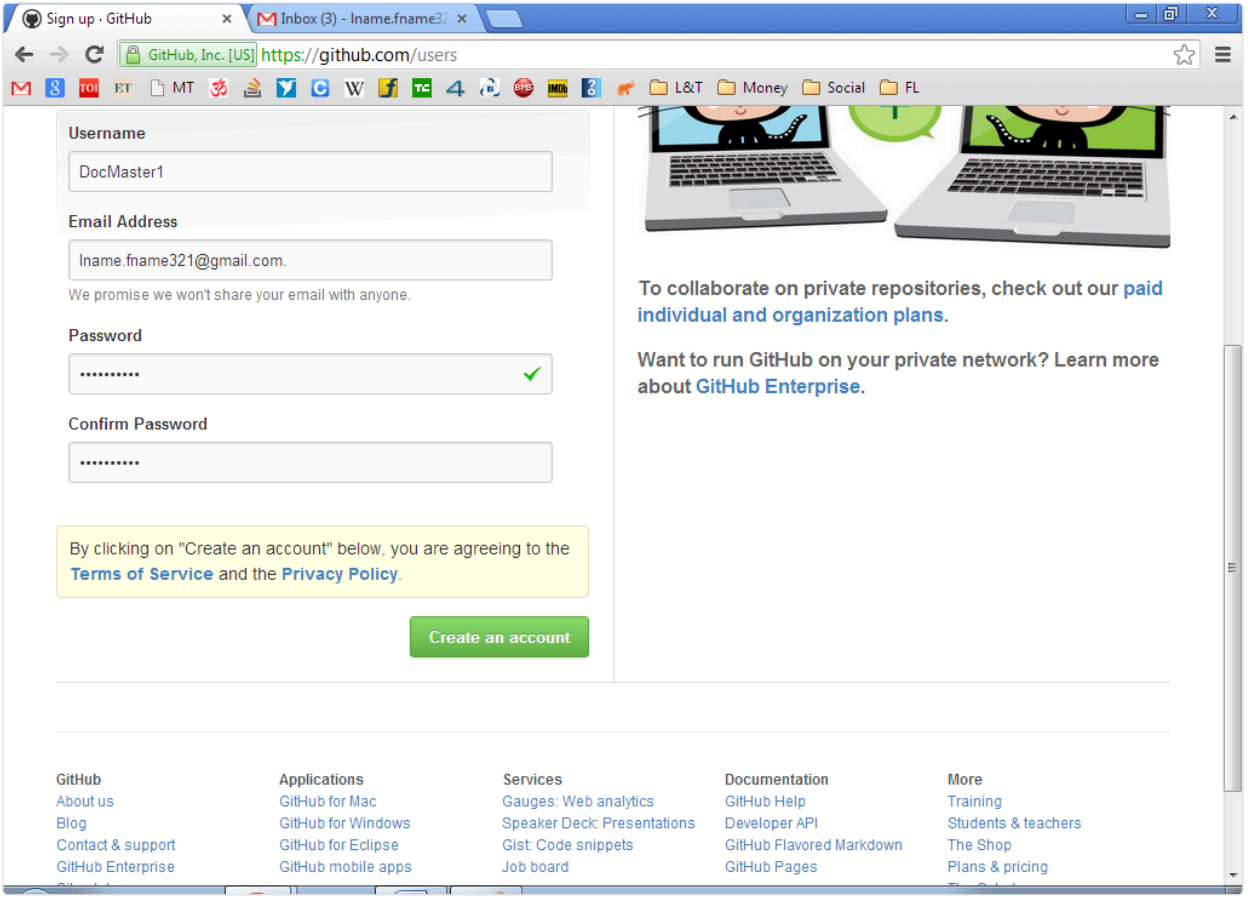
Go to github.com website.



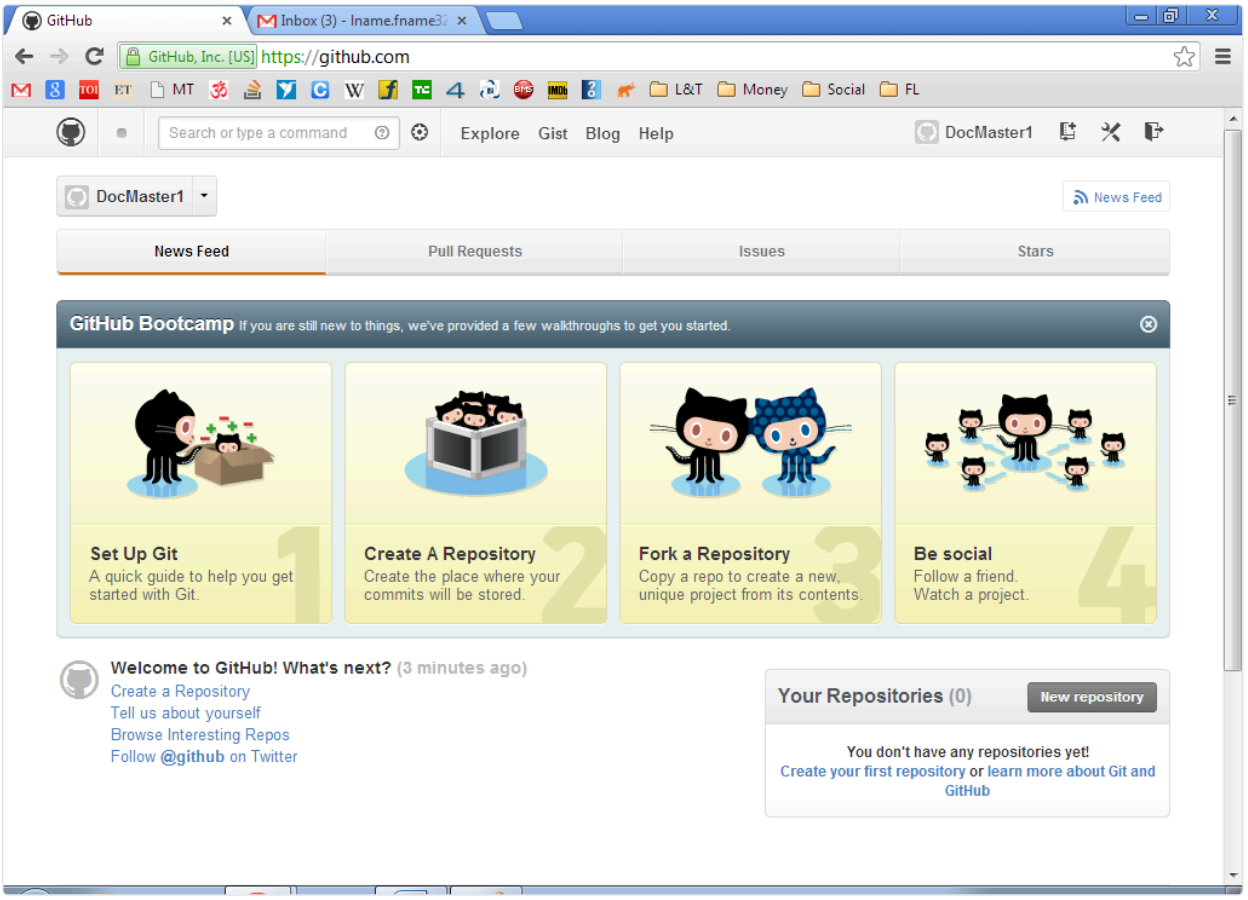
To setup github account, enter username, email id and password. Click 'Sign up for free'.



If there are any warnings, github shows them and takes you to the next page. Click on 'Create an account' button.



Now your github account is ready and you are good to go.



Setting up SSH key

To use your Github account on a Windows machine, you'll need to generate a new RSA key. You'll also need the following tools installed:

msysgit

TortoiseGit

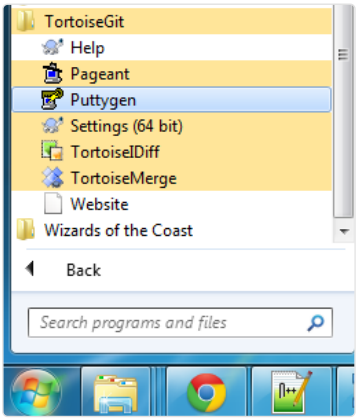
Generate a new RSA key

RSA is an asymmetric encryption algorithm used to authenticate users without using passwords. It's used instead of passwords simply because it provides a mechanism for verifying who you are without giving the other end of the connection information they can use to impersonate you.

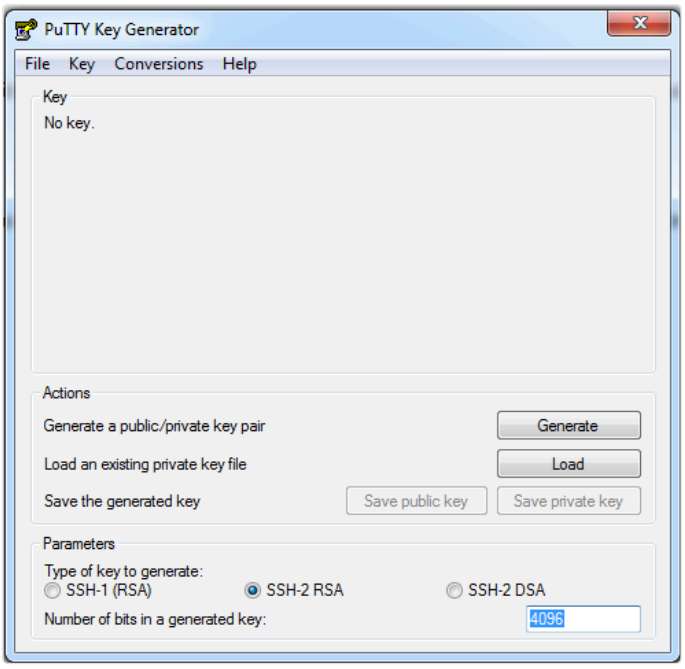
An RSA key consists of two halves: a public key and private key. You give your public key to other sites, which allows them to verify (using a lot of math) that you are the owner of your private key. You must keep your private key private.

Since a private key can theoretically be stolen, it's encrypted with a password (in RSA terms, a "passphrase"). So you need to make sure you both keep the private key in a safe place and remember the passphrase it's encrypted with.

To generate a new RSA key, you'll need to use a tool called Puttygen. It's installed along with TortoiseGit, and you may be able to find it in your start menu:

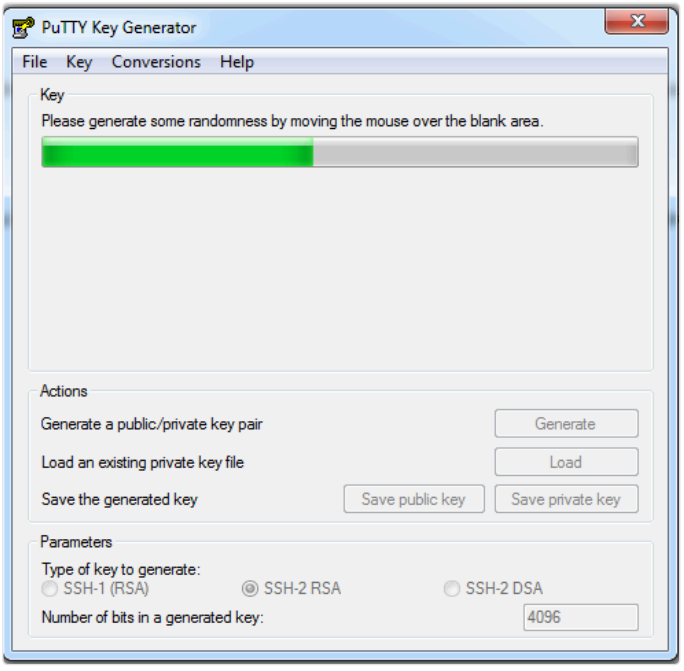


Once it is running, you will be presented with following screen:



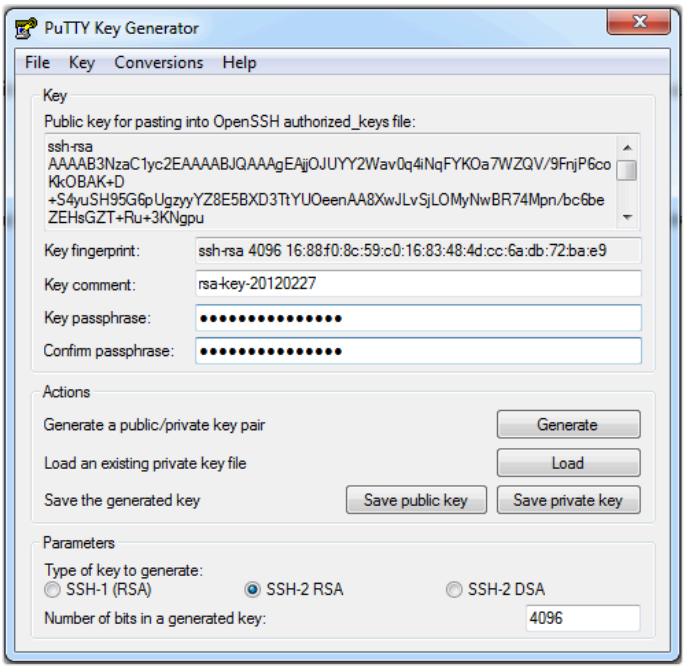
Change the number in the lower-right hand corner, labeled with Number of bits in a generated key, to 4096. This is the current acceptable length for RSA keys. Once this is done, hit the Generate button.

Following screen appears. As indicated in the screen, you are supposed to move mouse over blank area of the screen till progress bar reaches its 100 percent value.



The Puttygen program does not use Window's built-in cryptographically secure random number source. So, this is the fallback option to obtain entropy.

Following screen appears.



Do not close this window until the end.

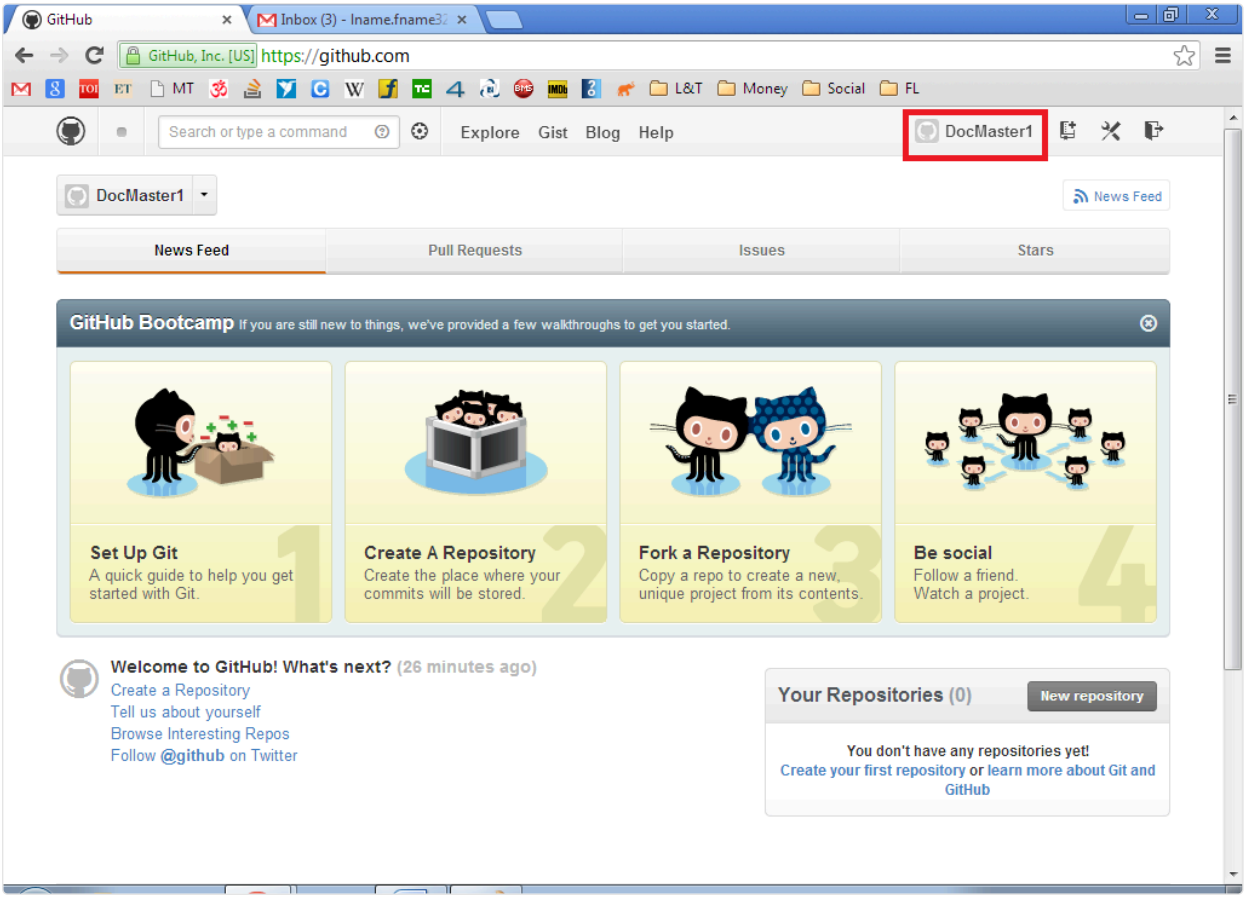
Enter your chosen passphrase into both the 'Key passphrase' and 'Confirm passphrase' box.

Then, click Save private key and save it somewhere safe.

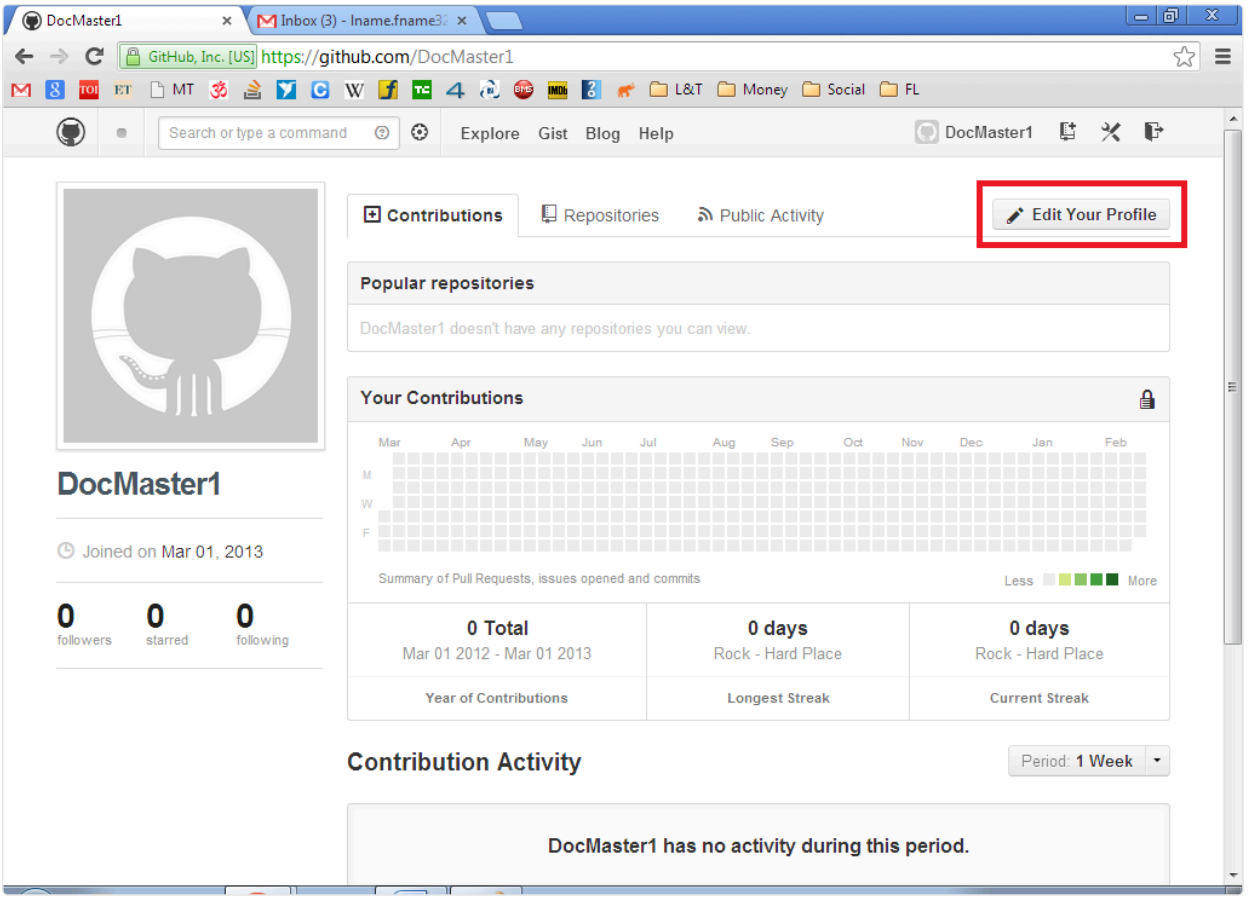
Don't close the window yet. You'll need all of the text in the Public key for pasting... box for the next step.

Give your public key to github

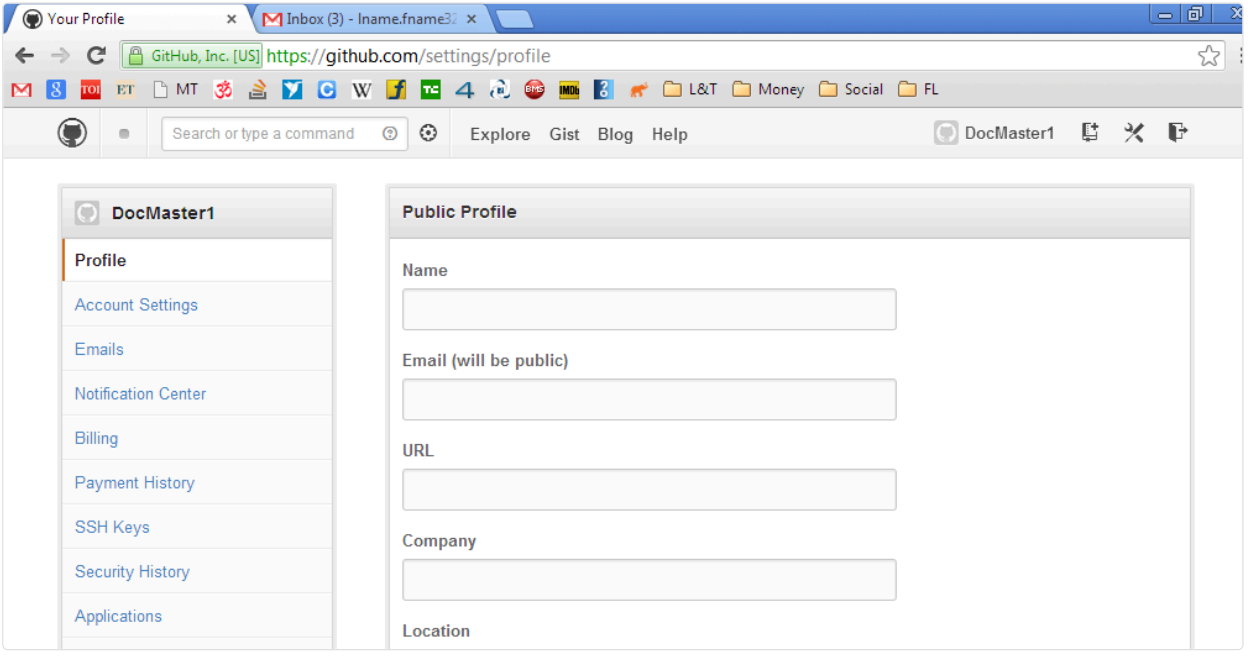
Open web browser. Log into your github account. In the top right corner click your username.



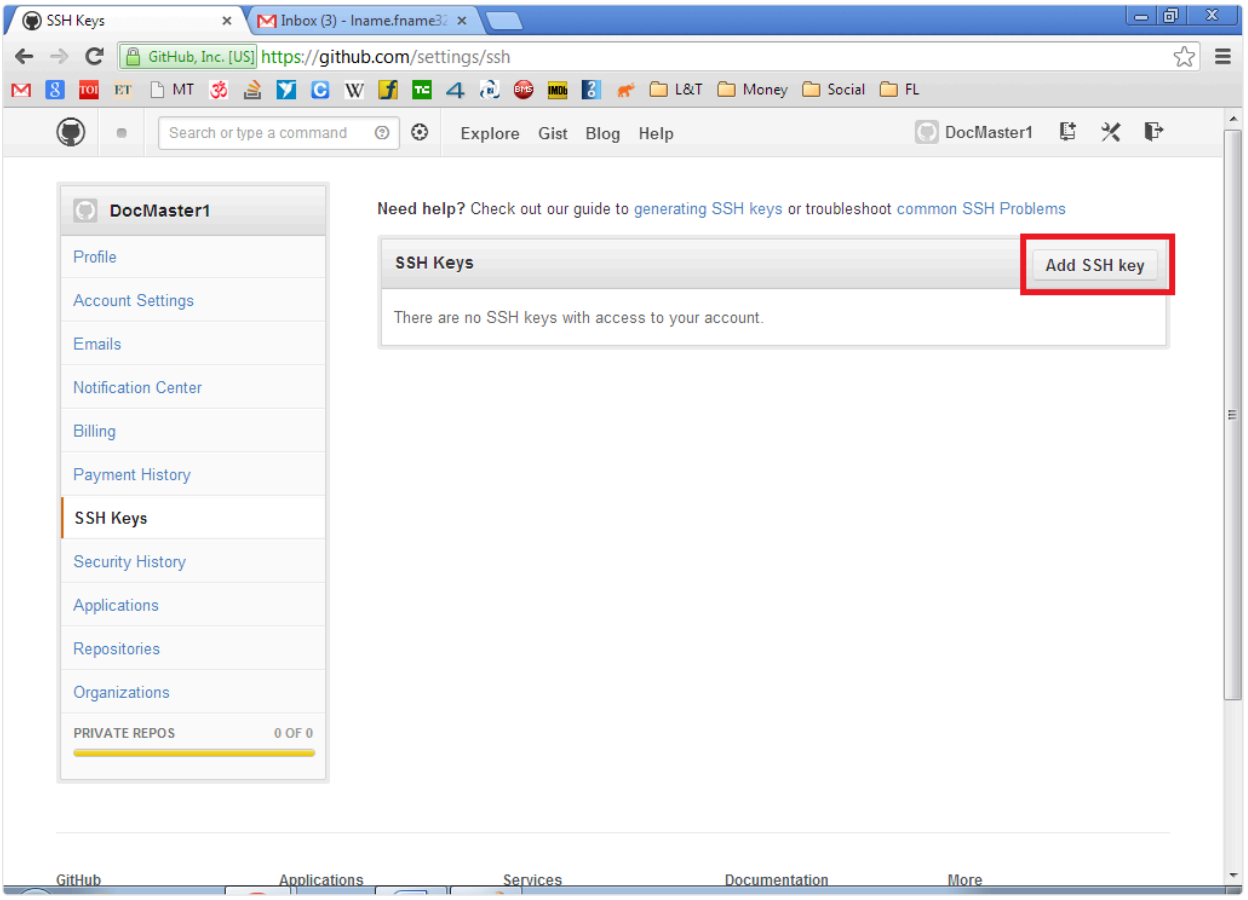
From the screen that appears, click on 'Edit your profile'.



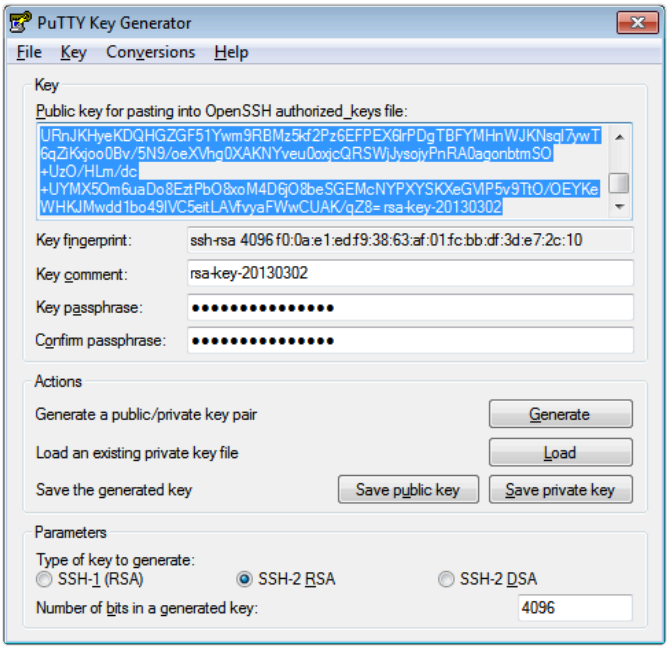
Then from the menu on left, click 'SSH Keys'.



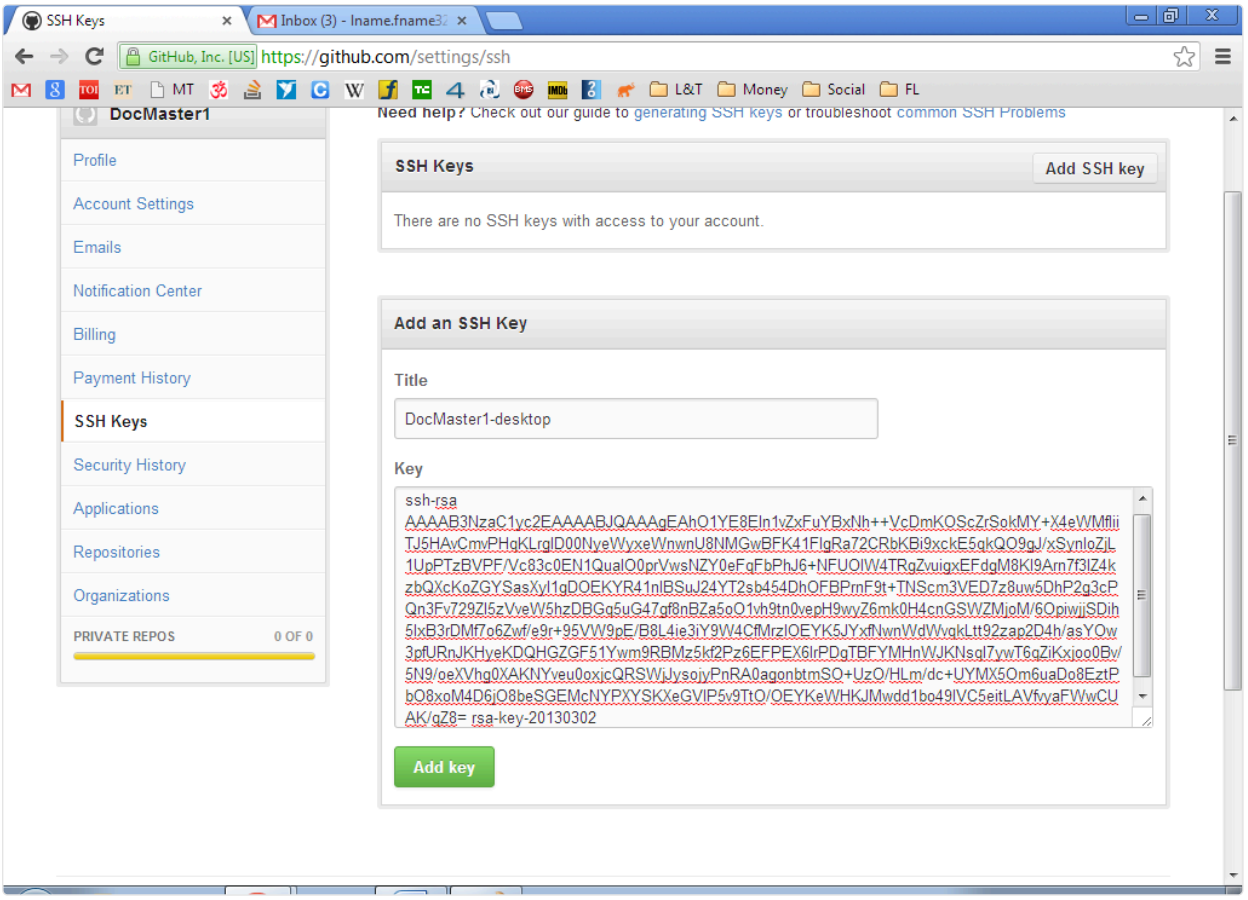
Click on the button 'Add SSH Key'.



Go back to Puttygen window and copy text as shown below. Right click -> Select All would be a good option.



Paste copied text in the textbox labeled Key. Also, enter a Title.



Finally click 'Add Key'.

That's it. Make sure to keep your private key somewhere safe. And do not forget your passphrase.

Github Workflow

Before starting to learn the workflow, make sure you have following things:

Installed msysgit, the windows version of Git (section 6.1)

Installed TortoiseGit, a friendly GUI wrapper around msysgit (section 6.2)

A github account setup with public half of your private key (section 6.3 and 6.4)

Private key and passphrase (section 6.4)

A word about git and github:

Git is a decentralized version control system. The "decentralized" bit sets it apart from other version control systems like CVS, SVNor Perforce. "Decentralized" means that there is no central repository -- each developer has a repository (or a few copies) that contains the entire history of everything. This has some advantages:

You can make commits without being network-connected

You can commit without worrying about build-breakage

In a disaster scenario, you're fine unless every copy of the repo is wiped out

Unfortunately, these come at the cost of additional complexity.

To simplify things a bit, in most real-world scenarios, one repository is designated as blessed [In our case, this blessed repo would be: temcocontrols / T3000_Building_Automation_System]. This blessed repository serves as a central, true copy. All the other repositories (or, "clones") still contain the same version history; the blessed repository simply serves as synchronization primitive to make the workflow cleaner.

You can do a variety of things with a repository. These include --

Add new commits to a local repository (on your machine)

Send commits to another repository (on your machine or the network)

Get commits from another repository (on your machine or the network)

In the setup we're using, each developer interacts with three repositories:

The repository on your local machine

Your "fork" on Github

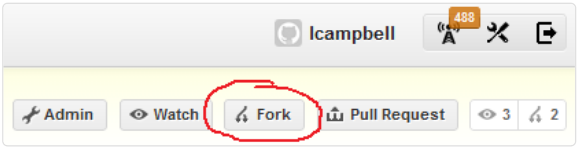
The "one true repo", which is the central, blessed repository

You're free to do whatever you want with the repository on your machine and your fork on Github. Typically, you'll make commits on your local repository and send them to your Github fork immediately (or, as soon as you're network-connected). Commits going into the blessed repository must go through a review process (using Github's wonderful interface) before they're accepted and merged into the codebase. This process is performed using a pull request on Github, which is detailed further down.

Forking:

Right now, we've only got the blessed repository. We need to create both your fork and your local copy set up. First, log into Github, navigate to our Github page and select a repository you'd like to work on. I'm going to assume for this case you want to work on T3000_Building_Automation_System.

Click the Fork button in the right-hand corner:



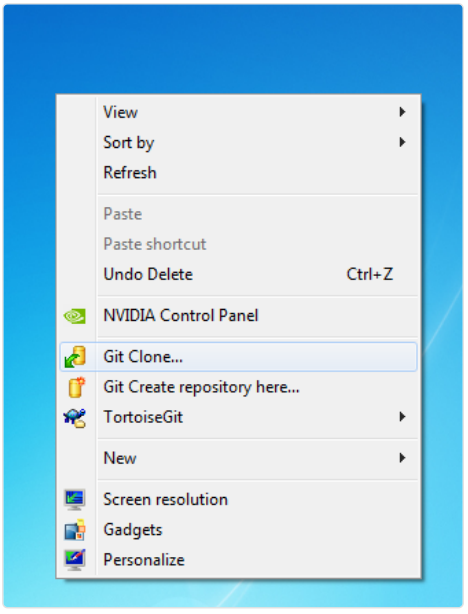
After some hardcore forking action, you'll be greeted with your Github fork. This is your personal copy of the blessed repository.

Assuming your Github username is Jay and the name of the upstream repository is T3000_Building_Automation_System, your fork will be located at https://github.com/Jay/ T3000_Building_Automation_System.

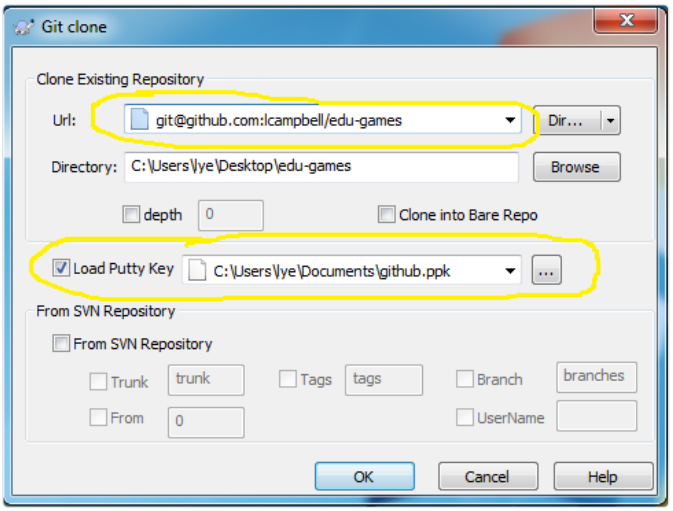
[Remember, these names are just for example. In reality, username Jay has already been taken up by someone else. Here, I am mentioning the same because it would be easier to understand. Also, the usernames and repo names shown in diagrams are different ones. Do not get confused by that. Just have a look at diagram for position of the buttons and conceptual understanding, etc.]

Cloning your fork into a local repository

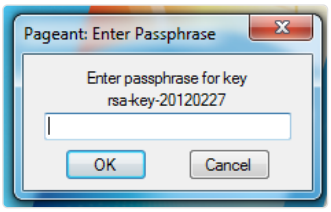
Git doesn't allow you to commit directly to a repository on the network -- you need to make a local copy before you can start doing work. Making a copy, in Git terminology, is called "cloning", and we're going to do it with TortoiseGit. Start by right-clicking on the Desktop (or other appropriate place) to bring up a context menu.



From this menu, select Git Clone... and you'll be greeted with the following dialog:

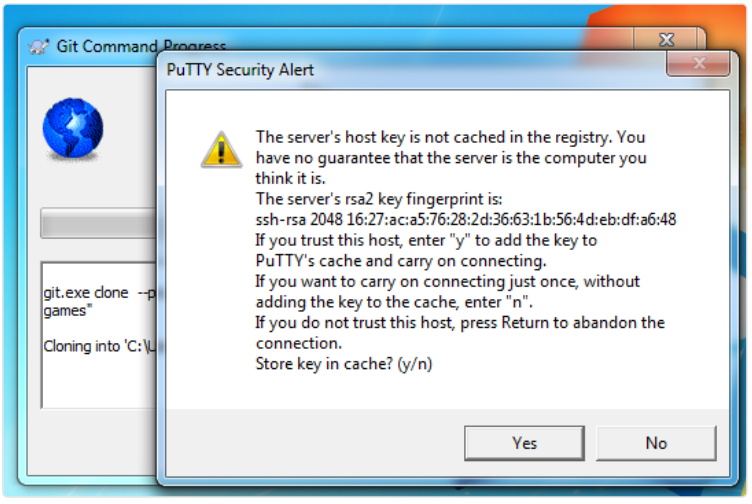


Here, you'll need to enter the URL for your Github fork, and the location of your private key. Do not enter the URL of the blessed, upstream repository. Once you've got the appropriate information inputted, click OK, and it will prompt you for your key's passphrase.



Toss in your passphrase and hit OK. TortoiseGit will save your passphrase for an indeterminate amount of time.

If this is your first time connecting to Github's servers, you'll get the following message:

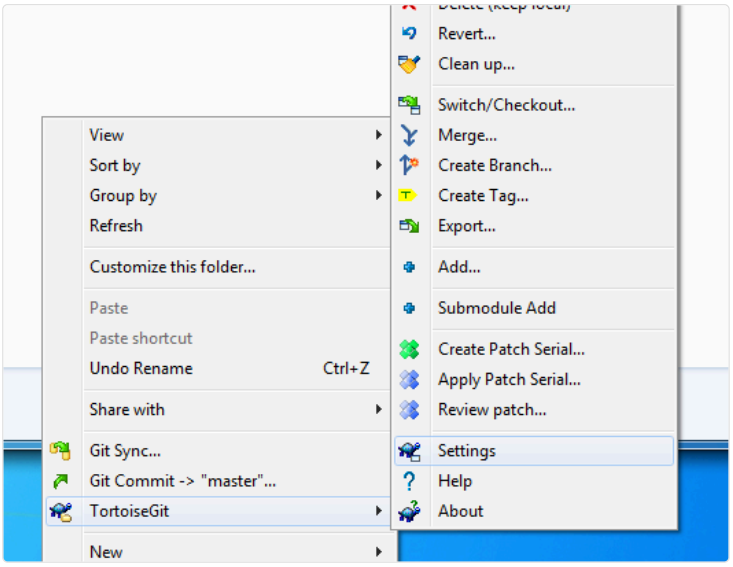


Check to make sure that the rsa2 key fingerprint the dialog tells you is the same as in the screenshot; if it's not, then something is terribly wrong (it's not Github you're communicating with). The reason you get this dialog is due to the nature of public key cryptography. You don't yet have Github's public key. Once you connect once, their public key is saved on your machine and you'll never be prompted again. If their public key changes for some reason, you'll know immediately and Git will cease communication before any damage is done.

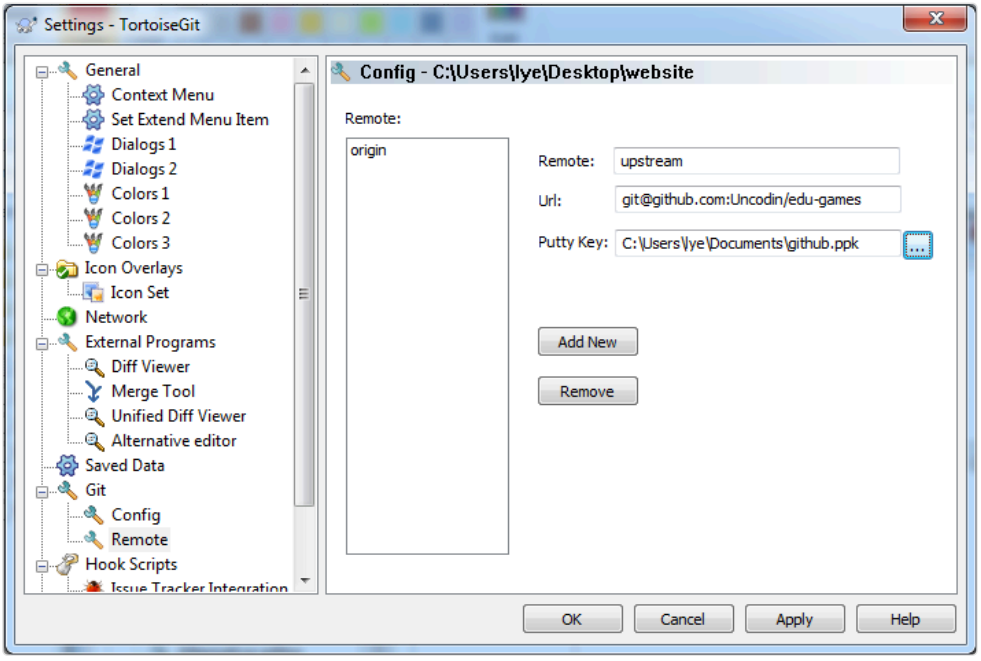
If the key fingerprint is correct, click Yes.

A window should pop up with a progress bar as the repository is copied to your machine (forked repo). Since it contains the entire history it can take a bit of time on a slow network connection.

Once it's finished downloading, we need to take one last step -- we need to tell your local repository about the blessed upstream repository (temcocontrols/T3000_Building_Automation_System). To start, open up the TortoiseGit settings by right-clicking in your repository and selecting Settings



Select Remote from the left menu. You'll get this dialog, albiet without the details filled in:



In the fields, put in

Remote: upstream

Url: The URL to the upstream repository:

git@github.com:temcocontrols/ T3000_Building_Automation_System

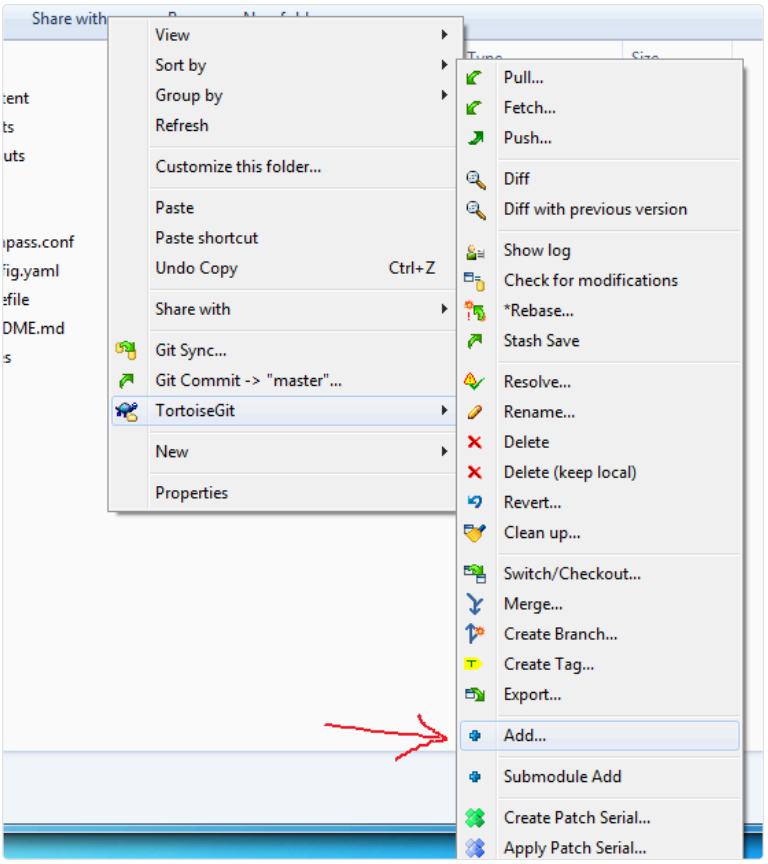
Putty Key: Your private key

Hit Add New, then OK. You're all set!

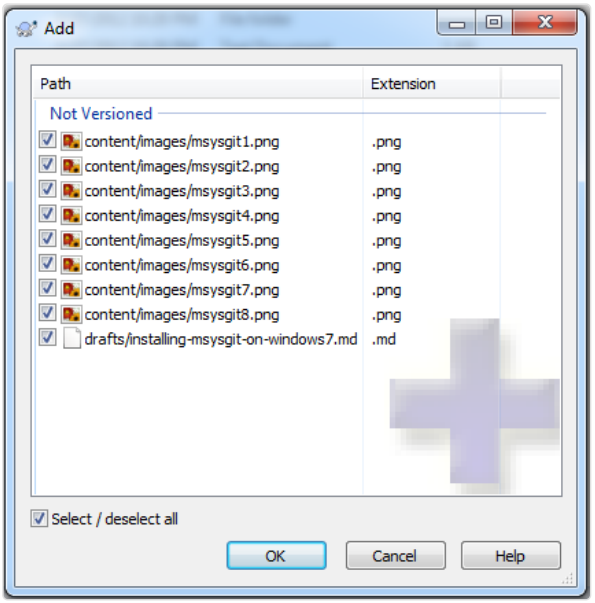
Making changes, adding commits

With your fresh clone, look around, make some changes. Add a file or something. Or get some actual work done.

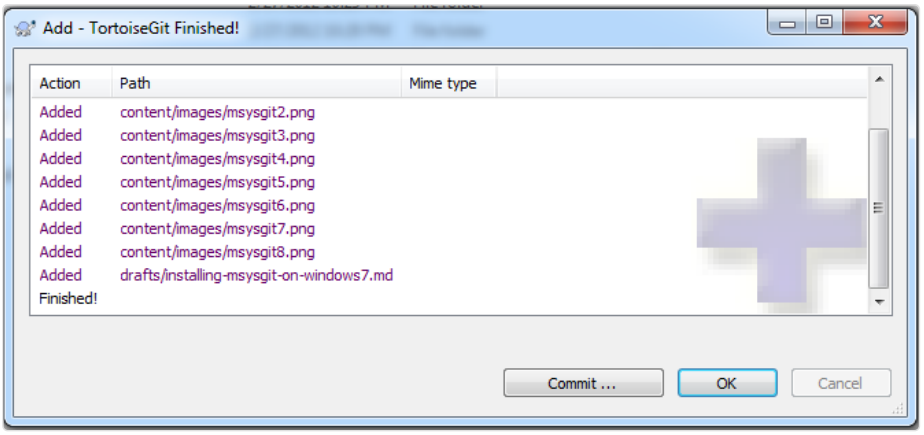
When you've changed something, go to the main folder of your repository and open up the right-click context menu. From that menu, select Add...



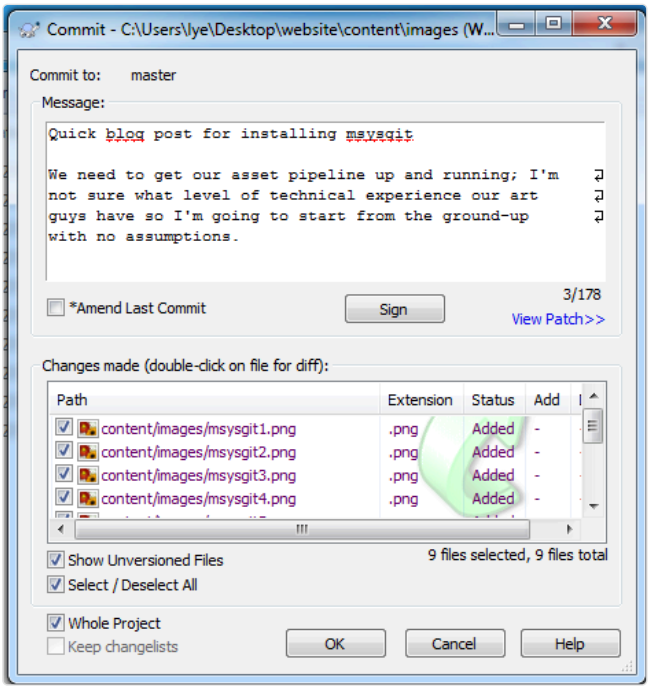
You'll get the dialog telling you which files have been added, removed or changed. Make sure they're all checked and hit OK. Do note that adding files does not commit them. You need to do that in a separate step.



Once you've clicked OK, you'll get this:



To hit the previous point home, this dialog gives you a helpful Commit... button to commit the files you've added/changed/removed. If you don't hit commit, you can select Commit from the right-click context menu at a later point in time. For now though, hit Commit... to arrive at this dialog:

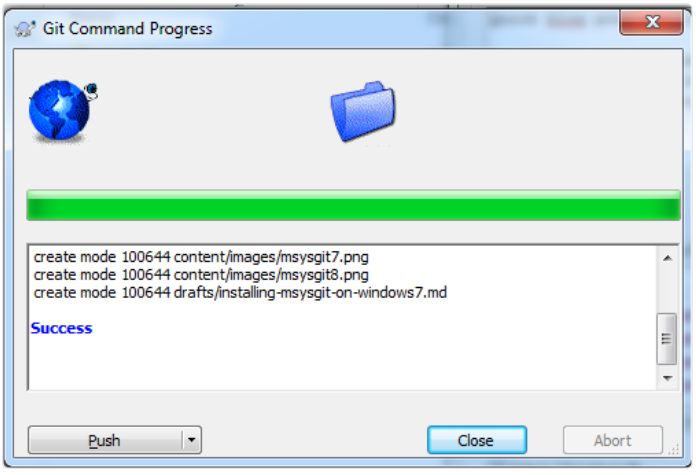


At this point, write a quick blurb about the changes you've made. The first line of your message should be no longer than 80 characters (basically, fit it in the one line of that text box). Each paragraph should be separated by exactly one blank line.

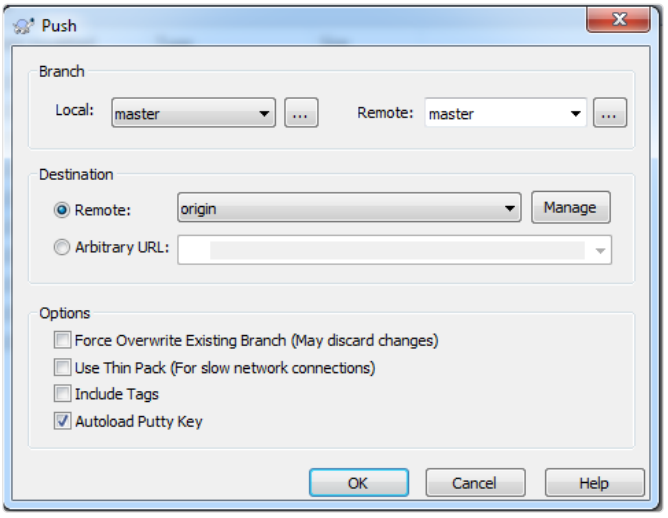
The defaults are usually what you want.

Hit OK to finalize your commit.

You should now be at this dialog:



At this point, we've made a commit on our local repository. We want to send that commit up to our fork [Jay/T3000_Building_Automation_System]; this action is called "Pushing". TortoiseGit gives us a helpful button to push our new commit to our fork. Click Push.

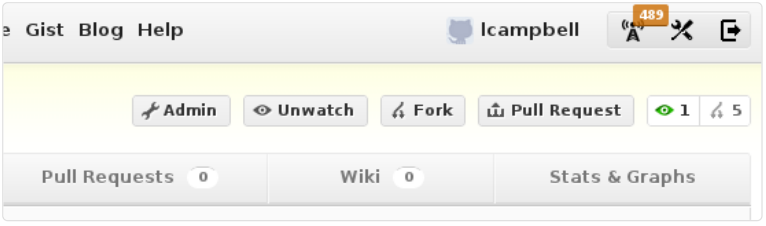


As usual, the defaults are probably what you want here. Make sure that Remote is set to origin, not upstream, then click OK.

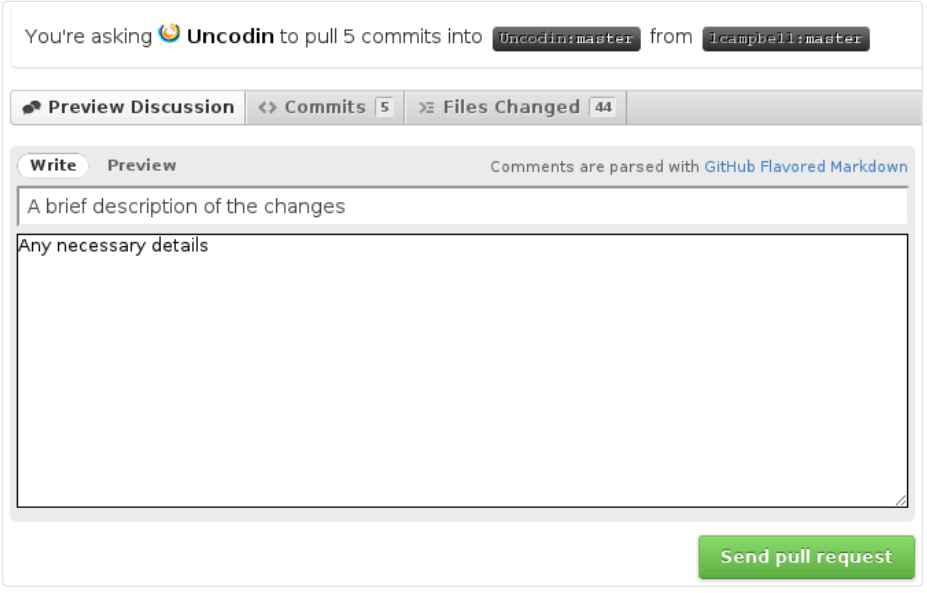
Once your commit is pushed to your Github fork, you can peek at your Github page with your browser to see the changes you've made. Note that they haven't been submitted for review for inclusion into the upstream repository. They're just on your fork.

Getting changes into the upstream repository

You've made changes to your Github fork [Jay/T3000_Building_Automation_System], but you need to get them into the upstream repository [temcocontrols/T3000_Building_Automation_System] for everyone else to have access to them. This is done by sending a pull request on the Github web interface. Log into your account and navigate to your fork's page. In the upper right-hand corner, there's a Pull Request button.



Click the Pull Request button and you'll be taken to the pull request page.



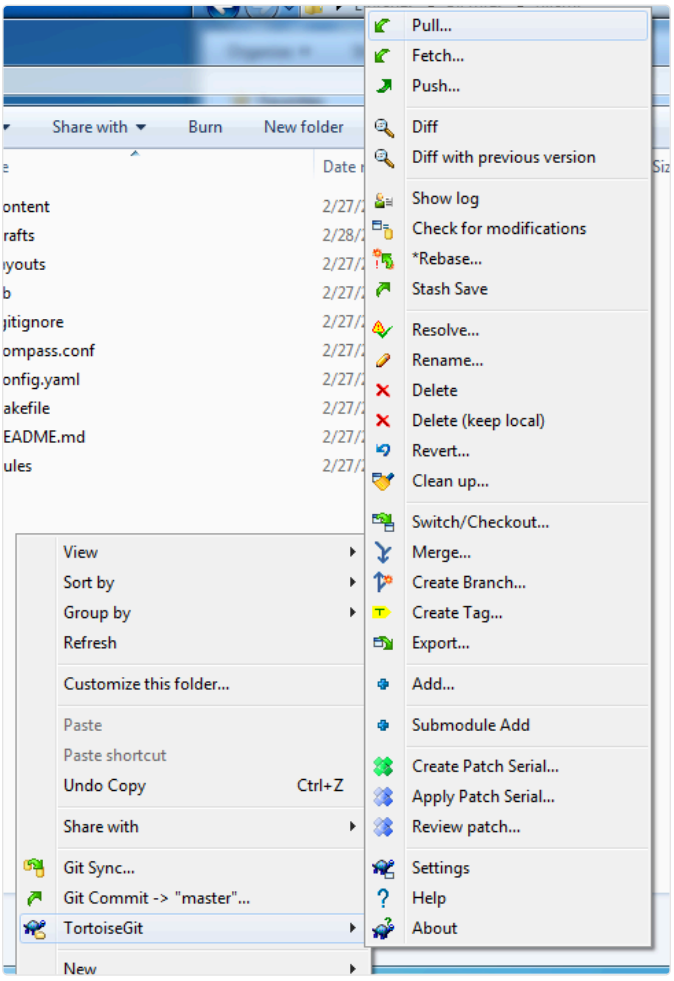
Throw in a title and a description for the pull request, and then hit Submit. We will be sent emails and will have a chance to add comments to your changes.

An important note is that any commits you send to your fork before the pull request is accepted will be merged into that pull request. This lets you make any changes in response to comments, but can be a bit of a surprise if you don't know about it ahead of time.

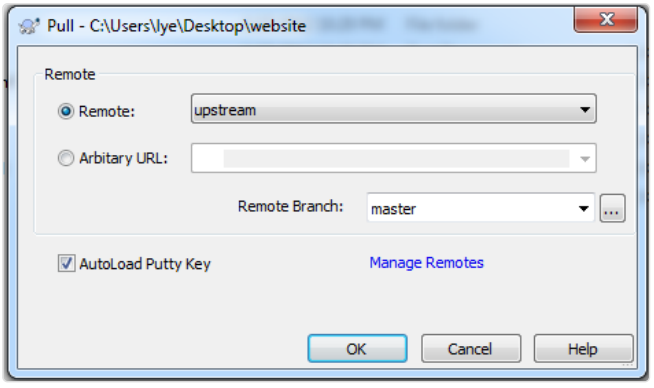
When the pull request is accepted, you'll be notified by email.

Getting upstream changes back into your copy

As all of this is going on, we'll be making changes and sending pull requests. To migrate those changes into your codebase, you'll need to use a Pull command. From the right-click context menu, select Pull.



In the dialog that pops up, select upstream as the remote.



Then just hit OK and any changes made by other people will be pulled into your local repository. They'll be sent to your Github fork when you next push to it.

Conclusion

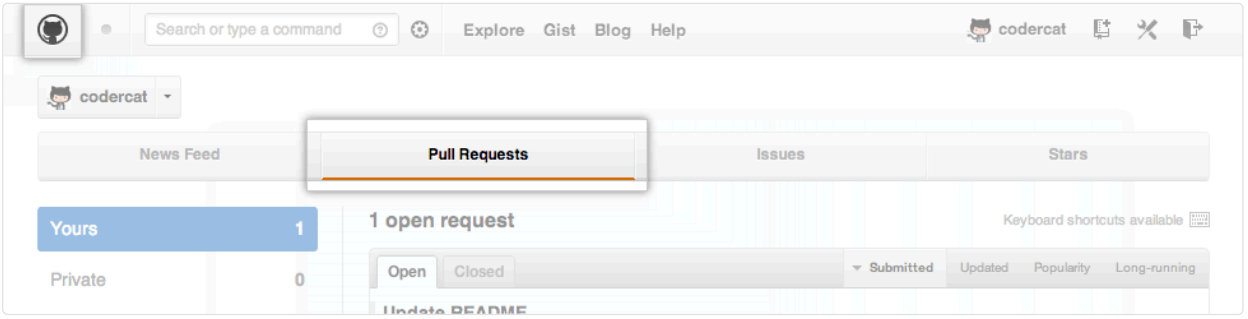
In this way, TortoiseGit and Github are connected. There are built-in tools to establish a proper code-review workflow.

Accepting Pull Requests and Merging changes to upstream repo

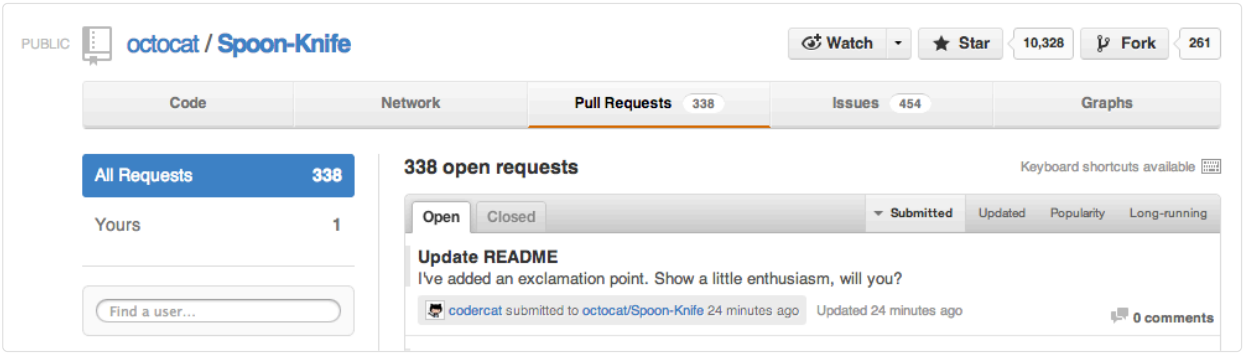
By [temcocontrols/T3000_Building_Automation_System]

Managing Pull Requests

All pull requests sent or received by you are browsable through the pull request dashboard.



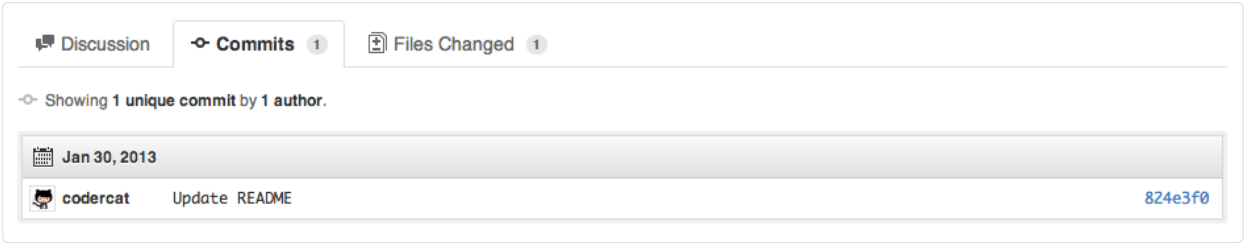
Pull requests for a specific repository are also browsable by anyone with access by visiting the Pull Requests page.



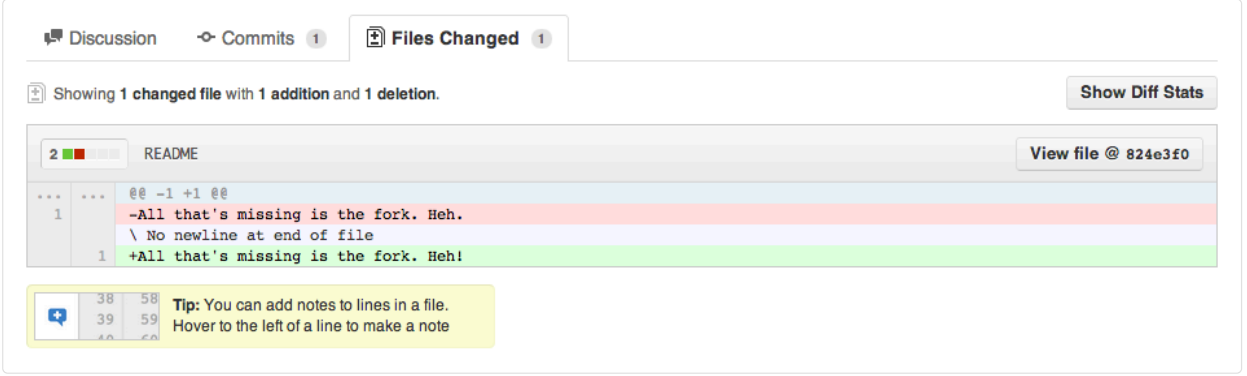
The pull request dashboard and the repository pull request list support a wide range of filtering and sorting controls. Use them to narrow down the list to the pull requests you're interested in.

Reviewing Proposed Changes

When you receive a pull request, the first thing to do is review the set of proposed changes. Pull requests are tightly integrated with the underlying git repository, so you can see exactly what commits would be merged should the request be accepted:



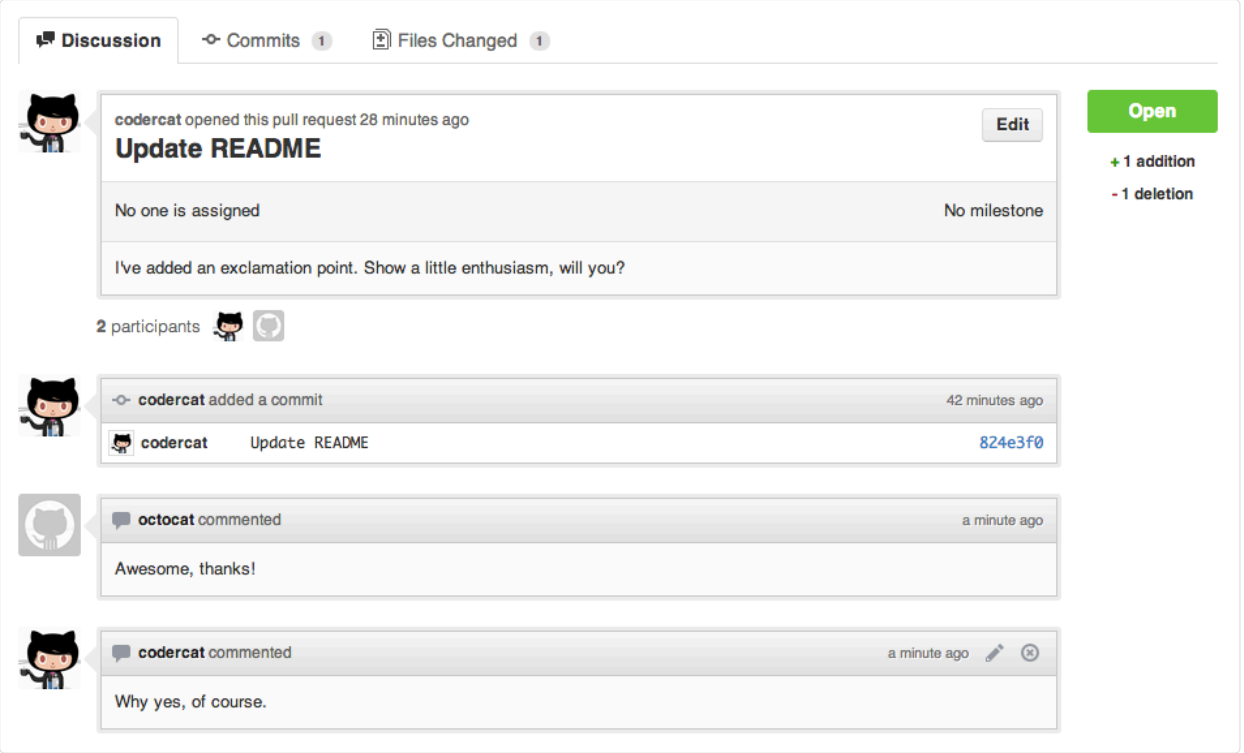
You can also review the cumulative diff of all file changes across all commits.



Pull Request Discussion

After reviewing the basic description, commits, and cumulative diff, the person tasked with applying the changes may have questions or comments. Perhaps the coding style doesn't match project guideline, or the change is missing unit tests, or maybe everything looks great and

some props are in order. The discussion view is designed to encourage and capture this type of discussion.



The discussion view starts with the pull request's original title and description and then captures additional activity to display chronologically from there. Any of the following types of activity are captured as they happen:

Comments left on the pull request itself.

Additional commits pushed to the pull request's branch.

File and line notes left on any of the commits included in the pull request's range.

Pull request comments are Markdown compatible, so you can embed images, use preformatted text blocks, and other formatting supported by Markdown.

Merging a Pull Request

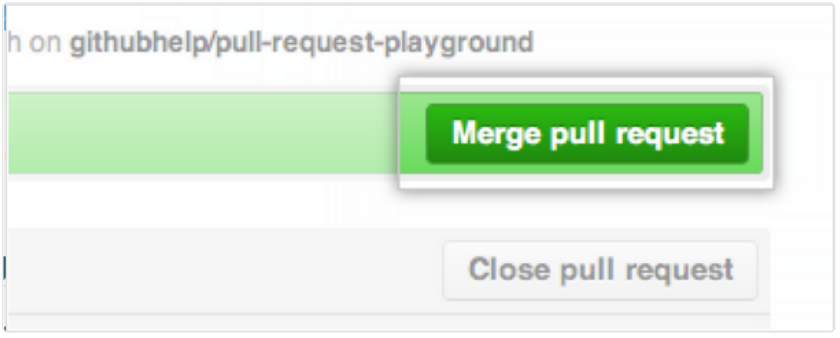
Once a pull request is deemed satisfactory, anyone with push access to the destination repository can perform the merge. There are a variety of ways to accomplish this. Two popular methods are described below.

Merge directly on Github

If the merge will not have any conflicts, you can merge the pull request online without ever typing a single git command.

Open the pull request's page

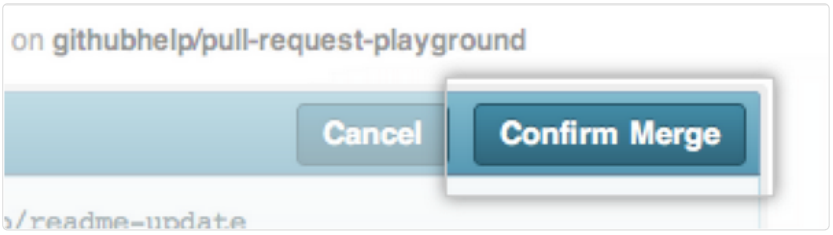
Click the "Merge pull request" button



Enter a commit message



Click “Confirm Merge”



TortoiseGit and Github workflow for T3000_Building_Automation_System project

Before reading this chapter, make sure you have read and understood chapter 6. This chapter only builds on the concepts explained in chapter 6 and makes use of those concepts w.r.t. T3000_Building_Automation_System project. This chapter would be mostly containing screen shots of T3000 project and related steps.

Github Terms:

Base repo: temcocontrols/T3000_Building_Automation_System. Base repo is the main repository of the code. This shall correspond to the finalized work of the team.

Fork repo: Each fork of the main repo corresponds to a team member’s work. In my case, this repo is: jay-github/ T3000_Building_Automation_System

Branch: Each branch within the fork and/or in the main repo can correspond to several kinds of things depending on how you want to work. Usually, each branch refers to a version of the project but can also correspond to different channels of development, like hotfixes or experimental work.

Pull Request: The pull request (in the Github ecosystem) corresponds to the task. Every time I want to contribute an isolated finished task to the main repo, I create a pull request corresponding to the commits made in that task. These commits are pulled from my fork to the main repo.

The ecosystem for Temco Controls software development is proposed as follows:

temcocontrols/T3000_Building_Automation_System remains the base repo controlled from a single point.

Each developer will fork his or her own repo. With this logic, my repository would be jay-github/ T3000_Building_Automation_System.

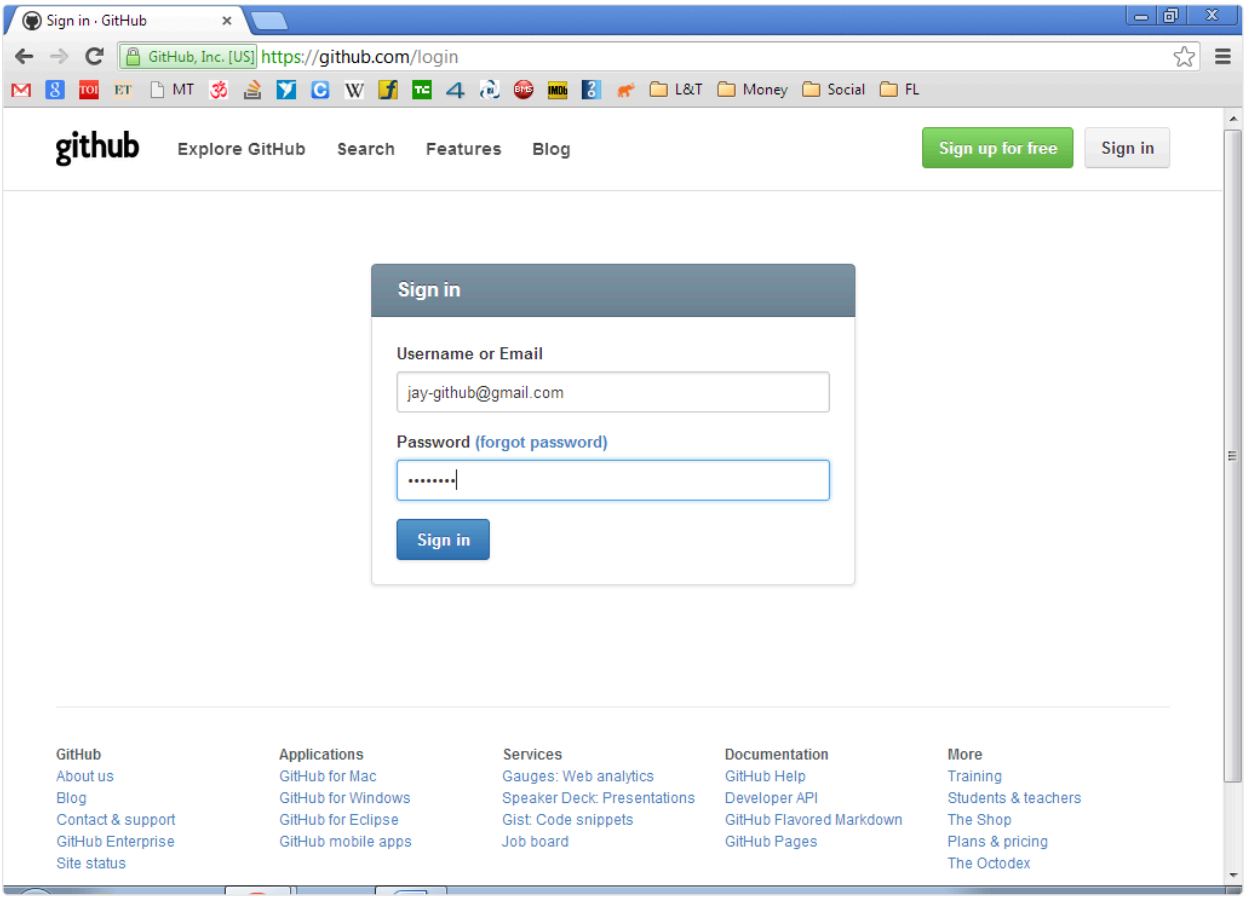
After forking the repo, each developer shall clone that repo in his own local PC. Work there and Push the changes back to his fork.

Once the changes are pushed to fork, developer shall send a Pull Request to temcocontrols on github.

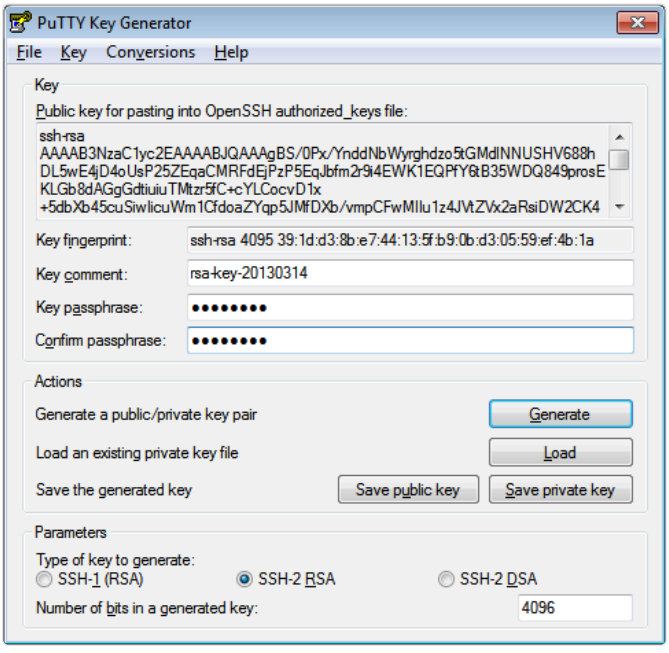
The single person controlling temcocontrols repo would be able to review the work done by developer and also able to accept and merge or reject the pull request.

Let’s see how this can be done.

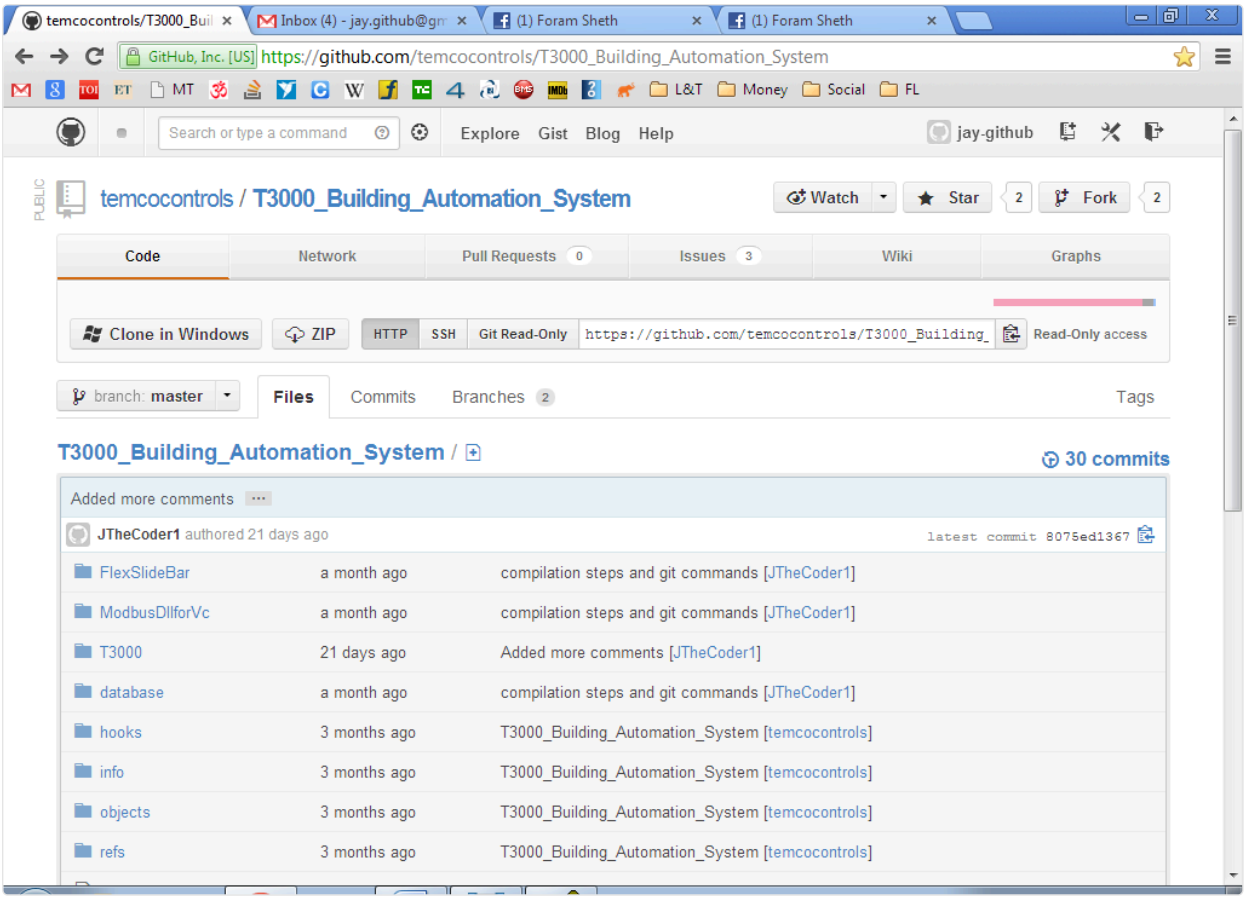
Sign in to the developer account.



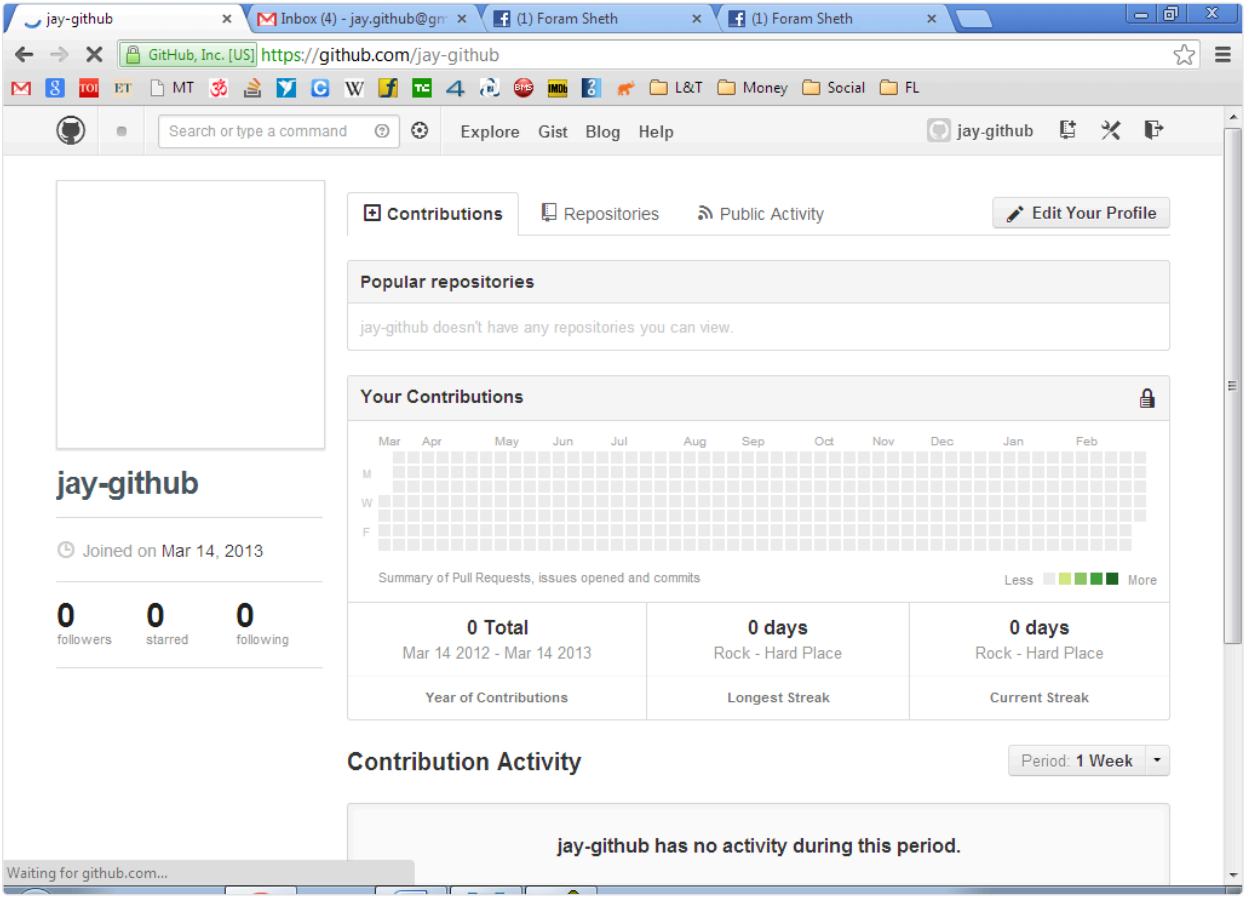
Generate SSH key.



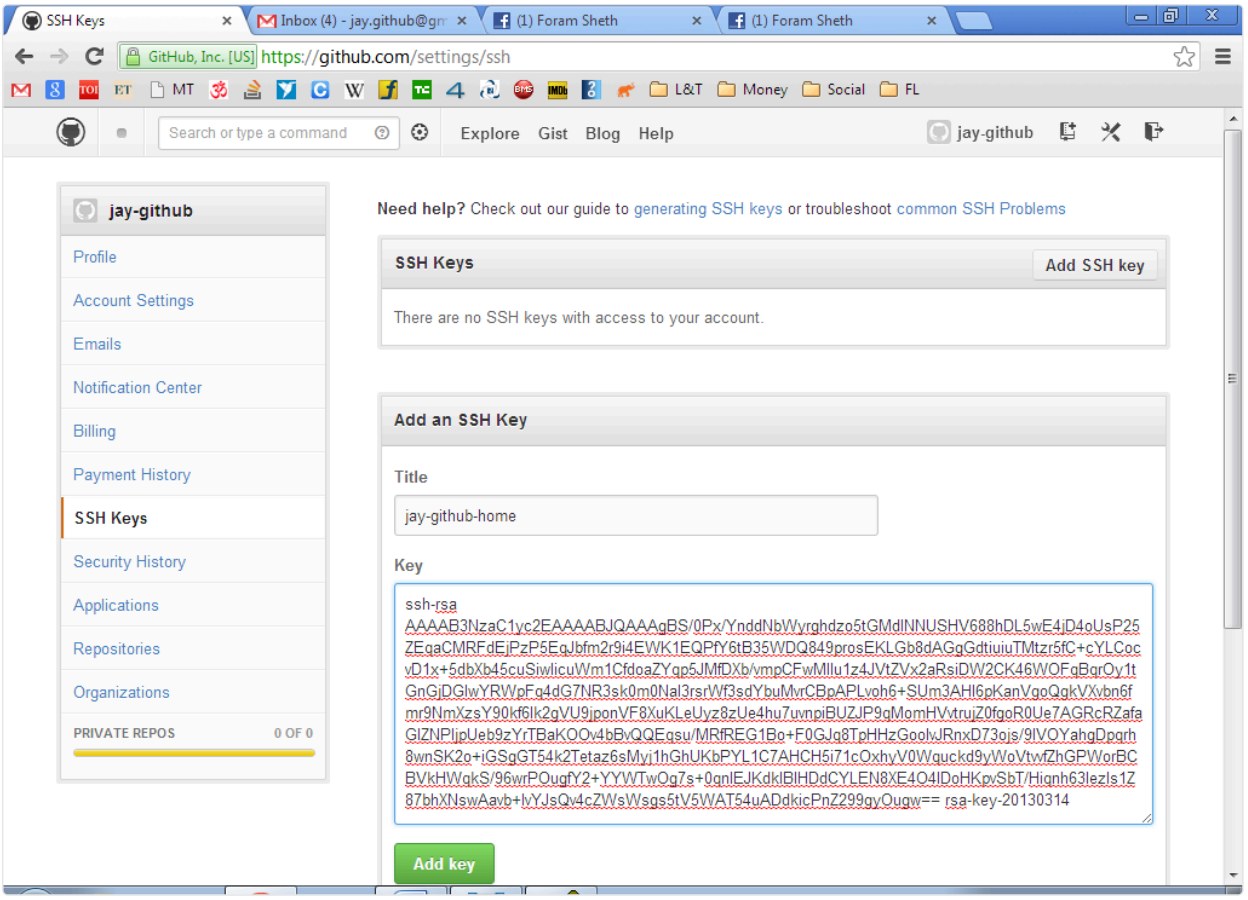
Screen below shows signed in home page for developer.



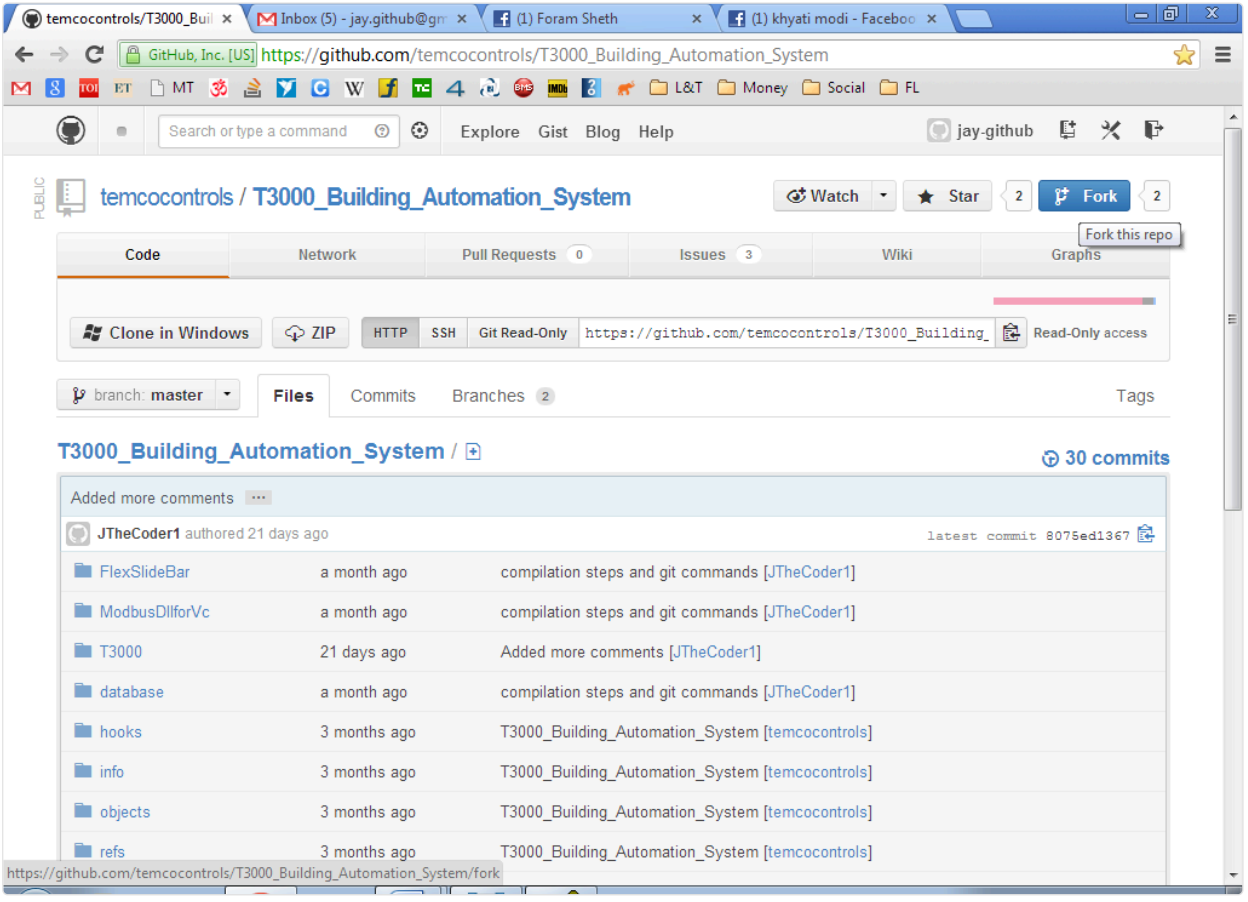
Go to Edit Your Profile.



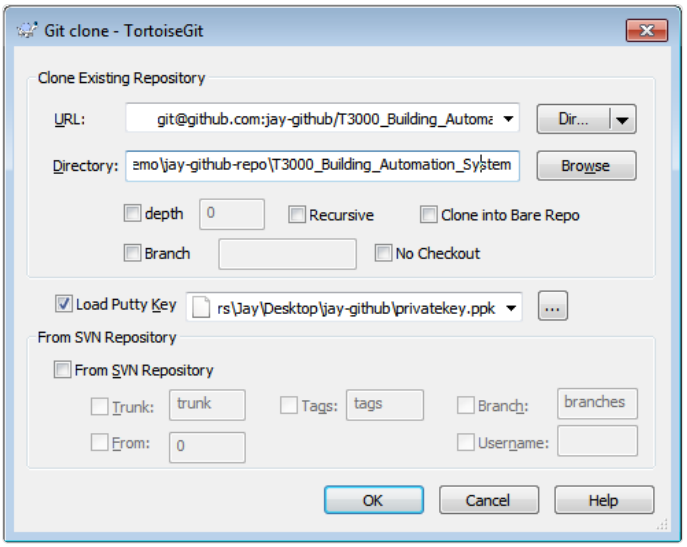
Add SSH key.



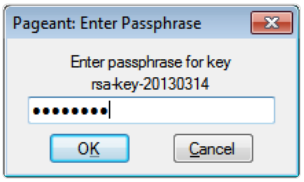
Fork temcocontrols/T3000... project



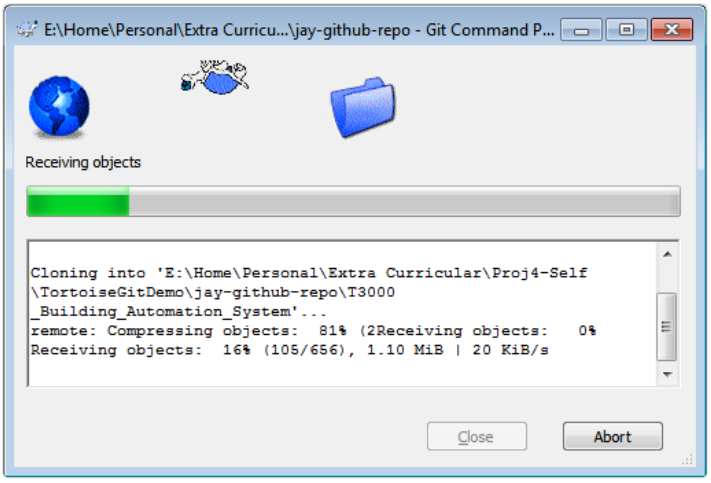
Clone fork into local PC using TortoiseGit.



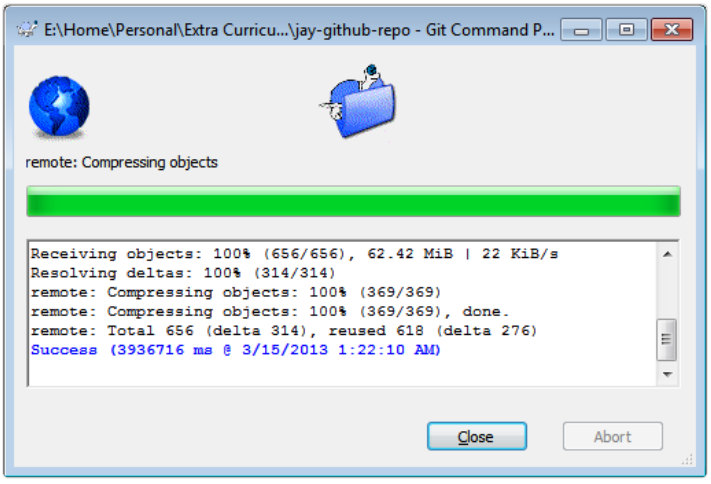
Enter passphrase.



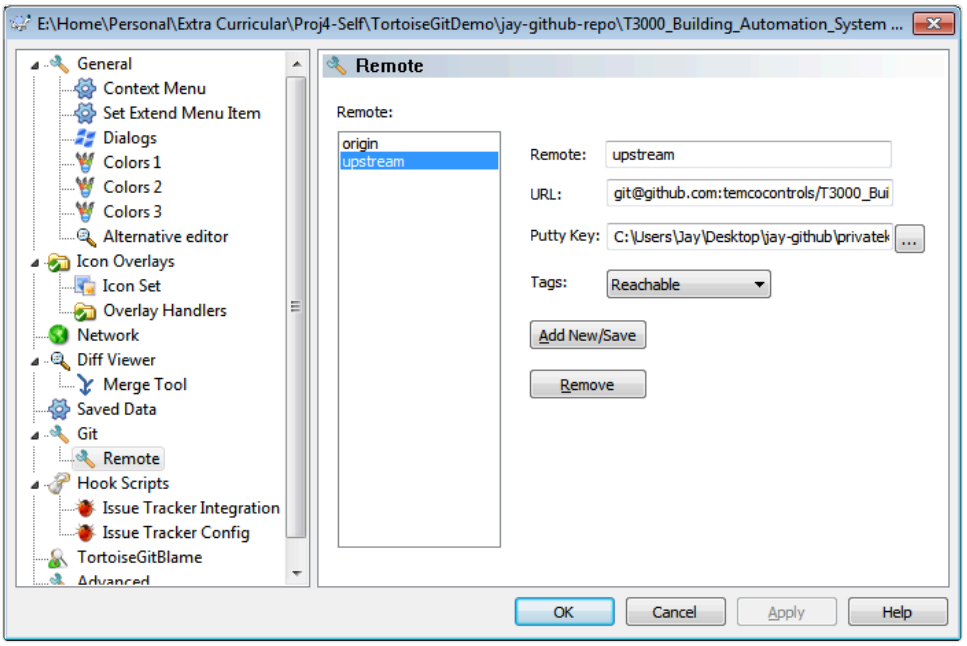
Cloning may take some time.



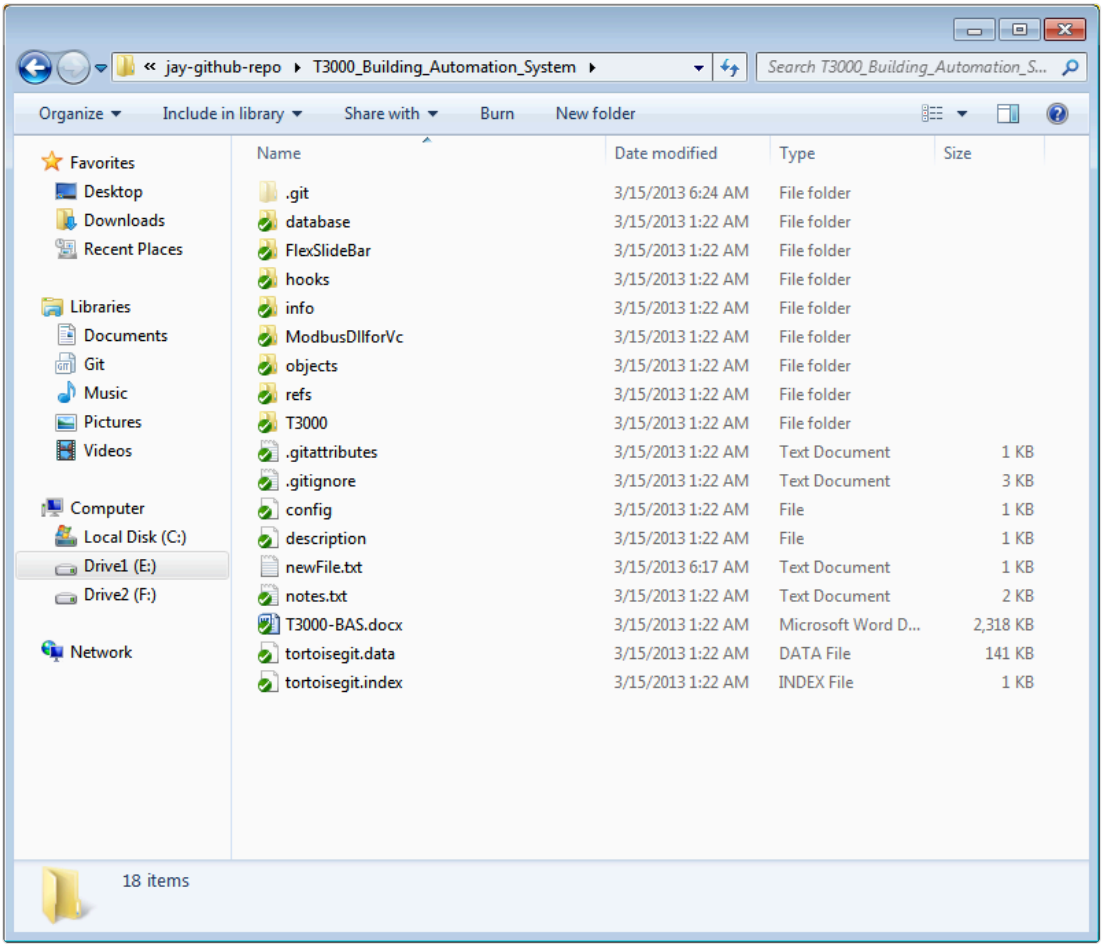
Eventually it will get over.



Enter details of main repo (temcocontrols/T3000... repo)

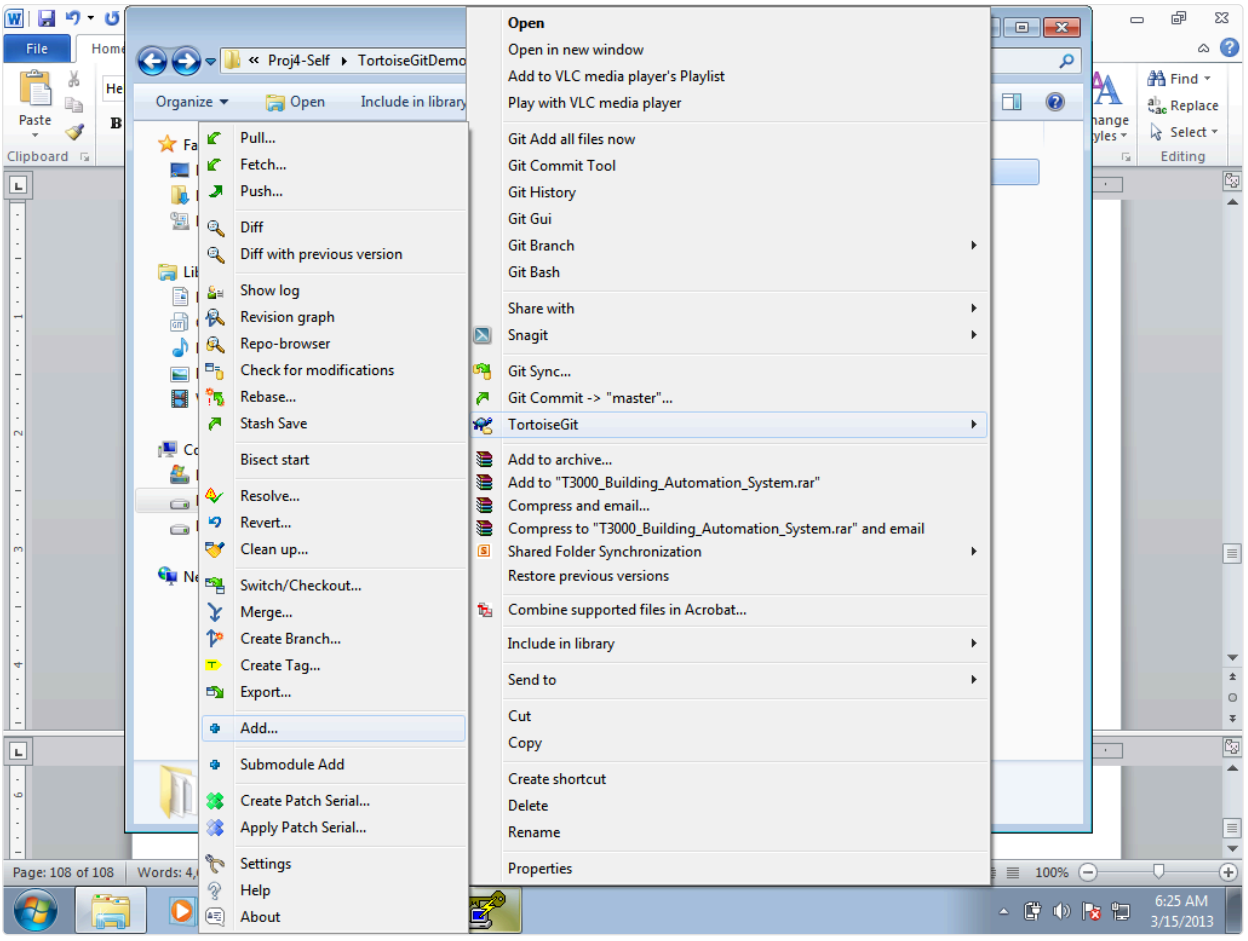


Here, for demo purpose, I have added newFile.txt in the repo directory on my local PC.

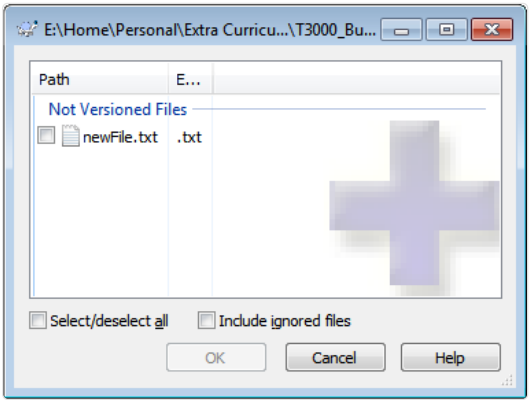


Now, it's time to send my changes to single person controlling temcocontrols repo. This is done by first committing changes, pushing them to own fork and then sending a pull request to temcocontrols.

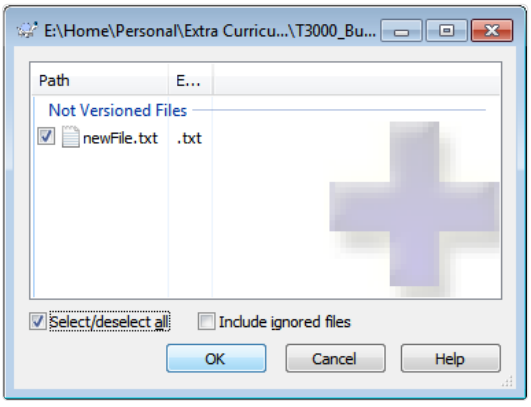
Add... is shown below.



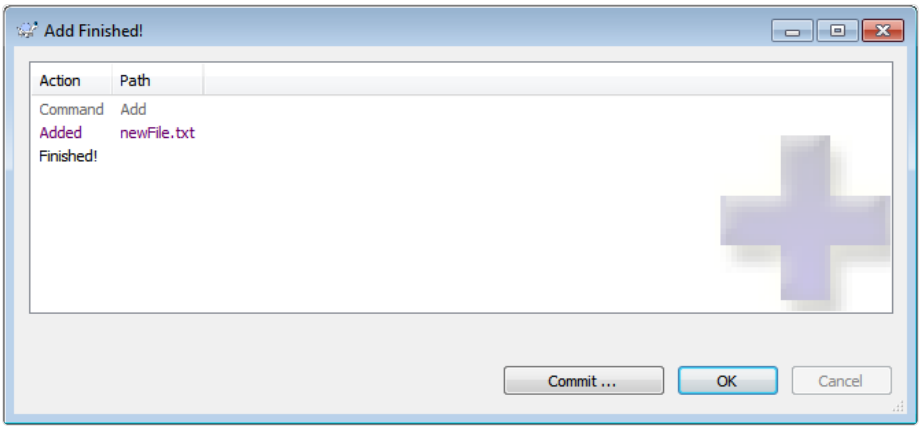
TortoiseGit confirms what needs to be added.



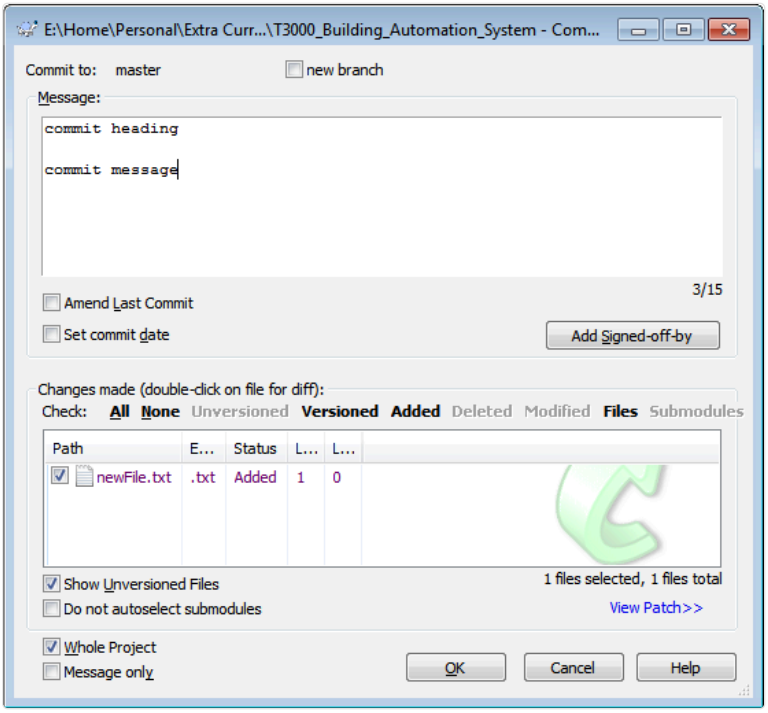
Select files to be added and click ok.



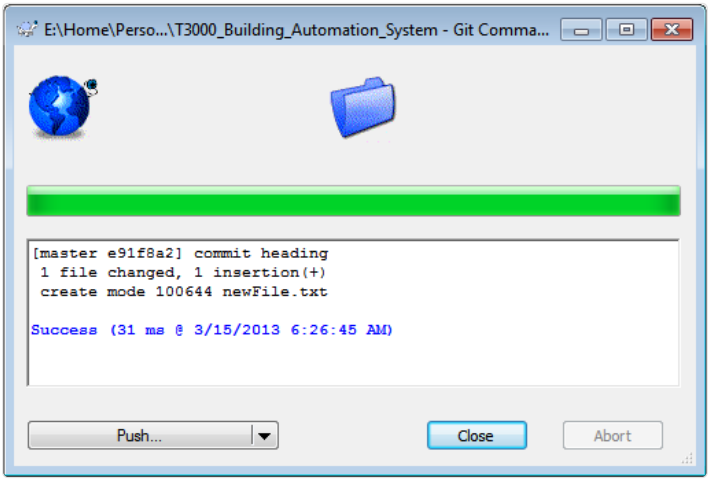
File is added and click on Commit... to commit changes on local PC.



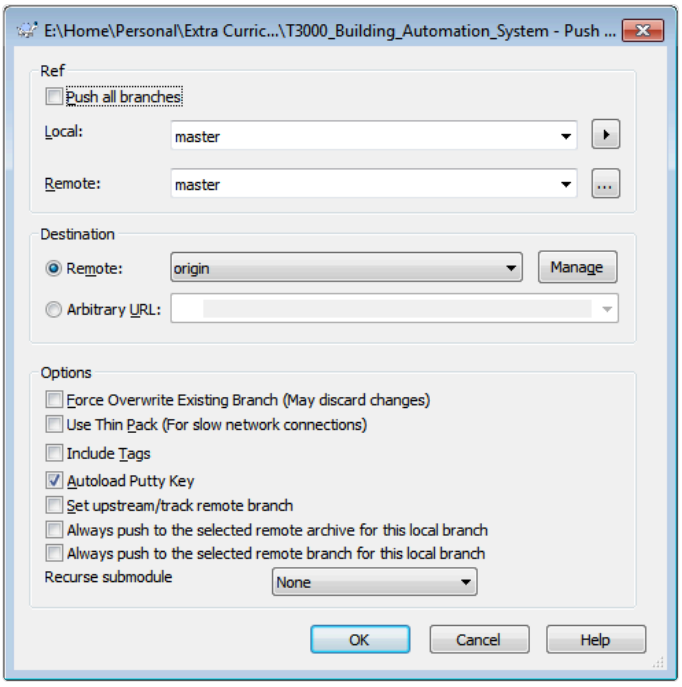
Commit dialog box appears. Enter commit message and click ok.



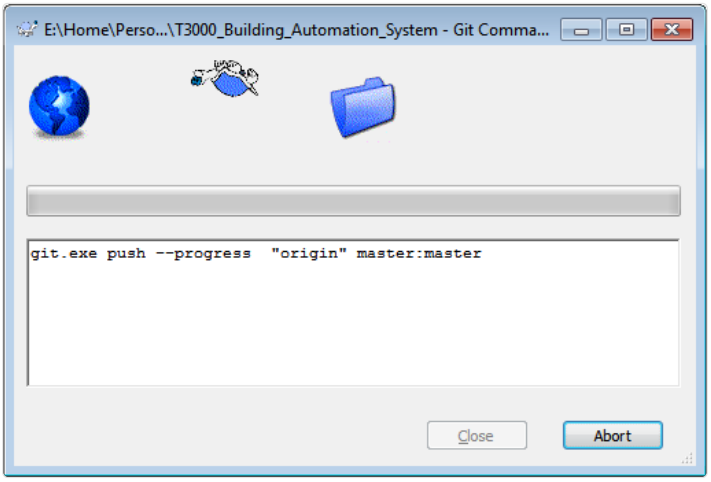
Once committed, if you want to send changes to your fork on github, click Push... or click Close.



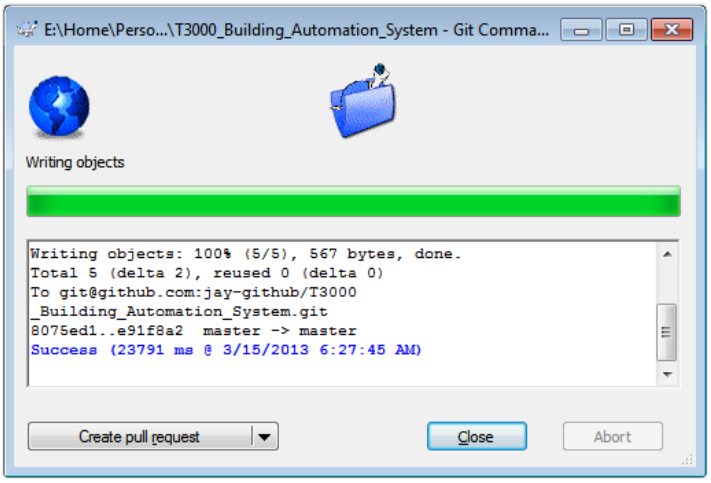
Select Destination as origin. Not upstream. Else changes would go to main repo instead of your fork and click ok.



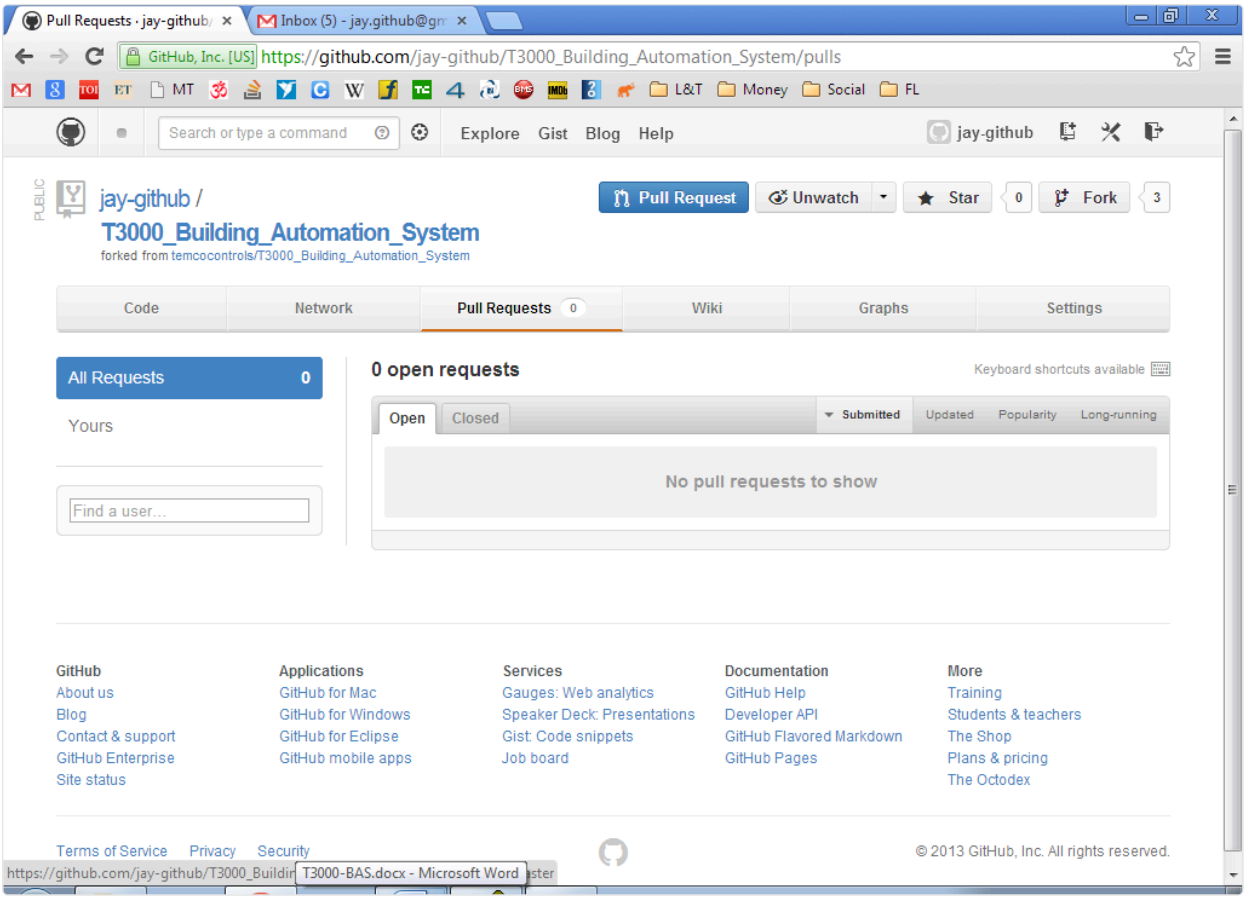
Some waiting...



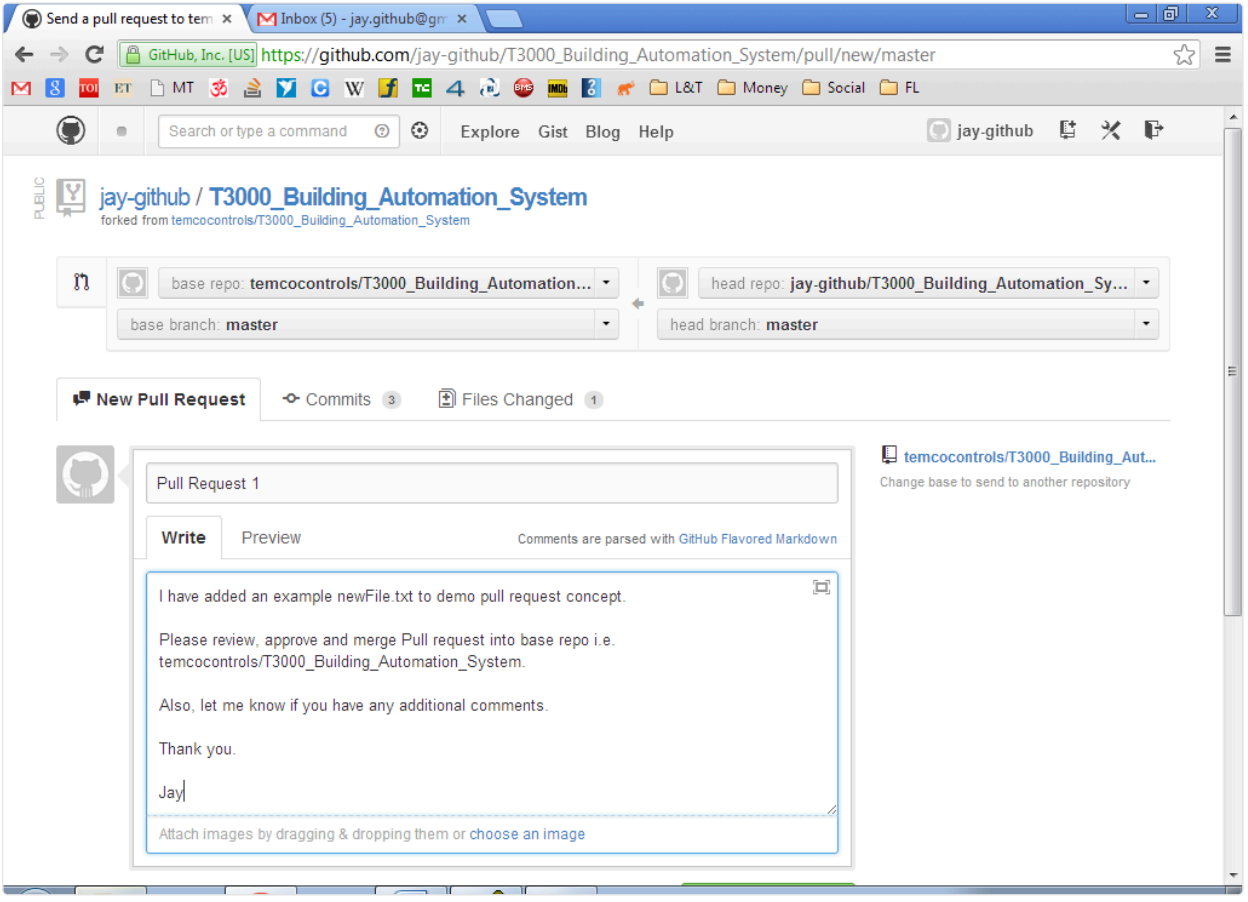
And pushing is done.



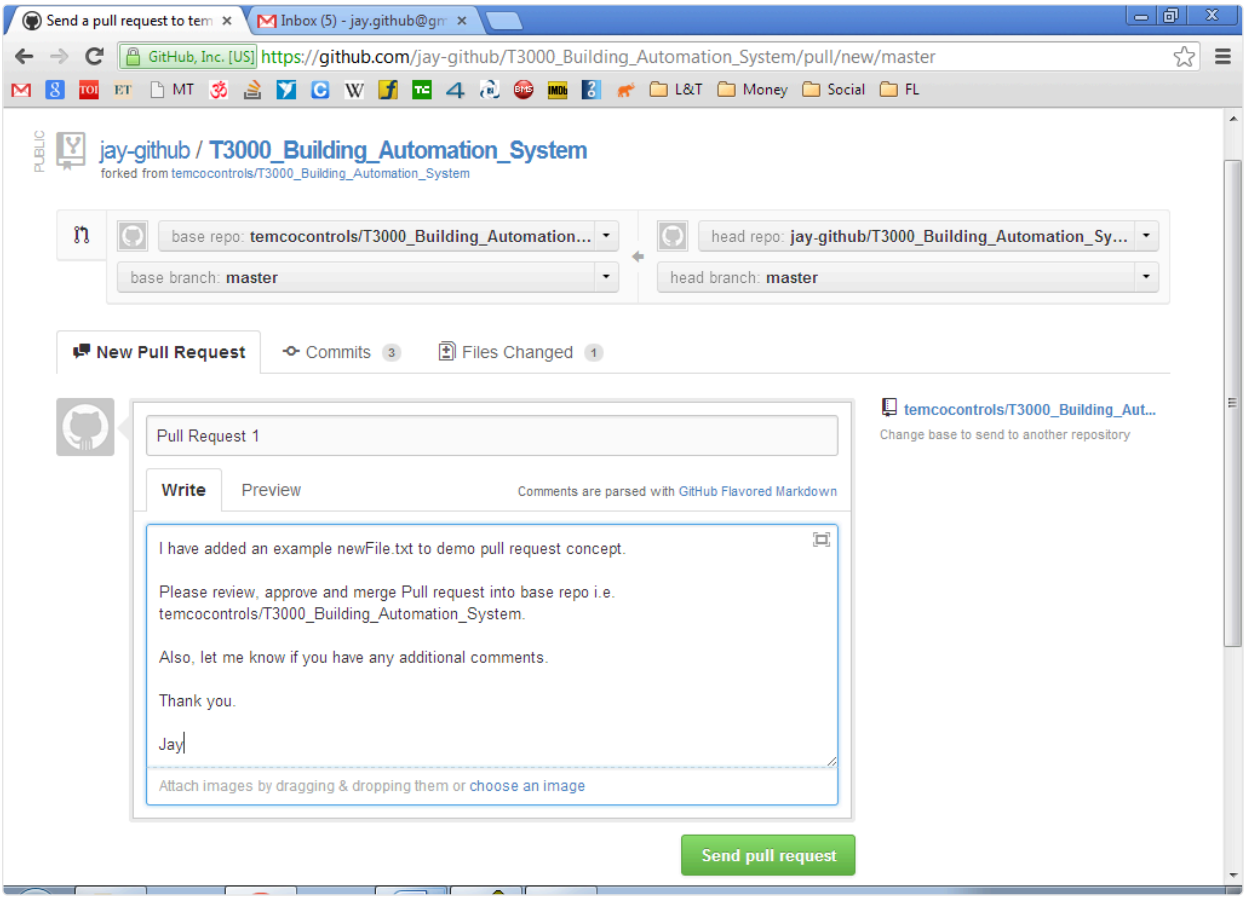
Now notify temcontrols of the changes by clicking Pull Request on github website.



Enter a title and a message.

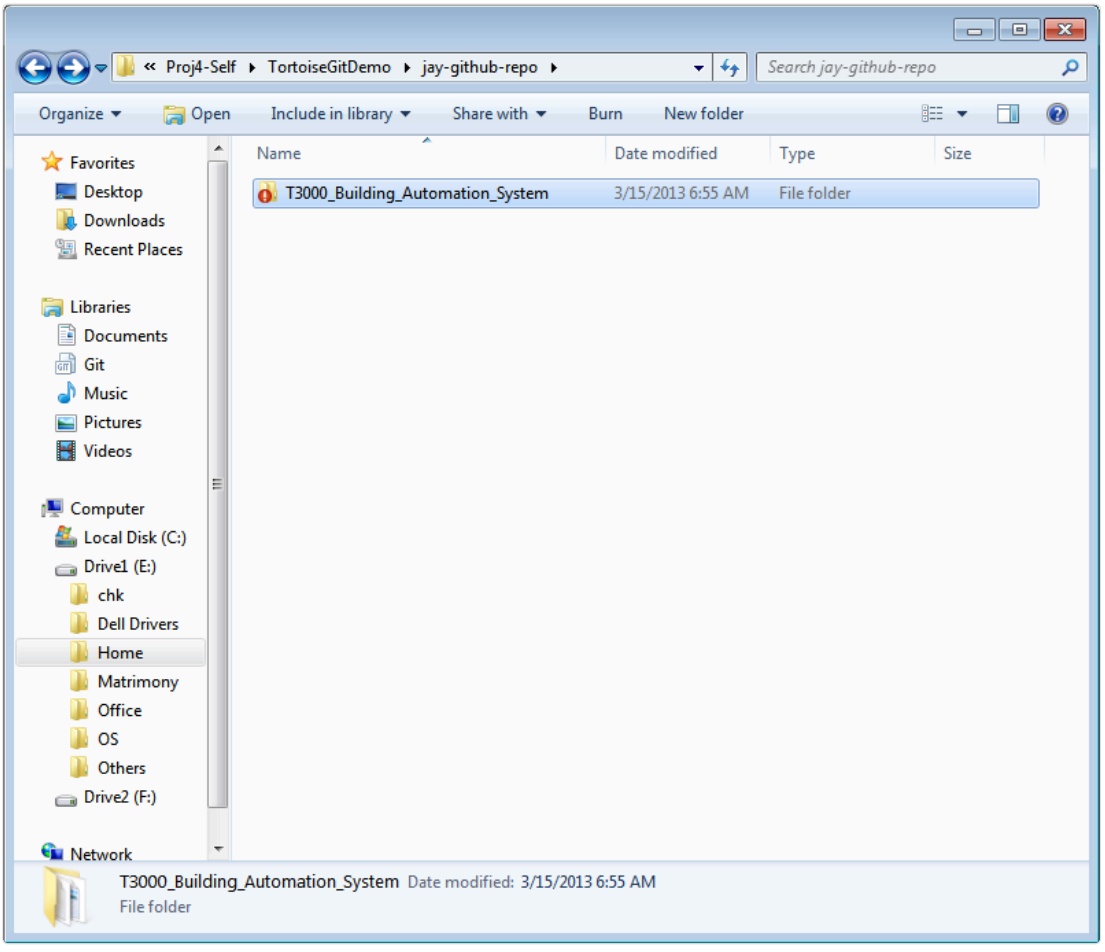


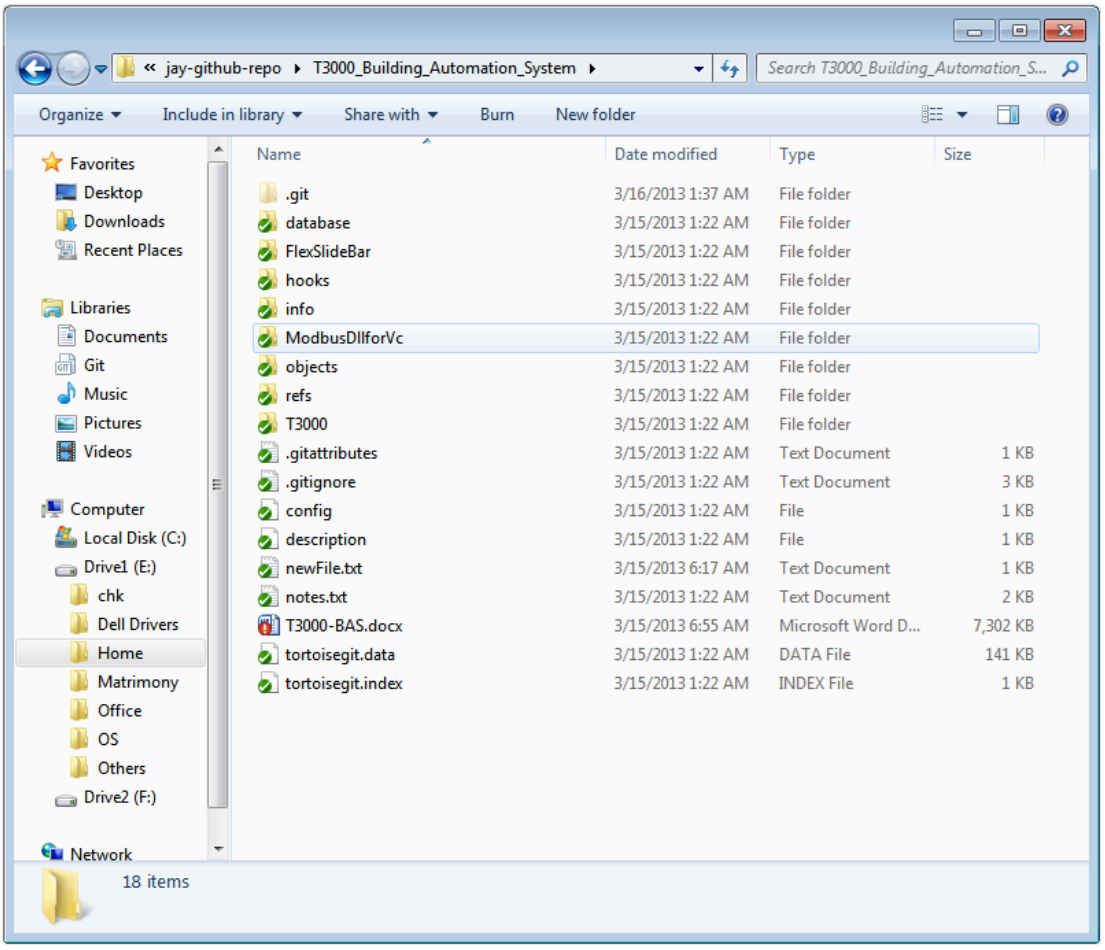
Click on Send Pull Request.



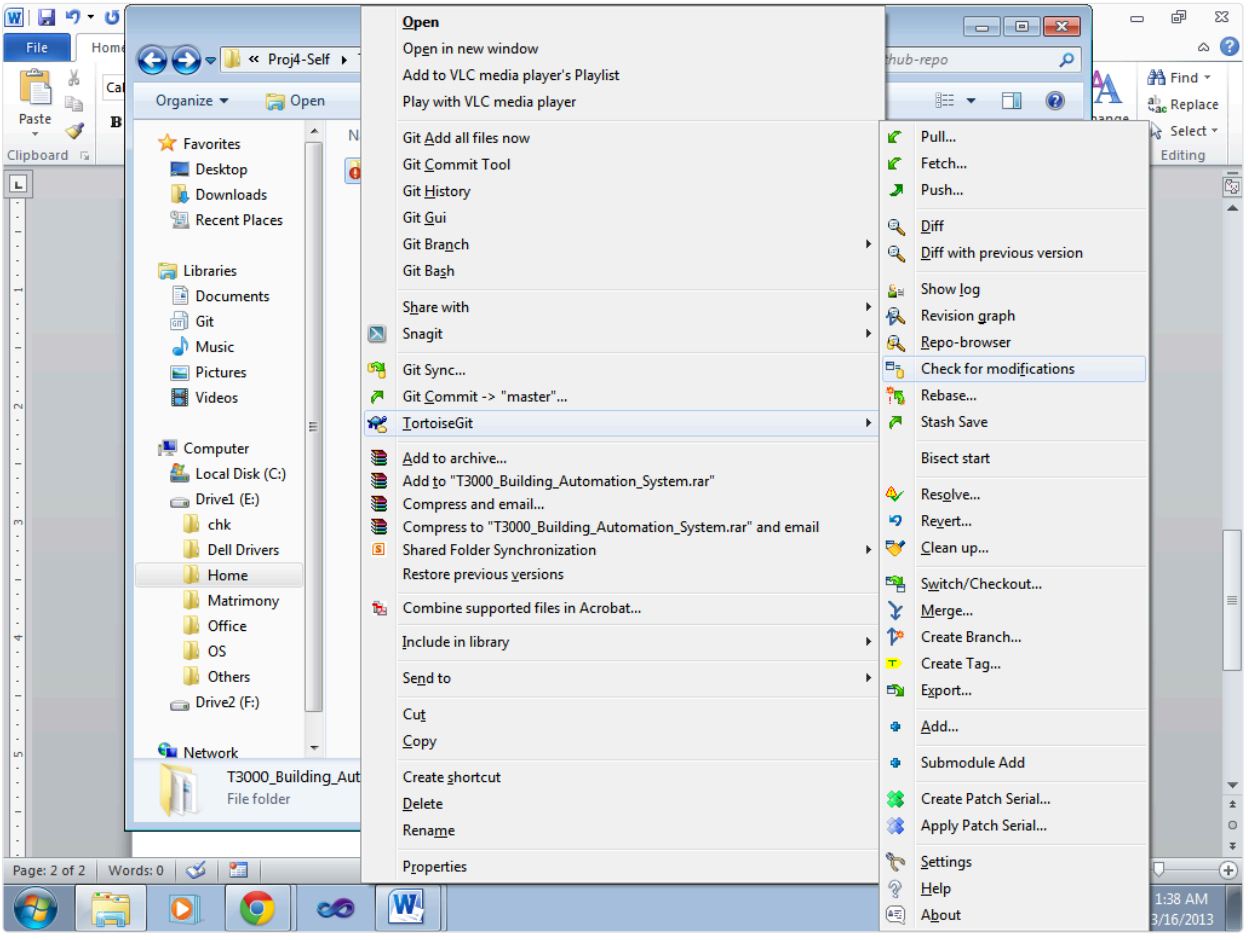
Now, single person controlling main repo shall be able to merge the pull request as described in last section of chapter 6.

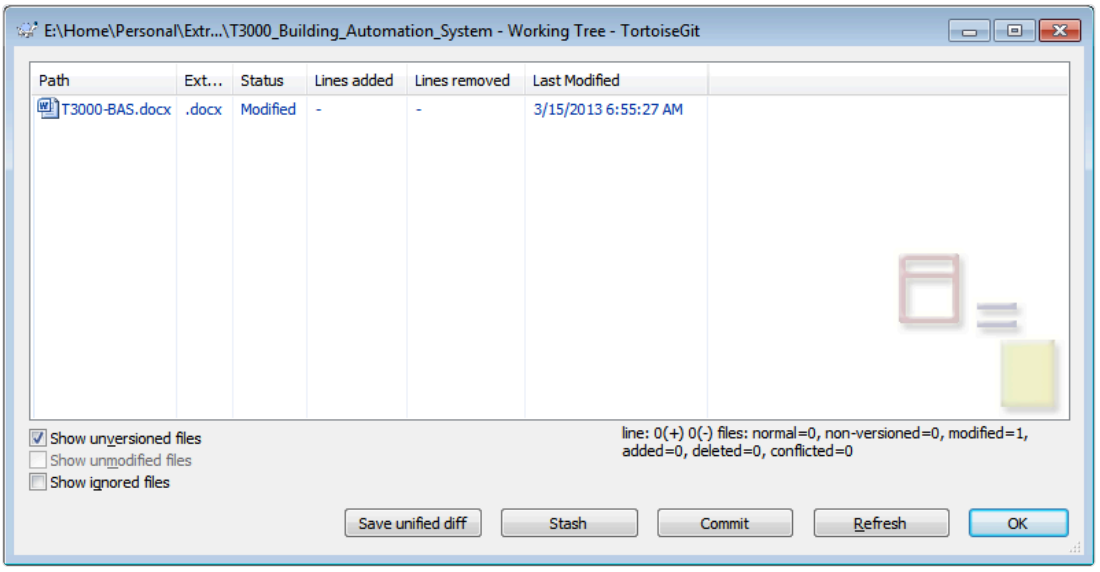
Following are screen shots of adding modifications. (In case Add... does not work)



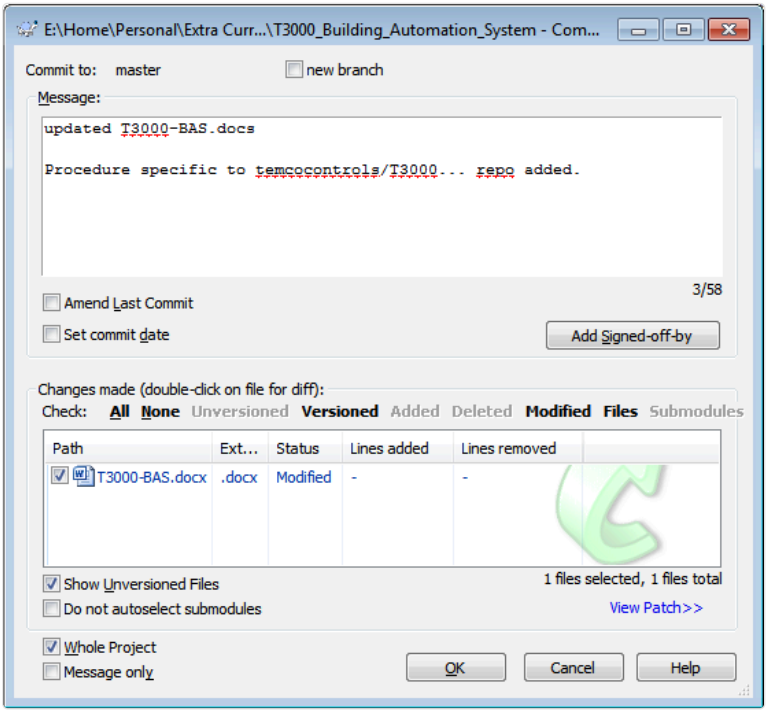


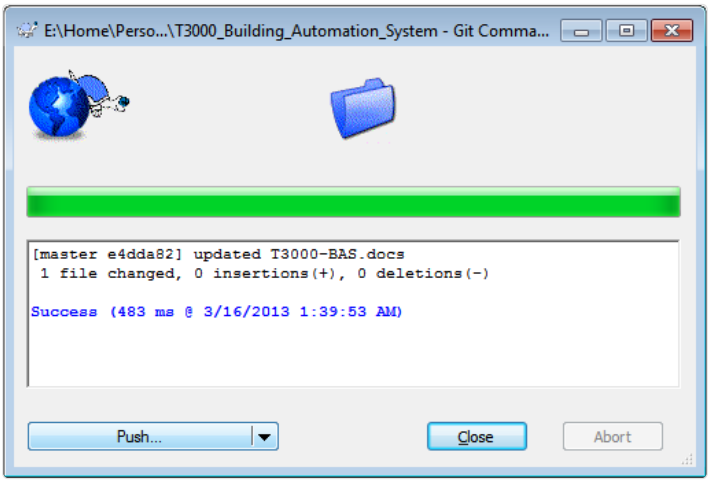
Click on TortoiseGit -> Check for modifications

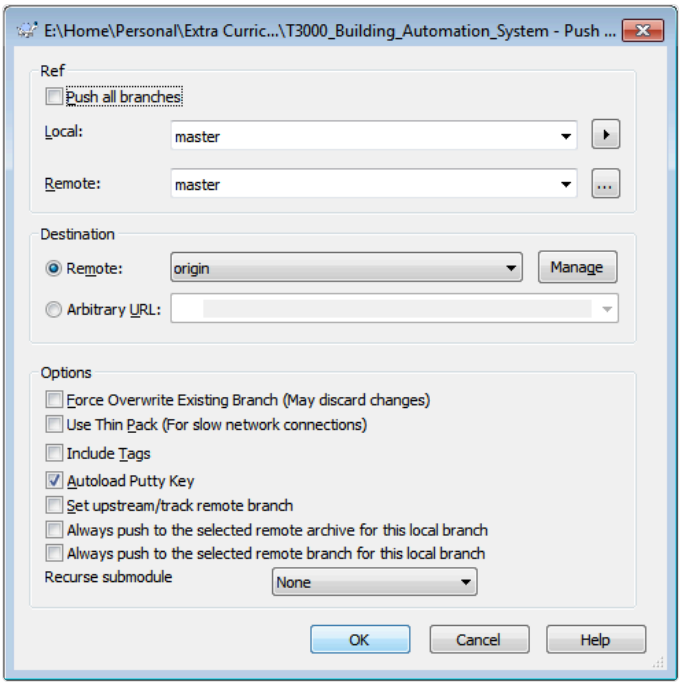


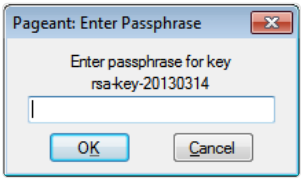


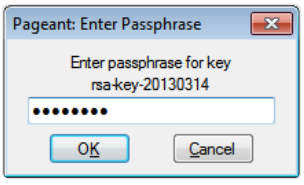
Click on Commit.

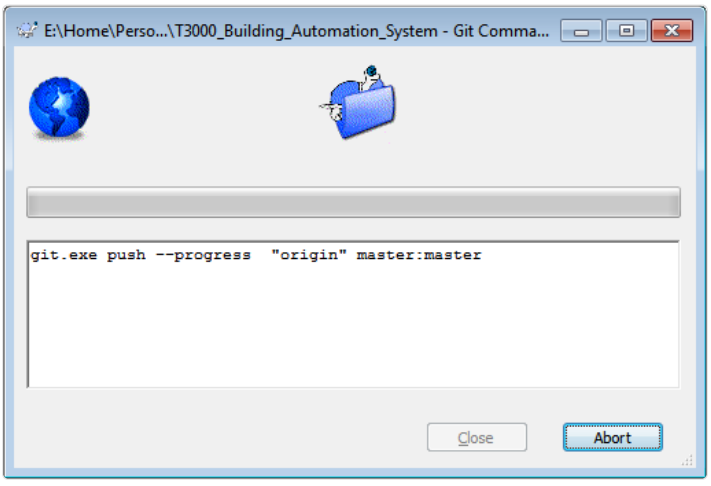


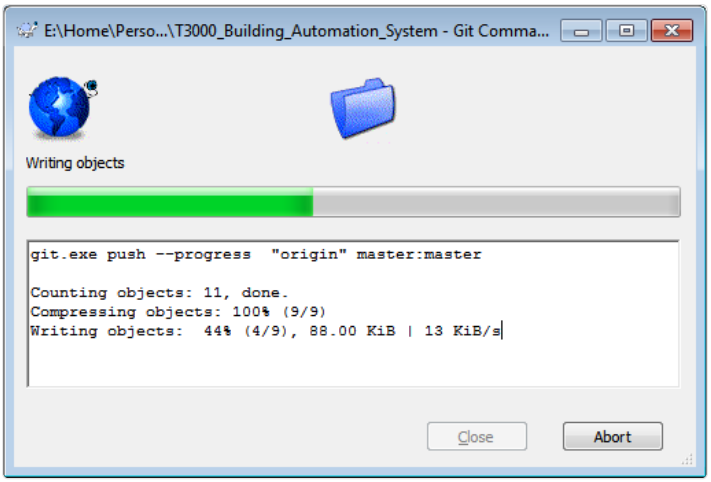


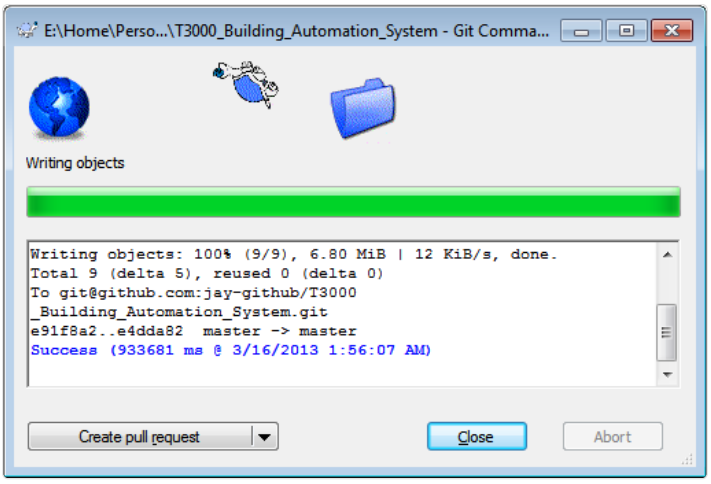

E:\Home\Personal\Extra Curric...\T3000_Building_Automation_System - Push ...
Ref
Push all branches
Local:
master
Remote:
master
Destination
Remote:
origin
Manage
Arbitrary URL:
Options
Force Overwrite Existing Branch (May discard changes)
Use Thin Pack (For slow network connections)
Include Tags
Autoload Putty Key
Set upstream/track remote branch
Always push to the selected remote archive for this local branch
Always push to the selected remote branch for this local branch
Recurse submodule
None
OK
Cancel
Help


Pageant: Enter Passphrase
Enter passphrase for key
rsa-key-20130314
OK
Cancel

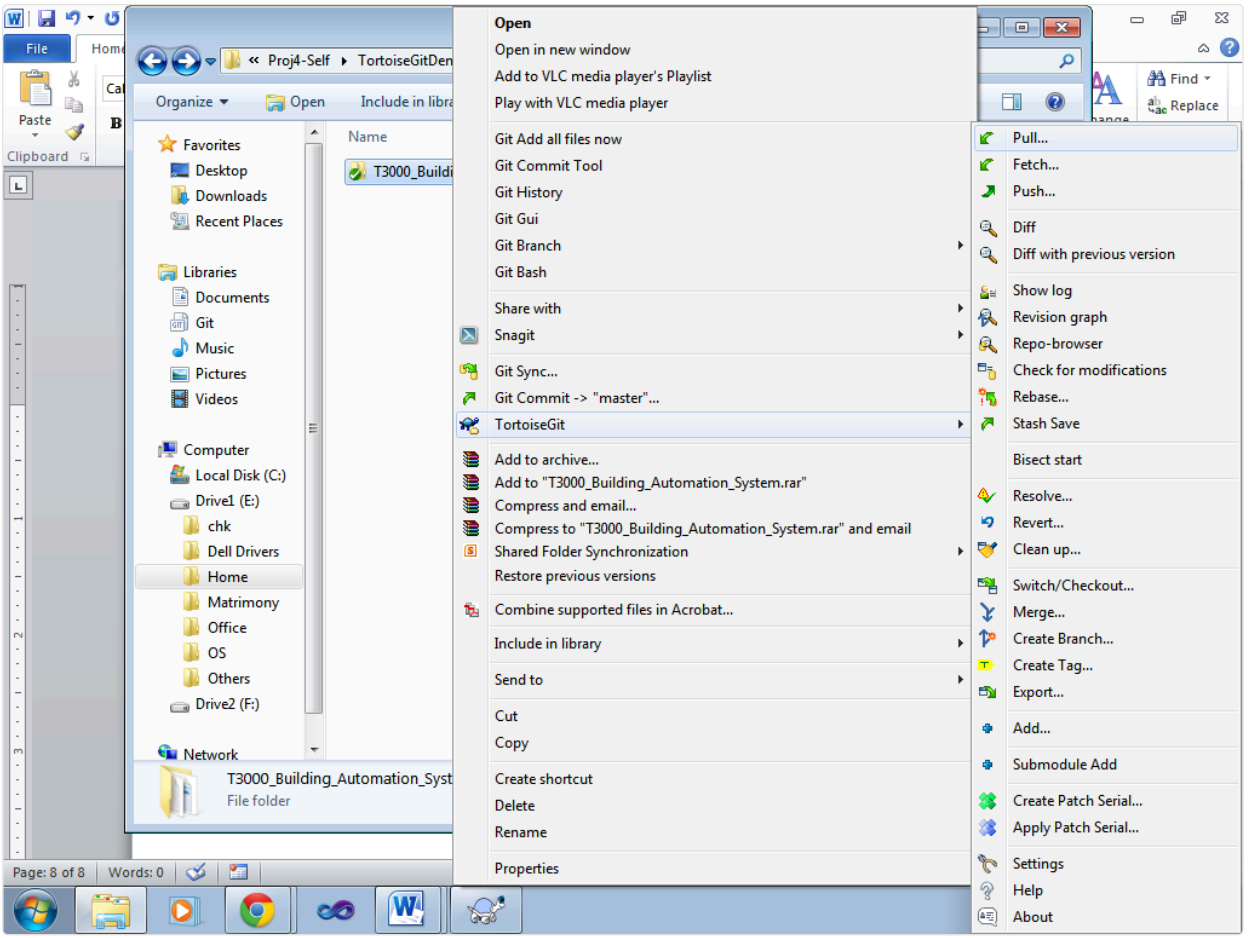

Pageant: Enter Passphrase
Enter passphrase for key
rsa-key-20130314
OK
Cancel

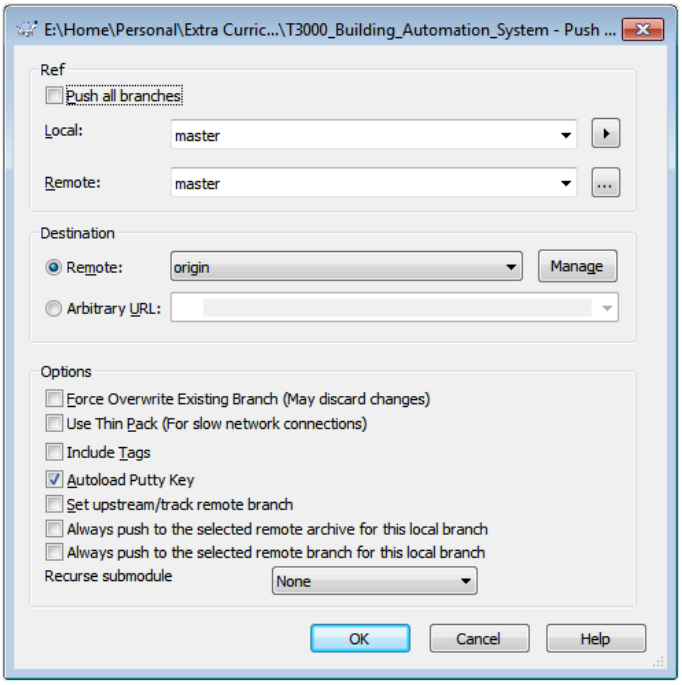

E:\Home\Perso...\T3000_Building_Automation_System - Git Comma...
git.exe push --progress "origin" master:master
Close
Abort

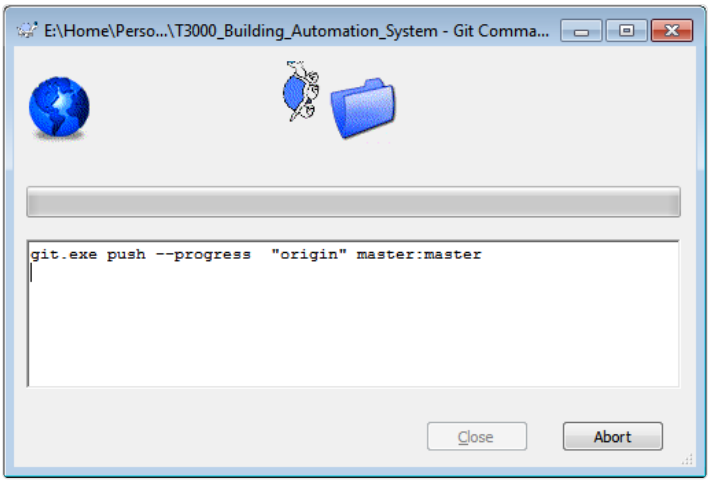


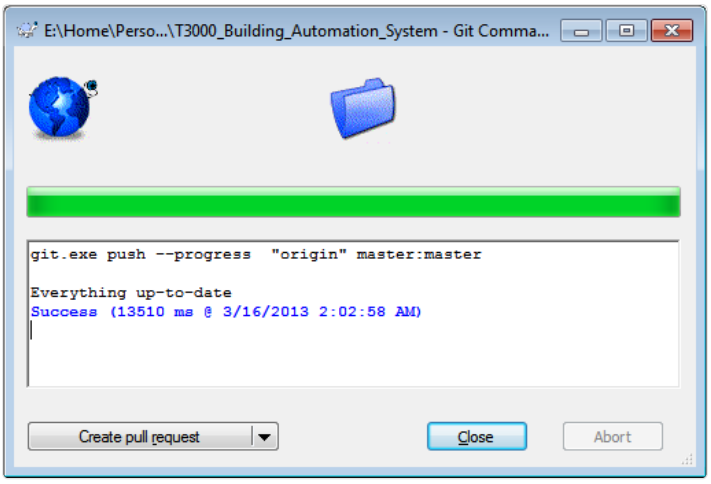


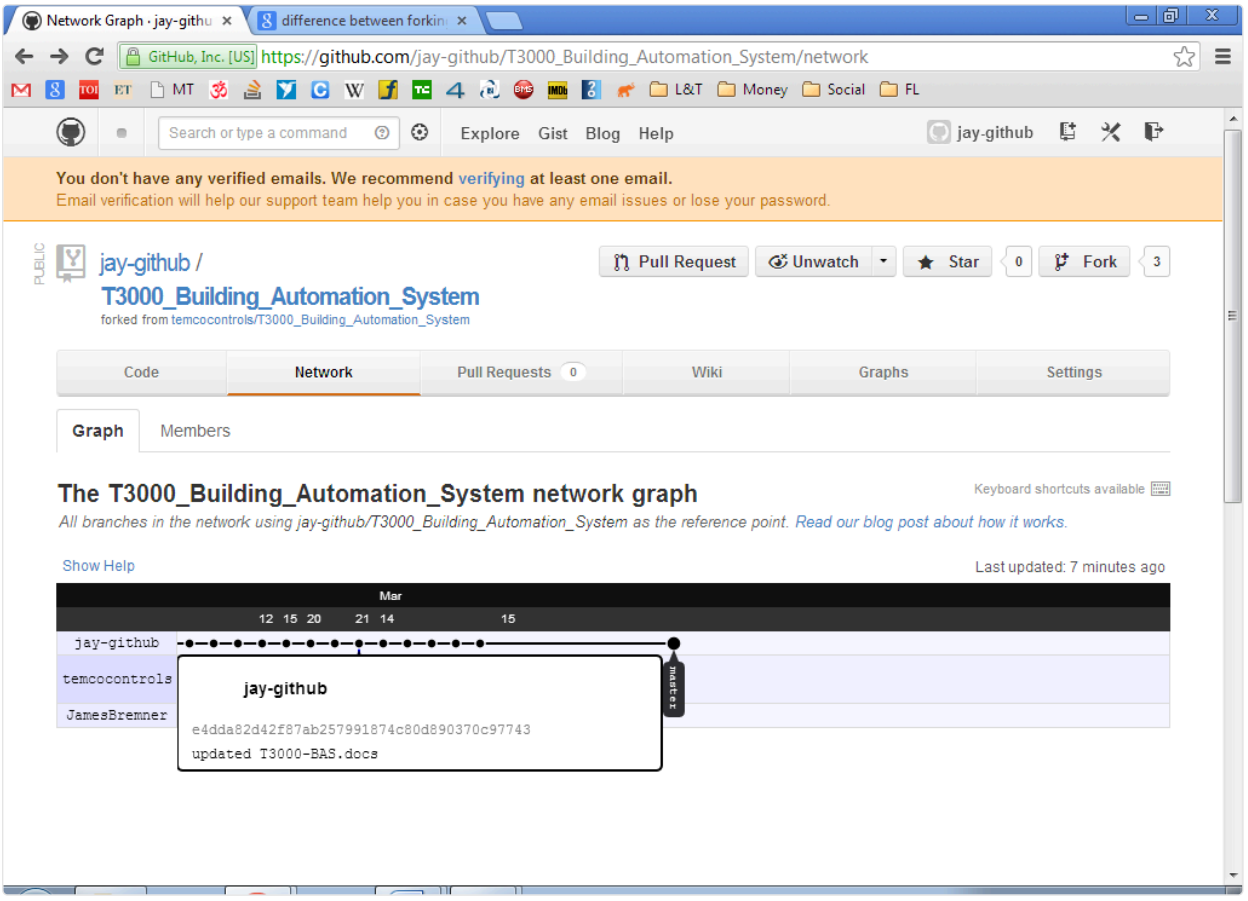
Push to github. TortoiseGit -> Push...

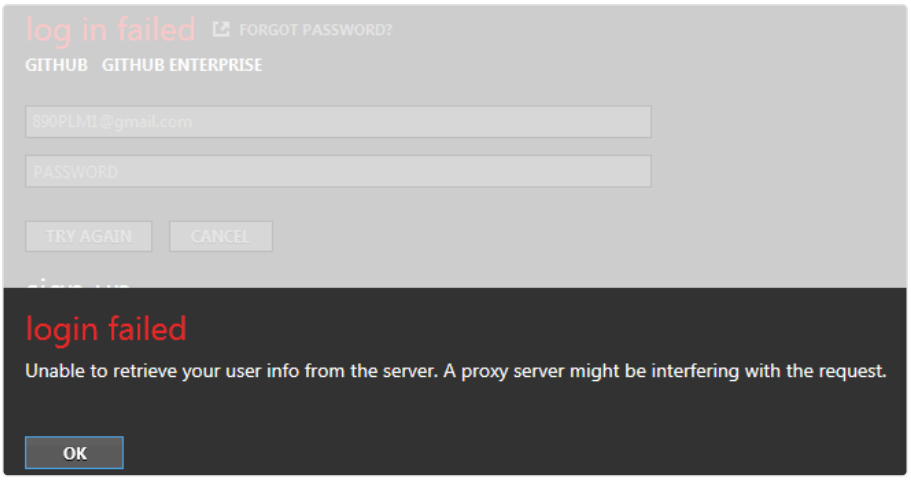

Open
Open in new window
Add to VLC media player's Playlist
Play with VLC media player
Git Add all files now
Git Commit Tool
Git History
Git Gui
Git Branch
Git Bash
Share with
Snagit
Git Sync...
Git Commit -> "master"...
TortoiseGit
Add to archive...
Add to "T3000_Building_Automation_System.rar"
Compress and email...
Compress to "T3000_Building_Automation_System.rar" and email
Shared Folder Synchronization
Restore previous versions
Combine supported files in Acrobat...
Include in library
Send to
Cut
Copy
Create shortcut
Delete
Rename
Properties
Pull...
Fetch...
Push...
Diff
Diff with previous version
Show log
Revision graph
Repo-browser
Check for modifications
Rebase...
Stash Save
Bisect start
Resolve...
Revert...
Clean up...
Switch/Checkout...
Merge...
Create Branch...
Create Tag...
Export...
Add...
Submodule Add
Create Patch Serial...
Apply Patch Serial...
Settings
Help
About
Favorites
Desktop
Downloads
Recent Places
Libraries
Documents
Git
Music
Pictures
Videos
Computer
Local Disk (C:)
Drive1 (E:)
chk
Dell Drivers
Home
Matrimony
Office
OS
Others
Drive2 (F:)
Network
T3000_Building_Automation_Syst
File folder
Page: 8 of 8
Words: 0


E:\Home\Personal\Extra Curric...\T3000_Building_Automation_System - Push ...
Ref
Push all branches
Local:
master
Remote:
master
Destination
Remote:
origin
Manage
Arbitrary URL:
Options
Force Overwrite Existing Branch (May discard changes)
Use Thin Pack (For slow network connections)
Include Tags
Autoload Putty Key
Set upstream/track remote branch
Always push to the selected remote archive for this local branch
Always push to the selected remote branch for this local branch
Recurse submodule
None
OK
Cancel
Help


E:\Home\Perso...\T3000_Building_Automation_System - Git Comma...
git.exe push --progress "origin" master:master
Close
Abort


E:\Home\Perso...\T3000_Building_Automation_System - Git Comma...
git.exe push --progress "origin" master:master
Everything up-to-date
Success (13510 ms @ 3/16/2013 2:02:58 AM)
Create pull request
Close
Abort


Network Graph · jay-github
difference between forkin
GitHub, Inc. [US] https://github.com/jay-github/T3000_Building_Automation_System/network
Search or type a command
Explore Gist Blog Help
jay-github
You don't have any verified emails. We recommend verifying at least one email.
Email verification will help our support team help you in case you have any email issues or lose your password.
jay-github / T3000_Building_Automation_System
forked from temcocontrols/T3000_Building_Automation_System
Pull Request Unwatch Star 0 Fork 3
Code Network Pull Requests 0 Wiki Graphs Settings
Graph Members
The T3000_Building_Automation_System network graph
Keyboard shortcuts available
All branches in the network using jay-github/T3000_Building_Automation_System as the reference point. Read our blog post about how it works.
Show Help
Last updated: 7 minutes ago
Mar
12 15 20 21 14 15
jay-github
temcocontrols
JamesBremner
jay-github
e4dda82d42f87ab257991874c80d890370c97743
updated T3000-BAS.docs
master



https://github.com/gitextensions/gitextensions/issues/2044

download and install

http://download.microsoft.com/download/2/B/F/2BF4D7D1-E781-4EE0-9E4F-FDD44A2F8934/NDP40-KB2468871-v2-x86.exe

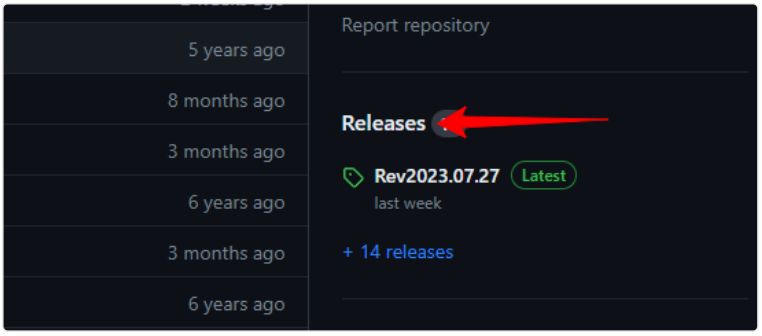
Creating a release

Turning a build into a published release: the tag, the assets that go with it, and what a customer downloads.

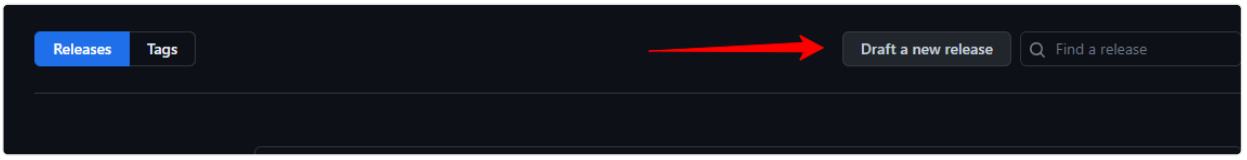
How to create a new release in github:

1- Go to the github repository and click release:

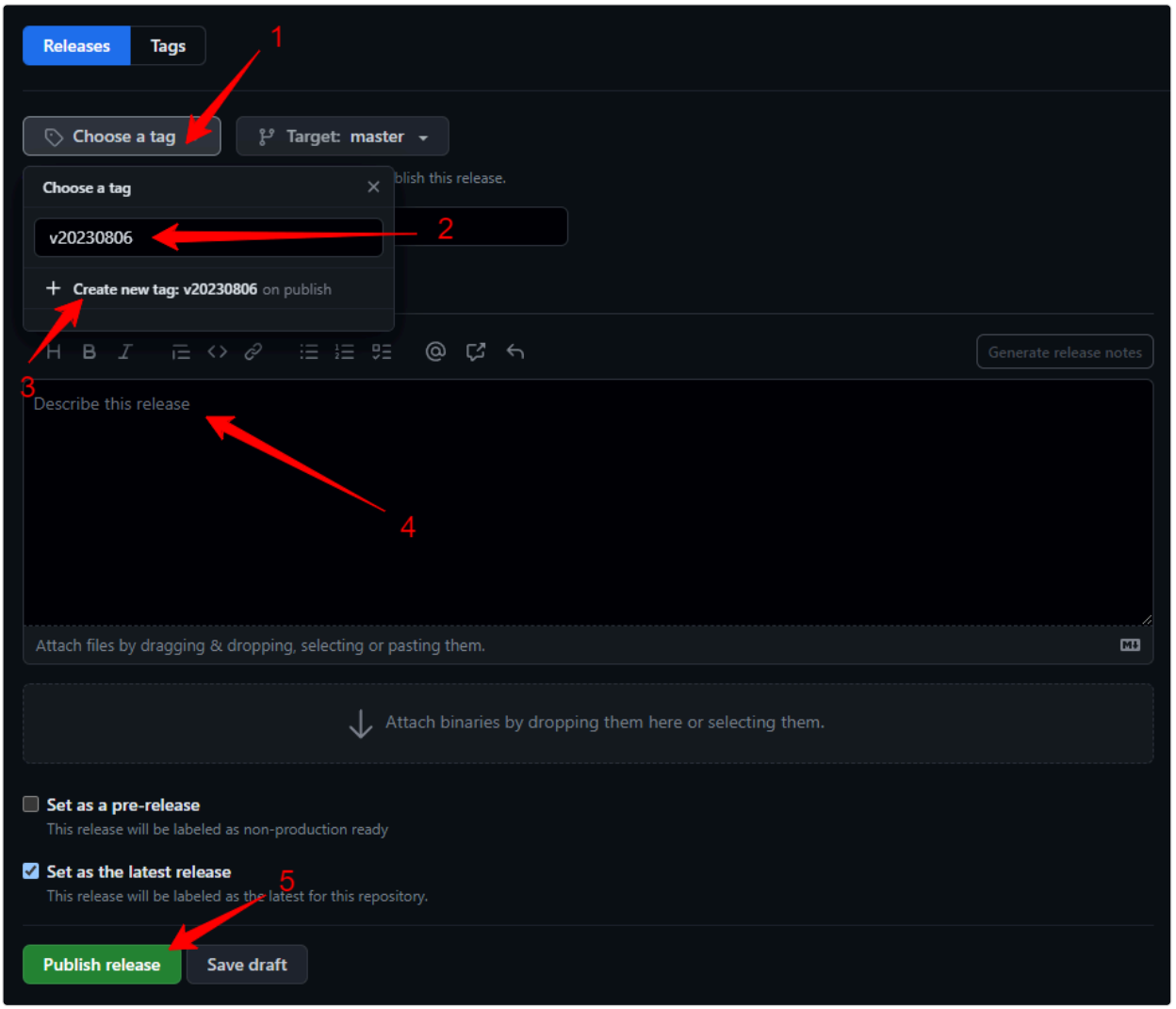
https://github.com/temcocontrols/T3000_Building_Automation_System



2- Click Draft a new release



3- fill the version number as the tag name in the usual format (we don't need the "v" letter but the github action will clean it so don't worry about it) and fill the release title and description (release notes) and then click "Publish release"



That is it, the github action will take care of everything (it need about ten minutes to finish), the version will auto updated in the ProductPath.ini file and the Update zip file will auto replaced in the ftp folder

TestTool: trend logs

Reading a controller's trend logs from the test tool rather than from the T3000 screens, which is how a fault is confirmed against raw history.

T3 Controller Manual Data Logging Guide

Since the firmware does not support most of the functions on the menu, the T3000 software cannot automatically generate the Trendlog view like products such as the T3-BB. Our newly developed T3-3IIC supports this set of functions. In this case, I can only guide you on how to manually record data using the Test Tool.

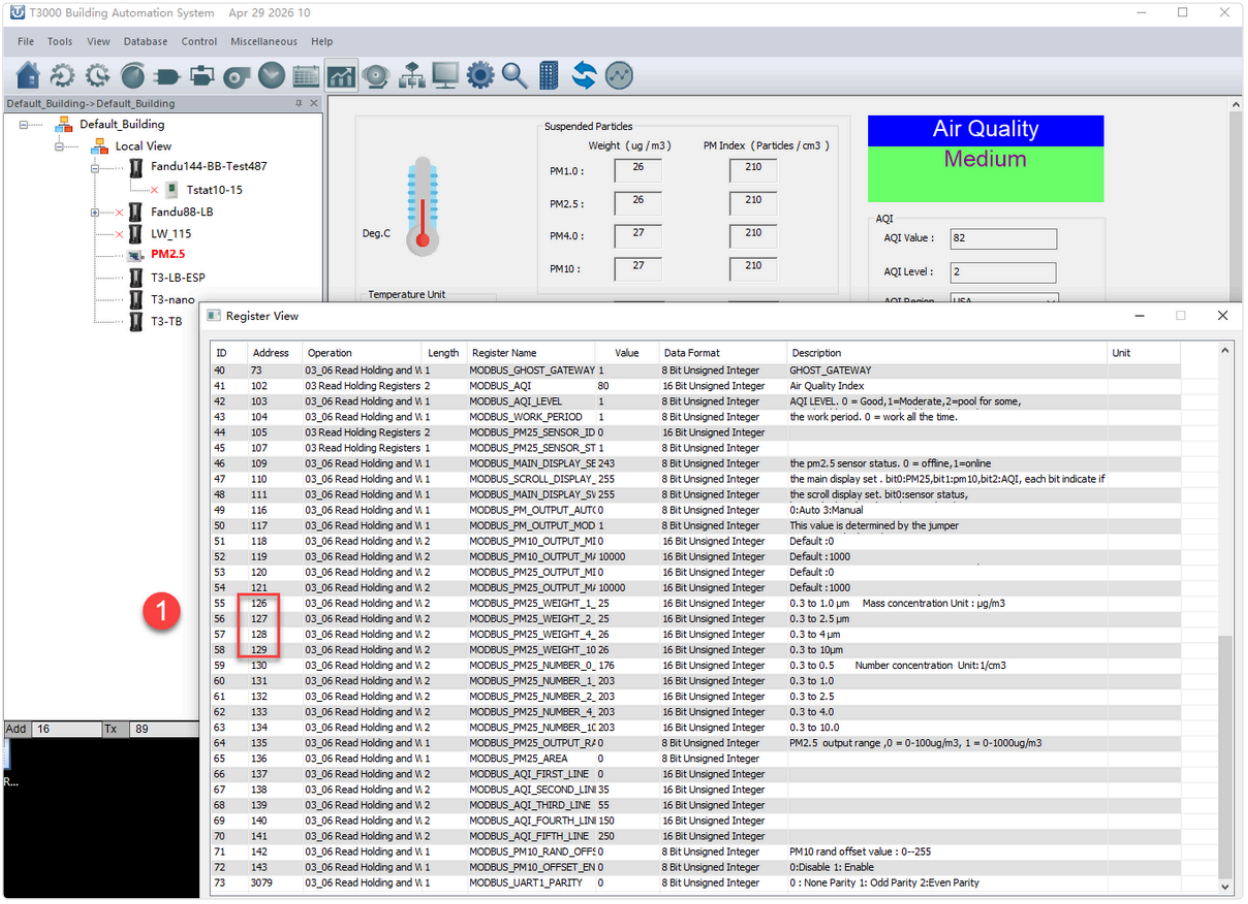
In Tab 1 of the Tool-Register viewer, confirm the registers you want to sample. For example, registers 126, 127, 128, and 129 correspond to PM2.5 values.

Open the TestTool located in the T3000 folder, then select 1.Register Monitor under Function.

Select the device IP address or COM port in Connection.

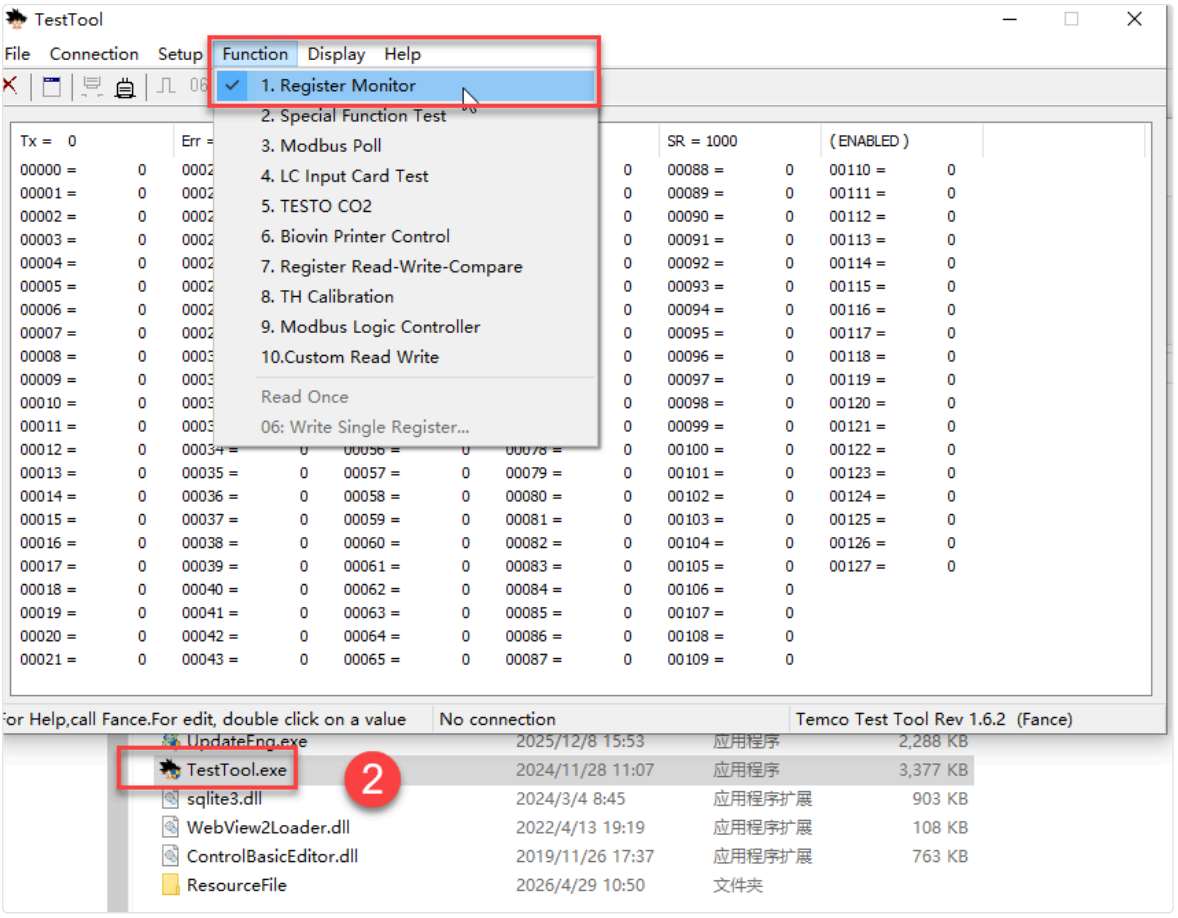
(Note: If using a COM port, please close the T3000 software first)

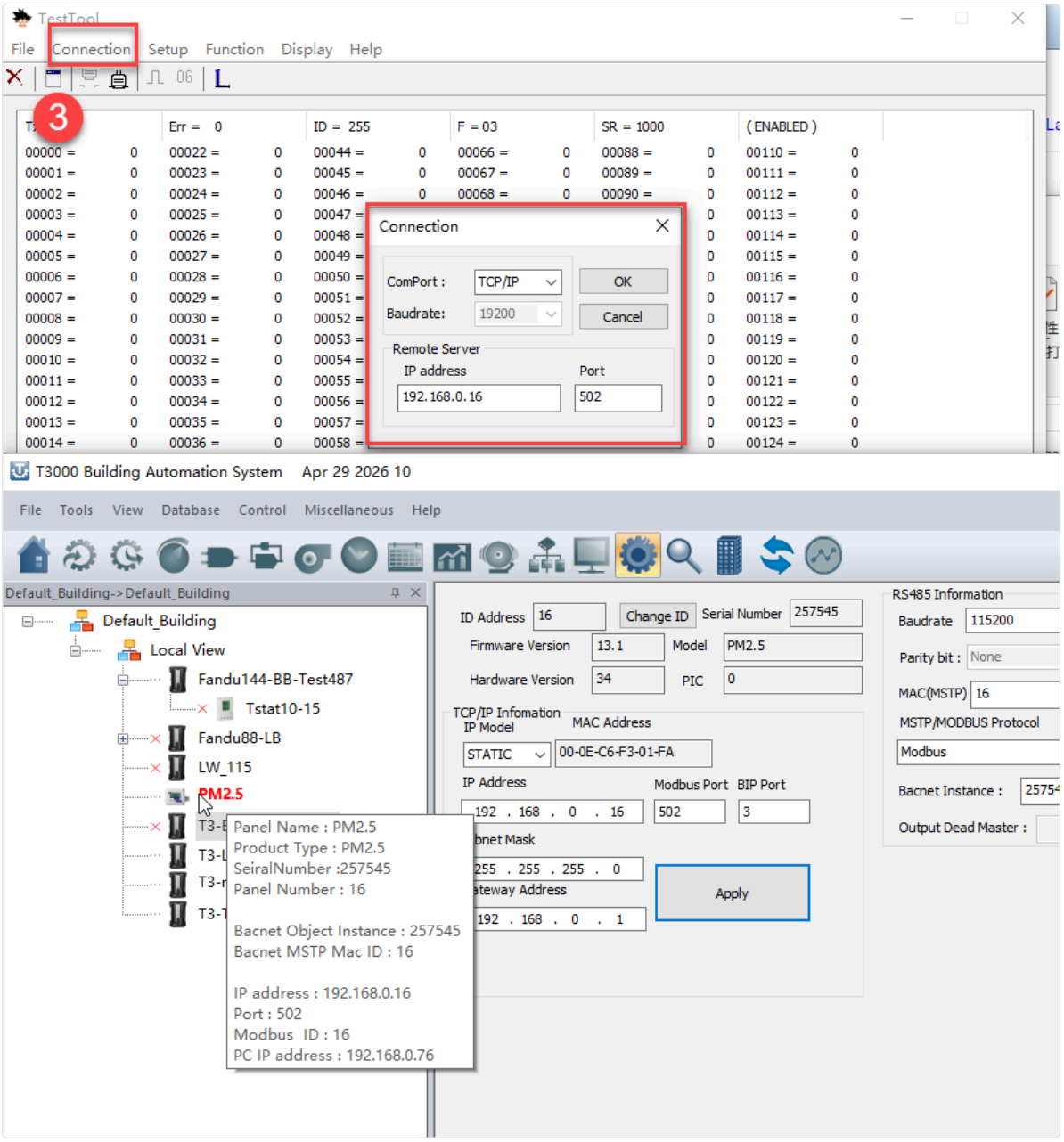
Enter the target registers and set the sampling interval time, then click the button to start sampling.

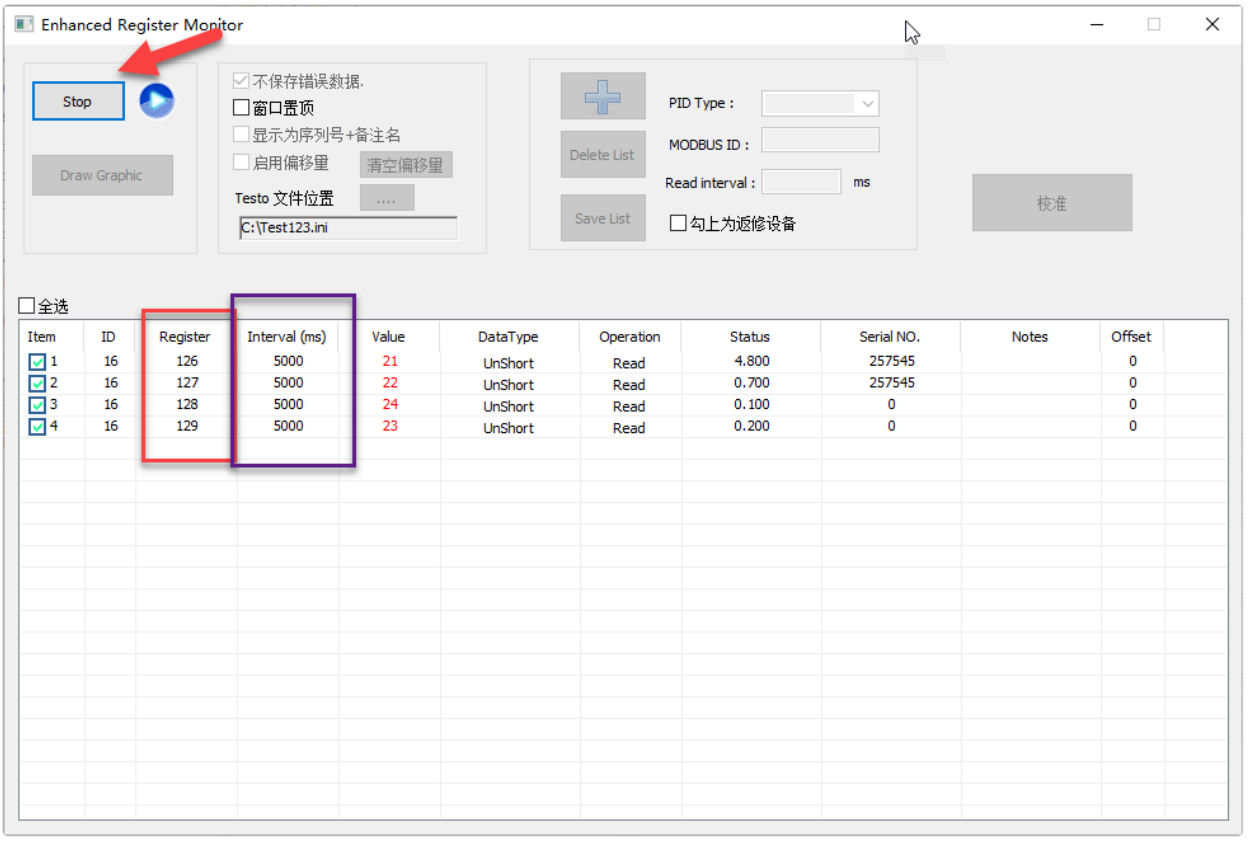


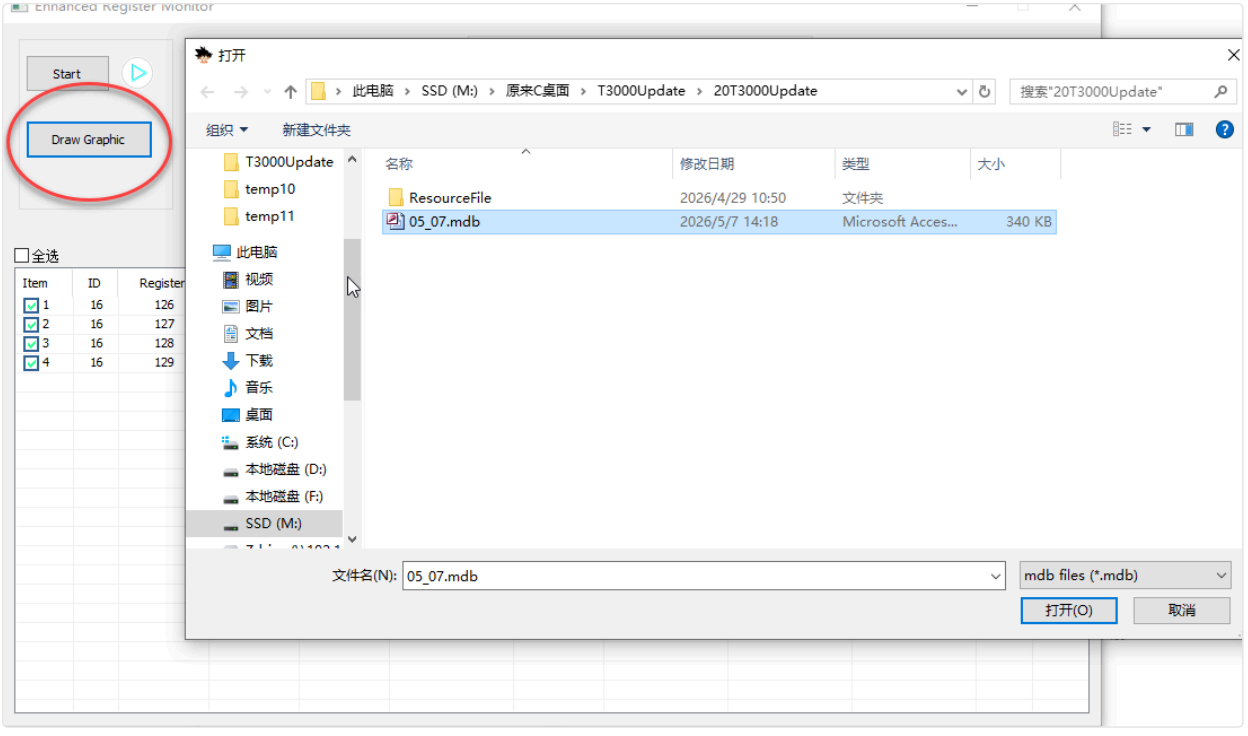
ID	Address	Operation	Length	Register Name	Value	Data Format	Description	Unit
40	73	03_06 Read Holding and W	1	MODBUS_GHOST_GATEWAY	1	8 Bit Unsigned Integer	GHOST_GATEWAY	
41	102	03 Read Holding Registers	2	MODBUS_AQI	80	16 Bit Unsigned Integer	Air Quality Index	
42	103	03_06 Read Holding and W	1	MODBUS_AQI_LEVEL	1	8 Bit Unsigned Integer	AQI LEVEL. 0 = Good,1=Moderate,2=pool for some,	
43	104	03_06 Read Holding and W	1	MODBUS_WORK_PERIOD	1	8 Bit Unsigned Integer	the work period. 0 = work all the time.	
44	105	03 Read Holding Registers	2	MODBUS_PM25_SENSOR_ID	0	16 Bit Unsigned Integer		
45	107	03 Read Holding Registers	1	MODBUS_PM25_SENSOR_ST	1	8 Bit Unsigned Integer		
46	109	03_06 Read Holding and W	1	MODBUS_MAIN_DISPLAY_SE	243	8 Bit Unsigned Integer	the pm2.5 sensor status. 0 = offline,1=online	
47	110	03_06 Read Holding and W	1	MODBUS_SCROLL_DISPLAY_	255	8 Bit Unsigned Integer	the main display set . bit0:PM25,bit1:pm10,bit2:AQI, each bit indicate if	
48	111	03_06 Read Holding and W	1	MODBUS_MAIN_DISPLAY_SV	255	8 Bit Unsigned Integer	the scroll display set. bit0:sensor status,	
49	116	03_06 Read Holding and W	1	MODBUS_PM_OUTPUT_AUT(	0	8 Bit Unsigned Integer	0:Auto 3:Manual	
50	117	03_06 Read Holding and W	1	MODBUS_PM_OUTPUT_MOD	1	8 Bit Unsigned Integer	This value is determined by the jumper	
51	118	03_06 Read Holding and W	2	MODBUS_PM10_OUTPUT_MI	0	16 Bit Unsigned Integer	Default :0	
52	119	03_06 Read Holding and W	2	MODBUS_PM10_OUTPUT_M/	10000	16 Bit Unsigned Integer	Default :1000	
53	120	03_06 Read Holding and W	2	MODBUS_PM25_OUTPUT_MI	0	16 Bit Unsigned Integer	Default :0	
54	121	03_06 Read Holding and W	2	MODBUS_PM25_OUTPUT_M/	10000	16 Bit Unsigned Integer	Default :1000	
55	126	03_06 Read Holding and W	2	MODBUS_PM25_WEIGHT_1_	25	16 Bit Unsigned Integer	0.3 to 1.0 μm Mass concentration Unit : μg/m3	
56	127	03_06 Read Holding and W	2	MODBUS_PM25_WEIGHT_2_	25	16 Bit Unsigned Integer	0.3 to 2.5 μm	
57	128	03_06 Read Holding and W	2	MODBUS_PM25_WEIGHT_4_	26	16 Bit Unsigned Integer	0.3 to 4 μm	
58	129	03_06 Read Holding and W	2	MODBUS_PM25_WEIGHT_10	26	16 Bit Unsigned Integer	0.3 to 10μm	
59	130	03_06 Read Holding and W	2	MODBUS_PM25_NUMBER_0_	176	16 Bit Unsigned Integer	0.3 to 0.5 Number concentration Unit: 1/cm3	
60	131	03_06 Read Holding and W	2	MODBUS_PM25_NUMBER_1_	203	16 Bit Unsigned Integer	0.3 to 1.0	
61	132	03_06 Read Holding and W	2	MODBUS_PM25_NUMBER_2_	203	16 Bit Unsigned Integer	0.3 to 2.5	
62	133	03_06 Read Holding and W	2	MODBUS_PM25_NUMBER_4_	203	16 Bit Unsigned Integer	0.3 to 4.0	
63	134	03_06 Read Holding and W	2	MODBUS_PM25_NUMBER_1(	203	16 Bit Unsigned Integer	0.3 to 10.0	
64	135	03_06 Read Holding and W	1	MODBUS_PM25_OUTPUT_R/	0	8 Bit Unsigned Integer	PM2.5 output range ,0 = 0-100ug/m3, 1 = 0-1000ug/m3	
65	136	03_06 Read Holding and W	1	MODBUS_PM25_AREA	0	8 Bit Unsigned Integer		
66	137	03_06 Read Holding and W	2	MODBUS_AQI_FIRST_LINE	0	16 Bit Unsigned Integer		
67	138	03_06 Read Holding and W	2	MODBUS_AQI_SECOND_LIN	35	16 Bit Unsigned Integer		
68	139	03_06 Read Holding and W	2	MODBUS_AQI_THIRD_LINE	55	16 Bit Unsigned Integer		
69	140	03_06 Read Holding and W	2	MODBUS_AQI_FOURTH_LIN	150	16 Bit Unsigned Integer		
70	141	03_06 Read Holding and W	2	MODBUS_AQI_FIFTH_LINE	250	16 Bit Unsigned Integer		
71	142	03_06 Read Holding and W	1	MODBUS_PM10_RAND_OFFS	0	8 Bit Unsigned Integer	PM10 rand offset value : 0--255	
72	143	03_06 Read Holding and W	1	MODBUS_PM10_OFFSET_EN	0	8 Bit Unsigned Integer	0:Disable 1: Enable	
73	3079	03_06 Read Holding and W	1	MODBUS_UART1_PARITY	0	8 Bit Unsigned Integer	0 : None Parity 1: Odd Parity 2:Even Parity	

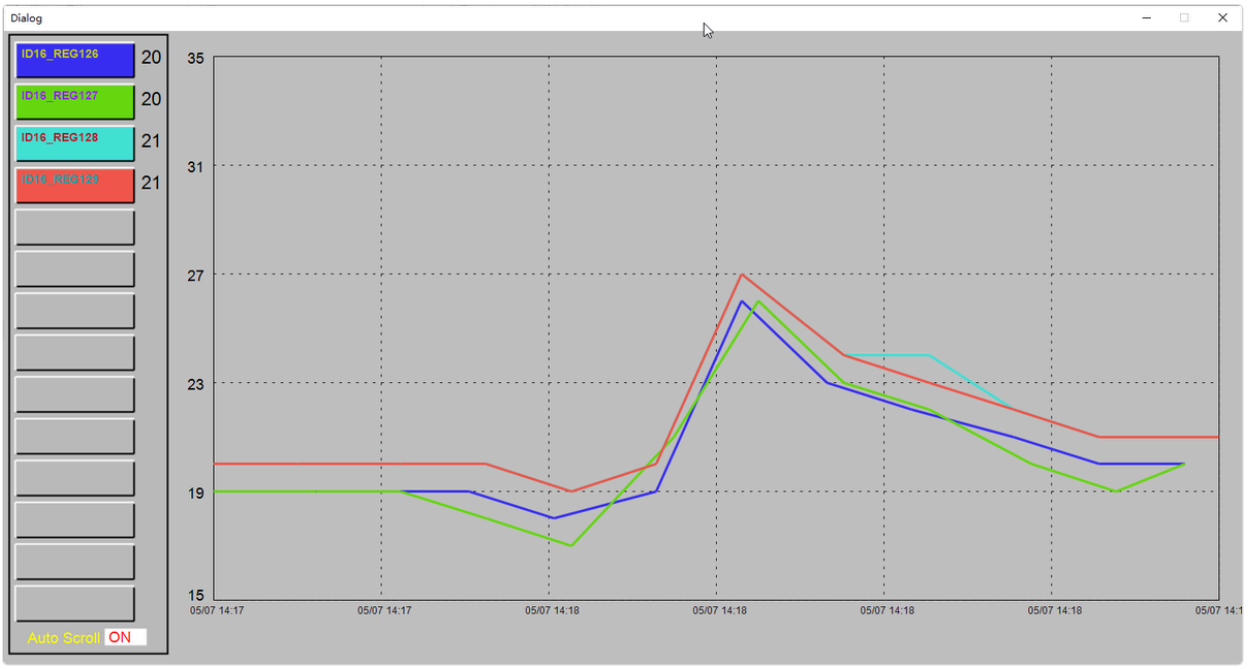
After finishing sampling, you can manually load the data to generate the view. You can also manually open the database file (Windows Office is required to open it normally).

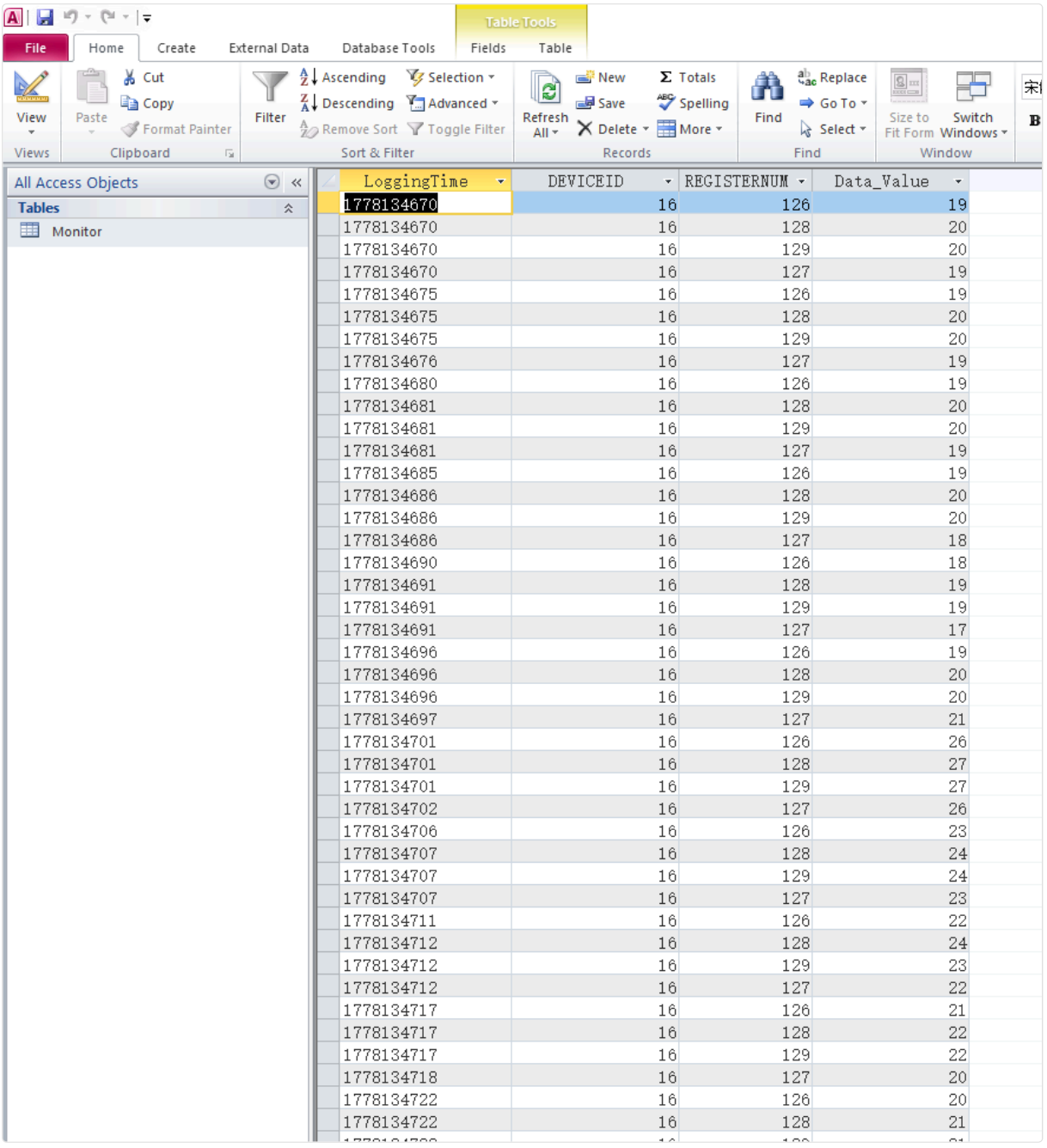

TestTool
File Connection Setup Function Display Help
1. Register Monitor
2. Special Function Test
3. Modbus Poll
4. LC Input Card Test
5. TESTO CO2
6. Biovin Printer Control
7. Register Read-Write-Compare
8. TH Calibration
9. Modbus Logic Controller
10.Custom Read Write
Read Once
06: Write Single Register...
Tx = 0
SR = 1000
(ENABLED)
For Help,call Fance.For edit, double click on a value
No connection
Temco Test Tool Rev 1.6.2 (Fance)
UpdateEng.exe 2025/12/8 15:53 应用程序 2,288 KB
TestTool.exe 2024/11/28 11:07 应用程序 3,377 KB
2
sqlite3.dll 2024/3/4 8:45 应用程序扩展 903 KB
WebView2Loader.dll 2022/4/13 19:19 应用程序扩展 108 KB
ControlBasicEditor.dll 2019/11/26 17:37 应用程序扩展 763 KB
ResourceFile 2026/4/29 10:50 文件夹


TestTool
File
Connection
Setup
Function
Display
Help
3
Connection
ComPort :
TCP/IP
Baudrate:
19200
OK
Cancel
Remote Server
IP address
192.168.0.16
Port
502
T3000 Building Automation System
Apr 29 2026 10
File
Tools
View
Database
Control
Miscellaneous
Help
Default_Building->Default_Building
Default_Building
Local View
Fandu144-BB-Test487
Tstat10-15
Fandu88-LB
LW_115
PM2.5
Panel Name : PM2.5
Product Type : PM2.5
SeiralNumber :257545
Panel Number : 16
Bacnet Object Instance : 257545
Bacnet MSTP Mac ID : 16
IP address : 192.168.0.16
Port : 502
Modbus ID : 16
PC IP address : 192.168.0.76
ID Address
16
Change ID
Serial Number
257545
Firmware Version
13.1
Model
PM2.5
Hardware Version
34
PIC
0
TCP/IP Infomation
IP Model
MAC Address
STATIC
00-0E-C6-F3-01-FA
IP Address
Modbus Port
BIP Port
502
3
Apply
RS485 Information
Baudrate
115200
Parity bit :
None
MAC(MSTP)
16
MSTP/MODBUS Protocol
Modbus
Bacnet Instance :
Output Dead Master :


Enhanced Register Monitor
Stop
Draw Graphic
不保存错误数据.
窗口置顶
显示为序列号+备注名
启用偏移量
清空偏移量
Testo 文件位置
....
C:\Test123.ini
Delete List
Save List
PID Type :
MODBUS ID :
Read interval :
ms
勾上为返修设备
校准
全选
Item ID Register Interval (ms) Value DataType Operation Status Serial NO. Notes Offset
1 16 126 5000 21 UnShort Read 4.800 257545 0
2 16 127 5000 22 UnShort Read 0.700 257545 0
3 16 128 5000 24 UnShort Read 0.100 0 0
4 16 129 5000 23 UnShort Read 0.200 0 0


Start
Draw Graphic
全选
打开
此电脑 › SSD (M:) › 原来C桌面 › T3000Update › 20T3000Update
搜索"20T3000Update"
组织
新建文件夹
T3000Update
temp10
temp11
此电脑
视频
图片
文档
下载
音乐
桌面
系统 (C:)
本地磁盘 (D:)
本地磁盘 (F:)
SSD (M:)
名称
修改日期
类型
大小
ResourceFile 2026/4/29 10:50 文件夹
05_07.mdb 2026/5/7 14:18 Microsoft Acces... 340 KB
文件名(N): 05_07.mdb
mdb files (*.mdb)
打开(O)
取消


Dialog
ID16_REG126
20
ID16_REG127
20
ID16_REG128
21
ID16_REG129
21
Auto Scroll
ON
35
31
27
23
19
15
05/07 14:17
05/07 14:17
05/07 14:18
05/07 14:18
05/07 14:18
05/07 14:18



LoggingTime	DEVICEID	REGISTERNUM	Data_Value
1778134670	16	126	19
1778134670	16	128	20
1778134670	16	129	20
1778134670	16	127	19
1778134675	16	126	19
1778134675	16	128	20
1778134675	16	129	20
1778134676	16	127	19
1778134680	16	126	19
1778134681	16	128	20
1778134681	16	129	20
1778134681	16	127	19
1778134685	16	126	19
1778134686	16	128	20
1778134686	16	129	20
1778134686	16	127	18
1778134690	16	126	18
1778134691	16	128	19
1778134691	16	129	19
1778134691	16	127	17
1778134696	16	126	19
1778134696	16	128	20
1778134696	16	129	20
1778134697	16	127	21
1778134701	16	126	26
1778134701	16	128	27
1778134701	16	129	27
1778134702	16	127	26
1778134706	16	126	23
1778134707	16	128	24
1778134707	16	129	24
1778134707	16	127	23
1778134711	16	126	22
1778134712	16	128	24
1778134712	16	129	23
1778134712	16	127	22
1778134717	16	126	21
1778134717	16	128	22
1778134717	16	129	22
1778134718	16	127	20
1778134722	16	126	20
1778134722	16	128	21

TestTool: recording data

Capturing a session of live point values so that a fault which has already passed can still be looked at.

Download TestTool.exe

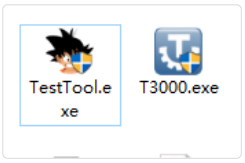
Please Update your T3000 software to fix this bugs .

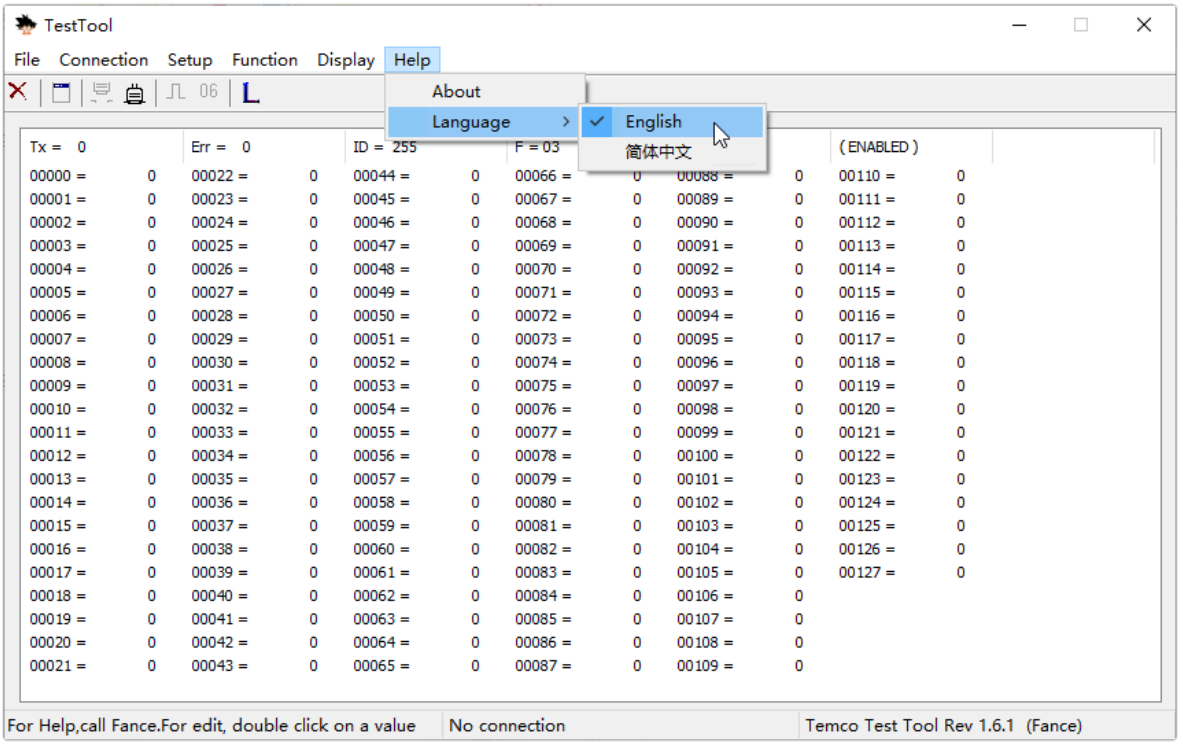
Help->Check for updates ->Update T3000 .

The T3000.exe directory will contain the latest version of TestTool.exe

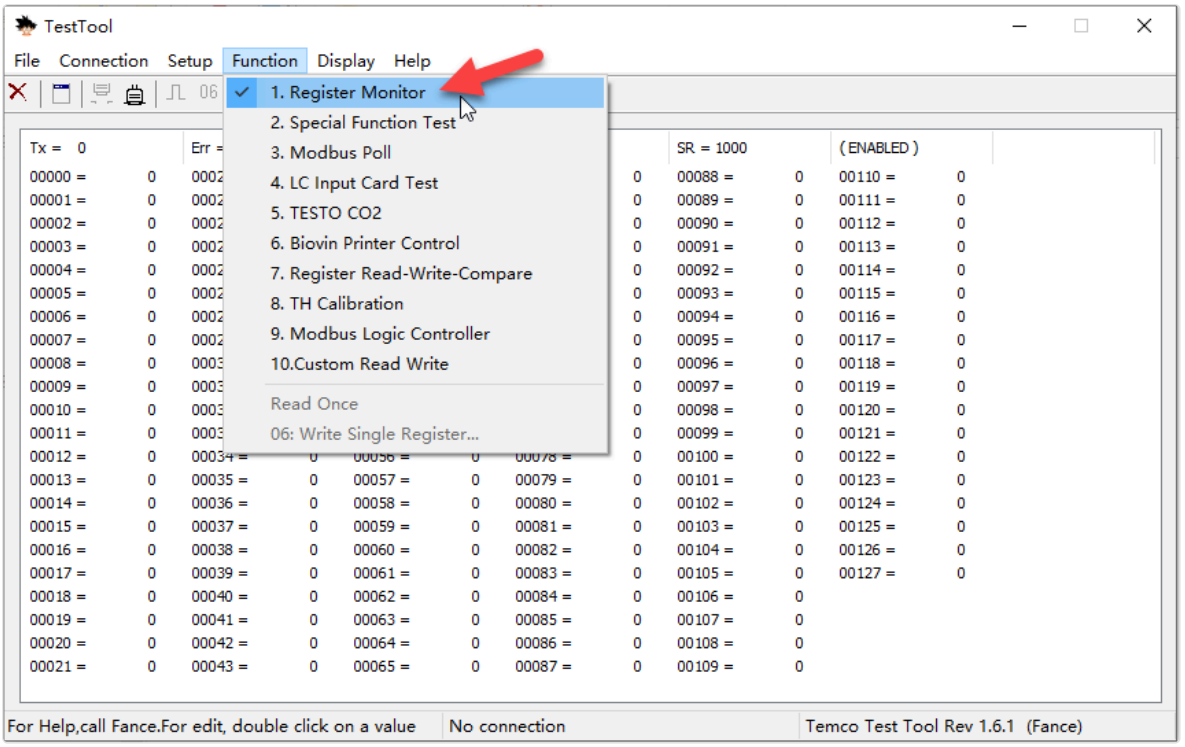
https://temcocontrols.com/ftp/software/20T3000Update.zip

You can also download it directly from here and extract it to any directory

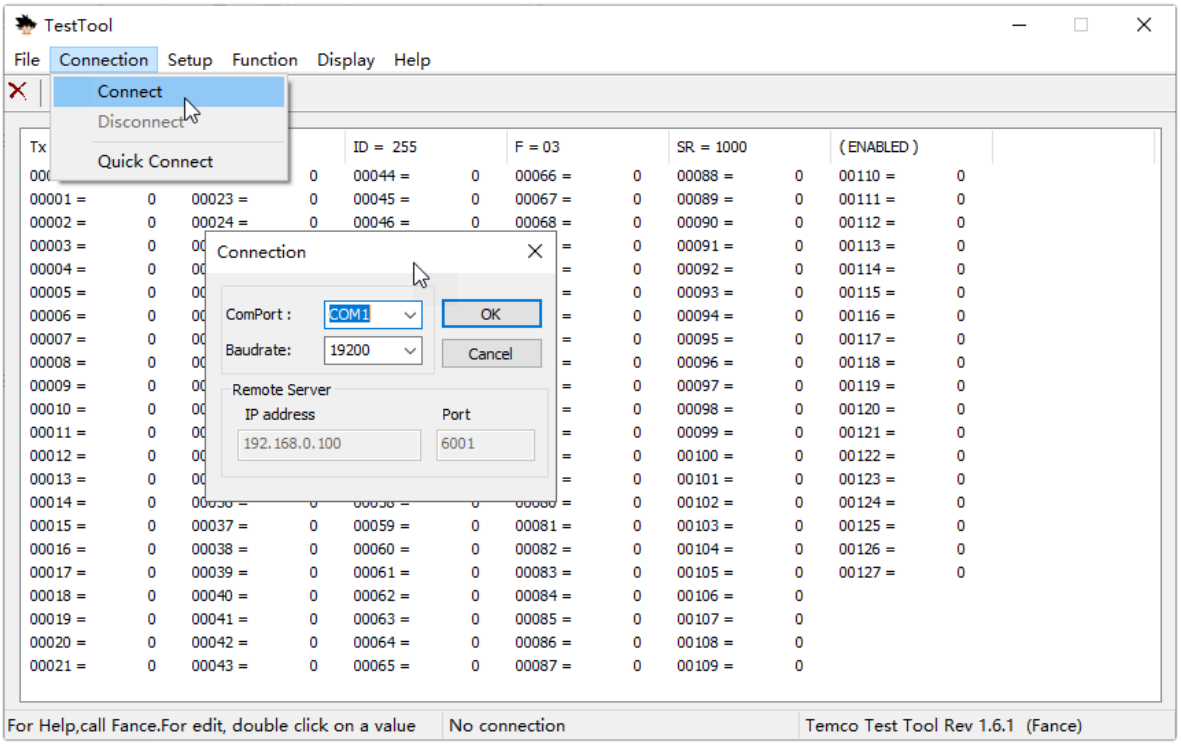




Select Function 1. Register Monitor



Connect your device

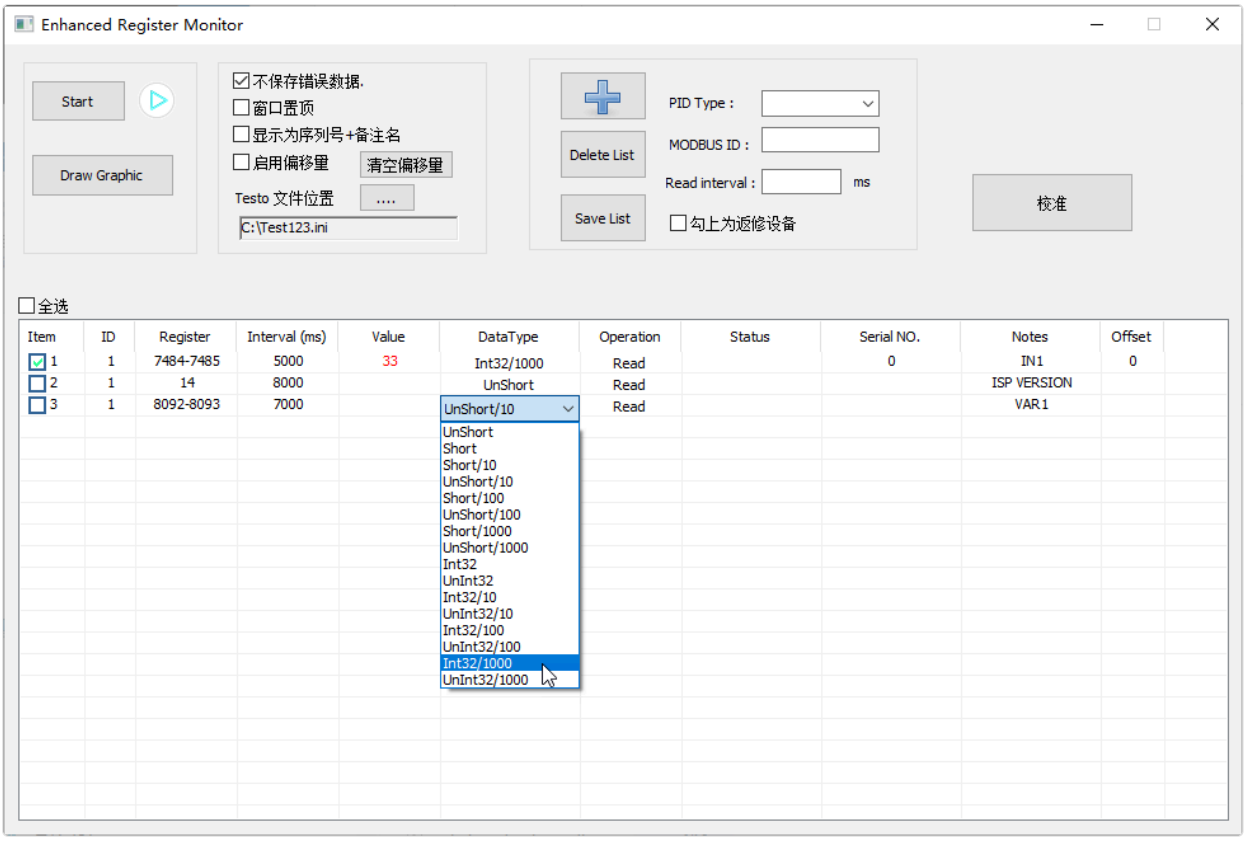


Fill in the Modbus ID, register, sampling time, data type that you want to monitor

Click the "+" sign to add a line

Input 1 value = (Reg7484 * 65536 + Reg7485) / 1000

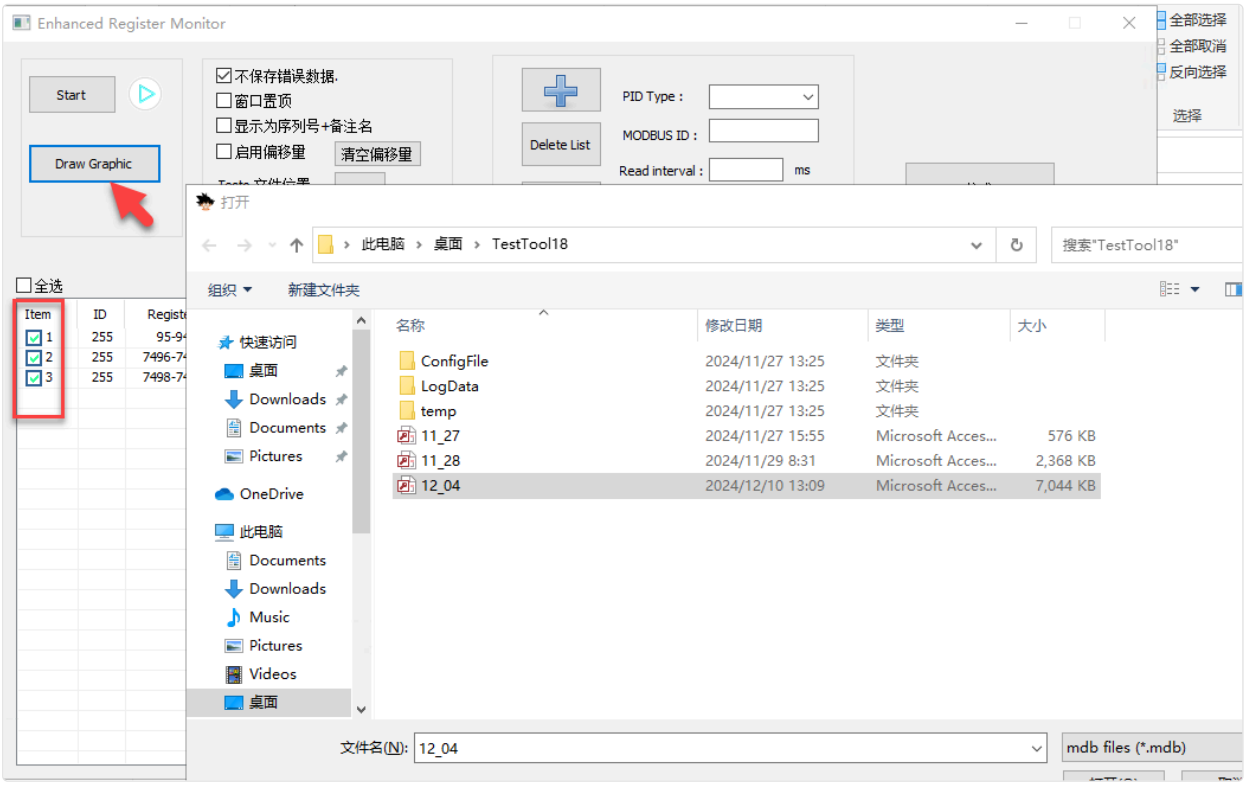
If it is a combination of two registers, please fill in "Register" with something like 7484-7485, or use the register number if it is a single register.



Click start to start recording

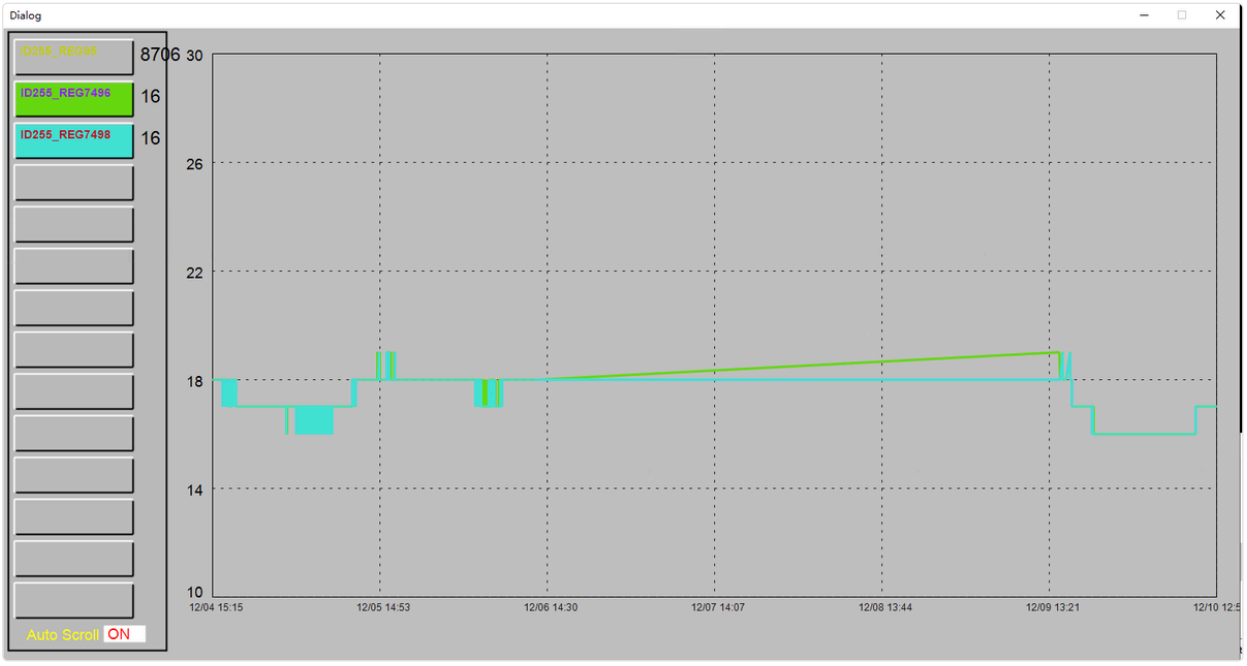
When the recording ends, an access database is generated in the TestTool.exe directory with the current date. Click Draw Graphic to select the database to draw the image

Select the row that you want to draw



You can also open *.mdb directly to see the results

LoggingTime	DEVICEID	REGISTERNUM	Data_Value
1733456739	255	95	88526
1733456733	255	95	88520
1733456727	255	95	88514
1733456721	255	95	88509
1733456715	255	95	88503
1733456710	255	95	88497
1733456704	255	95	88491
1733456698	255	95	88485
1733456692	255	95	88479
1733456686	255	95	88474
1733456680	255	95	88468
1733456675	255	95	88462
1733456669	255	95	88456
1733456663	255	95	88450
1733456657	255	95	88444
1733456651	255	95	88439



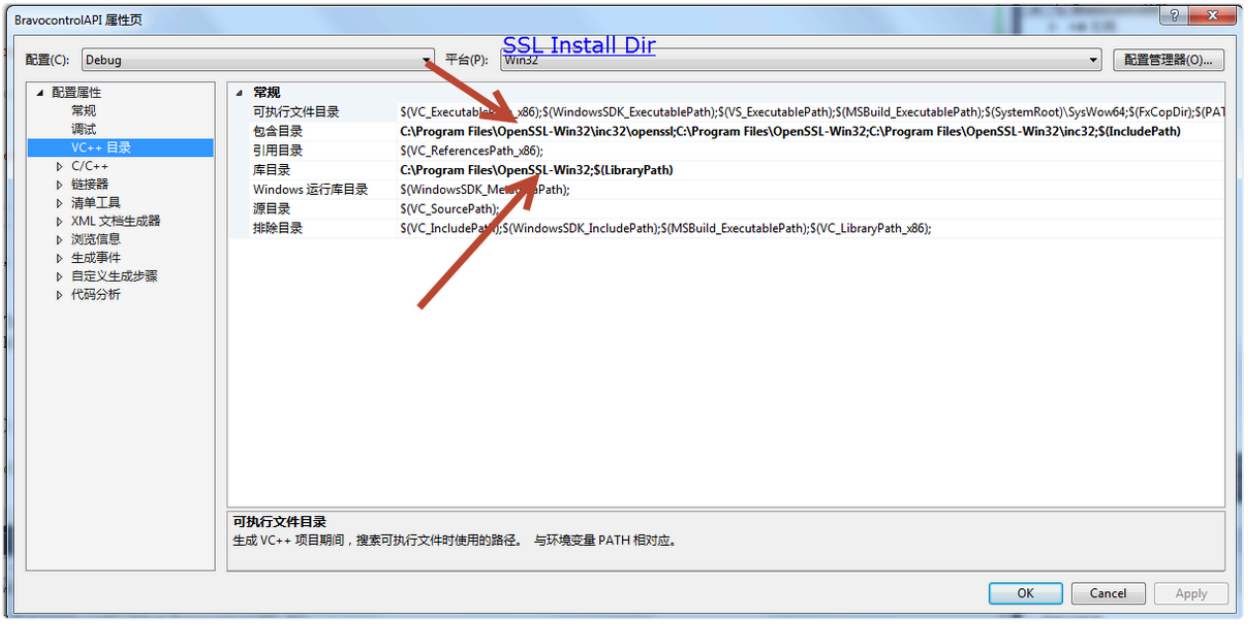
Building the Bravo Controls API

The Bravo Controls API is a separate project in the repository, built and pointed at a running T3000. The sources are in `broveControls_API` beside the solution.

Download SSL Win32 from website and install it ;

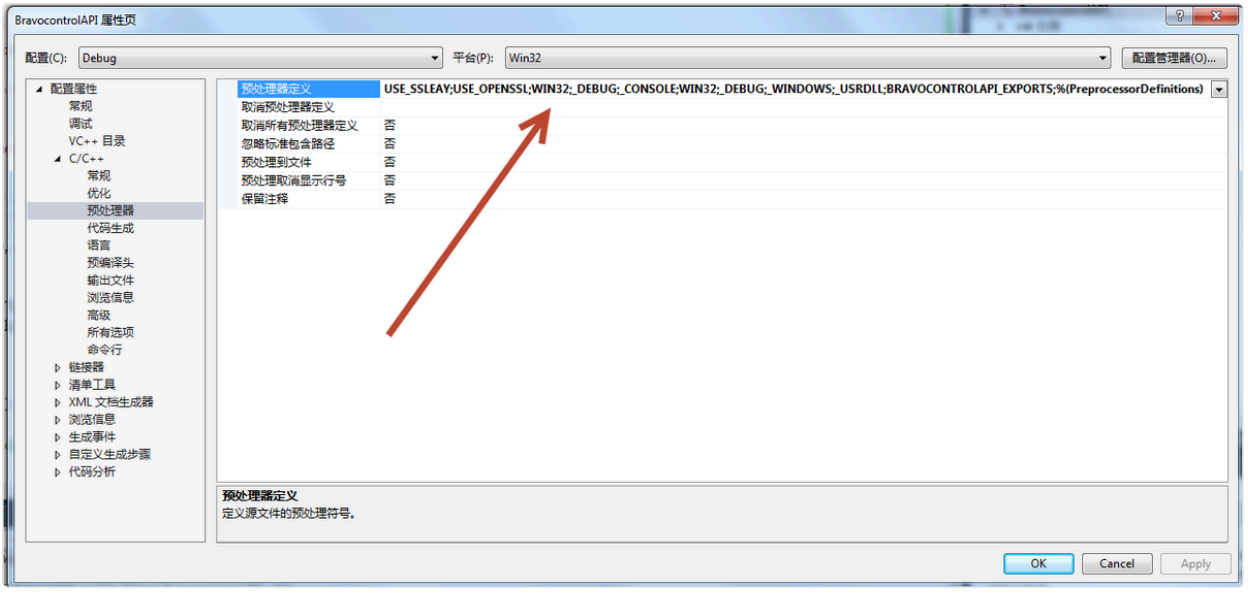
https://slproweb.com/

Project configuration



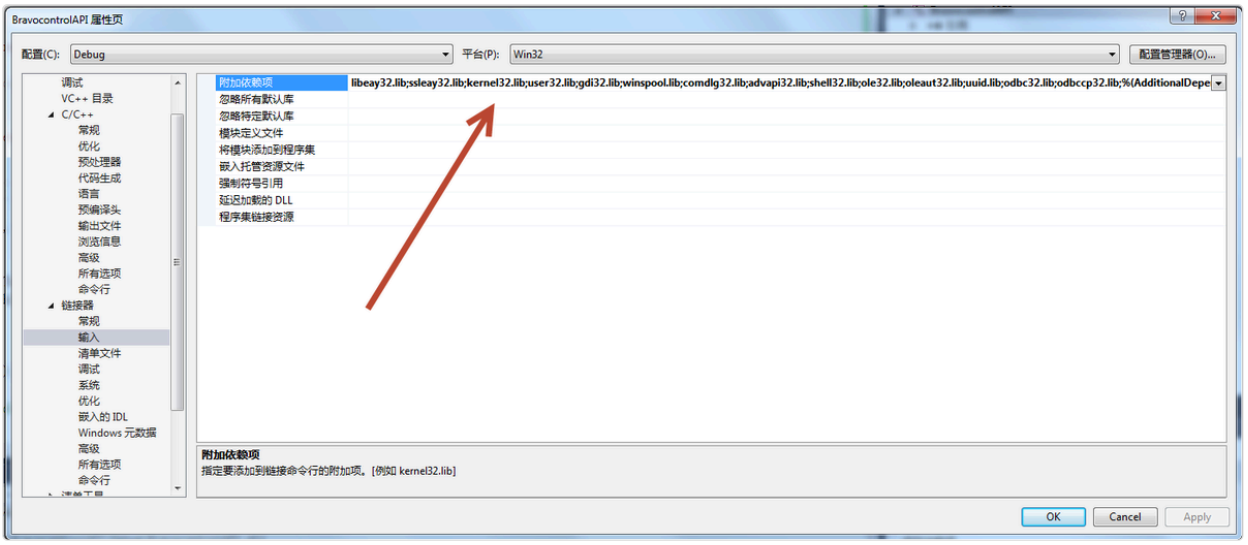
Change the pre define

USE_SSLEAY;USE_OPENSSL;WIN32;_DEBUG;_CONSOLE;WIN32;_DEBUG;_WINDOWS;_USRDLL;BRAVOCONTROLAPI_EXPORTS;%(PreprocessorDefinitions)



Change the additional libiary

libeay32.lib;ssleay32.lib;kernel32.lib;user32.lib;gdi32.lib;winspool.lib;comdlg32.lib;advapi32.lib;shell32.lib;ole32.lib;oleaut32.lib;uuid.lib;odbc32.lib;odbccp32.lib;%(AdditionalDependencies)

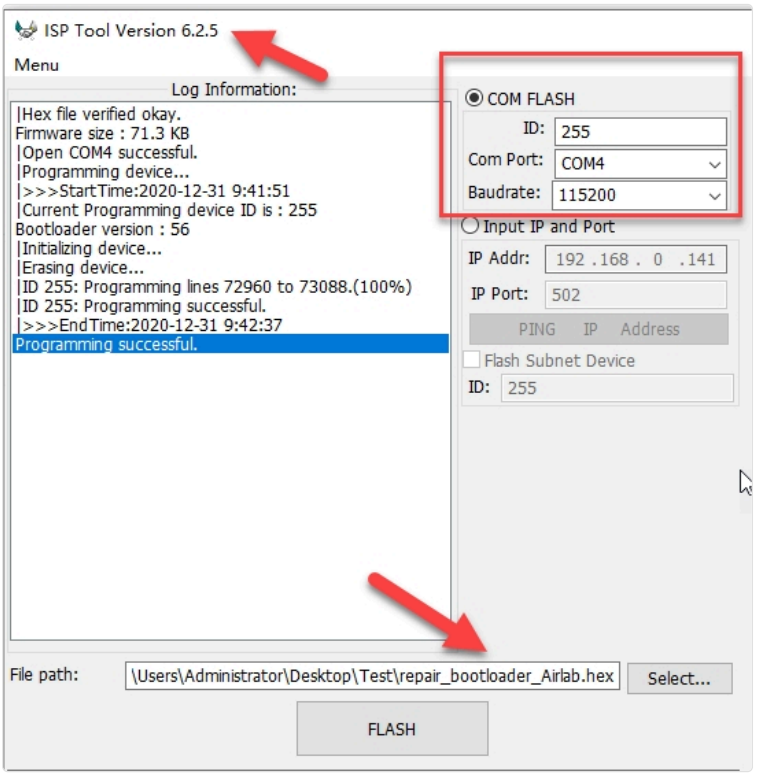

BravocontrolAPI 属性页
配置(C): Debug
平台(P): Win32
配置管理器(O)...
调试
VC++ 目录
C/C++
常规
优化
预处理器
代码生成
语言
预编译头
输出文件
浏览信息
高级
所有选项
命令行
链接器
常规
输入
清单文件
调试
系统
优化
嵌入的 IDL
Windows 元数据
高级
所有选项
命令行
附加依赖项
忽略所有默认库
忽略特定默认库
模块定义文件
将模块添加到程序集
嵌入托管资源文件
强制符号引用
延迟加载的 DLL
程序集链接资源
libeay32.lib;ssleay32.lib;kernel32.lib;user32.lib;gdi32.lib;winspool.lib;comdlg32.lib;advapi32.lib;shell32.lib;ole32.lib;oleaut32.lib;uuid.lib;odbc32.lib;odbccp32.lib;%(AdditionalDepe
附加依赖项
指定要添加到链接命令行的附加项。[例如 kernel32.lib]
OK
Cancel
Apply

Updating the Airlab bootloader

Airlab is a T3 variant with its own bootloader path. Doing it by hand is the recovery route when the normal update will not run.

The update of Airlab bootloader is non-mandatory, so we hope you can do the following manually。

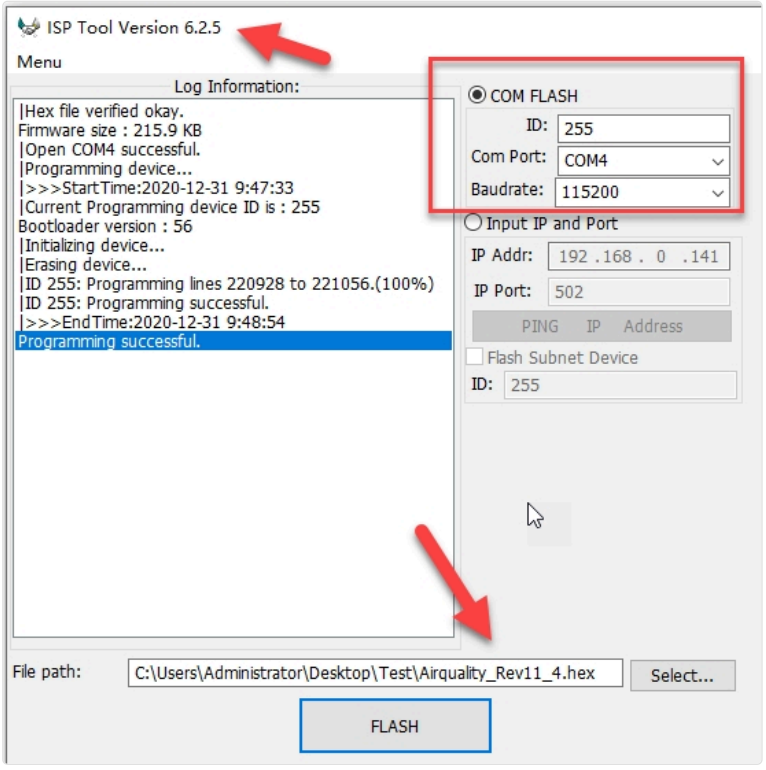
Step 1 . Use ISP Tool to select the repair_bootloader_airlab.hex in the attachment. Use RS485 port ,Select the corresponding ID and baudrate. After this step is updated, the device's screen will always be in a double-flash state.



Step 2 . Select the latest version of Airlab firmware, version 11.4, as attached

Also use RS485 to update the firmware airquality_rev11_4.hex

When the update is complete, the device is in normal working mode.



Step 3 . Using the T3000 software, you can know the device's Wifi IP Address

Then we can use this IP address to update the required firmware

